



SiFive P550 Core Complex Manual

21G2.01.00

Copyright © 2021 by SiFive, Inc. All rights reserved.

SiFive P550 Core Complex Manual

Proprietary Notice

Copyright © 2021 by SiFive, Inc. All rights reserved.

SiFive P550 Core Complex Manual by SiFive, Inc. is licensed under Attribution-NonCommercialNoDerivatives 4.0 International. To view a copy of this license, visit: <http://creativecommons.org/licenses/by-nc-nd/4.0>

Information in this document is provided “as is,” with all faults.

SiFive expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SiFive does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SiFive reserves the right to make changes without further notice to any products herein.

Contents

List of Tables	12
List of Figures	20
1 Introduction	25
1.1 About this Document	25
1.2 About this Release	26
1.3 P550 Core Complex Overview	26
1.4 P550 RISC-V Core	27
1.5 Memory System	28
1.6 Interrupts	28
1.7 Debug Support	28
1.8 Compliance	29
2 List of Abbreviations and Terms	30
3 P550 RISC-V Core	32
3.1 Supported Modes	33
3.2 Instruction Memory System	34
3.2.1 Execution Memory Space	34
3.2.2 L1 Instruction Cache	34
3.2.3 Cache Maintenance	35
3.2.4 Coherence with Higher Level Caches	35
3.2.5 Instruction Fetch Unit	35
3.2.6 Branch Prediction	36
3.3 Rename and Dispatch Unit	37
3.4 Execution Pipeline	37
3.5 Load/Store Unit	39
3.5.1 Hardware Prefetcher	39

3.6	Data Memory System	41
3.6.1	L1 Data Cache	41
3.6.2	Cache Maintenance Operations	42
3.6.3	Coherence with Higher Level Caches	42
3.7	Atomic Memory Operations	42
3.8	Floating-Point Unit (FPU)	43
3.9	Virtual Memory Support	43
3.9.1	Address and Page Table Formats	44
3.9.2	Supervisor Address Translation and Protection Register (SATP)	47
3.9.3	Supervisor Memory-Management Fence Instruction (SFENCE.VMA)	49
3.9.4	Scenarios that Require SFENCE.VMA Instruction	50
3.9.5	Trap Virtual Memory	51
3.9.6	Virtual Address Translation Process	51
3.9.7	Virtual-to-Physical Mapping Example	52
3.9.8	MMU at Reset	55
3.10	Physical Memory Protection (PMP)	55
3.10.1	PMP Functional Description	55
3.10.2	PMP Region Locking	56
3.10.3	PMP Registers	56
3.10.4	PMP and PMA	58
3.10.5	PMP Programming Overview	58
3.10.6	PMP and Paging	60
3.10.7	PMP Limitations	60
3.10.8	Behavior for Regions without PMP Protection	61
3.10.9	Cache Flush Behavior on PMP Protected Region	61
3.11	Hardware Performance Monitor	61
3.11.1	Performance Monitoring Counters Reset Behavior	61
3.11.2	Fixed-Function Performance Monitoring Counters	61
3.11.3	Event-Programmable Performance Monitoring Counters	62
3.11.4	Event Selector Registers	62
3.11.5	Event Selector Encodings	62
3.11.6	Counter-Enable Registers	71
3.12	Ports	72

3.12.1	Front Port	72
3.12.2	Memory Port	72
3.12.3	Peripheral Port	72
3.12.4	System Port	73
4	Physical Memory Attributes and Memory Map	74
4.1	Physical Memory Attributes Overview	74
4.2	Memory Map	76
5	Programmer's Model.....	77
5.1	Base Instruction Formats.....	77
5.2	I Extension: Standard Integer Instructions	78
5.2.1	R-Type (Register-Based) Integer Instructions	79
5.2.2	I-Type Integer Instructions	80
5.2.3	I-Type Load Instructions.....	81
5.2.4	S-Type Store Instructions	82
5.2.5	Unconditional Jumps	83
5.2.6	Conditional Branches.....	84
5.2.7	Upper-Immediate Instructions.....	85
5.2.8	Memory Ordering Operations	85
5.2.9	Environment Call and Breakpoints	86
5.2.10	NOP Instruction.....	86
5.3	M Extension: Multiplication Operations.....	86
5.3.1	Division Operations	87
5.4	A Extension: Atomic Operations	87
5.4.1	Atomic Load-Reserve and Store-Conditional Instructions	87
5.4.2	Atomic Memory Operations (AMOs)	88
5.5	F Extension: Single-Precision Floating-Point Instructions	89
5.5.1	Floating-Point Control and Status Registers	89
5.5.2	Rounding Modes	90
5.5.3	Single-Precision Floating-Point Load and Store Instructions	90
5.5.4	Single-Precision Floating-Point Computational Instructions	91
5.5.5	Single-Precision Floating-Point Conversion and Move Instructions.....	91

5.5.6	Single-Precision Floating-Point Compare Instructions	94
5.6	D Extension: Double-Precision Floating-Point Instructions	95
5.6.1	Double-Precision Floating-Point Load and Store Instructions	95
5.6.2	Double-Precision Floating-Point Computational Instructions	96
5.6.3	Double-Precision Floating-Point Conversion and Move Instructions	97
5.6.4	Double-Precision Floating-Point Compare Instructions	100
5.6.5	Double-Precision Floating-Point Classify Instruction	101
5.7	C Extension: Compressed Instructions	101
5.7.1	Compressed 16-bit Instruction Formats	101
5.7.2	Stack-Pointed-Based Loads and Stores	102
5.7.3	Register-Based Loads and Stores	103
5.7.4	Control Transfer Instructions	104
5.7.5	Integer Computational Instructions	105
5.8	B Extension: Bit Manipulation Instructions	108
5.8.1	Zba Extension	108
5.8.2	Zbb Extension	109
5.9	Zicsr Extension: Control and Status Register Instructions	113
5.9.1	Control and Status Registers	115
5.9.2	Defined CSRs	115
5.9.3	CSR Access Ordering	119
5.9.4	SiFive RISC-V Implementation Version Registers	119
5.9.5	Custom CSRs	121
5.10	Base Counters and Timers	122
5.10.1	Timer Register	123
5.10.2	Timer API	123
5.11	Privileged Instructions	124
5.11.1	Machine-Mode Privileged Instructions	124
5.11.2	Supervisor-Mode Privileged Instructions	125
5.12	ABI - Register File Usage and Calling Conventions	125
5.12.1	RISC-V Assembly	127
5.12.2	Assembler to Machine Code	127
5.12.3	Calling a Function (Calling Convention)	129
5.13	Memory Ordering - FENCE Instructions	131

5.14	Boot Flow	132
5.15	Linker File	133
5.15.1	Linker File Symbols	134
5.16	RISC-V Compiler Flags	135
5.16.1	arch, abi, and mtune.....	135
5.17	Compilation Process	139
5.18	Large Code Model Workarounds	139
5.18.1	RISC-V Code Model Summary	140
5.18.2	Enabling the Compact Code Model	140
5.19	Pipeline Hazards.....	141
5.19.1	Read-After-Write Hazards	141
5.19.2	Write-After-Write Hazards.....	142
5.20	Reading CSRs.....	142
6	Custom Instructions and CSRs.....	144
6.1	CFLUSH.D.L1.....	144
6.2	CDISCARD.D.L1	144
6.3	CEASE	145
6.4	PAUSE	145
6.5	Branch Prediction Mode CSR.....	145
6.6	SiFive Feature Disable CSRs.....	146
6.7	Other Custom Instructions	150
7	Interrupts and Exceptions.....	151
7.1	Interrupt Concepts	151
7.2	Exception Concepts	151
7.3	Trap Concepts	153
7.4	Interrupt Block Diagram	154
7.5	Local Interrupts.....	154
7.6	Interrupt Operation	155
7.6.1	Interrupt Entry and Exit	155
7.7	Interrupt Control and Status Registers	156
7.7.1	Machine Status Register (mstatus).....	156

7.7.2	Machine Trap Vector (mtvec).....	156
7.7.3	Machine Interrupt Enable (mie).....	158
7.7.4	Machine Interrupt Pending (mip).....	158
7.7.5	Machine Cause (mcause)	159
7.7.6	Minimum Interrupt Configuration	160
7.8	Supervisor Mode Interrupts	161
7.8.1	Delegation Registers (mideleg and medeleg)	161
7.8.2	Supervisor Status Register (sstatus)	162
7.8.3	Supervisor Interrupt Enable Register (sie)	163
7.8.4	Supervisor Interrupt Pending (sip).....	163
7.8.5	Supervisor Cause Register (scause).....	164
7.8.6	Supervisor Trap Vector (stvec)	165
7.8.7	Delegated Interrupt Handling	166
7.9	Interrupt Priorities	167
7.10	Interrupt Latency	167
7.11	Non-Maskable Interrupt	168
7.11.1	Handler Addresses	168
7.11.2	RNMI CSRs	168
7.11.3	MNRET Instruction	169
7.11.4	RNMI Operation	169
8	Core-Local Interruptor (CLINT).....	170
8.1	CLINT Priorities and Preemption	170
8.2	CLINT Vector Table	171
8.3	CLINT Interrupt Sources	173
8.4	CLINT Interrupt Attribute	173
8.5	CLINT Memory Map	174
8.6	Register Descriptions	174
8.6.1	MSIP Registers	174
8.6.2	Timer Registers.....	174
8.7	Supervisor Mode Delegation	175
9	Platform-Level Interrupt Controller (PLIC)	176

9.1	Memory Map	176
9.2	Interrupt Sources	178
9.3	Interrupt Priorities	178
9.4	Interrupt Pending Bits	179
9.5	Interrupt Enables	180
9.6	PLIC Clock Gate Disable	181
9.7	Priority Thresholds	181
9.8	Interrupt Claim Process	181
9.9	Interrupt Completion	182
9.10	Example PLIC Interrupt Handler	182
10	TileLink Error Device	184
11	Bus-Error Unit	185
11.1	Bus-Error Unit Overview	185
11.2	Memory Map	186
11.3	Reportable Errors	186
11.4	Functional Description	186
11.4.1	BEU Global Interrupt	187
11.4.2	BEU Local Interrupt	187
11.4.3	Global Interrupt Configuration	187
12	Private Level 2 Cache	188
12.1	Functional Description	189
12.1.1	Coherence and Ordering	189
12.1.2	Cache Maintenance	189
12.1.3	Error Correction Codes (ECC)	190
12.1.4	Private L2 Performance Monitor	191
12.2	Memory Map	195
12.3	Register Descriptions	196
12.3.1	PL2 Cache Configuration Register (Config)	196
12.3.2	ECC Error Injection Register (ECCInjectError)	197
12.3.3	ECC Error Injection Done Register (EccInjectErrorDone)	197

12.3.4	PL2 Cache Flush Register (Cflush64)	197
12.3.5	Flush Count Register (FlushCount)	199
12.3.6	PL2 Configuration Register (pL2Config).....	199
13	Level 3 Cache Controller	201
13.1	Level 3 Cache Controller Overview	201
13.2	Functional Description	201
13.2.1	Way Enable and the L3 Loosely-Integrated Memory (L3 LIM)	202
13.2.2	Way Masking and Locking	203
13.2.3	L3 Zero Device.....	203
13.2.4	L3 Features Access Summary	205
13.2.5	L3 Performance Monitor	205
13.2.6	Error Correction Codes (ECC)	210
13.2.7	Coherence	211
13.3	Memory Map	211
13.4	Register Descriptions	213
13.4.1	L3 Cache Configuration Register (Config).....	213
13.4.2	Way Enable Register (WayEnable)	213
13.4.3	ECC Error Injection Register (ECCInjectError).....	213
13.4.4	ECC Directory Fix Registers (DirECCFix*)	214
13.4.5	ECC Directory Fail Registers (DirECCFail*).....	214
13.4.6	ECC Data Fix Registers (DatECCFix*)	214
13.4.7	ECC Data Fail Registers (DatECCFail*).....	214
13.4.8	L3 Cache ECC Error Injection and Correction	215
13.4.9	Cache Flush Registers (Flush*)	215
13.4.10	Way Mask Registers (WayMask*)	216
13.5	Procedure to Flush the L3 Cache	217
14	Power Management.....	218
14.1	Power Modes	218
14.2	Run Mode	218
14.2.1	Power Control	218
14.3	WFI Clock Gate Mode	219

14.3.1	WFI Wake Up.....	219
14.4	CEASE Instruction for Power Down	220
14.5	Hardware Reset.....	220
14.6	Early Boot Flow.....	220
14.7	Interrupt State During Early Boot	221
14.8	Other Boot Time Considerations.....	222
14.9	Power-Down Flow	222
15	Debug	224
15.1	Debug Module	224
15.2	Debug and Trigger Registers.....	227
15.2.1	Debug Control and Status Register (dcsr)	227
15.2.2	Debug PC (dpc)	228
15.2.3	Debug Scratch (dscratch).....	228
15.2.4	Trigger Select Register (tselect)	229
15.2.5	Trigger Data Registers (tdata1-3).....	229
15.3	Breakpoints	230
15.3.1	Breakpoint Match Control Register (mcontrol).....	230
15.3.2	Breakpoint Match Address Register (maddress)	233
15.3.3	Breakpoint Execution	233
15.3.4	Sharing Breakpoints Between Debug and Machine Mode	233
15.4	Debug Memory Map	233
15.4.1	Debug RAM and Program Buffer (0x300–0x3FF)	234
15.4.2	Debug ROM (0x800–0xFFF)	234
15.4.3	Debug Flags (0x100–0x110, 0x400–0x7FF)	234
15.4.4	Safe Address	234
15.5	Instruction Trace Interface.....	234
15.6	Debug Module Interface.....	236
15.6.1	Debug Module Status Register (dmstatus)	237
15.6.2	Debug Module Control Register (dmcontrol)	239
15.6.3	Hart Info Register (hartinfo).....	239
15.6.4	Abstract Control and Status Register (abstractcs)	241
15.6.5	Abstract Command Register (command)	242

15.6.6	Abstract Command Autoexec Register (abstractauto)	242
15.6.7	Debug Module Control and Status 2 Register (dmcs2)	242
15.6.8	Abstract Commands	243
15.6.9	System Bus Access	245
15.7	Debug Module Operational Sequences	245
15.7.1	Entering Debug Mode	245
15.7.2	Exiting Debug Mode	245
16	SiFive Trace System	247
16.1	Introduction	247
16.2	Trace Control Interface	247
16.3	Instruction Trace Encoder	249
16.3.1	Ingress Port	249
16.3.2	BTM Generator	249
16.3.3	In-Circuit Trace Message	252
16.3.4	TE Trace Packer	252
16.3.5	Register Interface	253
16.4	Instrumentation Trace Component (ITC)	259
16.4.1	ITC Register Interface	259
16.4.2	Using the ITC	260
16.5	Timestamp Unit	261
16.5.1	Timestamp Register Interface	262
16.6	External Triggers	264
16.6.1	Register Descriptions	265
16.6.2	Using External Trigger Inputs	265
16.7	Core Watchpoints	266
16.7.1	Register Descriptions	266
16.8	Watchpoint Precedence	267
16.9	PC Sampling Unit	267
16.9.1	Register Descriptions	268
16.10	Nexus Messages	271
16.10.1	Nexus Message Formats	271
16.10.2	Address Compression	274

16.10.3	Timestamp Compression	275
16.10.4	Nexus Transmission Protocol	275
16.11	PIB Trace Sink	276
16.11.1	Register Descriptions	276
16.11.2	Programming Recommendations	278
16.11.3	Calibration Mode	278
16.11.4	SWT Manchester Protocol	279
16.11.5	SWT UART Protocol	279
16.11.6	PIB Parallel Protocol	280
16.12	Nexus Branch History Trace Encoder	281
16.12.1	History Buffer	281
16.12.2	Return-Address Stack Optimization	282
16.12.3	Repeat Branch Optimization	283
16.12.4	Sync Messages	284
16.12.5	HTM Nexus Message Formats	285
17	Error Correction Codes (ECC)	287
17.1	ECC Configuration	287
17.1.1	ECC Initialization	288
17.2	ECC Interrupt Handling and Error Injection	288
17.3	Hardware Operation Upon ECC Error	289
A	SiFive Core Complex Configuration Options	290
A.1	P550 Series	290
B	SiFive RISC-V Implementation Registers	292
B.1	Machine Architecture ID Register (marchid)	292
B.2	Machine Implementation ID Register (mimpid)	293
C	Revision History	294
	References	295

Tables

Table 1	P550 Core Complex Feature Set	25
Table 2	RISC-V Specification Compliance	29
Table 3	Abbreviations and Terms.....	31
Table 4	P550 Feature Set	32
Table 5	Executable Memory Regions for the P550 Core Complex	34
Table 6	Hardware Prefetcher CSR.....	40
Table 7	Hardware Prefetcher 2 CSR	41
Table 8	PTE Configuration Bits.....	46
Table 9	PTE Encoding fields	47
Table 10	satp.MODE Values	48
Table 11	pmpXcfg Bitfield Description	57
Table 12	pmpaddrX Encoding Examples for A=NAPOT	58
Table 13	mhpmevent Register	63
Table 14	Physical Memory Attributes for External Regions.....	75
Table 15	Physical Memory Attributes for Internal Regions.....	75
Table 16	P550 Core Complex Memory Map. Physical Memory Attributes: R –Read, W –Write, X –Execute, I –Instruction Cacheable, D –Data Cacheable, A –Atomics.....	76
Table 17	Base Instruction Formats	77
Table 18	R-Type Integer Instructions.....	79
Table 19	R-Type Integer Instruction Description	79
Table 20	I-Type Integer Instructions	80
Table 21	I-Type Integer Instruction Description	81
Table 22	I-Type Load Instructions	82
Table 23	I-Type Load Instruction Description	82
Table 24	S-Type Store Instructions	83
Table 25	S-Type Store Instruction Description	83
Table 26	J-Type Instruction Description.....	84
Table 27	B-Type Instructions	84
Table 28	B-Type Instruction Description	84

Table 29	RISC-V Base Instruction to Assembly Pseudoinstruction Example	85
Table 30	Multiplication Operation Description	86
Table 31	Division Operation Description	87
Table 32	Atomic Load-Reserve and Store-Conditional Instruction Description.....	88
Table 33	Atomic Memory Operation Description.....	89
Table 34	Accrued Exception Flags.....	89
Table 35	Floating-Point Rounding Modes	90
Table 36	Single-Precision FP Load and Store Instructions Description	90
Table 37	Single-Precision FP Computational Instructions Description	91
Table 38	Single-Precision FP Conversion Instructions Description.....	92
Table 39	Single-Precision FP to FP Sign-Injection Instructions Description.....	93
Table 40	RISC-V Base Instruction to Assembly Pseudoinstruction Example	93
Table 41	Single-Precision FP Move Instructions Description	93
Table 42	Single-Precision FP Compare Instructions Description	94
Table 43	Single-Precision FP Classify Instruction Description	94
Table 44	Floating-Point Number Classes.....	95
Table 45	Double-Precision FP Load and Store Instructions Description.....	95
Table 46	Double-Precision FP Computational Instructions Description	96
Table 47	Double-Precision FP Conversion Instructions Description	98
Table 48	Double-Precision FP to FP Sign-Injection Instructions Description	99
Table 49	RISC-V Base Instruction to Assembly Pseudoinstruction Example	99
Table 50	Double-Precision FP Move Instructions Description	100
Table 51	Double-Precision FP Compare Instructions Description.....	100
Table 52	Double-Precision FP Classify Instruction Description	101
Table 53	Stack-Pointed-Based Load Instruction Description.....	102
Table 54	Stack-Pointed-Based Store Instruction Description	103
Table 55	Register-Based Load Instruction Description	103
Table 56	Register-Based Store Instruction Description	104
Table 57	Unconditional Jump Instruction Description	104
Table 58	Unconditional Control Transfer Instruction Description	104
Table 59	Conditional Control Transfer Instruction Description.....	105
Table 60	Integer Constant-Generation Instruction Description	105
Table 61	Integer Register-Immediate Operation Description.....	106

Table 62	Integer Register-Immediate Operation Description (cont.)	106
Table 63	Integer Register-Immediate Operation Description (cont.)	106
Table 64	Integer Register-Immediate Operation Description (cont.)	106
Table 65	Integer Register-Immediate Operation Description (cont.)	107
Table 66	Integer Register-Register Operation Description	107
Table 67	Integer Register-Register Operation Description (cont.).....	107
Table 68	Address Calculation Instructions Description	108
Table 69	Add/Shift with Prefix Zero-Extend Instructions Description	109
Table 70	Count Leading/Trailing Zeroes Instructions Description	110
Table 71	Count Population Instructions Description	110
Table 72	Logic-With-Negate Instructions Description	111
Table 73	Comparison Instructions Description	111
Table 74	Sign- and Zero-Extend Instructions	112
Table 75	Bitwise Rotation Instructions Description	113
Table 76	OR Combine Instruction Description.....	113
Table 77	Byte-Reverse Instruction Description	113
Table 78	Control and Status Register Instruction Description	114
Table 79	CSR Reads and Writes	115
Table 80	User Mode CSRs	116
Table 81	Supervisor Mode CSRs.....	117
Table 82	Machine Mode CSRs	118
Table 83	Debug Mode Registers	119
Table 84	Core Generator Encoding of marchid.....	120
Table 85	Generator Release Encoding of mimpid.....	121
Table 86	Timer and Counter Pseudoinstruction Description.....	122
Table 87	Timer and Counter CSRs	123
Table 88	RISC-V Registers	126
Table 89	RISC-V Assembly and C Examples	127
Table 90	RISC-V Code Model Table	140
Table 91	Branch Prediction Mode CSR	146
Table 92	SiFive Feature Disable CSR 1	147
Table 93	SiFive Feature Disable CSR 1 Usage	148
Table 94	SiFive Feature Disable CSR 2	149

Table 95	Exception Priority	152
Table 96	Summary of Exception and Interrupt CSRs	153
Table 97	Machine Status Register (partial)	156
Table 98	Machine Trap Vector Register	157
Table 99	Encoding of <code>mtvec.MODE</code>	157
Table 100	Machine Interrupt Enable Register	158
Table 101	Machine Interrupt Pending Register	159
Table 102	Machine Cause Register	159
Table 103	<code>mcause</code> Exception Codes	160
Table 104	Machine Interrupt Delegation Register	162
Table 105	Machine Exception Delegation Register	162
Table 106	Supervisor Status Register (partial)	163
Table 107	Supervisor Interrupt Enable Register	163
Table 108	Supervisor Interrupt Pending Register	164
Table 109	Supervisor Cause Register	164
Table 110	<code>scause</code> Exception Codes	165
Table 111	Supervisor Trap Vector Register	166
Table 112	Encoding of <code>stvec.MODE</code>	166
Table 113	RNMI CSRs	168
Table 115	P550 Core Complex Interrupt IDs	173
Table 116	CLINT Memory Map	174
Table 117	PLIC Memory Map	177
Table 118	PLIC Interrupt Source Mapping	178
Table 119	Mapping of <code>global_interrupts</code> Signal Bits to PLIC Interrupt ID	178
Table 120	PLIC Interrupt Priority Register	179
Table 121	PLIC Interrupt Pending Register 1	179
Table 122	PLIC Interrupt Pending Register 5	180
Table 123	PLIC Interrupt Enable Register 1 for Hart 0 M-Mode	180
Table 124	PLIC Interrupt Enable Register 5 for Hart 0 M-Mode	180
Table 125	PLIC Clock Gate Disable Register	181
Table 126	PLIC Interrupt Priority Threshold Register	181
Table 127	PLIC Claim/Complete Register for Hart 0 M-Mode	182
Table 128	Bus-Error Unit Memory Map	186

Table 129	Bus-Error Unit Error Events	186
Table 130	Private L2 Cache Instance.....	188
Table 131	Private L2 Cache Feature Set	189
Table 132	PL2 Performance Monitor Event Selectors	191
Table 133	PL2 Performance Monitor Counter Client Filter	191
Table 134	Master IDs in the Private L2 Cache	192
Table 135	PL2 Performance Monitor Counters.....	192
Table 136	PL2PM pmEventSelect Register	193
Table 137	Private L2 Cache Memory Map	196
Table 138	PL2 Cache Configuration Register	196
Table 139	ECC Error Injection Register	197
Table 140	ECC Error Injection Done Register	197
Table 141	PL2 Cache Flush Register	198
Table 142	Cache Flush Commands	198
Table 143	PL2 Flush Count Register.....	199
Table 144	PL2 Configuration 0 Register	200
Table 145	L3 Features Access Summary	205
Table 146	L3 Performance Monitor Event Selectors	206
Table 147	L3 Performance Monitor Counter Client Filter	206
Table 148	L3 Performance Monitor Counters.....	207
Table 149	L3PM pmEventSelect Register.....	208
Table 150	L3 Cache Controller Memory Map	212
Table 151	L3 Cache Configuration Register.....	213
Table 152	Way Enable Register	213
Table 153	ECC Error Injection Register	214
Table 154	Way Mask 0 Register	216
Table 155	Master IDs in the L3 Cache Controller	216
Table 156	Power Dial Register	219
Table 157	Debug Module Memory Map Seen from the Debug Module Interface	225
Table 158	Debug Module Memory Map from the Perspective of the Core	226
Table 159	Debug and Trigger Registers	227
Table 160	Debug Control and Status Register	228
Table 161	Trigger Select Register	229

Table 162	Trigger Data Register 1	229
Table 163	Trigger Data Registers 2 and 3	230
Table 164	Trigger CSRs When Used as Breakpoints	230
Table 165	Breakpoint Match Control Register	231
Table 166	NAPOT Size Encoding	232
Table 167	Encoding of <code>priv</code>	235
Table 168	SiFive Instruction Trace Interface Fields.....	236
Table 169	Debug Module Interface Signals	237
Table 170	Debug Module Status Register	238
Table 171	Debug Module Control Register	239
Table 172	Hart Info Register	240
Table 173	Abstract Control and Status Register	241
Table 174	Abstract Command Register	242
Table 175	Abstract Command Autoexec Register	242
Table 176	Debug Module Control and Status 2 Register	243
Table 177	Debug Abstract Commands	243
Table 178	Abstract Command Example for 32-bit Block Write	244
Table 179	System Bus vs. Program Buffer Comparison	245
Table 180	Trace Encoder Memory Map	248
Table 181	Core to Trace Encoder Interface	249
Table 182	Branch Trace Messages Description	250
Table 183	SYNC Types	251
Table 184	EVCODE and PTCM Description.....	251
Table 185	CKSRC Description	252
Table 186	Trace Encoder Control Register	254
Table 187	Trace Encoder Implementation Register	257
Table 188	Trace Encoder Sink Base Register	258
Table 189	Trace Encoder Sink Base High Bits Register	258
Table 190	Trace Encoder Sink Limit Register.....	258
Table 191	Trace Encoder Sink Write Pointer Register.....	258
Table 192	Trace Encoder SRAM Sink Access Pointer Register	259
Table 193	Trace Encoder SRAM Sink Data Register	259
Table 194	Instrumentation Trace Enable Register	259

Table 195	Instrumentation Trace Stimulus Register	260
Table 196	Timestamp Control Register	263
Table 197	Timestamp Lower Bits Register	264
Table 198	Timestamp Upper Bits Register	264
Table 199	External Trigger Input Control Register	265
Table 200	Watchpoint Action Identifiers	266
Table 201	Core Watchpoint Control Register	266
Table 202	PC Sample Control Register	268
Table 203	PC Capture and Sample Implementation Register	269
Table 204	PC Capture Register	269
Table 205	PC Capture High Register	270
Table 206	PC Sample Register	270
Table 207	PC Sample High Register	271
Table 208	Direct Branch Message Description	271
Table 209	Indirect Branch Message Description	272
Table 210	Direct Branch with Sync Message Description	272
Table 211	Indirect Branch with Sync Message Description	272
Table 212	Program Trace Synchronization Message Description	273
Table 213	Error Message Description	273
Table 214	Program Trace Correlation Message Description	273
Table 215	In-Circuit Trace Message Description (Reports PC)	273
Table 216	Data Acquisition Message (from ITC) Description	274
Table 217	Ownership Trace Message (from ITC) Description	274
Table 218	Address Compression Example	274
Table 219	Timestamp Compression Description	275
Table 220	Nexus Transmission Protocol Description	276
Table 221	PIB Sink Control Register	277
Table 222	PIB Port Width Configuration Description	278
Table 223	Calibration Mode Description	279
Table 224	SWT Manchester Protocol Description	279
Table 225	RAS Optimization Example	282
Table 226	Repeat Branch Optimization Example 1	283
Table 227	Repeat Branch Optimization Example 2	283

Table 228	Sync Messages	284
Table 229	Indirect Branch History Message	285
Table 230	Indirect Branch History with Sync Message	285
Table 231	Resource Full Message	286
Table 232	Program Trace Correlation Message (HTM format)	286
Table 233	Memory Protection Summary	287
Table 234	Core Generator Encoding of marchid	292
Table 235	Generator Release Encoding of mimpid	293
Table 236	P550 Core Complex Manual Revision History	294

Figures

Figure 1	P550 Series Block Diagram	27
Figure 2	P550 Block Diagram	33
Figure 3	P550 Pipeline Stages.....	33
Figure 4	P550 Integer Execution Pipeline	38
Figure 5	TLB Update Flow.....	44
Figure 6	Sv48 Virtual Address	44
Figure 7	Sv48 Physical Address.....	45
Figure 8	Sv48 PTE Format	45
Figure 9	RV64 Supervisor Address Translation Register (satp)	48
Figure 10	SFENCE.VMA Instruction	49
Figure 11	Linux User Application Memory Map Example	53
Figure 12	Hardware Table Walk Example.....	54
Figure 13	RV64 pmpcfg0 Register.....	56
Figure 14	RV64 pmpcfg2 Register.....	56
Figure 15	RV64 pmpXcfg bitfield	57
Figure 16	RV64 pmpaddrX Register.....	58
Figure 17	PMP Example Block Diagram	59
Figure 18	Event Selector Fields	62
Figure 19	R-Type.....	77
Figure 20	I-Type	78
Figure 21	S-Type.....	78
Figure 22	B-Type.....	78
Figure 23	U-Type.....	78
Figure 24	J-Type	78
Figure 25	ADD Instruction Example.....	79
Figure 26	ADDI Instruction Example.....	81
Figure 27	LW Instruction Example	82
Figure 28	Store Instructions.....	82
Figure 29	SW Instruction Example	83

Figure 30	JAL Instruction.....	83
Figure 31	JALR Instruction	83
Figure 32	Branch Instructions	84
Figure 33	Upper-Immediate Instructions	85
Figure 34	FENCE Instructions	85
Figure 35	NOP Instructions	86
Figure 36	Multiplication Operations	86
Figure 37	Division Operations.....	87
Figure 38	Atomic Operations	87
Figure 39	Atomic Memory Operations.....	88
Figure 40	Floating-Point Control and Status Register	89
Figure 41	Single-Precision FP Load Instruction	90
Figure 42	Single-Precision FP Store Instruction.....	90
Figure 43	Single-Precision FP Computational Instructions.....	91
Figure 44	Single-Precision FP Fused Computational Instructions.....	91
Figure 45	Single-Precision FP to Integer and Integer to FP Conversion Instructions	91
Figure 46	Single-Precision FP to FP Sign-Injection Instructions	92
Figure 47	Single-Precision FP Move Instructions	93
Figure 48	Single-Precision FP Compare Instructions	94
Figure 49	Single-Precision FP Classify Instruction	94
Figure 50	Double-Precision FP Load Instruction	95
Figure 51	Double-Precision FP Store Instruction	95
Figure 52	Double-Precision FP Computational Instructions.....	96
Figure 53	Double-Precision FP Fused Computational Instructions.....	96
Figure 54	Double-Precision FP to Integer and Integer to FP Conversion Instructions	97
Figure 55	Double-Precision to Single-Precision and Single-Precision to Double-Precision FP Conversion Instructions	97
Figure 56	Double-Precision FP to FP Sign-Injection Instructions	99
Figure 57	Double-Precision FP Move Instructions.....	99
Figure 58	Double-Precision FP Compare Instructions	100
Figure 59	Double-Precision FP Classify Instruction.....	101
Figure 60	CR Format - Register	101
Figure 61	CI Format - Immediate	101

Figure 62	CSS Format - Stack-relative Store	101
Figure 63	CIW Format - Wide Immediate	101
Figure 64	CL Format - Load.....	102
Figure 65	CS Format - Store.....	102
Figure 66	CA Format - Arithmetic	102
Figure 67	CJ Format - Jump	102
Figure 68	Stack-Pointed-Based Loads.....	102
Figure 69	Stack-Pointed-Based Stores	102
Figure 70	Register-Based Loads.....	103
Figure 71	Register-Based Stores	103
Figure 72	Unconditional Jump Instructions.....	104
Figure 73	Unconditional Control Transfer Instructions	104
Figure 74	Conditional Control Transfer Instructions.....	105
Figure 75	Integer Constant-Generation Instructions	105
Figure 76	Integer Register-Immediate Operations.....	105
Figure 77	Integer Register-Immediate Operations (cont.)	106
Figure 78	Integer Register-Immediate Operations (cont.)	106
Figure 79	Integer Register-Immediate Operations (cont.)	106
Figure 80	Integer Register-Immediate Operations (cont.)	107
Figure 81	Integer Register-Register Operations.....	107
Figure 82	Integer Register-Register Operations (cont.).....	107
Figure 83	Defined Illegal Instruction	108
Figure 84	Address Calculation Instructions	108
Figure 85	Address Calculation Instructions (cont.)	108
Figure 86	Add with Prefix Zero-Extend Instruction	109
Figure 87	Shift with Prefix Zero-Extend Instruction.....	109
Figure 88	Count Leading/Trailing Zeroes Instructions.....	109
Figure 89	Count Leading/Trailing Zeroes Instructions (cont.)	109
Figure 90	Count Population Instruction	110
Figure 91	Count Population Instruction (cont.).....	110
Figure 92	Logic-With-Negate Instructions	110
Figure 93	Comparison Instructions	111
Figure 94	Sign-Extend Instructions.....	111

Figure 95	Zero-Extend Instruction	111
Figure 96	Bitwise Rotation Instructions	112
Figure 97	Bitwise Rotation Instructions (cont.)	112
Figure 98	Bitwise Rotation Instructions (cont.)	112
Figure 99	Bitwise Rotation Instructions (cont.)	112
Figure 100	Zicsr Instructions.....	114
Figure 101	Timer and Counter Pseudoinstructions.....	122
Figure 102	ECALL and EBREAK Instructions.....	124
Figure 103	Wait for Interrupt Instruction	124
Figure 104	Supervisor Memory-Management Fence Instruction	125
Figure 105	RISC-V Assembly Example.....	127
Figure 106	RISC-V Assembly to Machine Code.....	128
Figure 107	One RISC-V Instruction	129
Figure 108	Stack Memory during Function Calls.....	131
Figure 109	P550 Core Complex Interrupt Architecture Block Diagram	154
Figure 110	CLINT Block Diagram.....	170
Figure 111	CLINT Interrupts and Vector Table	171
Figure 112	CLINT Vector Table Example	172
Figure 113	CLINT Interrupt Attribute Example	173
Figure 114	Bus-Error Unit Block Diagram	185
Figure 115	Cache Structure in the P550 Core Complex	188
Figure 116	Event Selector Fields	191
Figure 117	Organization of the SiFive L3 Cache Controller.....	202
Figure 118	Mapping of L3 Cache Ways to L3 LIM Addresses	203
Figure 119	Difference between L3 LIM and L3 Zero Device	204
Figure 120	Event Selector Fields	206
Figure 121	Trace Encoder Block Diagram.....	247
Figure 122	Trace Packer	253
Figure 123	Trace Encoder External Trigger Handshake	264
Figure 124	Trace Encoder External Triggers	264
Figure 125	SWT Manchester Protocol	279
Figure 126	SWT UART Protocol	280
Figure 127	PIB Parallel Protocol	280

Figure 128	PIB Parallel Protocol, pibRefCenter=1.....	281
-------------------	--	-----

Chapter 1

Introduction

SiFive's P550 Core Complex is a full-Linux-capable, cache-coherent 64-bit RISC-V processor available as an IP block. The SiFive P550 Core Complex is guaranteed to be compatible with all applicable RISC-V standards, and this document should be read together with the official RISC-V user-level, privileged, and external debug architecture specifications.



A summary of features in the P550 Core Complex can be found in Table 1.

P550 Core Complex Feature Set	
Feature	Description
Number of Harts	1 Hart.
P550 Core	1 × P550 RISC-V core.
PLIC Interrupts	127 Interrupt signals, which can be connected to off-core-complex devices.
PLIC Priority Levels	The PLIC supports 7 priority levels.
Level 3 Cache	2 MiB 8-way L3 Cache.
Hardware Breakpoints	4 hardware breakpoints.
Physical Memory Protection Unit	PMP with 8 regions and a minimum granularity of 4096 bytes.

Table 1: P550 Core Complex Feature Set

The P550 Core Complex also has a number of on-core-complex configurability options, allowing one to tune the design to a specific application. The configurable options are described in Appendix A.

1.1 About this Document

This document describes the functionality of the P550 Core Complex 21G2.01.00. To learn more about the Evaluation RTL deliverables of the P550 Core Complex, consult the P550 Core Complex User Guide.

1.2 About this Release

This release of P550 Core Complex 21G2.01.00 is intended for evaluation purposes only. As such, the RTL source code has been intentionally obfuscated, and its use is governed by your Evaluation License.

The full list of technical limitations of the Evaluation P550 Core Complex can be found in the P550 Core Complex User Guide.

1.3 P550 Core Complex Overview

The P550 Core Complex includes 1 × P550 64-bit RISC-V core, along with the necessary functional units required to support the core. These units include a Core-Local Interruptor (CLINT) to support local interrupts, a Platform-Level Interrupt Controller (PLIC) to support platform interrupts, physical memory protection, a Debug unit to support a JTAG-based debugger host connection, Nexus 5001 compliant instruction trace for non-invasive debug and performance profiling, and a local crossbar that integrates the various components together.

The P550 Core Complex memory system consists of a Data Cache, Instruction Cache, private L2 Cache, and L3 Cache. The P550 Core Complex also includes a Front Port, which allows external masters to be coherent with the L1 memory system and access to the TIMs, thereby removing the need to maintain coherence in software for any external agents.

All memories, including caches and TIMs, support Single Error Correction, Double Error Detection (SECDDED) ECC to provide improved reliability and address safety critical applications.

An overview of the SiFive P550 Series is shown in Figure 1. Refer to the docs/core_complex_configuration.txt file for a comprehensive summary of the P550 Core Complex configuration.

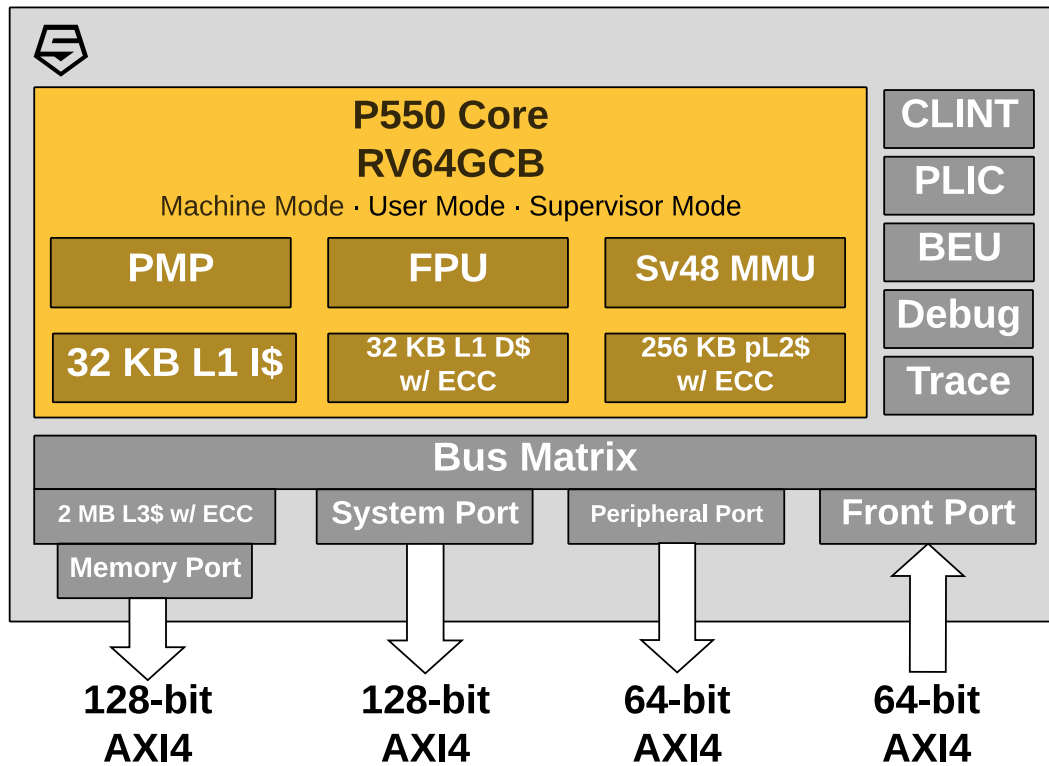


Figure 1: P550 Series Block Diagram

The P550 Core Complex memory map is detailed in Section 4.2, and the interfaces are described in full in the P550 Core Complex User Guide.

1.4 P550 RISC-V Core

The P550 Core Complex includes a 64-bit P550 RISC-V core, which has a triple-issue, out-of-order, 13 stage RISC-V processor. The SiFive P550 core is guaranteed to be compatible with all applicable RISC-V standards.

The P550 core is configured to support the RV64I base ISA, as well as the Multiply (M), Single-Precision Floating Point (F), Double-Precision Floating Point (D), Atomic (A), Compressed (C), and Bit Manipulation (B) RISC-V extensions. This is captured by the RISC-V extension string: RV64GCB. The base ISA and instruction extensions are described in Chapter 5.

The P550 also supports machine, supervisor, and user privilege modes, in conjunction with Physical Memory Protection (PMP), thereby allowing System-on-Chip (SoC) implementations to make the right area, power, and feature trade-offs.

The P550 core is designed to be feature rich, providing a very flexible memory system that includes L1 caches, standards-based configurable bus interfaces, and memory maps that provide a lot of flexibility for SoC integration.

The microarchitecture also incorporates a branch prediction unit that is composed of a 32-entry Branch Target Buffer (BTB), a 9.1 KiB-entry Branch History Table (BHT), a 16-entry Return Address Stack (RAS), 512-entry Indirect Jump Target Predictor (IJTP), and a 16-entry Return Instruction Predictor.

The P550 includes an IEEE 754-2008 compliant Floating-Point Unit. The floating-point pipeline sits in conjunction with the integer pipeline; however, it is longer, increasing the total pipeline stages to 7.

The P550 core is described in more detail in Chapter 3.

1.5 Memory System

The P550 Core Complex memory system has a Level 1 memory system optimized for high performance. The instruction subsystem consists of a 32 KiB, 4-way instruction cache. The data subsystem is comprised of a high performance 32 KiB, 4-way L1 data cache. In addition to the L1 memory system, the P550 Core Complex has a per hart 256 KiB, 8-way private L2 cache. The memory system is described in more detail in Chapter 3.

The P550 Core Complex also supports a shared, 2 MiB, 8-way L3 cache with 4 banks. The L3 Cache Controller is described in Chapter 13.

1.6 Interrupts

The P550 Core Complex provides the standard RISC-V M-mode timer and software interrupts via the Core-Local Interruptor (CLINT).

The P550 Core Complex also includes a RISC-V standard Platform-Level Interrupt Controller (PLIC), which supports 132 global interrupts with 7 priority levels pre-integrated with the on-core-complex peripherals.

Interrupts are described in Chapter 7. The CLINT is described in Chapter 8. The PLIC is described in Chapter 9.

1.7 Debug Support

The P550 Core Complex provides external debugger support over an industry-standard JTAG port, including 4 hardware-programmable breakpoints per hart.

Debug support is described in detail in Chapter 15, and the debug interface is described in the P550 Core Complex User Guide.

1.8 Compliance

The P550 Core Complex is compliant to the following versions of the various RISC-V specifications:

ISA	Version	Status
RV64I Base Integer Instruction Set	2.0	Frozen
Extensions	Version	Status
M Standard Extension for Integer Multiplication and Division	2.0	Ratified
A Standard Extension for Atomic Instruction	2.0	Frozen
F Standard Extension for Single-Precision Floating-Point	2.0	Frozen
D Standard Extension for Double-Precision Floating-Point	2.0	Frozen
C Standard Extension for Compressed Instruction	2.0	Ratified
Zba Standard Extension for Bit Manipulation	0.94	
Zbb Standard Extension for Bit Manipulation	0.94	
Privilege Mode	Version	Status
Machine-Level ISA	1.11	
User-Level ISA	1.11	
Supervisor-Level ISA	1.11	
Devices	Version	Status
The RISC-V Debug Specification	1.0	
RISC-V Platform-Level Interrupt Controller (PLIC) Specification	—	

Table 2: RISC-V Specification Compliance

Chapter 2

List of Abbreviations and Terms

Term	Definition
BHT	Branch History Table
BTB	Branch Target Buffer
CLIC	Core-Local Interrupt Controller. Configures priorities and levels for core-local interrupts.
CLINT	Core-Local Interruptor. Generates per hart software and timer interrupts.
DTIM	Data Tightly-Integrated Memory
Hart	HARdware Thread
IJTP	Indirect-Jump Target Predictor
ITIM	Instruction Tightly-Integrated Memory
JTAG	Joint Test Action Group
LIM	Loosely-Integrated Memory. Used to describe memory space delivered in a SiFive Core Complex that is not tightly integrated to a CPU core.
MDP	Memory Dependence Predictor
MSHR	Miss Status Handling Register
NLP	Next-Line Predictor
PLIC	Platform-Level Interrupt Controller. The global interrupt controller in a RISC-V system.
PMP	Physical Memory Protection
RAS	Return-Address Stack
RO	Used to describe a Read-Only register field
ROB	ReOrder Buffer
RS	Read/Set field. A register field that cannot be cleared by software, only reset will clear.
RW	Used to describe a Read/Write register field
RW1C	Used to describe a Read/Write-1-to-Clear register field
TileLink	A free and open interconnect standard originally developed at UC Berkeley
W1C	Used to describe a Write-1-to-Clear register field
WARL	Write-Any, Read-Legal field. A register field that can be written with any value, but returns only supported values when read.
WIRI	Writes-Ignored, Reads-Ignore field. A read-only register field reserved for future use. Writes to the field are ignored and reads should ignore the value returned.
WLRL	Write-Legal, Read-Legal field. A register field that should only be written with legal values and that only returns legal value if last written with a legal value.
WO	Used to describe a Write-Only register field
WPRI	Writes-Preserve, Reads-Ignore field. A register field that might contain unknown information. Reads should ignore the value returned, but writes to the whole register should preserve the original value.

Table 3: Abbreviations and Terms

Chapter 3

P550 RISC-V Core

This chapter describes the 64-bit P550 RISC-V processor core, instruction fetch and execution unit, rename and dispatch unit, L1, L2, and L3 memory systems, Physical Memory Protection unit, Hardware Performance Monitor, and external interfaces.

The P550 feature set is summarized in Table 4.

Feature	Description
ISA	RV64GCB
SiFive Custom Instruction Extension (SCIE)	Not Present
Modes	Machine mode, user mode, supervisor mode
L1 Instruction Cache	32 KiB 4-way instruction cache
L1 Data Cache	32 KiB 4-way data cache
Private L2 (pL2) Cache	256 KiB 8-way pL2 cache with 2 banks
L3 Cache	2 MiB 8-way L3 cache with 4 banks
ECC Support	Single error correction, double error detection on the L1 data cache, pL2 cache, and L3 cache
Physical Memory Protection	8 regions with a granularity of 4096 bytes
Memory Management Unit	Bare and Sv48 virtual memory support with fully-associative 64-entry L1 data and instruction TLBs, and a direct-mapped 512-entry unified TLB

Table 4: P550 Feature Set

The P550 processor core has the capability to execute instructions out-of-order.

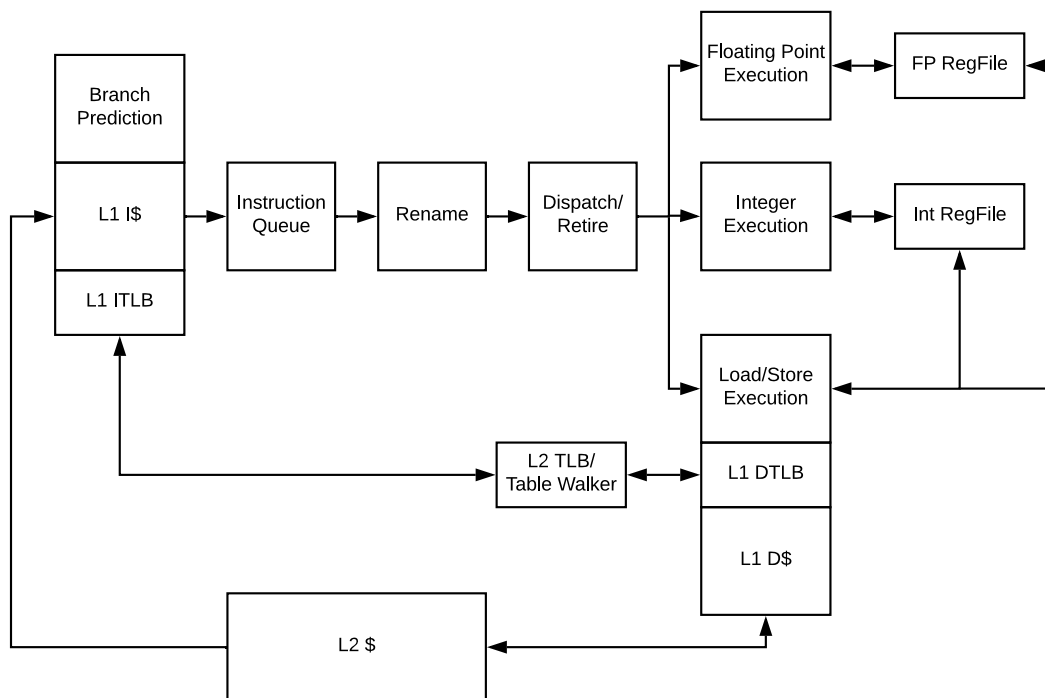


Figure 2: P550 Block Diagram

The P550 processor core has a 10 stage pipeline for integer operations, and a 13 stage pipeline for load instructions. Actual instruction latencies may be higher due to instruction dependencies, pipeline hazards, data cache misses, and memory dependencies. The integer pipeline is described in more detail in Section 3.4. The load/store pipeline is described in more detail in Section 3.5.

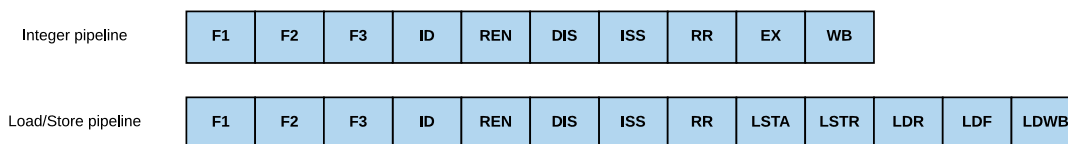


Figure 3: P550 Pipeline Stages

3.1 Supported Modes

The P550 supports RISC-V supervisor and user modes, providing three levels of privilege: machine (M), user (U), and supervisor (S). U-mode provides a mechanism to isolate application processes from each other and from trusted code running in M-mode. S-mode adds a number of additional CSRs and capabilities.

See *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10* for more information on the privilege modes.

3.2 Instruction Memory System

This section describes the instruction memory system of the P550 core.

3.2.1 Execution Memory Space

The regions of executable memory consist of all directly addressable memory in the system. The memory includes any volatile or non-volatile memory located off the Core Complex ports, and includes the on-core-complex L3 LIM and L3 Zero Device.

Table 5 shows the executable regions of the P550 Core Complex.

Base	Top	Description
0x0800_0000	0x081F_FFFF	L3 LIM
0x0A00_0000	0x0A1F_FFFF	L3 Zero Device
0x2000_0000	0x3FFF_FFFF	Peripheral Port (512 MiB)
0x4000_0000	0x5FFF_FFFF	System Port (512 MiB)
0x8000_0000	0x10_7FFF_FFFF	Memory Port (64 GiB)

Table 5: Executable Memory Regions for the P550 Core Complex

All executable regions are treated as instruction cacheable. There is no method to disable this behavior.

Trying to execute an instruction from a non-executable address results in an instruction access trap.

3.2.2 L1 Instruction Cache

The L1 instruction cache is a 32 KiB 4-way set-associative cache. It has a line size of 64 bytes and is read-allocate with a random replacement policy. A cache line fill triggers a burst access outside of the Core Complex, starting with the first address of the cache line. There are no write-backs to memory from the instruction cache and it is not kept coherent with rest of the platform memory system.

The access latency of all blocks in the instruction memory system is three cycles and is fully pipelined. The instruction cache data array is way predicted, and reads at most one way from the data array to reduce power consumption.

The L1 instruction cache has parity error protection support.

3.2.3 Cache Maintenance

The instruction cache supports the `FENCE.I` instruction, which invalidates the entire instruction cache, as described in Section 5.13. Writes to instruction memory from the core or another master must be synchronized with the instruction fetch stream by executing `FENCE.I`.

3.2.4 Coherence with Higher Level Caches

The L1 instruction cache is partially inclusive with the private L2 Cache (pL2), described in Chapter 12, and the L3 Cache, described in Chapter 13.

When a block of instruction memory is allocated to the L1 cache, it is also allocated to the pL2 and L3 caches if the access was from the Memory Port. Instruction accesses to all other ports will not allocate to the pL2 or L3 caches, only the L1 cache.

When a block is evicted from L1, it might still reside in pL2 or L3, which will reduce access time the next time the block is fetched.

If a hart modifies instruction memory (i.e., self-modifying code), then a `FENCE.I` instruction is required to synchronize the instruction and data streams. Even though `FENCE.I` targets the L1 instruction cache, no cache operation is required on the pL2 or L3 caches to maintain instruction coherency.

3.2.5 Instruction Fetch Unit

The P550 instruction fetch unit is responsible for keeping the pipeline fed with instructions from memory. The instruction fetch unit delivers up to 16 bytes of instructions per clock cycle to support superscalar instruction execution. Fetches are always word-aligned and there is a one-cycle penalty for branching to a 32-bit instruction that is not word-aligned.

The instruction fetch unit comprises four stages: three Fetch stages (F1-F3) and one Instruction Decode (ID) stage. The P550 instruction fetch unit contains sophisticated, predictive hardware to mitigate the performance impact of control hazards within the instruction stream. The instruction fetch unit is decoupled from the Rename and Dispatch Unit (RDU), so that correctly predicted control-flow events usually do not result in execution stalls. This decoupling is through a 6-entry fetch queue, which holds up to 16 bytes of instructions, and a 16-entry instruction queue.

The P550 Core Complex is a 3-wide superscalar decode pipeline. Up to three instructions per cycle will be decoded within the instruction fetch unit before being delivered to the RDU in the ID stage. Note that at most one `JALR` instruction can be decoded per cycle.

The P550 implements the standard Compressed (C) extension to the RISC-V architecture, which allows for 16-bit RISC-V instructions. As two 16-bit instructions can be fetched per cycle, the instruction fetch unit can be idle when executing programs comprised mostly of compressed 16-bit instructions. This reduces memory accesses and power consumption.

All branches must be aligned to half-word addresses. Otherwise, the fetch generates an instruction address misaligned trap. Trying to fetch from a non-executable or unimplemented address results in an instruction access trap.

3.2.6 Branch Prediction

The main predictive structures in the instruction fetch unit are as follows:

- A 32-entry Branch Target Buffer (BTB), which predicts the target of taken branches and direct jumps;
- A 16-entry Return Instruction Predictor, which predicts whether the current fetch will contain a procedure return;
- A 512-entry Indirect Jump Target Predictor (IJTP). The IJTP is a correlating predictor that makes use of branch history;
- A 16-entry Return Address Stack (RAS), which predicts the target of procedure returns;
- A 9.1 KiB Branch History Table (BHT), which predicts the direction of conditional branches. The BHT is a correlating predictor that supports long branch histories.

The BTB has a one-cycle latency, so that correctly predicted branches and direct jumps result in no penalty, provided the target is 16-byte aligned. Depending on the target alignment, fewer instructions may be fetched on the next cycle.

Direct jumps that miss in the BTB result in a two-cycle fetch bubble. This event might not result in any execution stalls if the fetch queue is sufficiently full.

The Return Instruction Predictor also has a one-cycle latency, so that correctly predicting fetching a procedure return will also result in no fetch bubble. If the Return Instruction Predictor predicts a return, the target address is forwarded from the RAS to reduce the return prediction latency.

The BHT, IJTP, and RAS take precedence over the BTB. If these structures' predictions disagree with the BTB's prediction, fetch bubbles result. A BHT or RAS prediction will result in a two-cycle fetch bubble, unless the Return Instruction Predictor correctly predicted that the fetch contained a procedure return. In this case, there is no fetch bubble. An IJTP prediction will result in a three-cycle fetch bubble. These bubbles may not result in an execution stall.

Mispredicted branches and indirect jumps have a minimum penalty of eight cycles; the branch may resolve later depending on instruction dependencies and other pipeline hazards.

Branch prediction is enabled out of reset and cannot be disabled. However, instruction speculation, fetching before a prediction is confirmed, must be enabled in the Feature Disable CSR, described in Chapter 6.

As instruction speculation can occur at any point after it has been enabled, data cacheable regions of memory (i.e., DDR) must be able to respond to instruction fetches immediately after instruction speculation is enabled. If DDR initialization is not completed before instruction specu-

lation is enabled, the memory system must return a decode error (DECERR) for accesses made to DDR. The fetch unit will ignore errors associated with speculative accesses and continue to operate normally.

The Branch Prediction Mode CSR, also described in Chapter 6, provides a means to customize the branch predictor behavior to trade average performance for more predictable execution time.

3.3 Rename and Dispatch Unit

The P550 is an out-of-order execution processor and its Rename and Dispatch Unit (RDU) renames architectural registers to a pool of physical registers. The integer and floating-point registers are in separate register files; the P550 Core Complex has 128 integer physical registers, and 64 floating-point physical registers.

The RDU comprises two stages: Rename (REN) and Dispatch (DIS), along with the renaming tracking structures and Reorder Buffer (ROB) used to maintain precise exceptions. The ROB has 64 entries, and each entry can track a group of up to seven instructions. Instructions that may cause exceptions, branch, or indirect jump target mispredictions will end a group.

In the Rename stage, physical registers will be allocated and instruction source operands will be renamed. In the Dispatch stage, instructions will be steered toward one of the appropriate execution pipelines: one of the integer pipelines, one of the floating-point pipelines, or to the Load/Store Unit.

3.4 Execution Pipeline

To support out-of-order execution, the P550 has an issue queue for each integer or floating-point execution pipeline. Instructions will be held in the issue queue until their source operands are available. The P550 Core Complex has three integer pipelines and two floating-point pipelines, with issue queues of eight entries each.

The execution pipeline consists of a minimum of four stages:

- Issue (ISS): The oldest ready instruction will be selected and issued from the issue queue
- Register Read (RR): Source operands will be read from the physical register file or forwarded directly from the EX or WB stages of the execution pipelines
- Execute (EX): The first execution cycle. Single-cycle instructions can forward their results in this cycle to dependent instructions.
- Writeback (WB): The result of the instruction is written back into the physical register file. Results can also be forwarded from this stage.

Most integer operations take 1 cycle to execute. Integer multiplication takes 3 EX cycles and is fully pipelined. Integer division takes between 3 and 64 EX cycles, depending on the operand value.

In the P550 Core Complex, the three integer pipelines support the following instructions, respectively:

- Branches and indirect jumps, and all non-control-flow integer operations except for multiply and divide
- All non-control-flow integer operations except for multiply and divide
- All non-control-flow integer operations including multiply and divide

Figure 4 illustrates the P550 integer pipelines. The P550 floating-point pipelines operate similarly.

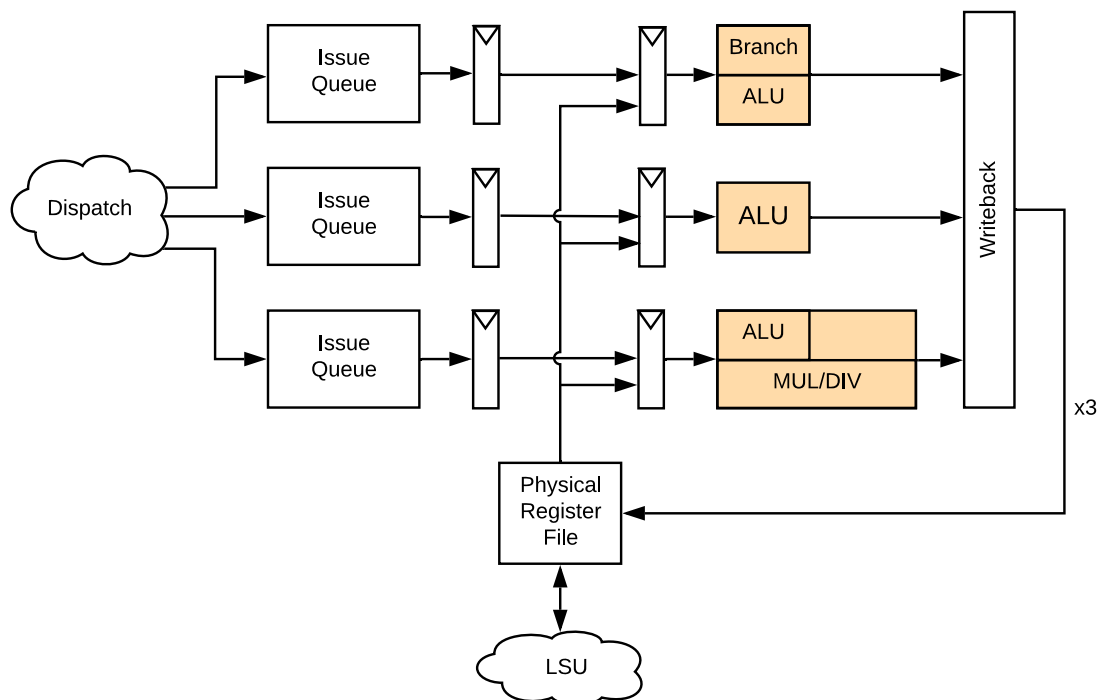


Figure 4: P550 Integer Execution Pipeline

Most floating-point operations take 2 EX cycles to execute. Single-precision and double-precision multiply and add/subtract instructions take 4 cycles and are fully pipelined.

The two P550 Core Complex floating-point pipelines support the following instructions, respectively:

- All instructions except for floating-point divide
- All instructions including floating-point divide

3.5 Load/Store Unit

The P550 supports out-of-order load/store issue, execution, and completion. The P550 Load/Store Unit (LSU) has a combined Issue Queue (LslssQ) with 16 entries for Load Address, Store Address, and Store Data pipelines. Instructions will be held in the issue queue until their source operands are available. Store Address operands may issue with or before their Store Data operand.

The LSU has a Load Address pipeline, Load Data return pipeline, Store Address pipeline, and Store Data pipeline, all of which may be utilized every cycle. Loads and stores retire their entries from the combined issue queue after execution. Long-latency loads wait in the load queue until completion. Store instructions buffer in the store queue until Reorder Buffer (ROB) retirement. The Miss Status Handling Register (MSHR) entries also include write combining buffer data.

The LSU data memory subsystem guarantees only weak memory ordering consistency between different memory addresses according to the RISC-V Weak Memory Ordering model (RVWMO). A FENCE instruction is required to guarantee ordering when desired between different memory addresses. Load/store execution in the P550 Core Complex takes advantage of nearly all of the allowed reorderings in RVWMO, including, but not limited to:

- Newer loads can read before older loads
- Newer loads can read before older stores
- Newer stores can write before older stores

With the exception of one: newer stores can write before older loads. The P550 LSU does not reorder newer stores before older loads. The P550 Core Complex pipelines memory accesses to non-idempotent I/O memory regions as far as the outer memory subsystem supports an in-order I/O memory channel for improved I/O throughput.

3.5.1 Hardware Prefetcher

The Hardware Prefetcher (HWPF) is a region-based sequential stride prefetcher and is tightly integrated in the LSU. It enables memory-level parallelism (MLP) and hides memory latency. The HWPF relies on pattern detection of loads and sends hints that either go to L1 MSRs or directly to L2. It is comprised of the HWPF module with individual prefetch engines and a HWPF issue queue.

The HWPF uses a single state machine that sends both L1 and L2 prefetches, depending on how far the tail pointer and respective demand pointer are separated.

The first L1 demand request allocates with the line address that consists of the region's virtual address and offset. The second L1 demand request calculates the stride based on the current and previous addresses. The third L1 demand request confirms the stride and starts sending prefetches equal to the number programmed in the `initialDist` of the HWPF CSR, up to a maximum of 64 L1 prefetch requests. Each L1 prefetch request establishes an MSHR and

sends an `AcquireBlock.NtoB` request to the downstream cache hierarchy. Each L2 prefetch request establishes an L2 HWPF queue and sends a Hint to L2.

Hardware Prefetcher CSRs

The following CSRs can be programmed at runtime to configure the HWPF.

Hardware Prefetcher CSR			
CSR	0x7C3		
Bits	Field Name	Attr.	Description
0	enable	RW	Enable hardware prefetcher
[6:1]	window	RW	Address comparison window
[12:7]	initialDist	RW	Prefetch beginning distance
[18:13]	maxAllowedDist	RW	Maximum allowed prefetch distance
[24:19]	linToExpThrd	RW	Threshold when HWPF distance increase changes from linear to exponential
[28:25]	qFullnessThrdL1	RW	Threshold fraction/16 of L1 MSHRs to stop sending hints to L1
[33:29]	hitCacheThrdL1	RW	Threshold # of L1 HWPF hits in L1 Cache Tag for evicting HWPF entry
[37:34]	hitMSHRThrdL1	RW	Threshold # of loads hitting L1 HWPF allocated L1 MSHRs or L1 HWPFs hitting demand allocated L1 MSHRs to increase aggressiveness
38	issueBubble	RW	Prefetch request to Load/Store tag read force block LS issue to inject a bubble
[44:39]	maxL1PFDist	RW	Threshold for directly sending prefetches to L2 instead of L1 MSHR
[46:45]	forgiveThrd	RW	Threshold for forgiving loads with mismatching strides when HWPF is in trained state. Setting to 0 (default) disables the feature.
[49:47]	numL1PFIssQEnt	RW	Number of HWPF Issue Queue entries dedicated for L1 prefetches
[63:50]	Reserved		

Table 6: Hardware Prefetcher CSR

Hardware Prefetcher 2 CSR			
CSR	0x7C4		
Bits	Field Name	Attr.	Description
0	l2pfEnable	RW	Enable L2 prefetches
[4:1]	qFullnessThrdL2	RW	Threshold fraction/16 of L2 MSHRs to stop sending hints to L2
[9:5]	hitCacheThrdL2	RW	Threshold # of L2 HWPF hits in L2 Cache Tag to stop sending hints to L2
[13:10]	hitMSHRThrdL2	RW	Threshold # of L1 HWPF hitting L2 HWPF L2 MSHRs to increase aggressiveness
[16:14]	numL2PFIssQEnt	RW	Number of HWPF Issue Queue entries dedicated for L2 prefetches
[63:17]	Reserved		

Table 7: Hardware Prefetcher 2 CSR

3.6 Data Memory System

The data memory system consists of on-core-complex data and the ports in the P550 Core Complex memory map, shown in Section 4.2. The on-core-complex data memory consists of a 32 KiB L1 data cache, 256 KiB private L2 cache, and 2 MiB L3 cache. The private L2 cache is described in Chapter 12.

Data accesses are classified as cacheable, for those targeting the Memory Port; or non-cacheable, for those targeting any other port in the Core Complex. Non-cacheable data accesses are collectively called memory-mapped I/O accesses, or MMIOs.

The P550 pipeline allows for multiple outstanding memory accesses. The number of outstanding cache line fills and MMIOs are implementation dependent. Misaligned accesses are not allowed to any memory region and result in a trap to allow for software emulation.

3.6.1 L1 Data Cache

The L1 data cache is a 32 KiB 4-way set-associative cache. It is virtually-indexed, physically-tagged with a line size of 64 bytes and is read/write-allocate with a PLRU replacement policy. The cache operates in write-back mode; this means that if a cache line is dirty, it is written back to memory when evicted. Out of reset, all lines of the cache are invalidated.

The L1 data cache supports ECC protection, as described in Chapter 17.

The Memory Port address range is the only cacheable region of memory. A cache line fill triggers a burst access starting with the first address of the cache line. The L1 data cache access latency is four clock cycles for aligned load-word and double-word, and five clock cycles for load-byte, half-word, and floating-point.

Stores are pipelined and commit on cycles where the data memory system is otherwise idle. Loads to addresses in flight in the store queue returns store-to-load-forward data with the same latency as an L1 data cache access.

The data cache cannot be disabled and the properties of the Memory Port cannot be modified to prevent cacheable accesses.

The L1 data cache data array is way predicted, and reads at most one way from the data array to reduce power consumption. The L1 data cache supports hit-under-miss and multiple outstanding miss-under-miss. The L1 data cache uses a MESI coherence protocol. The L1 data cache entries are allocated by loads, and stores configurably write-allocate (default to no-write-allocate).

Multiple retired store misses will gather within 64-byte cache line granules in MSHR entries before writing to the next-level cache. Loads to addresses in flight in the MSHR's write combining buffer will return buffer data at a slower latency than an L1 data cache access and a faster latency than a next-level cache access. The DTLB supports hit-under-miss and a single outstanding miss to page table walk. Virtual memory translation page table walks use the L1 data cache, but do not store-to-load-forward data from in-flight stores or the MSHR's write combining buffer data.

3.6.2 Cache Maintenance Operations

The data cache supports `CFLUSH.D.L1` and `CDISCARD.D.L1`. The instruction `CFLUSH.D.L1` cleans and invalidates the specified line or all cache lines. The instruction `CDISCARD.D.L1` invalidates the specified line or all cache lines.

These custom instructions are further described in Chapter 6.

3.6.3 Coherence with Higher Level Caches

The L1 data cache is inclusive with the private L2 Cache (pL2), described in Chapter 12, and the L3 Cache, described in Chapter 13.

When a block of data is allocated to the L1 cache, it is also allocated to the pL2 and L3 caches. When a block is evicted from the L1, the corresponding line in pL2 and L3 are then updated and marked dirty.

The custom instructions `CFLUSH.D.L1` and `CDISCARD.D.L1` only target the L1 data cache, and do not impact the pL2 and L3 caches. Flush capability of the pL2 Cache is described in Section 12.3.4. Flush capability of the L3 Cache Controller is described in Section 13.4.9.

3.7 Atomic Memory Operations

The P550 core supports the RISC-V standard Atomic (A) extension on the Memory Port, Peripheral Port, and internal memory regions.

Atomic instructions that target the Memory Port are implemented in the data cache and are not observable on the external data bus. The load-reserved (LR) and store-conditional (SC) instructions are special atomic instructions that are only supported in data cacheable regions. They will generate a precise access exception if targeted at uncachable data regions.

Atomic memory operations are not supported on the System Port. Atomic operations that target the System Port will generate a precise access exception.

See Section 5.4 for more information on the instructions added by this extension.

3.8 Floating-Point Unit (FPU)

The P550 FPU provides full hardware support for the IEEE 754-2008 floating-point standard for 32-bit single-precision and 64-bit double-precision arithmetic. The FPU includes a fully pipelined fused-multiply-add unit and an iterative divide and square-root unit, magnitude comparators, and float-to-integer conversion units, all with full hardware support for subnormals and all IEEE default values.

Section 5.5 describes the 32-bit single-precision instructions. Section 5.6 describes the 64-bit double-precision instructions.

The FPU comes up disabled on reset. First initialize `fcsr` and `mstatus.FS` prior to executing any floating-point instructions. In the `freedom-metal` startup code, write `mstatus.FS[1:0]` to `0x1`.

3.9 Virtual Memory Support

The P550 supports virtual memory through the use of a Memory Management Unit (MMU). The MMU supports the Bare and Sv48 modes as described in *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*. SiFive's Sv48 implementation provides a 48-bit virtual address space using 47-bits of physical address space. Supported page sizes include 4 KiB, 2 MiB, 1 GiB gigapage and 512 GiB terapages. The default Linux page size (PAGESIZE) is 4 KiB.

The translation lookaside buffers (TLBs) are address translation caches within the MMU. Translation is accomplished through page table entries (PTE) that reside in the TLB region. A hardware page-table walker refills the TLBs upon a cache miss. The PTE entries are fetched from a region defined by the root page table base address in the Supervisor Address Translation and Protection (satp) CSR. Each PTE contains the information necessary to translate the virtual memory address to a physical address on the design.

There are both level 1 and level 2 TLB entries. Level 1 entries contain separate instruction buffers (ITLB) and data buffers (DTLB) since they are accessed in different pipeline stages. The ITLB and DTLB each contain 64 entries, which are fully associative. Level 2 TLB entries are unified, and contain 512 direct-mapped TLB entries. Level 2 TLB are all 4 KiB pages. A block diagram of the instruction and data memory access from the L2 into the MMU TLB is shown below.

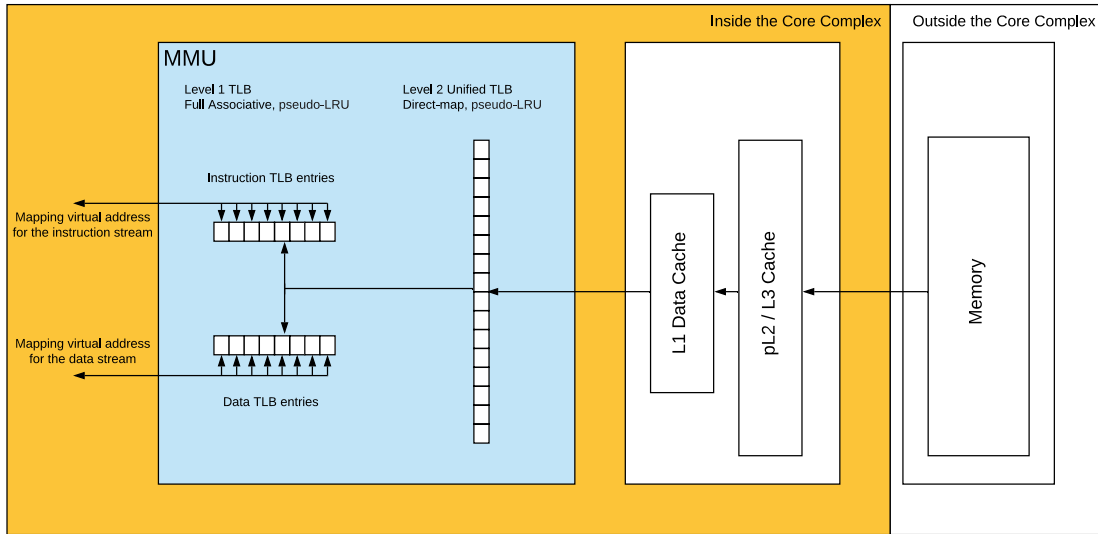


Figure 5: TLB Update Flow

Behaviors of the hardware are described below.

- When there is a TLB miss in the L1 ITLB or DTLB, the L2 unified TLB will populate the L1 TLB with the correct PTE, if it exists
- When there is a miss in both L1 and L2 TLB, a hardware page table walk will occur by the MMU to fill the TLB page table entry from the memory. The memory location where the hart will start fetching TLB page table entry from is determined by the physical page number (PPN) field in the Supervisor Address Translation and Protection (satp) CSR. The refill will occur from the data cache if it exists there, otherwise it will refill from the pL2 or L3 cache. If the pL2 or L3 cache does not contain the data, then it will be fetched from system memory.
- Both L1 and L2 unified TLB page table entry replacement policy is pseudo-LRU
- When L1 TLB entry is evicted, this entry is not updated in the L2 unified TLB
- When the L1 TLB entry is updated from L2, the entry will reside in L2 and will not be removed
- Executing the SFENCE.VMA instruction will invalidate both L1 and L2 TLB entries

3.9.1 Address and Page Table Formats

An Sv48 virtual address is partitioned as shown below. Note that address bits [63:48] of every instruction fetch, load, and store operation must be equal to bit 47, or else a page-fault exception will occur.

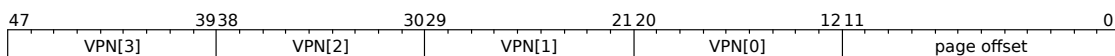


Figure 6: Sv48 Virtual Address

The 36-bit VPN is translated into a 44-bit PPN via a four-level page table, while the 12-bit page offset is untranslated.

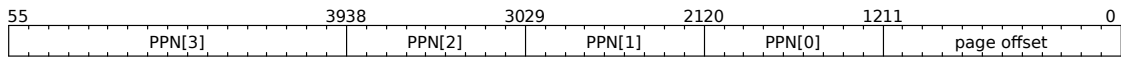


Figure 7: Sv48 Physical Address

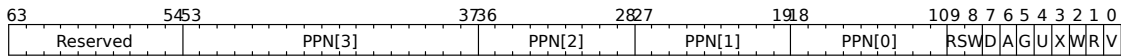


Figure 8: Sv48 PTE Format

A description of the PTE configuration bits can be found in Table 8.

Bits	Description
0	V: Valid 0x0 - Page table entry not valid 0x1 - Page table entry valid
1	R: Readable 0x0 - Page table entry not readable 0x1 - Page table entry readable
2	W: Writable 0x0 - Page table entry not writable 0x1 - Page table entry writable
3	X: Executable 0x0 - Page table entry not executable 0x1 - Page table entry executable
4	U: User mode access 0x0 - No access to user mode software 0x1 - Access granted to user mode software
5	G: Global mapping 0x0 - This mapping does not exist globally 0x1 - This mapping exists globally
6	A: Accessed 0x0 - Leaf page table entry has not been read, written, or fetched since the last time A was cleared 0x1 - Leaf page table entry has been read, written, or fetched since the last time A was cleared
7	D: Dirty 0x0 - The virtual page has not been written since the last time D was cleared 0x1 - The virtual page has been written since the last time D was cleared
[9:8]	RSW: Supervisor software use X - Open for supervisor software use

Table 8: PTE Configuration Bits

Page Table Configurations

Read, write, and execute permissions for Sv48 are summarized in Table 9. The value `PTE.V=1` indicates the PTE is valid, while `PTE.V=0` means all other bits in PTE are don't cares, and software can use these freely. The value `PTE.R=1` indicates the page is readable. Likewise, `PTE.W=1` indicates the page is writable, while `PTE.X=1` means the page is executable. When `PTE.V=0`, `PTE.R=0`, and `PTE.W=0`, this indicates the PTE is a pointer to the next level page table,

otherwise it is a leaf PTE. If a page is marked writable, it must also be marked readable. Combinations of `PTE.W=1` and `PTE.R=0` are not currently supported.

X	W	R	Meaning
0	0	0	Pointer to next level of page table
0	0	1	Read-only page
0	1	0	Reserved
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
1	1	0	Reserved
1	1	1	Read-write-execute page

Table 9: PTE Encoding fields

A fetch page-fault exception will occur if an instruction is fetched from a page that does not have execute permissions. A load page-fault exception will occur if a load or load-reserved instruction falls within a page without read permissions. A store page-fault exception will occur if a store, store-conditional, or AMO instruction falls within a page without write permissions.

The value `PTE.U=1` indicates the page is accessible to user mode. Supervisor mode software may also perform loads and stores to a page marked with `PTE.U=1`, but only if `sstatus.SUM=1`. The `sstatus.SUM` bit modifies the privilege of supervisor mode loads and stores to virtual memory. Supervisor mode software may not execute code on any page marked with `PTE.U=1`.

Two schemes to manage the A and D bits are permitted:

- When a virtual page is accessed and the A bit is clear, or is written and the D bit is clear, a page-fault exception is raised
- When a virtual page is accessed and the A bit is clear, or is written and the D bit is clear, the corresponding bit(s) are set in the PTE. The PTE update is atomic with respect to other accesses to the PTE, and memory access will not occur until the PTE update is visible globally.

For non-leaf PTEs, the D, A, and U bits are reserved for future use and must be cleared by software for forward compatibility.

It is important to note the P550 does not automatically set the accessed (A) and dirty (D) bits in a Sv48 Page Table Entry (PTE). Instead, the P550 MMU will raise a page fault exception for a read to a page with `PTE.A=0` or a write to a page with `PTE.D=0`.

3.9.2 Supervisor Address Translation and Protection Register (SATP)

The `satp` register is a 64-bit read/write register used to control supervisor address translation and protection.



Figure 9: RV64 Supervisor Address Translation Register (satp)

- The `satp.PPN` field holds the physical page number of the root page table, which is the supervisor physical address divided by 4 KiB
- The `satp.ASID` is an address space identifier used to facilitate address-translation fences on a per-address-space basis
- The `satp.MODE` field determines the selected address-translation scheme

Translation Modes

Possible values for `satp.MODE` include:

Value of <code>MODE</code>	Description
0x0	Bare mode - no translation enabled
0x1-0x7	Reserved
0x8	Page-based 39-bit virtual addressing (Sv39)
0x9	Page-based 48-bit virtual addressing (Sv48)
0xA	Reserved for page-based 57-bit virtual addressing
0xB	Reserved for page-based 64-bit virtual addressing
0xC-0xF	Reserved

Table 10: `satp.MODE` Values

When `satp.MODE=0x0`, supervisor virtual addresses are equal to supervisor physical addresses, and there is no additional memory protection beyond the physical memory protection scheme described in Section 3.10. In this case, the remaining fields in `satp` have no effect.

For RV64 architectures on SiFive designs, `satp.MODE=8` is used for Sv48 virtual addressing, and no other modes are currently supported.

Note that writing `satp` does not imply any ordering constraints between page-table updates and subsequent address translations. If the new address space's page tables have been modified, or if an ASID is reused, it may be necessary to execute an `SFENCE.VMA` instruction after writing `satp`, which will:

1. Synchronize page table writes and address translation hardware for higher privilege levels
2. Guarantee previous stores are ordered before all subsequent references from the hart to the memory management data structures
3. Flush L1 and L2 unified TLB entry

Note

Content from Section 3.9.3, Section 3.9.4, Section 3.9.5, and Section 3.9.6 are directly from *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

3.9.3 Supervisor Memory-Management Fence Instruction (SFENCE.VMA)

The supervisor memory-management fence instruction SFENCE.VMA is used to synchronize updates to in-memory memory-management data structures with current execution.

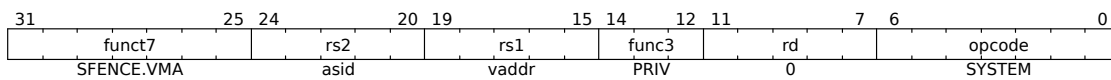


Figure 10: SFENCE.VMA Instruction

Instruction execution causes implicit reads and writes to these data structures; however, these implicit references are ordinarily not ordered with respect to explicit loads and stores. Executing an SFENCE.VMA instruction guarantees that any previous stores already visible to the current RISC-V hart are ordered before all subsequent implicit references from that hart to the memory-management data structures.

The SFENCE.VMA is used to flush any local hardware caches related to address translation. It is specified as a fence rather than a TLB flush to provide cleaner semantics with respect to which instructions are affected by the flush operation and to support a wider variety of dynamic caching structures and memory-management schemes. SFENCE.VMA is also used by higher privilege levels to synchronize page table writes and the address translation hardware.

SFENCE.VMA orders only the local hart's implicit references to the memory-management data structures.

Consequently, other harts must be notified separately when the memory-management data structures have been modified. One approach is to use 1) a local data fence to ensure local writes are visible globally, then 2) an interprocessor interrupt to the other thread, then 3) a local SFENCE.VMA in the interrupt handler of the remote thread, and finally 4) signal back to the originating thread that the operation is complete. This is, of course, the RISC-V analog to a TLB shutdown.

For the common case that the translation data structures have only been modified for a single address mapping (i.e., one page or superpage), rs1 can specify a virtual address within that mapping to affect a translation fence for that mapping only. Furthermore, for the common case that the translation data structures have only been modified for a single address-space identifier,

rs2 can specify the address space. The behavior of SFENCE.VMA depends on rs1 and rs2 as follows:

- If rs1=x0 and rs2=x0, the fence orders all reads and writes made to any level of the page tables, for all address spaces
- If rs1=x0 and rs2≠x0, the fence orders all reads and writes made to any level of the page tables, but only for the address space identified by integer register rs2. Accesses to global mappings are not ordered.
- If rs1≠x0 and rs2=x0, the fence orders only reads and writes made to the leaf page table entry corresponding to the virtual address in rs1, for all address spaces
- If rs1≠x0 and rs2≠x0, the fence orders only reads and writes made to the leaf page table entry corresponding to the virtual address in rs1, for the address space identified by integer register rs2. Accesses to global mappings are not ordered.

When rs2≠x0, bits [SXLEN-1:ASIDMAX] of the value held in rs2 are reserved for future use and should be zeroed by software and ignored by current implementations. Furthermore, if [ASIDLEN < ASIDMAX], the implementation shall ignore bits ASIDMAX-1:ASIDLEN of the value held in rs2.

3.9.4 Scenarios that Require SFENCE.VMA Instruction

The following common situations typically require executing an SFENCE.VMA instruction:

- When software recycles an ASID (i.e., reassociates it with a different page table), it should first change satp to point to the new page table using the recycled ASID, then execute SFENCE.VMA with rs1=x0 and rs2 set to the recycled ASID. Alternatively, software can execute the same SFENCE.VMA instruction while a different ASID is loaded into satp, provided the next time satp is loaded with the recycled ASID, it is simultaneously loaded with the new page table.
- If the implementation does not provide ASIDs, or software chooses to always use ASID 0, then after every satp write, software should execute SFENCE.VMA with rs1=x0. In the common case that no global translations have been modified, rs2 should be set to a register other than x0 but which contains the value zero, so that global translations are not flushed.
- If software modifies a non-leaf PTE, it should execute SFENCE.VMA with rs1=x0. If any PTE along the traversal path had its G bit set, rs2 must be x0; otherwise, rs2 should be set to the ASID for which the translation is being modified.
- If software modifies a leaf PTE, it should execute SFENCE.VMA with rs1 set to a virtual address within the page. If any PTE along the traversal path had its G bit set, rs2 must be x0; otherwise, rs2 should be set to the ASID for which the translation is being modified.
- For the special cases of increasing the permissions on a leaf PTE and changing an invalid PTE to a valid leaf, software may choose to execute the SFENCE.VMA lazily. After modifying the PTE but before executing SFENCE.VMA, either the new or old permissions will be used.

In the latter case, a page fault exception might occur, at which point software should execute `SFENCE.VMA` in accordance with the previous bullet point.

Speculation

The P550 will perform a speculative data access as a result of speculative ITLB refill. Changes in the `satp` register do not necessarily flush TLB entries. It is required to execute an `SFENCE.VMA` instruction after modifying page table entries in order to flush the cached translations. Exceptions only occur on accesses that are generated as a result of instruction execution, not access that are done speculatively.

ASID Usage for Supervisor Software

Supervisor software that uses ASIDs should use a nonzero ASID value to refer to the same address space across all harts in the supervisor execution environment (SEE) and should not use an ASID value of 0. If supervisor software does not use ASIDs, then the ASID field in the `satp` CSR should be set to 0.

3.9.5 Trap Virtual Memory

The `mstatus.TVM` (Trap Virtual Memory) bit supports intercepting supervisor virtual-memory management operations. When `TVM=1`, attempts to read or write the `satp` CSR or execute the `SFENCE.VMA` instruction while executing in S-mode will raise an illegal instruction exception. When `TVM=0`, these operations are permitted in supervisor mode. `TVM` is hard-wired to 0 when supervisor mode is not supported. The `TVM` mechanism improves virtualization efficiency by permitting guest operating systems to execute in supervisor mode, rather than classically virtualizing them in user mode. This approach obviates the need to trap accesses to most S-mode CSRs. Trapping `satp` accesses and the `SFENCE.VMA` instruction provides the hooks necessary to lazily populate shadow page tables.

3.9.6 Virtual Address Translation Process

For Sv48, `LEVELS` equals 4, and `PTESIZE` equals 8 in the steps below. A virtual address (`va`) is translated into a physical address (`pa`) as follows:

1. Let `a` be `satp.ppn × PAGESIZE`, and let `i = LEVELS - 1`.
2. Let `pte` be the value of the PTE at address `a + va.vpn[i] × PTESIZE`. If accessing `pte` violates a PMA or PMP check, raise an access exception corresponding to the original access type.
3. If `pte.v = 0`, or if `pte.r = 0` and `pte.w = 1`, stop and raise a page-fault exception corresponding to the original access type
4. Otherwise, the PTE is valid. If `pte.r = 1` or `pte.x = 1`, go to Step 5. Otherwise, this PTE is a pointer to the next level of the page table. Let `i = i - 1`. If `i < 0`, stop and raise a page-

fault exception corresponding to the original access type. Otherwise, let $a = \text{pte.ppn} \times \text{PAGESIZE}$ and go to Step 2.

5. A leaf PTE has been found. Determine if the requested memory access is allowed by the `pte.r`, `pte.w`, `pte.x`, and `pte.u` bits, given the current privilege mode and the value of the SUM and MXR fields of the `mstatus` register. If not, stop and raise a page-fault exception corresponding to the original access type.
6. If $i > 0$ and `pte.ppn[i-1:0]  $\neq$  0`, this is a misaligned superpage; stop and raise a page-fault exception corresponding to the original access type
7. If `pte.a = 0`, or if the memory access is a store and `pte.d = 0`, either raise a page-fault exception corresponding to the original access type, or:
 - a. Set `pte.a` to 1 and, if the memory access is a store, also set `pte.d` to 1
 - b. If this access violates a PMA or PMP check, raise an access exception corresponding to the original access type.
 - c. This update and the loading of `pte` in Step 2 must be atomic; in particular, no intervening store to the PTE may be perceived to have occurred in-between
8. The translation is successful. The translated physical address is given as follows:
 - a. $\text{pa.pgoff} = \text{va.pgoff}$
 - b. If $i > 0$, then this is a superpage translation and $\text{pa.ppn}[i-1:0] = \text{va.vpn}[i-1:0]$
 - c. $\text{pa.ppn}[\text{LEVELS}-1:i] = \text{pte.ppn}[\text{LEVELS}-1:i]$

3.9.7 Virtual-to-Physical Mapping Example

Note

The examples below assume a Sv39 configuration.

The following figure is a high-level view of how a virtual address is mapped to a physical address for a Linux application. When the Linux kernel creates a process, it will allocate multiple pages of physical memory to store the code and data. TLB MMU is used to:

- Translate the virtual addresses to physical addresses
- Provide uniform virtual memory layout for a user application
- Protect user applications unauthorized access to other address space

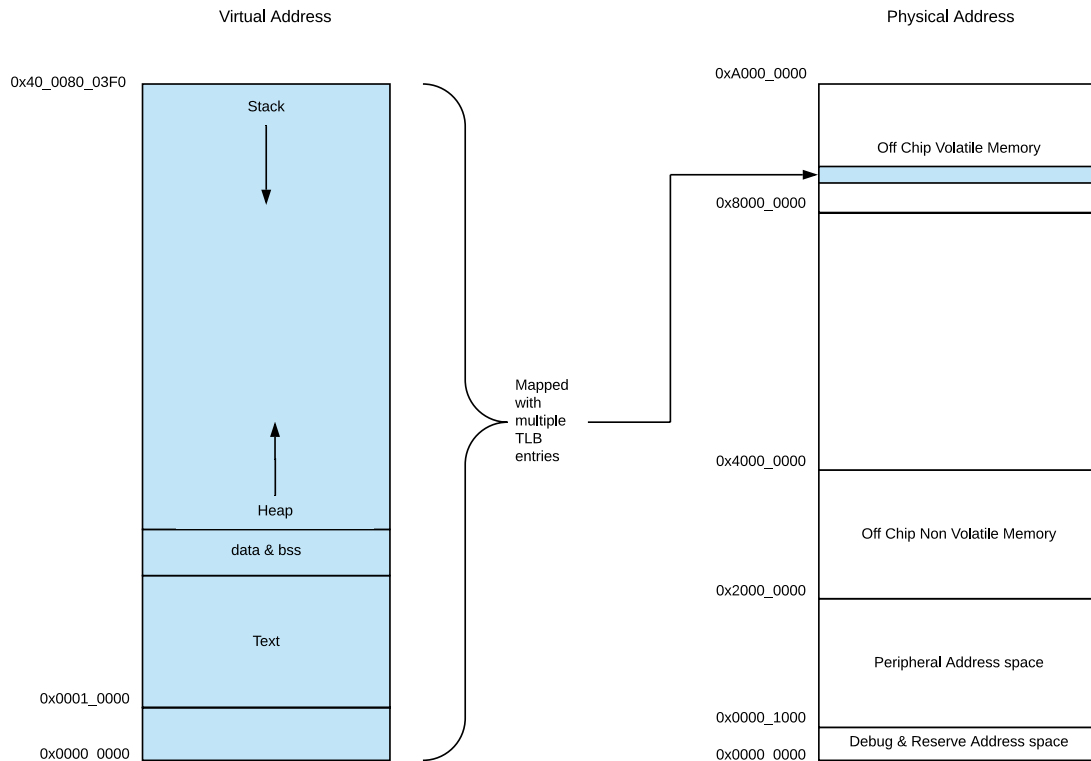


Figure 11: Linux User Application Memory Map Example

In this example, code beginning at VA=0x0001_0000 needs to be mapped to an address in off chip volatile memory.

When the hart tries to execute instructions at this address, it needs to use a matched TLB page table entry to do virtual address to physical address translation. If it cannot be located in the L1 instruction TLB, or the L2 unified TLB, the hart will start hardware table walk from the TLB page table base address. The page table base address is obtained by multiplying `satp.PPN` by the L2 PAGESIZE (4 KiB).

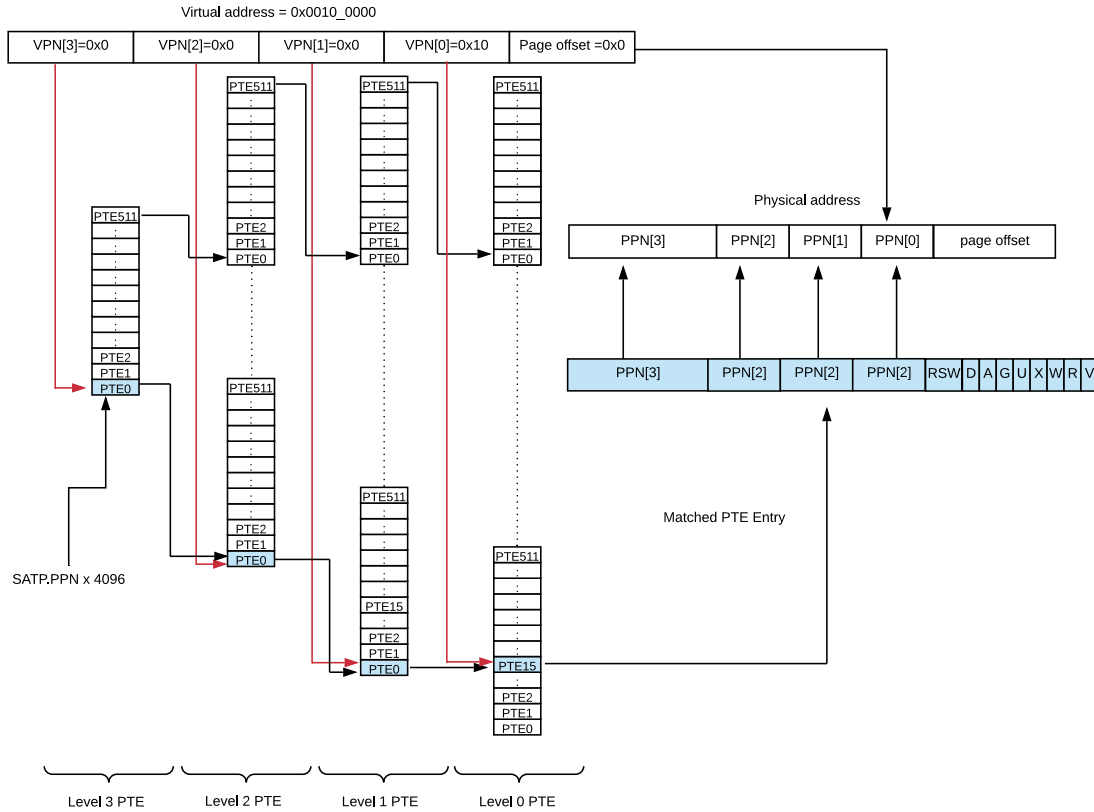


Figure 12: Hardware Table Walk Example

The TLB MMU will execute a page table walk in order to determine the correct mapping for a particular virtual address. Page table entries are pointers to the next level page table if the page is marked as not Readable (R=0), not Writable (W=0), and not Executable (X=0). Otherwise, it is a leaf PTE.

In this example, there are 4 levels of page table entries. The hart will start the hardware table walk from the level 3 page table entry. In the Sv48 scheme, there are 512 page table entries in level 3 page table entry. A hart can quickly locate the entry using VPN[3] number, in this case, entry 0. The hart will continue the hardware table walk to the L2 page table entry when the entry doesn't match.

There are 512 clusters of page table entries in L2 and each cluster also has 512 TLB page table entries. In this example, PPNs in the level 3 entry 0 is used to locate the right cluster in the L2 page entry. The hart will locate the entry using VPN[2] number, in this case, entry 0. The hart will continue a hardware table walk to L1 page table entry when this entry does not match.

There are 512×512 clusters of page table entries in L1 and each cluster also has 512 TLB page table entries. In this example, PPNs in the L2 entry 0 is used to locate the right cluster in the L1 page entry. The hart will locate the entry using VPN[1] number, in this case, entry 0. The hart will continue a hardware table walk to level 0 page table entry when this entry does not match.

At the level 0 page table entry, there are $512 \times 512 \times 512$ clusters of page table entries, and each cluster has 512 TLB page table entries. In this example, PPNs in the L1 entry 0 is used to locate the correct cluster in the level 0 page entry. The hart then finds the entry using the `VPN[0]` value, which in this case, corresponds to entry 15.

When there is a match in level 0 page table entry, virtual address will map to physical address. The physical page number is combined with the page offset to give the complete physical address.

3.9.8 MMU at Reset

The TLB MMU is disabled by default out of reset. All accessed regions have a 1:1 virtual-to-physical mapping when the MMU is disabled. If the PMP is not yet enabled, all access permissions out of reset are determined by the static PMA values.

3.10 Physical Memory Protection (PMP)

Machine mode is the highest privilege level and by default has read, write, and execute permissions across the entire memory map of the device. However, privilege levels below machine mode do not have read, write, or execute permissions to any region of the device memory map unless it is specifically allowed by the PMP. For the lower privilege levels, the PMP may grant permissions to specific regions of the device's memory map, but it can also revoke permissions when in machine mode.

When programmed accordingly, the PMP will check every access when the hart is operating in supervisor or user modes. For machine mode, PMP checks do not occur unless the lock bit (L) is set in the `pmpcfgY` CSR for a particular region.

PMP checks also occur on loads and stores when the machine previous privilege level is supervisor or user (`mstatus.MPP=0x1` or `mstatus.MPP=0x0`), and the Modify Privilege bit is set (`mstatus.MPRV=1`). For virtual address translation, PMP checks are also applied to page table accesses in supervisor mode.

The P550 PMP supports 8 regions with a minimum region size of 4096 bytes.

This section describes how PMP concepts in the RISC-V architecture apply to the P550. For additional information on the PMP refer to *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

3.10.1 PMP Functional Description

The P550 PMP unit has 8 regions and a minimum granularity of 4096 bytes. Access to each region is controlled by an 8-bit `pmpxcfg` field and a corresponding `pmpaddrX` register. Overlapping regions are permitted, where the lower numbered `pmpxcfg` and `pmpaddrX` registers take priority over higher numbered regions. The P550 PMP unit implements the architecturally

defined `pmpcfgY` CSR `pmpcfg0`, supporting 8 regions. `pmpcfg2` is implemented, but hardwired to zero. Access to `pmpcfg1` or `pmpcfg3` results in an illegal instruction exception.

The PMP registers may only be programmed in M-mode. Ordinarily, the PMP unit enforces permissions on S-mode and U-mode accesses. However, locked regions (see Section 3.10.2) additionally enforce their permissions on M-mode.

3.10.2 PMP Region Locking

The PMP allows for region locking whereby, once a region is locked, further writes to the configuration and address registers are ignored. Locked PMP entries may only be unlocked with a system reset. A region may be locked by setting the L bit in the `pmpXcfg` register.

In addition to locking the PMP entry, the L bit indicates whether the R/W/X permissions are enforced on machine mode accesses. When the L bit is clear, the R/W/X permissions apply to S-mode and U-mode.

3.10.3 PMP Registers

Each PMP region is described by an 8-bit `pmpXcfg` field, used in association with a 64-bit `pmpaddrX` register that holds the base address of the protected region. The range of each region depends on the Addressing (A) mode described in the next section. The `pmpXcfg` fields reside within 64-bit `pmpcfgY` CSRs.

Each 8-bit `pmpXcfg` field includes a read, write, and execute bit, plus a two bit address-matching field A, and a Lock bit, L. Overlapping regions are permitted, where the lowest numbered PMP entry wins for that region.

PMP Configuration Registers

For RV64 architectures, `pmpcfg1` and `pmpcfg3` are not implemented. This reduces the footprint since `pmpcfg2` already contains configuration fields `pmp8cfg` through `pmp11cfg` for both RV32 and RV64.

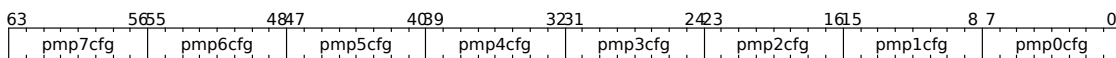


Figure 13: RV64 `pmpcfg0` Register

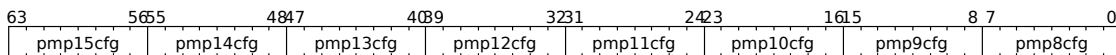


Figure 14: RV64 `pmpcfg2` Register

The `pmpcfgY` and `pmpaddrX` registers are only accessible via CSR specific instructions such as `csrr` for reads, and `csrw` for writes.

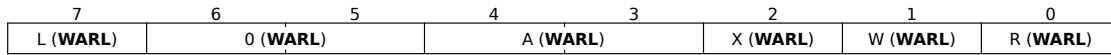


Figure 15: RV64 pmpXcfg bitfield

Bits	Description
0	R: Read Permissions 0x0 - No read permissions for this region 0x1 - Read permission granted for this region
1	W: Write Permissions 0x0 - No write permissions for this region 0x1 - Write permission granted for this region
2	X: Execute permissions 0x0 - No execute permissions for this region 0x1 - Execute permission granted for this region
[4:3]	A: Address matching mode 0x0 - PMP Entry disabled. No PMP protection applied for any privilege level. 0x1 - Top of range (TOR) region defined by two adjacent pmpaddr registers. The upper limit of region X is defined by pmpaddrX, and the base of the region is defined by pmpaddr(X-1). Address 'a' matches the region if $[pmpaddr(X-1) \leq a < pmpaddrX]$. If pmp0cfg defines a TOR region, then the base address of that region is 0x0, and pmpaddr0 defines the upper limit. Supports only a four byte granularity. 0x2 - Naturally aligned four-byte region (NA4). Supports only a four-byte region with four byte granularity. 0x3 - Naturally aligned power-of-two region (NAPOT), ≥ 8 bytes. When this setting is programmed, the low bits of the pmpaddrX register encode the size, while the upper bits encode the base address right shifted by two. There is a zero bit in between, we will refer to as the least significant zero bit (LSZB).
7	L: Lock Bit 0x0 - PMP Entry Unlocked, no permission restrictions applied to machine mode. PMP entry only applies to S and U modes. 0x1 - PMP Entry Locked, permissions enforced for all privilege levels including machine mode. Writes to pmpXcfg and pmpcfgY are ignored and can only be cleared with system reset.
Note: The combination of R=0 and W=1 is not currently implemented.	

Table 11: pmpXcfg Bitfield Description

Out of reset, the PMP register fields A and L are set to 0. All other hart state is unspecified by *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

Some examples follow using NAPOT address mode.

Base Address	Region Size*	LSZB Position	pmpaddrX Value
0x4000_0000	8 B	0	(0x1000_0000 1'b0)
0x4000_0000	32 B	2	(0x1000_0000 3'b011)
0x4000_0000	4 KB	9	(0x1000_0000 10'b01_1111_1111)
0x4000_0000	64 KB	13	(0x1000_0000 14'b01_1111_1111_1111)
0x4000_0000	1 MB	17	(0x1000_0000 18'b01_1111_1111_1111_1111)
*Region size is $2^{(LSZB+3)}$.			

Table 12: pmpaddrX Encoding Examples for A=NAPOT

PMP Address Registers

The PMP has 8 address registers. Each address register pmpaddrX correlates to the respective pmpXcfg field. Each address register contains the base address of the protected region right shifted by two, for a minimum 4-byte alignment.

The maximum encoded address bits per *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10* are [55:2].

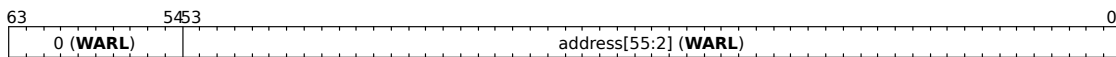


Figure 16: RV64 pmpaddrX Register

3.10.4 PMP and PMA

The PMP values are used in conjunction with the Physical Memory Attributes (PMAs) described in Section 4.1. Since the PMAs are static and not configurable, the PMP can only revoke read, write, or execute permissions to the PMA regions if those permissions already apply statically.

3.10.5 PMP Programming Overview

The PMP registers can only be programmed in machine mode. The pmpaddrX register should be first programmed with the base address of the protected region, right shifted by two. Then, the pmpcfgY register should be programmed with the properly configured 64-bit value containing each properly aligned 8-bit pmpXcfg field. Fields that are not used can be simply written to 0, marking them unused.

PMP Programming Example

The following example shows a machine mode only configuration where PMP permissions are applied to three regions of interest, and a fourth region covers the remaining memory map. Recall that lower numbered pmpXcfg and pmpaddrX registers take priority over higher numbered regions. This rule allows higher numbered PMP registers to have blanket coverage over the

entire memory map while allowing lower numbered regions to apply permissions to specific regions of interest. The following example shows a 64 KB Flash region at base address 0x0, a 32 KB RAM region at base address 0x2000_0000, and finally a 4 KB peripheral region at base address base 0x3000_0000. The rest of the memory map is reserved space.

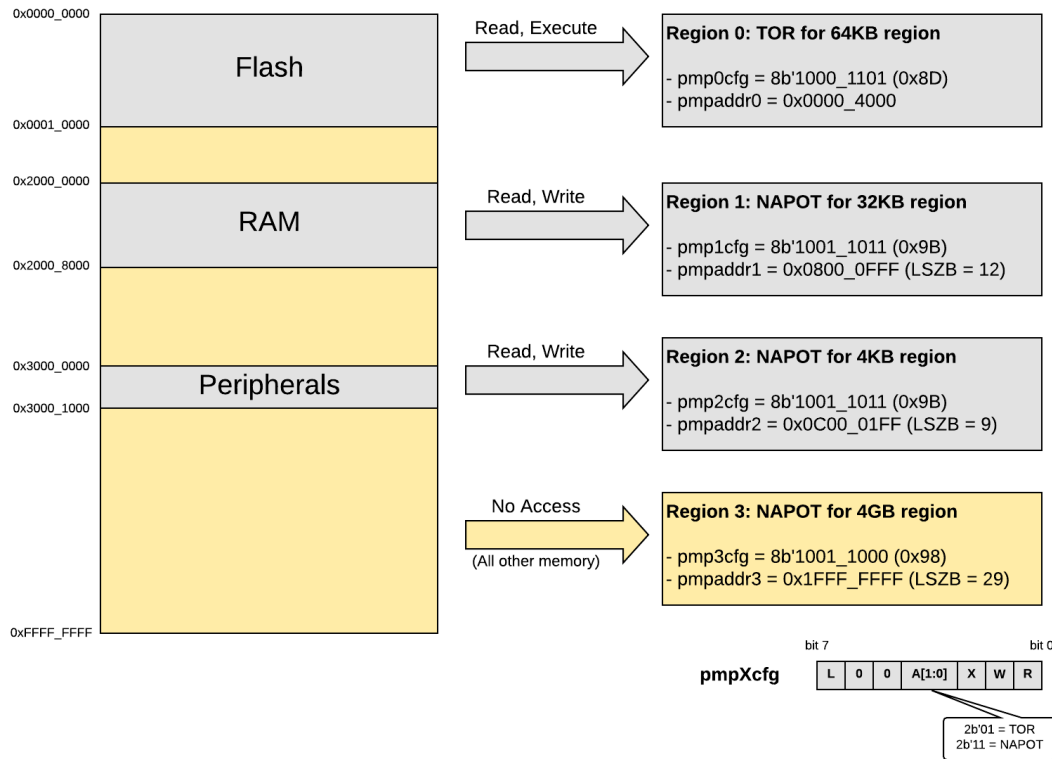


Figure 17: PMP Example Block Diagram

PMP Access Scenarios

The L, R, W, and X bits only determine if an access succeeds if all bytes of that access are covered by that PMP entry. For example, if a PMP entry is configured to match the four-byte range 0xC–0xF, then an 8-byte access to the range 0x8–0xF will fail, assuming that PMP entry is the highest-priority entry that matches those addresses.

While operating in machine mode when the lock bit is clear (L=0), if a PMP entry matches all bytes of an access, the access succeeds. If the lock bit is set (L=1) while in machine mode, then the access depends on the permissions set for that region. Similarly, while in Supervisor mode or User mode, the access depends on permissions set for that region.

Failed read or write accesses generate a load or store access exception, and an instruction access fault would occur on a failed instruction fetch. When an exception occurs while attempting to execute from a region without execute permissions, the fault occurs on the fetch and not

the branch, so the mepc CSR will reflect the value of the targeted protected region, and not the address of the branch.

It is possible for a single instruction to generate multiple accesses, which may not be mutually atomic. If at least one access generated by an instruction fails, then an exception will occur. It might be possible that other accesses from a single instruction will succeed, with visible side effects. For example, references to virtual memory may be decomposed into multiple accesses.

On some implementations, misaligned loads, stores, and instruction fetches may also be decomposed into multiple accesses, some of which may succeed before an access exception occurs. In particular, a portion of a misaligned store that passes the PMP check may become visible, even if another portion fails the PMP check. The same behavior may manifest for floating-point stores wider than XLEN bits (e.g., the FSD instruction in RV32D), even when the store address is naturally aligned.

3.10.6 PMP and Paging

The Physical Memory Protection mechanism is designed to compose with the page-based virtual memory systems described in *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*. When paging is enabled, instructions that access virtual memory may result in multiple physical-memory accesses, including implicit references to the page tables. The PMP checks apply to all of these accesses. The effective privilege mode for implicit page-table accesses is supervisor mode.

Implementations with virtual memory are permitted to perform address translations speculatively and earlier than required by an explicit virtual-memory access. The PMP settings for the resulting physical address may be checked at any point between the address translation and the explicit virtual-memory access. A mis-predicted branch to a non-executable address range does not generate a trap. Hence, when the PMP settings are modified in a manner that affects either the physical memory that holds the page tables or the physical memory to which the page tables point, M-mode software must synchronize the PMP settings with the virtual memory system. This is accomplished by executing an SFENCE.VMA instruction with `rs1=x0` and `rs2=x0`, after the PMP CSRs are written.

If page-based virtual memory is not implemented, or when it is disabled, memory accesses check the PMP settings synchronously, so no fence is needed.

3.10.7 PMP Limitations

In a system containing multiple harts, each hart has its own PMP device. The PMP permissions on a hart cannot be applied to accesses from other harts in a multi-hart system. In addition, SiFive designs may contain a Front Port to allow external bus masters access to the full memory map of the system. The PMP cannot prevent access from external bus masters on the Front Port.

3.10.8 Behavior for Regions without PMP Protection

If a non-reserved region of the memory map does not have PMP permissions applied, then by default, supervisor or user mode accesses will fail, while machine mode access will be allowed. Access to reserved regions within a device's memory map (an interrupt controller for example) will return 0x0 on reads, and writes will be ignored. Access to reserved regions outside of a device's memory map without PMP protection will result in a bus error. The bus error can generate an interrupt to the hart using the Bus-Error Unit (BEU). See Chapter 11 for more information.

3.10.9 Cache Flush Behavior on PMP Protected Region

When a line is brought into cache and the PMP is set up with the lock (L) bit asserted to protect a part of that line, a data cache flush instruction will generate a store access fault exception if the flush includes any part of the line that is protected. The cache flush instruction does an invalidate and write-back, so it is essentially trying to write back to the memory location that is protected. If a cache flush occurs on a part of the line that was not protected, the flush will succeed and not generate an exception. If a data cache flush is required without a write-back, use the cache discard instruction instead, as this will invalidate but not write back the line.

3.11 Hardware Performance Monitor

The P550 processor core supports a basic hardware performance monitoring (HPM) facility. The performance monitoring facility is divided into two classes of counters: fixed-function and event-programmable counters. These classes consist of a set of fixed counters and their counter-enable registers, as well as a set of event-programmable counters and their event selector registers. The registers are available to control the behavior of the counters. Performance monitoring can be useful for multiple purposes, from optimization to debug.

3.11.1 Performance Monitoring Counters Reset Behavior

The `instret` and `cycle` counters are initialized to zero on system reset. The hardware performance monitor event counters are not initialized on system reset, and thus have an arbitrary value. Users can write desired values to the counter control and status registers (CSRs) to start counting at a given, known value.

3.11.2 Fixed-Function Performance Monitoring Counters

A fixed-function performance monitor counter is hardware wired to only count one specific event type. That is, they cannot be reconfigured with respect to the event type(s) they count. The only modification to the fixed-function performance monitoring counters that can be done is to enable or disable counting, and write the counter value itself.

The P550 processor core contains two fixed-function performance monitoring counters.

Fixed-Function Cycle Counter (`mcycle`)

The fixed-function performance monitoring counter `mcycle` holds a count of the number of clock cycles the hart has executed since some arbitrary time in the past. The `mcycle` counter is read-write and 64 bits wide. Reads of `mcycle` return all 64 bits of the `mcycle` CSR.

Fixed-Function Instructions-Retired Counter (`minstret`)

The fixed-function performance monitoring counter `minstret` holds a count of the number of instructions the hart has retired since some arbitrary time in the past. The `minstret` counter is read-write and 64 bits wide. Reads of `minstret` return all 64 bits of the `minstret` CSR.

3.11.3 Event-Programmable Performance Monitoring Counters

Complementing the fixed-function counters are a set of programmable event counters. The P550 HPM includes four additional event counters, `mhpcounter3`, `mhpcounter4`, `mhpcounter5`, and `mhpcounter6`. These programmable event counters are read-write and 64 bits wide. The hardware counters themselves are implemented as 40-bit counters on the P550 core series. These hardware counters can be written to in order to initialize the counter value.

3.11.4 Event Selector Registers

To control the event type to count, event selector CSRs `mhpmevent3`, `mhpmevent4`, `mhpmevent5`, and `mhpmevent6` are used to program the corresponding event counters. These event selector CSRs are 64-bit **WARL** registers.

The event selectors are partitioned into two fields; the lower 8 bits select an event class, and the upper bits form a mask of events in that class.

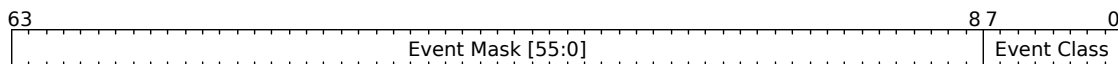


Figure 18: Event Selector Fields

The counter increments if the event corresponding to any set mask bit occurs. For example, if `mhpmevent3` is set to `0x4200`, then `mhpcounter3` will increment when either a load instruction or a conditional branch instruction retires. An event selector of 0 means "count nothing".

3.11.5 Event Selector Encodings

Table 13 describes the event selector encodings available. Events are categorized into classes based on the Event Class field encoded in `mhpmeventX[7:0]`. One or more events can be programmed by setting the respective Event Mask bit for a given event class. An event selector encoding of 0 means "count nothing". Multiple events will cause the counter to increment any time any of the selected events occur.

Machine Hardware Performance Monitor Event Register	
Instruction Commit Events, mhpmeventX[7:0]=0x0	
Bits	Description
8	Exception taken
9	Integer load instruction retired
10	Integer store instruction retired
11	Atomic memory operation retired
12	System instruction retired
13	Integer arithmetic instruction retired
14	Conditional branch retired
15	JAL instruction retired
16	JALR instruction retired
17	Integer multiplication instruction retired
18	Integer division instruction retired
19	Floating-point load instruction retired
20	Floating-point store instruction retired
21	Floating-point addition retired
22	Floating-point multiplication retired
23	Floating-point fused multiply-add retired
24	Floating-point division or square-root retired
25	Other floating-point instruction retired
Microarchitectural Events, mhpmeventX[7:0]=0x1	
Bits	Description
8	Address-generation interlock
9	Long-latency interlock
10	CSR read interlock
11	Instruction cache/ITIM busy
12	Data cache/DTIM busy
13	Branch direction misprediction
14	Branch/jump target misprediction
15	Pipeline flush from CSR write
16	Pipeline flush from other event
17	Integer multiplication interlock
18	Floating-point interlock
Memory System Events, mhpmeventX[7:0]=0x2	
Bits	Description
8	Instruction cache miss
9	Data cache miss or memory-mapped I/O access
10	Data cache write-back
11	Instruction TLB miss
12	Data TLB miss
13	UTLB miss
14	UTLB hit

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
15	PTE cache miss
16	PTE cache hit
Instruction Fetch Unit Events, mhpmeventX[7:0]=0x3	
Bits	Description
8	Branch direction predictor misprediction
9	IJTP misprediction
10	IJTP hint wrong
11	Instruction cache way predictor misprediction
12	Instruction cache prefetch request
13	Instruction cache fetch request
14	Instruction fetch unit stalled due to instruction cache miss
15	Instruction fetch unit stalled
16	Instruction cache fill
17	Instruction TLB miss
18	Branch direction predictor trained
19	Branch direction predictor training stalled
20	Stalled due to instruction TLB miss
21	Stalled due to full fetch queue
22	Decoded less than maximum operations
23	Next-line predictor high target redirection
24	Next-line predictor low target redirection
25	Next-line predictor not taken but redirected
26	Next-line predictor taken but redirected
27	RAS predicted a return
28	RAS forwarded a return prediction
29	RAS empty for return
30	RAS mispredicted target
31	RAS overflow
32	IJTP predicted a JALR
33	Branch direction predictor predicted taken on a branch
Rename and Dispatch Unit Events, mhpmeventX[7:0]=0x4	
Bits	Description
8	Load/store unit dispatch busy
9	ROB flush due to CSR write
10	ROB flush due to other event
11	Move eliminated
12	Conditional move converted
13	LUI fused
14	AUIPC fused
15	Integer pipeline dispatch buffer stall
16	Load/Store Unit dispatch buffer stall
17	Floating-point pipeline dispatch buffer stall

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
18	Rename history file stall
19	ROB stall
20	PC map table stall
21	Integer pipeline free list stall
22	Floating-point pipeline free list stall
23	Branch resolution queue stall
24	MDP allocate
25	MDP over-allocate when full
26	MDP update useful increment
27	MDP update useful increment maxed
28	MDP update not useful decrement
29	MDP update not useful decrement zeroed
30	MDP update multi-set set
31	MDP update multi-set clear
32	MDP age decayed
33	Instruction decode stall from rename history file rewind
34	ROB flush due to ECALL/EBREAK
35	ROB flush due to FENCE . I
36	ROB flush-before from LSU
37	ROB flush-after from LSU
Integer Pipeline Events, mhpmeventX[7:0]=0x5	
Bits	Description
8	Reserved
9	Branch/jump target misprediction
LS Tag L1 Data Cache Vector Events, mhpmeventX[7:0]=0x6	
Bits	Description
8	L1 data cache tag miss
9	L1 data cache tag hit
10	Reserved
11	L1 data cache way predictor miss
12	L1 data cache way predictor hit
13	L1 data cache way predictor unknown
14	Memory non-cacheable
15	L1 data cache load tag miss
16	L1 data cache load tag hit
17	Reserved
18	L1 data cache load way predictor miss
19	L1 data cache load way predictor hit
20	L1 data cache load way predictor unknown
21	Load non-cacheable
22	L1 data cache store tag miss
23	L1 data cache store tag hit

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
24	Reserved
25	Store non-cacheable
26	L1 data cache AMO/LR/SC tag miss
27	L1 data cache AMO/LR/SC tag hit
28	Reserved
29	L1 data cache AMO/LR/SC way predictor miss
30	L1 data cache AMO/LR/SC way predictor hit
31	L1 data cache AMO/LR/SC way predictor unknown
32	AMO/LR/SC non-cacheable
33	L1 data cache PTW tag miss
34	L1 data cache PTW tag hit
35	L1 data cache PTW way predictor miss
36	L1 data cache PTW way predictor hit
37	L1 data cache PTW way predictor unknown
38	PTW non-cacheable
39	L1 data cache HWPF tag miss
40	L1 data cache HWPF tag hit
Load/Store Tag Pipe Vector Events, mhpmeventX[7:0]=0x7	
Bits	Description
8	L1 data cache write block issue stage
9	L1 data cache LS tag read blocked
10	PTW block issue stage
11	Data TLB miss
12	Data TLB hit
13	Data TLB load miss
14	Data TLB load hit
15	Data TLB store miss
16	Data TLB store hit
17	Data TLB AMO/LR/SC miss
18	Data TLB AMO/LR/SC hit
19	Data TLB HWPF miss
20	Data TLB HWPF hit
21	LS tag replay address operand
22	LS tag replay in-order operation
23	LS tag confirm ROB resolve
24	LS tag confirm ROB exception
25	SFENCE.VMA TLB invalidate
26	LS tag confirm ROB flush newer (after self)
27	LS tag confirm ROB flush before (punt self)
Load/Store Tag Order Stage Events, mhpmeventX[7:0]=0x8	
Bits	Description
8	MSHR establish

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
9	MSHR reuse
10	MSHR load establish
11	MSHR load reuse
12	MSHR store establish
13	MSHR store reuse
14	MSHR AMO/LR/SC establish
15	MSHR PTW establish
16	MSHR HWPF establish
17	MSHR HWPF reuse
18	MSHR fence has dependency
19	MSHR non-fence has dependency
20	Load queue establish
21	Load queue needs data return
22	Load queue request data return
23	Load queue request data return from L1 data cache
24	Load queue request data return from MSHR
25	Load queue request data return from store-to-load forwarding
26	Load queue known read-after-read hazard
27	Load queue known read-after-write hazard
28	Load queue potential read-after-read hazard
29	Load queue potential read-after-write hazard
30	Store queue establish
31	LSTO op wants but cannot establish load queue
32	LSTO op wants but cannot establish store queue
33	LSTO op wants but cannot establish MSHR
34	Reserved
35	LSTO flush newest load queue entry
36	LSTO flush newer for MSHR capacity
37	LSTO flush newer for MSHR dependency
38	LSTO flush newer for self replay
39	LSTO L1 data cache tag correctable error
40	LSTO L1 data cache tag uncorrectable error
Load Data Pipe Events, mhpmeventX[7:0]=0x9	
Bits	Description
8	L1 data cache data tag search write read blocked
9	L1 data cache data store queue write blocked
10	L1 data cache data load queue read blocked
11	L1 data cache data MSHR data forward write blocked
12	LR/SC reservation set
13	LR/SC reservation cleared
Load Data Pipe Vector Events, mhpmeventX[7:0]=0xA	
Bits	Description

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
8	L1 data forward replay dependencies
9	L1 data write-back replay dependencies
10	L1 data write-back search hit L1 data cache data
11	L1 data write-back search forward from LS tag match
12	L1 data write-back search forward from LS tag order
13	L1 data write-back search forward from store queue entry
14	L1 data write-back search forward from store queue write gather
15	L1 data write-back load read from L1 data cache data hit
16	L1 data write-back load forward from store queue entry
17	L1 data write-back load forward from store queue write gather
18	L1 data write-back load read from MSHR data
19	L1 data write-back SC pass
20	L1 data write-back SC fail
21	L1 data write-back atomic
22	L1 data write-back L1 data cache data partial granule write
23	L1 data write-back ROB resolve
24	L1 data write-back ROB flush before (punt self)
25	L1 data write-back FS dirty
26	L1 data cache data correctable error
27	L1 data cache data uncorrectable error
Store Data Pipe Vector Events, mhpmeventX[7:0]=0xB	
Bits	Description
8	SD store data issued after address
9	SD replay store data operand
LSU Load/Store Queue Events, mhpmeventX[7:0]=0xC	
Bits	Description
8	LSU issue queue waiting for data TLB
9	LSU issue queue waiting for store queue
10	LSU issue queue waiting for load queue
11	LSU issue queue waiting for MSHR
12	Load queue return hold for L1 data cache tag
13	Load queue return retry for no data
14	Load queue return from MSHR data
15	Load queue return from L2 data fast
16	Load queue ROB resolve
17	Load queue ROB flush before
18	Load queue waiting for cacheable
19	Load queue waiting for non-cacheable
20	Load queue waiting for MSHR dependency
21	Store queue retire entry
22	Store queue retire write L1 data cache data
23	Store queue retire write MSHR data

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
24	Store queue retire write L1 data cache data L1HitCohClean
25	Store queue retire write MSHR data L1HitCohClean
26	Store queue retire hold for L1 data cache tag
27	Store queue retire hold for MSHR Dependency
28	LSU issue queue waiting for address source
29	LSU issue queue waiting for store data source
30	LSU issue queue waiting for older in-order op
31	LSU issue queue in-order op waiting for older ops
32	LSU issue queue waiting for older non-speculative op
33	LSU issue queue waiting for older stores to stack
34	LSU issue queue waiting for one older predicted MDP hazard store
35	LSU issue queue waiting for multiple older predicted MDP hazard stores
LSU Tag Search/MSHR Events, mhpmeventX[7:0]=0xD	
Bits	Description
8	L1 data cache tag MSHR data forward write blocked
9	L1 data cache tag MSHR L1 arbitration winner read blocked
10	Probe L1 data cache miss
11	Probe L1 data cache hit
12	Probe MSHR match
13	Tag search write hold for L1 data cache tag protected
14	Tag search write cannot establish release queue full
15	Tag search write cannot establish release queue match
16	Tag search write L1 data cache tag correctable error
17	Tag search write L1 data cache tag uncorrectable error
18	MSHR allocate L1 data cache
19	MSHR allocate L1 data cache over
20	MSHR upgrade L1 data cache tag
21	MSHR write L1 data cache way predictor blocked
22	MSHR data tag search write read blocked
23	MSHR data MSHR data read blocked
24	MSHR data store queue write gather write blocked
25	MSHR data MSHR L2 arbitration winner read blocked
26	MSHR data load queue write-back read blocked
27	MSHR data MSHR L1 arbitration winner read blocked
28	MSHR fence waiting for predecessor
29	MSHR successor waiting for fence
30	L1 data cache tag search write blocked
Load/Store Unit to L2 Events, mhpmeventX[7:0]=0xE	
Bits	Description
8	L2 request blocked
9	L2 request Acquire fromN
10	L2 request Acquire fromB

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
11	L2 request AcquirePerm no data
12	L2 request GetData
13	L2 request PutFull
14	L2 request PutPartial
15	L2 request Atomic
16	L2 request Hint
17	Probe request blocked
18	Evict request or Probe response blocked
19	Evict release dirty data
20	Evict release clean no data
21	Probe ack hit dirty data
22	Probe ack hit clean no data
23	Probe ack miss
24	Probe match release queue
25	L2 response bus error corrupt
26	L2 response bus error denied
LSU Hardware Prefetcher Events, mhpmeventX[7:0]=0xF	
Bits	Description
8	L1 HWPF sent
9	L2 HWPF sent
10	L3 HWPF sent
11	Demand (hit/miss) matched an engine and caused any number of prefetches
12	Late: demand matched an existed MSHR that was allocated by L1 HWPF
13	Very late: L1 HWPF reused an existing MSHR that was allocated by demand
14	New streams started
15	Increase distance aggressiveness
16	Stream replaced by new allocation
17	Stream untrained by window match and stride mismatch
18	L1 HWPF request blocked by L1 MSHR threshold
19	L2 HWPF request blocked by L2 MSHR threshold
20	L3 HWPF request blocked by L3 MSHR threshold
21	HWPF issue blocked
22	HWPF issue queue waiting for data TLB
23	HWPF issue queue waiting for MSHR
24	HWPF issue queue waiting for L2 queue
25	Stream stopped by HitL1Cache threshold
26	L2 HWPF stopped by HitL2Cache threshold
27	L3 HWPF stopped by HitL3Cache threshold
28	L1 HWPF hit L2 cache tag array
29	Unnecessary L2 HWPF hit L2 cache tag array
30	Late L2 HWPF: L1 HWPF match L2 MSHR in progress
31	Late: L2 HWPF match L2 MSHR in progress

Table 13: mhpmevent Register

Machine Hardware Performance Monitor Event Register	
32	L1 HWPf request blocked by HWPf issue queue threshold
33	L2 HWPf request blocked by HWPf issue queue threshold

Table 13: mhpmevent Register

Event mask bits that are writable for any event class are writable for all classes. Setting an event mask bit that does not correspond to an event defined in Table 13 has no effect for current implementations. However, future implementations may define new events in that encoding space, so it is not recommended to program unsupported values into the mhpmevent registers.

Combining Events

It is common usage to directly count each respective event. Additionally, it is possible to use combinations of these events to count new, unique events. For example, to determine the average cycles per load from a data memory subsystem, program one counter to count "Data cache/DTIM busy" and another counter to count "Integer load instruction retired". Then, simply divide the "Data cache/DTIM busy" cycle count by the "Integer load instruction retired" instruction count and the result is the average cycle time for loads in cycles per instruction.

It is important to be cognizant of the event types being combined; specifically, event types counting occurrences and event types counting cycles.

3.11.6 Counter-Enable Registers

The 32-bit counter-enable registers mcounteren and scounteren control the availability of the hardware performance-monitoring counters to the next-lowest privileged mode.

The settings in these registers only control accessibility. The act of reading or writing these enable registers does not affect the underlying counters, which continue to increment when not accessible.

When any bit in the mcounteren register is clear, attempts to read the cycle, time, instruction retire, or hpmcounterX register while executing in S-mode will cause an illegal instruction exception. When one of these bits is set, access to the corresponding register is permitted in the next implemented privilege mode, S-mode.

The same bit positions in the scounteren register analogously control access to these registers while executing in U-mode. If S-mode is permitted to access a counter register and the corresponding bit is set in scounteren, then U-mode is also permitted to access that register.

mcounteren and scounteren are **WARL** registers. Any of the bits may contain a hardwired value of zero, indicating reads to the corresponding counter will cause an illegal instruction exception when executing in a less-privileged mode.

3.12 Ports

This section describes the Port interfaces to the P550 core.

3.12.1 Front Port

The Front Port can be used by external masters to read from and write into the memory system utilizing any port in the Core Complex.

If a Front Port access targets the Memory Port, a coherency manager is responsible for maintaining coherency with the L1, pL2, and L3 caches. A read access can be returned directly from the caches without generating an external bus access. If a write from the Front Port targets a location in the L1 data cache or pL2 cache, it results in the line being evicted and invalidated. The write will then allocate to the L3 cache.

Any Front Port access that targets the Memory Port and results in an L1 and pL2 cache miss will allocate to the L3 cache.

The P550 Core Complex User Guide describes the implementation details of the Front Port.

3.12.2 Memory Port

The Memory Port is used to interface with memory that offers the highest performance for the P550 Core Complex, such as DDR. It supports cacheable accesses for data and instructions.

Consult Section 4.1 for further information about the Memory Port and its Physical Memory Attributes.

See the P550 Core Complex User Guide for a description of the Memory Port implementation in the P550 Core Complex.

3.12.3 Peripheral Port

The Peripheral Port is used to interface with lower speed peripherals and also supports code execution. When a device is attached to the Peripheral Port, it is expected that there are no other masters connected to that device.

Consult Section 4.1 for further information about the Peripheral Port and its Physical Memory Attributes.

See the P550 Core Complex User Guide for a description of the Peripheral Port implementation in the P550 Core Complex.

3.12.4 System Port

The System Port is used to interface with lower performance memory, like SRAM, memory-mapped I/O (MMIO), and higher speed peripherals. The System Port also supports code execution.

Consult Section 4.1 for further information about the System Port and its Physical Memory Attributes.

See the P550 Core Complex User Guide for a description of the System Port implementation in the P550 Core Complex.

Chapter 4

Physical Memory Attributes and Memory Map

This chapter describes the P550 Core Complex physical memory attributes and memory map.

4.1 Physical Memory Attributes Overview

The memory map is divided into different regions covering on-core-complex memory, system memory, peripherals, and empty holes. Physical memory attributes (PMAs) describe the properties of the accesses that can be made to each region in the memory map. These properties encompass the type of access that may be performed: execute, read, or write. As well as other optional attributes related to the access, such as supported access size, alignment, atomic operations, and cacheability.

RISC-V utilizes a simpler approach than other processor architectures in defining the attributes of memory accesses. Instead of defining access characteristics in page table descriptors or memory protection logic, the properties are fixed for memory regions or may only be modified in platform-specific control registers. As most systems don't require the ability to modify PMAs, SiFive cores only support fixed PMAs, which are set at design time. This results in a simpler design with lower gate count and power savings, and an easier programming interface.

External memory map regions are accessed through a specific port type and that port type is used to define the PMAs. The port types are Memory, Peripheral, and System. Memory map regions defined for internal memory and internal control regions also have a predefined PMA based on the underlying contents of the region.

The assigned PMA properties and attributes for P550 Core Complex memory regions are shown in Table 14 and Table 15 for external and internal regions, respectively.

The configured memory regions of the P550 Core Complex are listed with their attributes in Table 16.

Port Type	Access Properties	Attributes
Memory Port	Read, Write, Execute	Atomics+LR/SC, Data Cacheable, Instruction Cacheable, Instruction Speculation
Peripheral Port	Read, Write, Execute	Atomics, Instruction Cacheable
System Port	Read, Write, Execute	Instruction Cacheable

Table 14: Physical Memory Attributes for External Regions

Region	Access Properties	Attributes
Bus-Error Unit	Read, Write	Atomics
CLINT	Read, Write	Atomics
Debug	None	N/A
Error Device	Read, Write, Execute	Atomics
Hart 0 Private L2 Cache	Read, Write	Atomics
L3 Cache Controller	Read, Write	Atomics
L3 LIM	Read, Write, Execute	Atomics
L3 Zero Device	Read, Write, Execute	Atomics, Instruction Cacheable
PLIC	Read, Write	Atomics
Reserved	None	N/A
Trace Encoder	Read, Write	Atomics

Table 15: Physical Memory Attributes for Internal Regions

All memory map regions support word, half-word, and byte size data accesses.

Atomic access support enables the RISC-V standard Atomic (A) Extension for atomic instructions. These atomic instructions are further documented in Section 3.7 for the P550 core. The load-reserved (LR) and store-conditional (SC) instructions are only supported on the data cacheable region, marked in Table 14 with "Atomics+LR/SC".

No region supports unaligned accesses. An unaligned access will generate the appropriate trap: instruction address misaligned, load address misaligned, or store/AMO address misaligned.

The Physical Memory Protection unit is capable of controlling access properties based on address ranges, not ports. It has no control over the attributes of an address range, however.

Note

The Debug and Error Device regions have special behavior. The Debug region is reserved for use from a Debugger, and all accesses to it from the core in non-Debug mode will trap. The Error Device will also trap all accesses, as described in Chapter 10.

4.2 Memory Map

The memory map of the P550 Core Complex is shown in Table 16.

Base	Top	PMA	Description
0x00_0000_0000	0x00_0000_0FFF		Debug
0x00_0000_1000	0x00_0000_2FFF		Reserved
0x00_0000_3000	0x00_0000_3FFF	RWX A	Error Device
0x00_0000_4000	0x00_016F_FFFF		Reserved
0x00_0170_0000	0x00_0170_0FFF	RW A	Bus-Error Unit
0x00_0170_1000	0x00_01FF_FFFF		Reserved
0x00_0200_0000	0x00_0200_FFFF	RW A	CLINT
0x00_0201_0000	0x00_0201_3FFF	RW A	L3 Cache Controller
0x00_0201_4000	0x00_07FF_FFFF		Reserved
0x00_0800_0000	0x00_081F_FFFF	RWX A	L3 LIM
0x00_0820_0000	0x00_09FF_FFFF		Reserved
0x00_0A00_0000	0x00_0A1F_FFFF	RWXI A	L3 Zero Device
0x00_0A20_0000	0x00_0BFF_FFFF		Reserved
0x00_0C00_0000	0x00_0FFF_FFFF	RW A	PLIC
0x00_1000_0000	0x00_1000_0FFF	RW A	Trace Encoder
0x00_1000_1000	0x00_1010_3FFF		Reserved
0x00_1010_4000	0x00_1010_7FFF	RW A	Hart 0 Private L2 Cache
0x00_1010_8000	0x00_1FFF_FFFF		Reserved
0x00_2000_0000	0x00_3FFF_FFFF	RWXI A	Peripheral Port (512 MiB)
0x00_4000_0000	0x00_5FFF_FFFF	RWXI	System Port (512 MiB)
0x00_6000_0000	0x00_7FFF_FFFF		Reserved
0x00_8000_0000	0x10_7FFF_FFFF	RWXIDA	Memory Port (64 GiB)
0x10_8000_0000	0xFF_FFFF_FFFF		Reserved

Table 16: P550 Core Complex Memory Map. Physical Memory Attributes:
R–Read, **W**–Write, **X**–Execute, **I**–Instruction Cacheable, **D**–Data Cacheable,
A–Atomics

Chapter 5

Programmer's Model

The P550 Core Complex implements the 64-bit RISC-V architecture. The following chapter provides a reference for programmers and an explanation of the extensions supported by RV64GCB.

This chapter contains a high-level discussion of the RISC-V instruction set architecture and additional resources which will assist software developers working with RISC-V products. The P550 Core Complex is an implementation of the RISC-V RV64GCB architecture, and is guaranteed to be compatible with all applicable RISC-V standards. RV64GCB can emulate almost any other RISC-V ISA extension.

5.1 Base Instruction Formats

RISC-V base instructions are fixed to 32 bits in length and must be aligned on a four-byte boundary in memory. RISC-V ISA keeps the source (*rs1* and *rs2*) and destination (*rd*) registers at the same position in all formats to simplify decoding, with the exception of the 5-bit immediates used in CSR instructions.

The various formats are described in Table 17 below.

Format	Description
R	Format for register-register arithmetic/logical operations.
I	Format for register-immediate ALU operations and loads.
S	Format for stores.
B	Format for branches.
U	Format for 20-bit upper immediate instructions.
J	Format for jumps.

Table 17: Base Instruction Formats

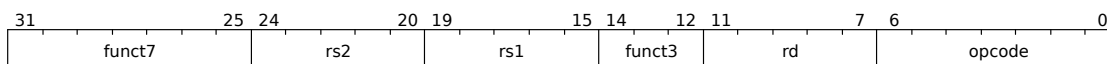


Figure 19: R-Type

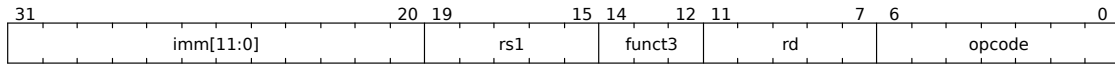


Figure 20: I-Type

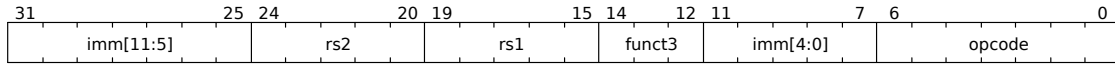


Figure 21: S-Type

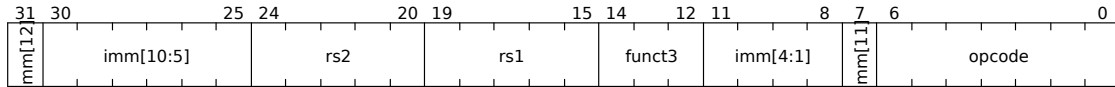


Figure 22: B-Type

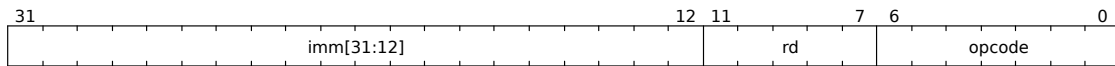


Figure 23: U-Type

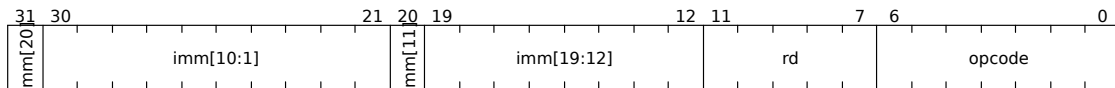


Figure 24: J-Type

The **opcode** field partially specifies an instruction, combined with **funct7** + **funct3** which describe what operation to perform. Each register field (**rs1**, **rs2**, **rd**) holds a 5-bit unsigned integer (0-31) corresponding to a register number (x0 - x31). Sign-extension is one of the most critical operations on immediates (particularly for XLEN>32), and in RISC-V the sign bit for all immediates is always held in bit 31 of the instruction to allow sign-extension to proceed in parallel with instruction decoding.

5.2 I Extension: Standard Integer Instructions

This section discusses the standard integer instructions supported by RISC-V. Integer computational instructions don't cause arithmetic exceptions.

5.2.1 R-Type (Register-Based) Integer Instructions

funct7			funct3		opcode	Instruction
00000000	rs2	rs1	000	rd	0110011	ADD
01000000	rs2	rs1	000	rd	0110011	SUB
00000000	rs2	rs1	001	rd	0110011	SLL
00000000	rs2	rs1	010	rd	0110011	SLT
00000000	rs2	rs1	011	rd	0110011	SLTU
00000000	rs2	rs1	100	rd	0110011	XOR
00000000	rs2	rs1	101	rd	0110011	SRL
01000000	rs2	rs1	101	rd	0110011	SRA
00000000	rs2	rs1	110	rd	0110011	OR
00000000	rs2	rs1	111	rd	0110011	AND

Table 18: R-Type Integer Instructions

Instruction	Description
ADD rd, rs1, rs2	Performs the addition of rs1 and rs2, result stored in rd.
SUB rd, rs1, rs2	Performs the subtraction of rs2 from rs1, result stored in rd.
SLL rd, rs1, rs2	Logical left shift (zeros are shifted into the lower bits) shift amount is encoded in the lower 5 bits of rs2.
SLT rd, x0, rs2	Signed and compare sets rd to 1 if rs2 is not equal to zero, otherwise sets rd to zero.
SLTU rd, x0, rs2	Unsigned compare sets rd to 1 if rs2 is not equal to zero, otherwise sets rd to zero.
SRL rd, rs1, rs2	Logical right shift (zeros are shifted into the lower bits) shift amount is encoded in the lower 5 bits of rs2.
SRA rd, rs1, rs2	Arithmetic right shift, shift amount is encoded in the lower 5 bits of rs2.
OR rd, rs1, rs2	Bitwise logical OR.
AND rd, rs1, rs2	Bitwise logical AND.
XOR rd, rs1, rs2	Bitwise logical XOR.

Table 19: R-Type Integer Instruction Description

Below is an example of an ADD instruction.

add x18, x19, x10

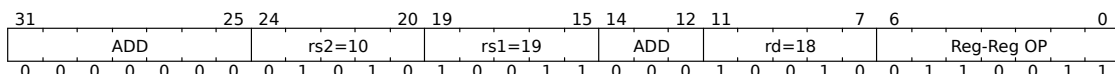


Figure 25: ADD Instruction Example

5.2.2 I-Type Integer Instructions

For I-Type integer instruction, one field is different from R-format. `rs2` and `func7` are replaced by the 12-bit signed immediate, `imm[11:0]`, which can hold values in range `[-2048, +2047]`. The immediate is always sign-extended to 32-bits before being used in an arithmetic operation. Bits `[31:12]` receive the same value as bit 11.

imm			func3		opcode	Instruction
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
00000000	shamnt	rs1	001	rd	0010011	SLLI
00000000	shamnt	rs1	101	rd	0010011	SRLI
01000000	shamnt	rs1	001	rd	0010011	SRAI

Table 20: I-Type Integer Instructions

One of the higher-order immediate bits is used to distinguish "shift right logical" (SRLI) from "shift right arithmetic" (SRAI).

Instruction	Description
ADDI	Adds the sign-extended 12-bit immediate to register <i>rs1</i> . Arithmetic overflow is ignored and the result is simply the low 64-bits of the result. <code>ADDI rd, rs1, 0</code> is used to implement the <code>MV rd, rs1</code> assembler pseudoinstruction.
SLTI	Set less than immediate. Places the value 1 in register <i>rd</i> if register <i>rs1</i> is less than the sign extended immediate when both are treated as signed numbers, else 0 is written to <i>rd</i> .
SLTIU	Compares the values as unsigned numbers (i.e., the immediate is first sign-extended to 64-bits then treated as an unsigned number). Note: <code>SLTIU rd, rs1, 1</code> sets <i>rd</i> to 1 if <i>rs1</i> equals zero, otherwise sets <i>rd</i> to 0 (assembler pseudo instruction <code>SEQZ rd, rs</code>).
XORI	Bitwise XOR on register <i>rs1</i> and the sign-extended 12-bit immediate and place the result in <i>rd</i> .
ORI	Bitwise OR on register <i>rs1</i> and the sign-extended 12-bit immediate and place the result in <i>rd</i> .
ANDI	Bitwise AND on register <i>rs1</i> and the sign-extended 12-bit immediate and place the result in <i>rd</i> .
SLLI	Shift Left Logical. The operand to be shifted is in <i>rs1</i> , and the shift amount is encoded in the lower 5 bits of the I-immediate field.
SRLI	Shift Right Logical. The operand to be shifted is in <i>rs1</i> , and the shift amount is encoded in the lower 5 bits of the I-immediate field.
SRAI	Shift Right Arithmetic. The operand to be shifted is in <i>rs1</i> , and the shift amount is encoded in the lower 5 bits of the I-immediate field (the original sign bit is copied into the vacated upper bits).

Table 21: I-Type Integer Instruction Description

Shift-by-immediate instructions only use lower 5 bits of the immediate value for shift amount (can only shift by 0-31 bit positions).

Below is an example of an ADDI instruction.

addi x15, x1, -50

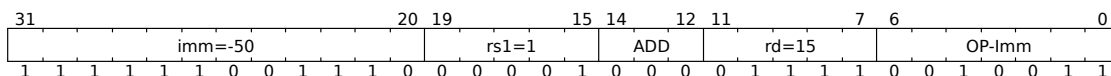


Figure 26: ADDI Instruction Example

5.2.3 I-Type Load Instructions

For I-Type load instructions, a 12-bit signed immediate is added to the base address in register *rs1* to form the memory address. In Table 22 below, **funct3** field encodes size and signedness of load data.

imm		func3		opcode	Instruction
imm[11:0]	rs1	000	rd	00000011	LB
imm[11:0]	rs1	001	rd	00000011	LH
imm[11:0]	rs1	010	rd	00000011	LW
imm[11:0]	rs1	100	rd	00000011	LBU
imm[11:0]	rs1	101	rd	00000011	LHU

Table 22: I-Type Load Instructions

Instruction	Description
LB rd, rs1, imm	Load Byte, loads 8 bits (1 byte) and sign-extends to fill destination 32-bit register.
LH rd, rs1, imm	Load Half-Word. Loads 16 bits (2 bytes) and sign-extends to fill destination 32-bit register.
LW rd, rs1, imm	Load Word, 32 bits.
LBU rd, rs1, imm	Load Unsigned Byte (8-bit).
LHU rd, rs1, imm	Load Unsigned Half-Word, which zero-extends 16 bits to fill destination 32-bit register.

Table 23: I-Type Load Instruction Description

Below is an example of a LW instruction.

lw x14, 8(x2)

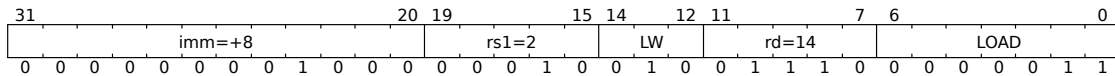


Figure 27: LW Instruction Example

5.2.4 S-Type Store Instructions

Store instructions need to read two registers: *rs1* for base memory address and *rs2* for data to be stored, as well as an immediate offset. The effective byte address is obtained by adding register *rs1* to the sign-extended 12-bit offset. Note that stores don't write a value to the register file, as there is no *rd* register used by the instruction. In RISC-V, the lower 5 bits of immediate are moved to where the *rd* field was in other instructions, and the *rs1/rs2* fields are kept in same place. The registers are kept always in the same place because a critical path for all operations includes fetching values from the registers. By always placing the read sources in the same place, the register file can read the registers without hesitation. If the data ends up being unnecessary (e.g., I-Type), it can be ignored.

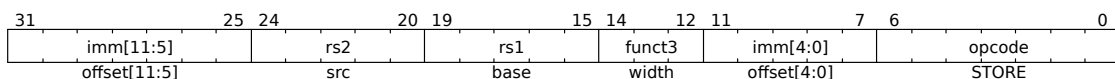


Figure 28: Store Instructions

imm			func3	imm	opcode	Instruction
imm[11:5]	rs2	rs1	000	imm[4:0]	01000011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	01000011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	01000011	SW

Table 24: S-Type Store Instructions

Instruction	Description
SB <i>rs2</i> , imm[11:0](<i>rs1</i>)	Store 8-bit value from the low bits of register <i>rs2</i> to memory.
SH <i>rs2</i> , imm[11:0](<i>rs1</i>)	Store 16-bit value from the low bits of register <i>rs2</i> to memory.
SW <i>rs2</i> , imm[11:0](<i>rs1</i>)	Store 32-bit value from the low bits of register <i>rs2</i> to memory.

Table 25: S-Type Store Instruction Description

Below is an example SW instruction.

SW x14, 8(x2)

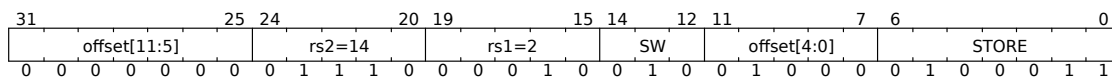


Figure 29: SW Instruction Example

5.2.5 Unconditional Jumps

The jump and link (JAL) instruction uses the J-type format, where the J-immediate encodes a signed offset in multiples of 2 bytes. The offset is sign-extended and added to the address of the jump instruction to form the jump target address. Jumps can therefore target a ± 1 MiB range. JAL stores the address of the instruction following the jump (pc+4) into register rd. The standard software calling convention uses x1 as the return address register and x5 as an alternate link register.

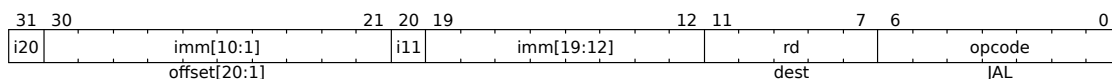


Figure 30: JAL Instruction

The indirect jump instruction JALR (jump and link register) uses the I-type encoding. The target address is obtained by adding the sign-extended 12-bit I-immediate to the register rs1, then setting the least-significant bit of the result to zero. The address of the instruction following the jump (pc+4) is written to register rd. Register x0 can be used as the destination if the result is not required.

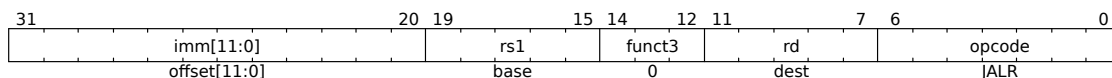


Figure 31: JALR Instruction

Both JAL and JALR instructions will generate an instruction-address-misaligned exception if the target address is not aligned to a four-byte boundary.

Instruction	Description
JAL rd, imm[20:1]	Jump and link
JALR rd, rs1, imm[11:0]	Jump and link register

Table 26: J-Type Instruction Description

5.2.6 Conditional Branches

All branch instructions use the B-Type instruction format. The 12-bit immediate represents values -4096 to +4094 in 2-byte increments. The offset is sign-extended and added to the address of the branch instruction to give the target address. The conditional branch range is ± 4 KiB.

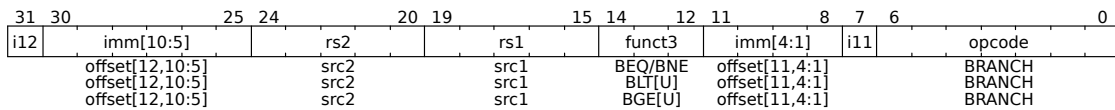


Figure 32: Branch Instructions

imm			func3	imm	opcode	Instruction
imm[12,10:5]	rs2	rs1	000	imm[4:1,11]	110011	BEQ
imm[12,10:5]	rs2	rs1	001	imm[4:1,11]	110011	BNE
imm[12,10:5]	rs2	rs1	100	imm[4:1,11]	110011	BLT
imm[12,10:5]	rs2	rs1	101	imm[4:1,11]	110011	BGE
imm[12,10:5]	rs2	rs1	110	imm[4:1,11]	110011	BLTU
imm[12,10:5]	rs2	rs1	111	imm[4:1,11]	110011	BGEU

Table 27: B-Type Instructions

Instruction	Description
BEQ rs1, rs2, imm[12:1]	Take the branch if registers rs1 and rs2 are equal.
BNE rs1, rs2, imm[12:1]	Take the branch if registers rs1 and rs2 are unequal.
BLT rs1, rs2, imm[12:1]	Take the branch if rs1 is less than rs2.
BGE rs1, rs2, imm[12:1]	Take the branch if rs1 is greater than or equal to rs2.
BLTU rs1, rs2, imm[12:1]	Take the branch if rs1 is less than rs2 (unsigned).
BGEU rs1, rs2, imm[12:1]	Take the branch if rs1 is greater than or equal to rs2 (unsigned).

Table 28: B-Type Instruction Description

ISA Base Instruction	Pseudoinstruction	Description
BEQ <i>rs</i> , <i>x0</i> , <i>offset</i>	BEQZ <i>rs</i> , <i>offset</i>	Take the branch if <i>rs</i> is equal to zero.

Table 29: RISC-V Base Instruction to Assembly Pseudoinstruction Example

Note

Software should be optimized such that the sequential code path is the most common path, with less-frequently taken code paths placed out of line. Software should also assume that backward branches will be predicted taken and forward branches as not taken, at least the first time they are encountered. Dynamic predictors should quickly learn any predictable branch behavior.

5.2.7 Upper-Immediate Instructions



Figure 33: Upper-Immediate Instructions

LUI (load upper immediate) is used to build 32-bit constants and uses the U-type format. LUI places the U-immediate value in the top 20 bits of the destination register *rd*, filling in the lowest 12 bits with zeros. Together with an ADDI to set low 12 bits, can create any 32-bit value in a register using two instructions (LUI/ADDI).

For example:

LUI x10, 0x87654 # x10 = 0x8765_4000

ADDI x10, x10, 0x321 # x10 = 0x8765_4321

AUIPC (add upper immediate to pc) is used to build pc-relative addresses and uses the U-type format. AUIPC forms a 32-bit offset from the 20-bit U-immediate, filling in the lowest 12 bits with zeros, and adds this offset to the address of the AUIPC instruction, then places the result in register *rd*.

5.2.8 Memory Ordering Operations

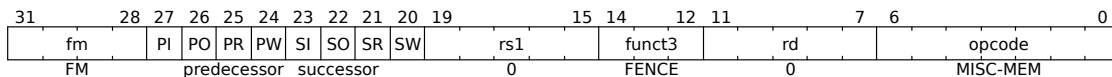


Figure 34: FENCE Instructions

The FENCE instruction is used to order device I/O and memory accesses as viewed by other RISC-V harts and external devices or coprocessors. Any combination of device input (I), device

output (O), memory reads (R), and memory writes (W) may be ordered with respect to any combination of the same. These operations are discussed further in Section 5.13.

5.2.9 Environment Call and Breakpoints

SYSTEM instructions are used to access system functionality that might require privileged access and are encoded using the I-type instruction format. These can be divided into two main classes: those that atomically read-modify-write control and status registers (CSRs), and all other potentially privileged instructions.

5.2.10 NOP Instruction

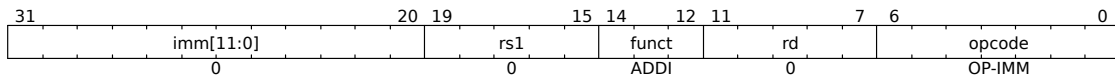


Figure 35: NOP Instructions

The NOP instruction does not change any architecturally visible state, except for advancing the pc and incrementing any applicable performance counters. NOP is encoded as **ADDI x0, x0, 0**.

5.3 M Extension: Multiplication Operations

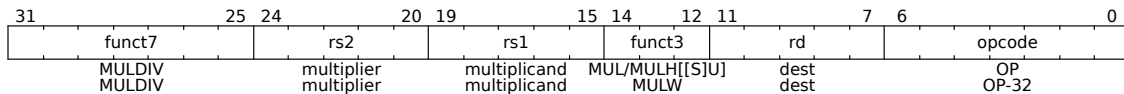


Figure 36: Multiplication Operations

Instruction	Description
MUL rd, rs1, rs2	Multiplication of rs1 by rs2 and places the lower 64-bits in the destination register.
MULH rd, rs1, rs2	Multiplication that return the upper 64-bits of the full 2×64-bit product.
MULHU rd, rs1, rs2	Unsigned multiplication that return the upper 64-bits of the full 2×64-bit product.
MULHSU rd, rs1, rs2	Signed rs1 multiple unsigned rs2 that return the upper 64-bits of the full 2×64-bit product.
MULW rd, rs1, rs2	RV64 instruction that multiplies the lower 32 bits of the source registers, placing the sign-extension of the lower 32 bits of the result into the destination register.

Table 30: Multiplication Operation Description

Combining MUL and MULH together creates one multiplication operation.

5.3.1 Division Operations

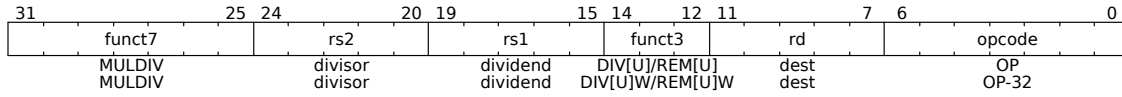


Figure 37: Division Operations

Instruction	Description
DIV rd, rs1, rs2	64-bits by 64-bits signed division of r1 by rs2 rounding towards zero.
DIVU rd, rs1, rs2	64-bits by 64-bits unsigned division of r1 by rs2 rounding towards zero.
REM rd, rs1, rs2	Remainder of the corresponding division.
REMU rd, rs1, rs2	Unsigned remainder of the corresponding division.
DIVW rd, rs1, rs2	RV64 instruction. Signed divide the lower 32 bits of rs1 by the lower 32 bits of rs2.
DIVUW rd, rs1, rs2	RV64 instruction. Unsigned divide the lower 32 bits of rs1 by the lower 32 bits of rs2.
REMW rd, rs1, rs2	Singed remainder.
REMUW rd, rs1, rs2	Unsigned remainder sign-extend the 32-bit result to 64 bits, including on a divide by zero.
MULDIV rd, rs1, rs	Multiply Divide.

Table 31: Division Operation Description

Combining DIV and REM together creates one division operation.

5.4 A Extension: Atomic Operations

Atomic operations are defined as operations that automatically read-modify-write memory to support synchronization between multiple RISC-V harts running in the same memory space.

5.4.1 Atomic Load-Reserve and Store-Conditional Instructions

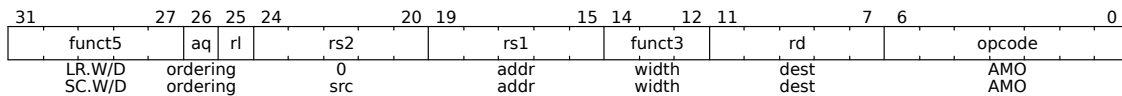


Figure 38: Atomic Operations

Instruction	Description
LR.W	Load Reserve. Loads a word from the address in rs1, places the sign-extended value in rd, and registers a reservation set—a set of bytes that subsumes the bytes in the addressed word.
SC.W	Store Conditional Conditionally writes a word in rs2 to the address in rs1: the SC.W succeeds only if the reservation is still valid and the reservation set contains the bytes being written. If the SC.W succeeds, the instruction writes the word in rs2 to memory, and it writes zero to rd. If the SC.W fails, the instruction does not write to memory, and it writes a nonzero value to rd. Executing an SC.W instruction invalidates any reservation held by this hart.
LR.D	RV64 - Loads doubleword.
SC.D	RV64 - Stores doubleword.

Table 32: Atomic Load-Reserve and Store-Conditional Instruction Description

For RV64, the sign-extended value of LR.W and SC.W is placed in rd.

Note

Only cores with data caches support the LR/SC instructions used by the A-Extension. Cores with DTIMs will *NOT*.

5.4.2 Atomic Memory Operations (AMOs)

The atomic memory operation (AMO) instructions perform read-modify-write operations for multiprocessor synchronization. These AMO instructions atomically load a data value from the address in rs1, place the value into register rd, apply a binary operator to the loaded value and the original value in rs2, then store the result back to the address in rs1.

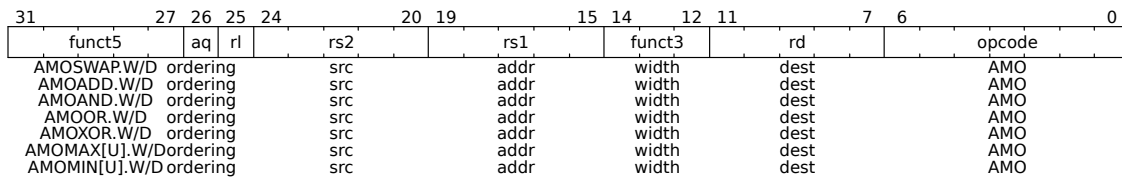


Figure 39: Atomic Memory Operations

Instruction	Description
AMOSWAP.W/D	Word / doubleword swap.
AMOADD.W/D	Word / doubleword add.
AMOAND.W/D	Word / doubleword and.
AMOOR.W/D	Word / doubleword or.
AMOXOR.W/D	Word / doubleword xor.
AMOMIN.W/D	Word / doubleword minimum.
AMOMINU.W/D	Unsigned word / doubleword minimum.
AMOMAX.W/D	Word / doubleword maximum.
AMOMAXU.W/D	Unsigned word / doubleword maximum.

Table 33: Atomic Memory Operation Description

For RV64, 32-bit AMOs always sign-extend the value placed in `rd`.

5.5 F Extension: Single-Precision Floating-Point Instructions

The F Extension implements single-precision floating-point computational instructions compliant with the IEEE 754-2008 arithmetic standard. The F Extension adds 32 floating-point registers, `f0–f31`, each 32 bits wide, and a floating-point control and status register `fcsr`. Floating-point load and store instructions transfer floating-point values between registers and memory, and instructions to transfer values to and from the integer register file are also provided.

5.5.1 Floating-Point Control and Status Registers

Floating-Point Control and Status Register, `fcsr`, is a RISC-V control and status register (CSR). The register selects the dynamic rounding mode for floating-point arithmetic operations and holds the accrued exception flags.



Figure 40: Floating-Point Control and Status Register

Flag Mnemonic	Flag Meaning
NV	Invalid Operation
DZ	Divide by Zero
OF	Overflow
UF	Underflow
NX	Inexact

Table 34: Accrued Exception Flags

The `fcsr` register can be read and written with the `FRCSR` and `FSCSR` instructions. The `FRRM` instruction reads the Rounding Mode field `frm`. `FSRM` swaps the value in `frm` with an integer register. `FRFLAGS` and `FSFLAGS` are defined analogously for the Accrued Exception Flags field `fflags`.

5.5.2 Rounding Modes

Floating-point operations use either a static rounding mode encoded in the instruction, or a dynamic rounding mode held in `frm`. A value of 111 in the instruction's `rm` field selects the dynamic rounding mode held in `frm`. If `frm` is set to an invalid value (101–111), any subsequent attempt to execute a floating-point operation with a dynamic rounding mode will raise an illegal instruction exception. Some instructions, including widening conversions, have the `rm` field, but are nevertheless unaffected by the rounding mode. Software should set their `rm` field to `RNE` (000).

Mode	Mnemonic	Meaning
000	RNE	Round to Nearest, ties to Even.
001	RTZ	Round towards Zero.
010	RDN	Round Down (towards $-\infty$).
011	RUP	Round Up (towards $+\infty$).
100	RMM	Round to Nearest, ties to Max Magnitude.
101		<i>Invalid. Reserved for future use.</i>
110		<i>Invalid. Reserved for future use.</i>
111	DYN	In instruction's <code>rm</code> field, selects dynamic rounding mode; In Rounding Mode register, <i>Invalid</i> .

Table 35: Floating-Point Rounding Modes

5.5.3 Single-Precision Floating-Point Load and Store Instructions

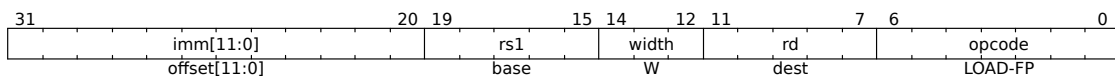


Figure 41: Single-Precision FP Load Instruction

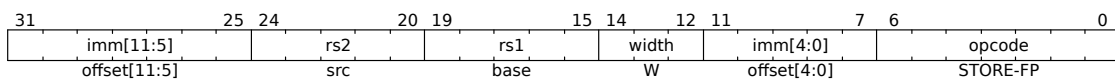


Figure 42: Single-Precision FP Store Instruction

Instruction	Operation	Description
FLW <code>rd, rs1, imm</code>	$f[rd] = M[x[rs1] + sext(offset)][31:0]$	Loads a single-precision floating-point value from memory into floating-point register <code>rd</code> .
FSW <code>imm, rs1, rs2</code>	$M[x[rs1] + sext(offset)] = f[rs2][31:0]$	Stores a single-precision value from floating-point register <code>rs2</code> to memory.

Table 36: Single-Precision FP Load and Store Instructions Description

5.5.4 Single-Precision Floating-Point Computational Instructions

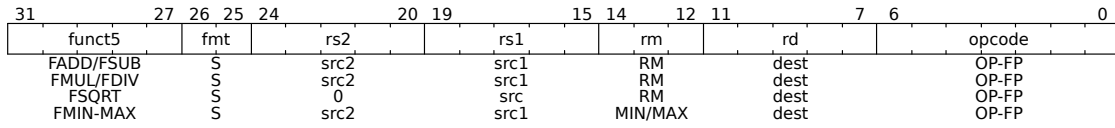


Figure 43: Single-Precision FP Computational Instructions

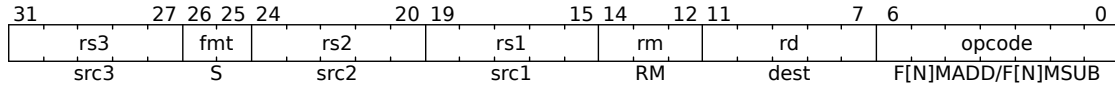


Figure 44: Single-Precision FP Fused Computational Instructions

Instruction	Operation	Description
FADD.S rd,rs1,rs2	$f[rd] = f[rs1] + f[rs2]$	Single-precision floating-point addition.
FSUB.S rd,rs1,rs2	$f[rd] = f[rs1] - f[rs2]$	Single-precision floating-point subtraction.
FMUL.S rd,rs1,rs2	$f[rd] = f[rs1] \times f[rs2]$	Single-precision floating-point multiplication.
FDIV.S rd,rs1,rs2	$f[rd] = f[rs1] \div f[rs2]$	Single-precision floating-point division.
FSQRT.S rd,rs1	$f[rd] = \sqrt{f[rs1]}$	Single-precision floating-point square root.
FMIN.S rd,rs1,rs2	$f[rd] = \min(f[rs1], f[rs2])$	Single-precision floating-point minimum-number.
FMAX.S rd,rs1,rs2	$f[rd] = \max(f[rs1], f[rs2])$	Single-precision floating-point maximum-number.
FMADD.S rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	Single-precision floating-point multiply and add.
FMSUB.S rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	Single-precision floating-point multiply and subtract.
FNMADD.S rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	Single-precision floating-point multiply, negate, and add.
FNMSUB.S rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	Single-precision floating-point multiply, negate, and subtract.

Table 37: Single-Precision FP Computational Instructions Description

5.5.5 Single-Precision Floating-Point Conversion and Move Instructions

Single-Precision Floating-Point Conversion Instructions

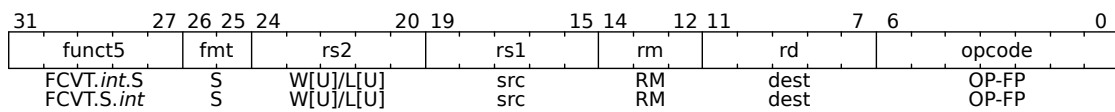


Figure 45: Single-Precision FP to Integer and Integer to FP Conversion Instructions

Table 38: Single-Precision FP Conversion Instructions Description

Single-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions

31	27	26	25	24	20	19	15	14	12	11	7	6	0
funct5				fmt	rs2		rs1		rm	rd		opcode	
FSGNI				S	src2		src1		I/N/I/X	dest		OP-FP	

Figure 46: Single-Precision FP to FP Sign-Injection Instructions

Instruction	Operation	Description
FSGNJ.S rd,rs1,rs2	$f[rd] = \{f[rs2][31], f[rs1][30:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is rs2's sign bit.
FSGNJS rd,rs1,rs2	$f[rd] = \{-f[rs2][31], f[rs1][30:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is the opposite of rs2's sign bit.
FSGNJX.S rd,rs1,rs2	$f[rd] = \{f[rs1][31] \wedge f[rs2][31], f[rs1][30:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The sign bit is the XOR of the sign bits of rs1 and rs2.

Table 39: Single-Precision FP to FP Sign-Injection Instructions Description

ISA Base Instruction	Pseudoinstruction	Description
FSGNJ.S rx,ry,ry	FMV.S rx,ry	Moves ry to rx.
FSGNJS rx,ry,ry	FNEG.S rx,ry	Moves the negation of ry to rx.
FSGNJX.S rx,ry,ry	FABS.S rx,ry	Moves the absolute value of ry to rx.

Table 40: RISC-V Base Instruction to Assembly Pseudoinstruction Example

Single-Precision Floating-Point Move Instructions

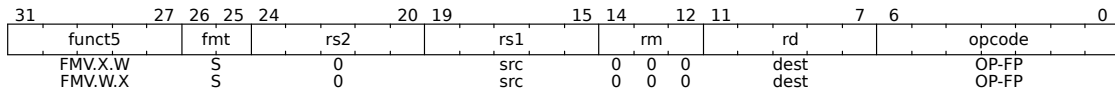


Figure 47: Single-Precision FP Move Instructions

Instruction	Operation	Description
FMV.X.W rd,rs1	$x[rd] = \text{sext}(f[rs1][31:0])$	Moves the single-precision value in floating-point register rs1 represented in IEEE 754-2008 encoding to the lower 32 bits of integer register rd. The higher 32 bits of the destination register are filled with copies of the floating-point number's sign bit.
FMV.W.X rd,rs1	$f[rd] = x[rs1][31:0]$	Moves the single-precision value encoded in IEEE 754-2008 standard encoding from the lower 32 bits of integer register rs1 to the floating-point register rd.

Table 41: Single-Precision FP Move Instructions Description

5.5.6 Single-Precision Floating-Point Compare Instructions

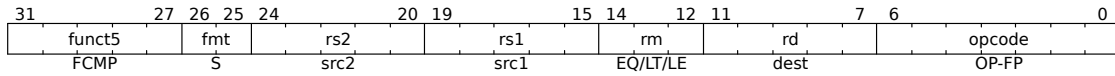


Figure 48: Single-Precision FP Compare Instructions

Instruction	Operation	Description
FEQ.S rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	Writes 1 to the integer register rd if rs1 is equal to rs2, 0 otherwise. Performs a quiet comparison; only sets the invalid operation exception flag if either input is a signaling NaN.
FLT.S rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	Writes 1 to the integer register rd if rs1 less than rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.
FLE.S rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	Writes 1 to the integer register rd if rs1 less than or equal to rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.

Table 42: Single-Precision FP Compare Instructions Description

Single-Precision Floating-Point Classify Instruction

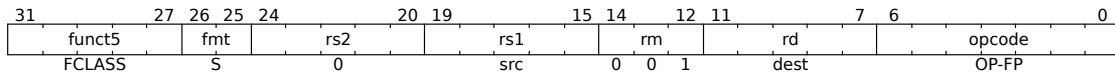


Figure 49: Single-Precision FP Classify Instruction

Instruction	Operation	Description
FCLASS.S rd, rs1	$x[rd] = \text{classify}_s(f[rs1])$	Examines the value in floating-point register rs1 and writes to integer register rd a 10-bit mask that indicates the class of the floating-point number.

Table 43: Single-Precision FP Classify Instruction Description

rd bit	Meaning
0	rs1 is $-\infty$
1	rs1 is negative normal number
2	rs1 is a negative subnormal number
3	rs1 is -0
4	rs1 is +0
5	rs1 is a positive subnormal number
6	rs1 is a positive normal number
7	rs1 is $+\infty$
8	rs1 is a signaling NaN
9	rs1 is a quiet NaN

Table 44: Floating-Point Number Classes

5.6 D Extension: Double-Precision Floating-Point Instructions

The D extension widens the 32 floating-point registers, f_0 – f_{31} , to 64 bits. The f registers can now hold either 32-bit or 64-bit floating-point values. When multiple floating-point precisions are supported, then valid values of narrower n-bit types, $n < \text{FLEN}$, are represented in the lower n bits of an FLEN-bit. Any operation that writes a narrower result to an f register must write all 1s to the uppermost FLEN-n bits to yield a legal NaN-boxed value. Floating-point n-bit transfer operations move external values held in IEEE standard formats into and out of the f registers, and comprise floating-point loads and stores and floating-point move instructions.

5.6.1 Double-Precision Floating-Point Load and Store Instructions

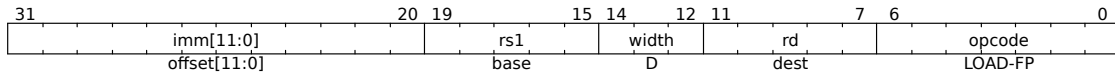


Figure 50: Double-Precision FP Load Instruction

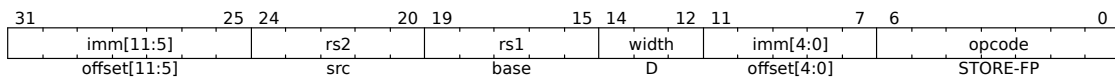


Figure 51: Double-Precision FP Store Instruction

Instruction	Operation	Description
FLD rd, rs1, imm	$f[\text{rd}] = M[x[\text{rs1}] + \text{sext}(\text{offset})][63:0]$	Loads a double-precision floating-point value from memory into floating-point register rd.
FSD imm, rs1, rs2	$M[x[\text{rs1}] + \text{sext}(\text{offset})] = f[\text{rs2}][63:0]$	Stores a double-precision value from the floating-point register rs2 to memory.

Table 45: Double-Precision FP Load and Store Instructions Description

FLD and FSD are only guaranteed to execute atomically if the effective address is naturally aligned and $XLEN \geq 64$. These instructions do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

5.6.2 Double-Precision Floating-Point Computational Instructions

The double-precision floating-point computational instructions are defined analogously to their single-precision counterparts, but operate on double-precision operands and produce double-precision results.

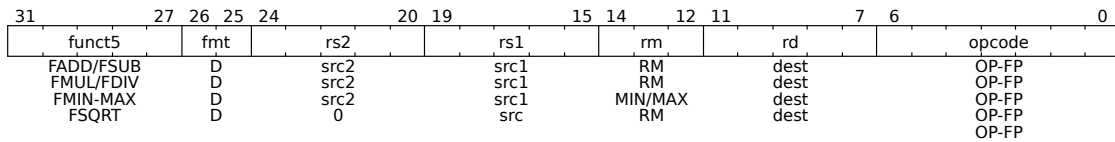


Figure 52: Double-Precision FP Computational Instructions

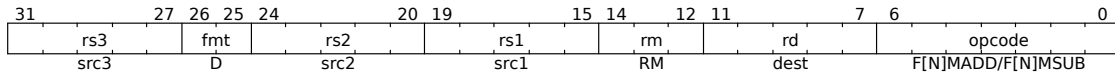


Figure 53: Double-Precision FP Fused Computational Instructions

Instruction	Operation	Description
FADD.D rd,rs1,rs2	$f[rd] = f[rs1] + f[rs2]$	Double-precision floating-point addition.
FSUB.D rd,rs1,rs2	$f[rd] = f[rs1] - f[rs2]$	Double-precision floating-point subtraction.
FMUL.D rd,rs1,rs2	$f[rd] = f[rs1] \times f[rs2]$	Double-precision floating-point multiplication.
FDIV.D rd,rs1,rs2	$f[rd] = f[rs1] \div f[rs2]$	Double-precision floating-point division.
FSQRT.D rd,rs1	$f[rd] = \sqrt{f[rs1]}$	Double-precision floating-point square root.
FMIN.D rd,rs1,rs2	$f[rd] = \min(f[rs1], f[rs2])$	Double-precision floating-point minimum-number.
FMAX.D rd,rs1,rs2	$f[rd] = \max(f[rs1], f[rs2])$	Double-precision floating-point maximum-number.
FMADD.D rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	Double-precision floating-point multiply and add.
FMSUB.D rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	Double-precision floating-point multiply and subtract.
FNMADD.D rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	Double-precision floating-point multiply, negate, and add.
FNMSUB.D rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	Double-precision floating-point multiply, negate, and subtract.

Table 46: Double-Precision FP Computational Instructions Description

5.6.3 Double-Precision Floating-Point Conversion and Move Instructions

Double-Precision Floating-Point Conversion Instructions

All floating-point to integer and integer to floating-point conversion instructions round according to the *rm* field.

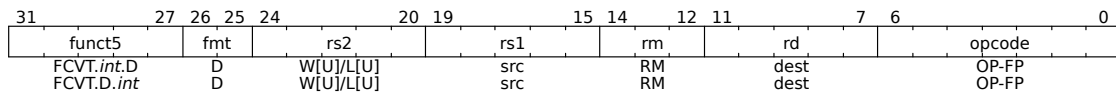


Figure 54: Double-Precision FP to Integer and Integer to FP Conversion Instructions

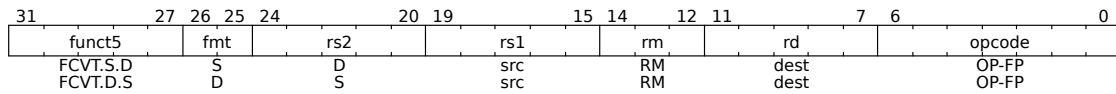


Figure 55: Double-Precision to Single-Precision and Single-Precision to Double-Precision FP Conversion Instructions

Instruction	Operation	Description
FCVT.W.D rd, rs1	$x[rd] = \text{sext}(s32_{f64}(f[rs1]))$	Converts a double-precision floating-point number to a signed 32-bit integer. Sign-extends the 32-bit result to the destination register width.
FCVT.D.W rd, rs1	$f[rd] = f64_{s32}(x[rs1])$	Converts a signed 32-bit integer to a double-precision floating-point number. Always produces an exact result and is unaffected by rounding mode.
FCVT.WU.D rd, rs1	$x[rd] = \text{sext}(u32_{f64}(f[rs1]))$	Converts a double precision floating-point number to an unsigned 32-bit integer. Sign-extends the 32-bit result to the destination register width.
FCVT.D.WU rd, rs1	$f[rd] = f64_{u32}(x[rs1])$	Converts an unsigned 32-bit integer to a double-precision floating-point number. Always produces an exact result and is unaffected by rounding mode.
FCVT.L.D rd, rs1	$x[rd] = s64_{f64}(f[rs1])$	Converts a double-precision floating-point number to a signed 64-bit integer.
FCVT.D.L rd, rs1	$f[rd] = f64_{s64}(x[rs1])$	Converts a signed 64-bit integer to a double-precision floating-point number.
FCVT.LU.D rd, rs1	$x[rd] = u64_{f64}(f[rs1])$	Converts a double-precision floating-point number to an unsigned 64-bit integer.
FCVT.D.LU rd, rs1	$f[rd] = f64_{u64}(x[rs1])$	Converts an unsigned 64-bit integer to a double-precision floating-point number.
FCVT.S.D rd, rs1	$f[rd] = f32_{f64}(f[rs1])$	Converts a double-precision floating-point number to a single-precision floating-point number.
FCVT.D.S rd, rs1	$f[rd] = f64_{f32}(f[rs1])$	Converts a single-precision floating-point number to a double-precision floating-point number.

Table 47: Double-Precision FP Conversion Instructions Description

Double-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions

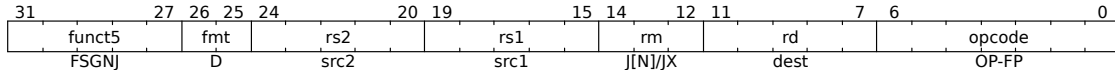


Figure 56: Double-Precision FP to FP Sign-Injection Instructions

Instruction	Operation	Description
FSGNJD rd,rs1,rs2	$f[rd] = \{f[rs2][63], f[rs1][62:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is rs2's sign bit.
FSGNJND rd,rs1,rs2	$f[rd] = \{\sim f[rs2][63], f[rs1][62:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is the opposite of rs2's sign bit.
FSGNJXD rd,rs1,rs2	$f[rd] = \{f[rs1][63] \wedge f[rs2][63], f[rs1][62:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The sign bit is the XOR of the sign bits of rs1 and rs2.

Table 48: Double-Precision FP to FP Sign-Injection Instructions Description

ISA Base Instruction	Pseudoinstruction	Description
FSGNJD rx,ry,ry	FMV.D rx,ry	Moves ry to rx.
FSGNJND rx,ry,ry	FNEG.D rx,ry	Moves the negation of ry to rx.
FSGNJXD rx,ry,ry	FABS.D rx,ry	Moves the absolute value of ry to rx.

Table 49: RISC-V Base Instruction to Assembly Pseudoinstruction Example

Double-Precision Floating-Point Move Instructions

The RV64 architecture provides instructions to move bit patterns between the floating-point and integer registers.

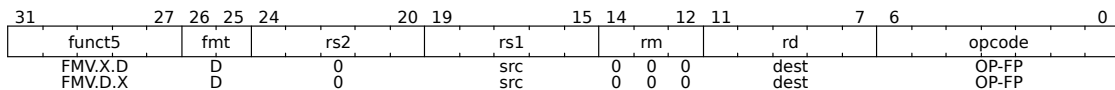


Figure 57: Double-Precision FP Move Instructions

Instruction	Operation	Description
FMV.X.D rd, rs1	$x[rd] = f[rs1][63:0]$	Moves the double-precision value in floating-point register rs1 to a representation in IEEE 754-2008 standard encoding in integer register rd.
FMV.D.X rd, rs1	$f[rd] = x[rs1][63:0]$	Moves the double-precision value encoded in IEEE 754-2008 standard encoding from the integer register rs1 to the floating-point register rd.

Table 50: Double-Precision FP Move Instructions Description

FMV.X.D and FMV.D.X do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

5.6.4 Double-Precision Floating-Point Compare Instructions

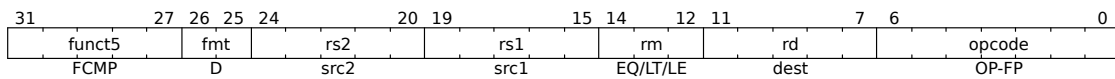


Figure 58: Double-Precision FP Compare Instructions

Instruction	Operation	Description
FEQ.D rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	Writes 1 to the integer register rd if rs1 is equal to rs2, 0 otherwise. Performs a quiet comparison; only sets the invalid operation exception flag if either input is a signaling NaN.
FLT.D rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	Writes 1 to the integer register rd if rs1 less than rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.
FLE.D rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	Writes 1 to the integer register rd if rs1 less than or equal to rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.

Table 51: Double-Precision FP Compare Instructions Description

5.6.5 Double-Precision Floating-Point Classify Instruction

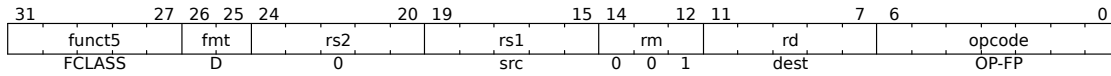


Figure 59: Double-Precision FP Classify Instruction

Instruction	Operation	Description
FCLASS.D rd, rs1	$x[rd] = \text{classify}_d(f[rs1])$	Examines the value in floating-point register rs1 and writes to integer register rd a 10-bit mask that indicates the class of the floating-point number.

Table 52: Double-Precision FP Classify Instruction Description

5.7 C Extension: Compressed Instructions

The C Extension reduces static and dynamic code size by adding short 16-bit instruction encodings for common operations. The C extension can be added to any of the base ISAs (RV32, RV64, RV128), and we use the generic term "RVC" to cover any of these. Typically, 50%–60% of the RISC-V instructions in a program can be replaced with RVC instructions, resulting in a 25%–30% code-size reduction. The C extension is compatible with all other standard instruction extensions. The C extension allows 16-bit instructions to be freely intermixed with 32-bit instructions, with the latter now able to start on any 16-bit boundary, i.e., IALIGN=16. With the addition of the C extension, no instructions can raise instruction-address-misaligned exceptions. It is important to note that the C extension is not designed to be a stand-alone ISA, and is meant to be used alongside a base ISA. The compressed 16-bit instruction format is designed around the assumption that x1 is the return address register and x2 is the stack pointer.

5.7.1 Compressed 16-bit Instruction Formats

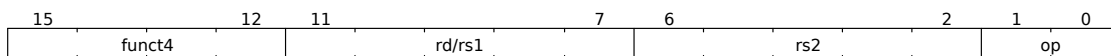


Figure 60: CR Format - Register

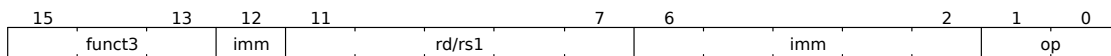


Figure 61: CI Format - Immediate

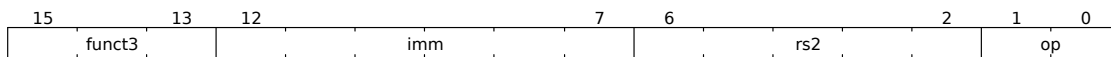


Figure 62: CSS Format - Stack-relative Store

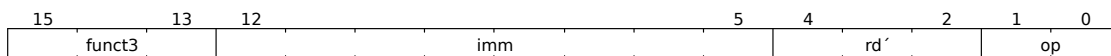


Figure 63: CIW Format - Wide Immediate

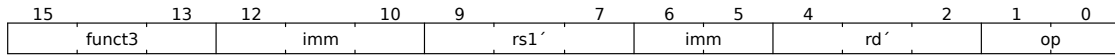


Figure 64: CL Format - Load

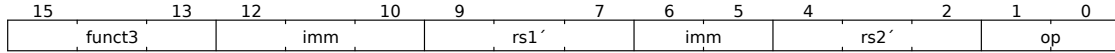


Figure 65: CS Format - Store

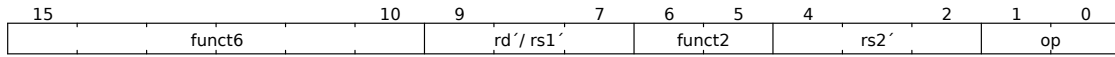


Figure 66: CA Format - Arithmetic

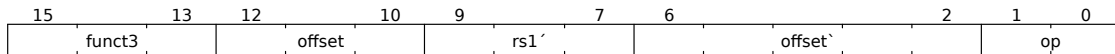


Figure 67: CJ Format - Jump

5.7.2 Stack-Pointed-Based Loads and Stores

The compressed load instructions are expressed in CI format.

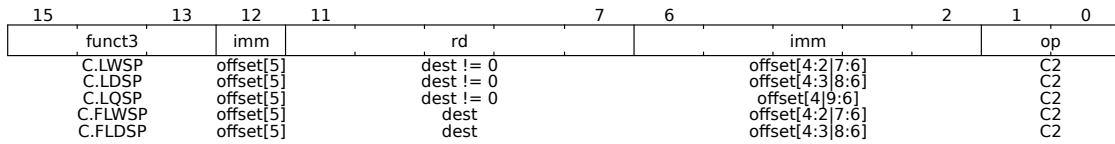


Figure 68: Stack-Pointed-Based Loads

Instruction	Description
C.LWSP	Loads a 32-bit value from memory into register rd.
C.LDSP	RV64C Instruction which loads a 64-bit value from memory into register rd.
C.LQSP	RV128C loads a 128-bit value from memory into register rd.
C.FLWSP	RV32FC Instruction that loads a single-precision floating-point value from memory into floating-point register rd.
C.FLDSP	RV32DC/RV64DC Instruction that loads a double-precision floating-point value from memory into floating-point register rd.

Table 53: Stack-Pointed-Based Load Instruction Description

The compressed store instructions are expressed in CSS format.

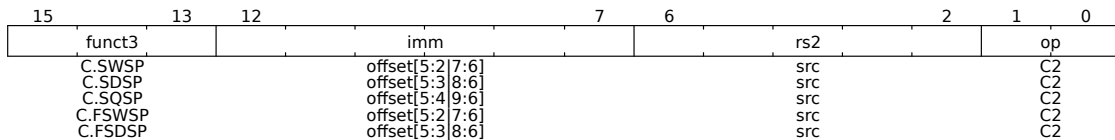


Figure 69: Stack-Pointed-Based Stores

Instruction	Description
C.LWSP	Loads a 32-bit value from memory into register rd.
C.SWSP	Stores a 32-bit value in register rs2 to memory.
C.SDSP	RV64C/RV128C instruction that stores a 64-bit value in register rs2 to memory.
C.SQSP	RV128C instruction that stores a 128-bit value in register rs2 to memory.
C.FSWSP	RV32FC instruction that stores a single-precision floating-point value in floating-point register rs2 to memory.
C.FSDSP	RV32DC/RV64DC instruction that stores a double-precision floating-point value in floating-point register rs2 to memory.

Table 54: Stack-Pointed-Based Store Instruction Description

5.7.3 Register-Based Loads and Stores

The compressed register-based load instructions are expressed in CL format.

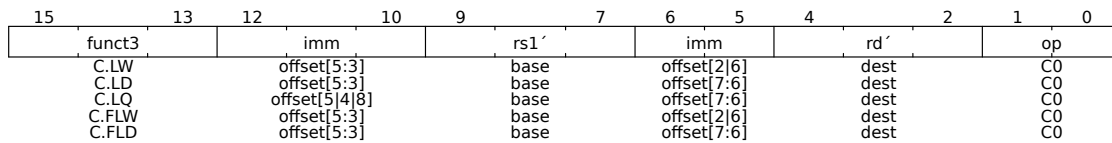


Figure 70: Register-Based Loads

Instruction	Description
C.LW	Loads a 32-bit value from memory into register rd.
C.LD	RV64C/RV128C-only instruction that loads a 64-bit value from memory into register rd.
C.LQ	RV128C-only instruction that loads a 128-bit value from memory into register rd.
C.FLW	RV32FC-only instruction that loads a single-precision floating-point value from memory into floating-point register rd.
C.FLD	RV32DC/RV64DC-only instruction that loads a double-precision floating-point value from memory into floating-point register rd.

Table 55: Register-Based Load Instruction Description

The compressed register-based store instructions are expressed in CS format.

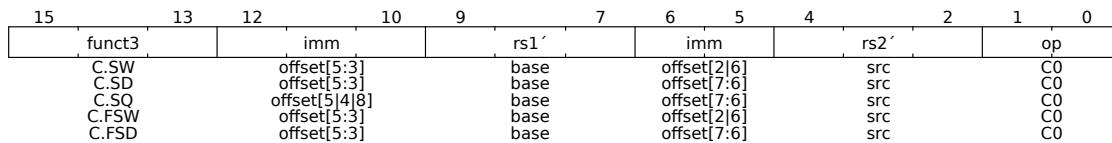


Figure 71: Register-Based Stores

Instruction	Description
C.SW	Stores a 32-bit value in register rs2 to memory.
C.SD	RV64C/RV128C instruction that stores a 64-bit value in register rs2 to memory.
C.SQ	RV128C instruction that stores a 128-bit value in register rs2 to memory.
C.FSW	RV32FC instruction that stores a single-precision floating-point value in floating-point register rs2 to memory.
C.FSD	RV32DC/RV64DC instruction that stores a double-precision floating-point value in floating-point register rs2 to memory.

Table 56: Register-Based Store Instruction Description

5.7.4 Control Transfer Instructions

RVC provides unconditional jump instructions and conditional branch instructions.

The unconditional jump instructions are expressed in CJ format.

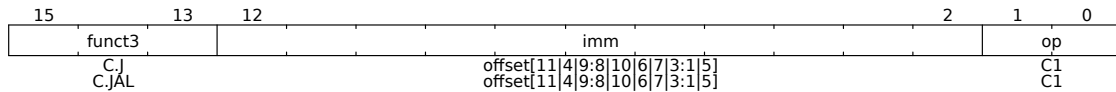


Figure 72: Unconditional Jump Instructions

Instruction	Description
C.J	Unconditional control transfer.
C.JAL	RV32C instruction that performs the same operation as C.J, but additionally writes the address of the instruction following the jump (pc+2) to the link register, x1.

Table 57: Unconditional Jump Instruction Description

The unconditional control transfer instructions are expressed in CR format.

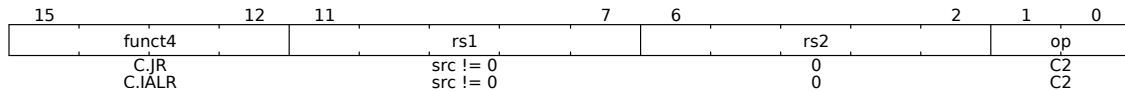


Figure 73: Unconditional Control Transfer Instructions

Instruction	Description
C.JR	Performs an unconditional control transfer to the address in register rs1.
C.JALR	Performs the same operation as C.JR, but additionally writes the address of the instruction following the jump (pc+2) to the link register, x1.

Table 58: Unconditional Control Transfer Instruction Description

The conditional control transfer instructions are expressed in CB format.

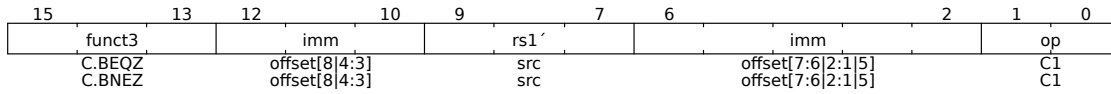


Figure 74: Conditional Control Transfer Instructions

Instruction	Description
C.BEQZ	Conditional control transfers. Takes the branch if the value in register <code>rs1'</code> is zero.
C.BNEZ	Conditional control transfers. Takes the branch if <code>rs1'</code> contains a nonzero value.

Table 59: Conditional Control Transfer Instruction Description

5.7.5 Integer Computational Instructions

Integer Constant-Generation Instructions

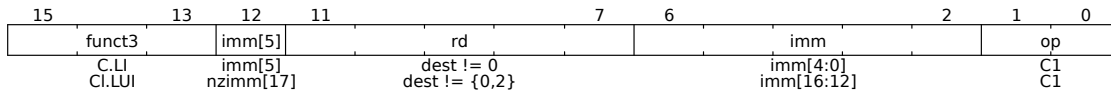


Figure 75: Integer Constant-Generation Instructions

Instruction	Description
C.LI	Loads the sign-extended 6-bit immediate, <code>imm</code> , into register <code>rd</code> .
C.LUI	Loads the non-zero 6-bit immediate field into bits 17–12 of the destination register, clears the bottom 12 bits, and sign-extends bit 17 into all higher bits of the destination

Table 60: Integer Constant-Generation Instruction Description

Integer Register-Immediate Operations

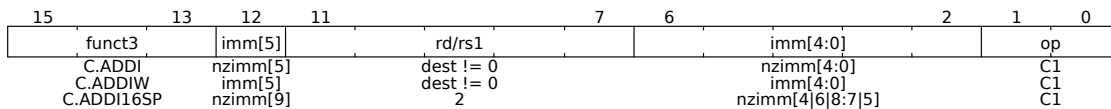


Figure 76: Integer Register-Immediate Operations

Instruction	Description
C.ADDI	Adds the non-zero sign-extended 6-bit immediate to the value in register <i>rd</i> then writes the result to <i>rd</i> .
C.ADDIW	RV64C/RV128C instruction that performs the same computation but produces a 32-bit result, then sign-extends result to 64 bits.
C.ADDI16SP	Adds the non-zero sign-extended 6-bit immediate to the value in the stack pointer (<i>sp</i> = <i>x2</i>), where the immediate is scaled to represent multiples of 16 in the range (-512,496). C.ADDI16SP is used to adjust the stack pointer in procedure prologues and epilogues.

Table 61: Integer Register-Immediate Operation Description

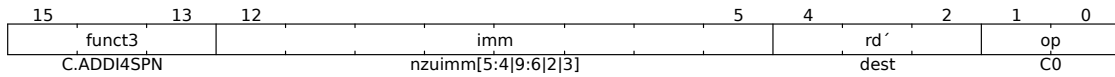


Figure 77: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.ADDI4SPN	Adds a zero-extended non-zero immediate, scaled by 4, to the stack pointer, <i>x2</i> , and writes the result to <i>rd\'</i> .

Table 62: Integer Register-Immediate Operation Description (cont.)



Figure 78: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.SLLI	Performs a logical left shift of the value in register <i>rd</i> then writes the result to <i>rd</i> . The shift amount is encoded in the <i>shamt</i> field.

Table 63: Integer Register-Immediate Operation Description (cont.)

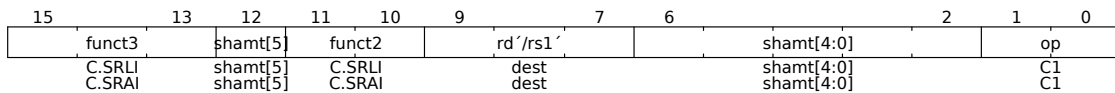


Figure 79: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.SRLI	Logical right shift of the value in register <i>rd\'</i> then writes the result to <i>rd\'</i> . The shift amount is encoded in the <i>shamt</i> field.
C.SRAI	Arithmetic right shift of the value in register <i>rd\'</i> then writes the result to <i>rd\'</i> . The shift amount is encoded in the <i>shamt</i> field.

Table 64: Integer Register-Immediate Operation Description (cont.)

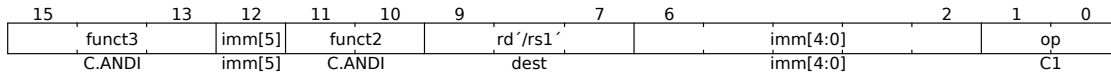


Figure 80: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.ANDI	Computes the bitwise AND of the value in register <i>rd'</i> and the sign-extended 6-bit immediate, then writes the result to <i>rd'</i> .

Table 65: Integer Register-Immediate Operation Description (cont.)

Integer Register-Register Operations

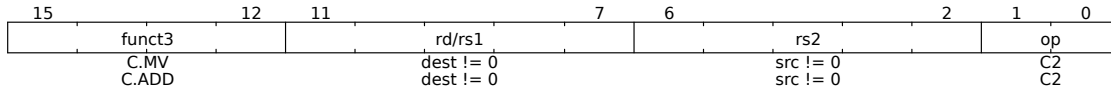


Figure 81: Integer Register-Register Operations

Instruction	Description
C.MV	Copies the value in register <i>rs2</i> into register <i>rd</i> .
C.ADD	Adds the values in registers <i>rd</i> and <i>rs2</i> and writes the result to register <i>rd</i> .

Table 66: Integer Register-Register Operation Description

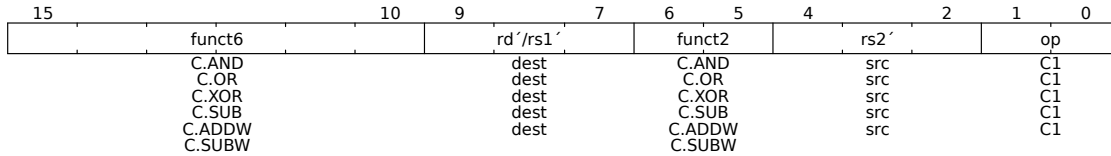


Figure 82: Integer Register-Register Operations (cont.)

Instruction	Description
C.AND	Computes the bitwise AND of the values in registers <i>rd'</i> and <i>rs2'</i> .
C.OR	Computes the bitwise OR of the values in registers <i>rd'</i> and <i>rs2'</i> .
C.XOR	Computes the bitwise XOR of the values in registers <i>rd'</i> and <i>rs2'</i> .
C.SUB	Subtracts the value in register <i>rs2'</i> from the value in register <i>rd'</i> .
C.ADDW	RV64C/RV128C-only instruction that adds the values in registers <i>rd'</i> and <i>rs2'</i> , then sign-extends the lower 32 bits of the sum before writing the result to register <i>rd</i> .
C.SUBW	RV64C/RV128C-only instruction that subtracts the value in register <i>rs2'</i> from the value in register <i>rd'</i> , then sign-extends the lower 32 bits of the difference before writing the result to register <i>rd</i> .

Table 67: Integer Register-Register Operation Description (cont.)

Defined Illegal Instruction

A 16-bit instruction with all bits zero is permanently reserved as an illegal instruction.

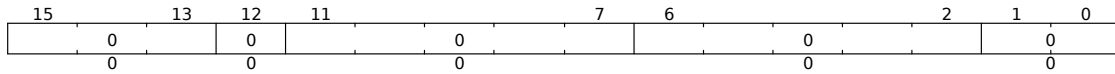


Figure 83: Defined Illegal Instruction

5.8 B Extension: Bit Manipulation Instructions

This section discusses the bit manipulation instructions supported by RISC-V. SiFive implements the Zba and Zbb instructions of the B extension.

5.8.1 Zba Extension

The Zba instructions are used to accelerate the generation of addresses that index into arrays of basic types (halfword, word, doubleword) using both unsigned word-sized and 64-sized indices; that is, a shifted index is added to a base address.

Address Calculation Instructions

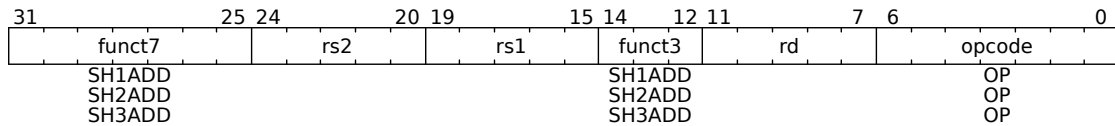


Figure 84: Address Calculation Instructions

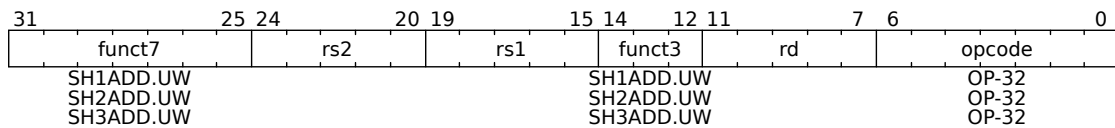


Figure 85: Address Calculation Instructions (cont.)

Instruction	Description
SH1ADD rd, rs1, rs2	Shifts rs1 by 1 bit, then adds the result to rs2
SH2ADD rd, rs1, rs2	Shifts rs1 by 2 bits, then adds the result to rs2
SH3ADD rd, rs1, rs2	Shifts rs1 by 3 bits, then adds the result to rs2
SH1ADD.UW rd, rs1, rs2	Performs an 64-wide addition of rs2, and the unsigned value formed by extracting the least-significant word of rs1 and shifting it left by 1 bit
SH2ADD.UW rd, rs1, rs2	Performs an 64-wide addition of rs2, and the unsigned value formed by extracting the least-significant word of rs1 and shifting it left by 2 bits
SH3ADD.UW rd, rs1, rs2	Performs an 64-wide addition of rs2, and the unsigned value formed by extracting the least-significant word of rs1 and shifting it left by 3 bits

Table 68: Address Calculation Instructions Description

Add/Shift with Prefix Zero-Extend Instructions

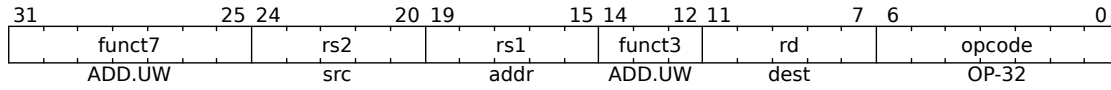


Figure 86: Add with Prefix Zero-Extend Instruction

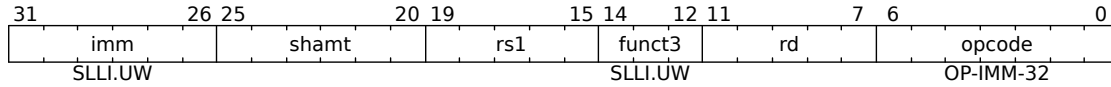


Figure 87: Shift with Prefix Zero-Extend Instruction

Instruction	Description
ADD.UW rd, rs1, rs2	Performs a 64-wide addition between rs2 and the zero-extended least-significant word of rs1
SLLI.UW rd, rs1, shamt	Takes the least-significant word of rs1, zero-extends it, and shifts it left by the immediate

Table 69: Add/Shift with Prefix Zero-Extend Instructions Description

5.8.2 Zbb Extension

The Zbb instructions are used for basic bit manipulation.

Count Leading/Trailing Zeroes Instructions

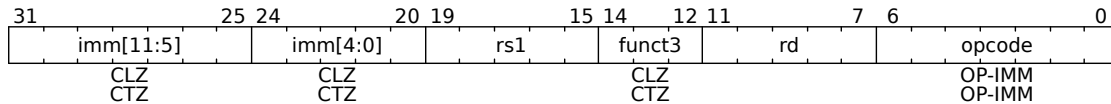


Figure 88: Count Leading/Trailing Zeroes Instructions

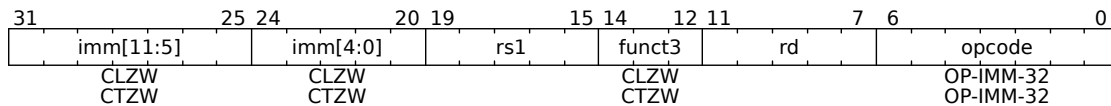


Figure 89: Count Leading/Trailing Zeroes Instructions (cont.)

Instruction	Description
CLZ rd, rs	Counts the number of 0 bits before the first 1 bit, starting at the most-significant bit and progressing to bit 0. If the input is 0, the output is 64. If the most-significant bit of the input is 1, the output is 0.
CTZ rd, rs	Counts the number of 0 bits before the first 1 bit, starting at the least-significant bit and progressing to the most-significant bit. If the input is 0, the output is 64. If the least-significant bit of the input is 1, the output is 0.
CLZW rd, rs	Counts the number of 0 bits before the first 1 bit, starting at bit 31 and progressing to bit 0. If the least-significant word is 0, the output is 32. If the most-significant bit of the word is 1, the output is 0.
CTZW rd, rs	Counts the number of 0 bits before the first 1 bit, starting at the least-significant bit and progressing to the most-significant word. If the least-significant word is 0, the output is 32. If the least-significant bit of the input is 1, the output is 0.

Table 70: Count Leading/Trailing Zeroes Instructions Description

Count Population Instructions

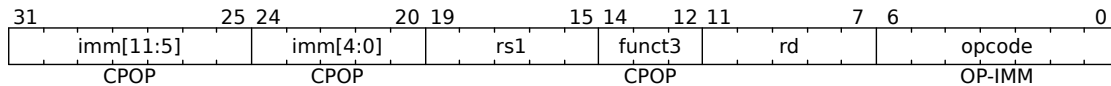


Figure 90: Count Population Instruction

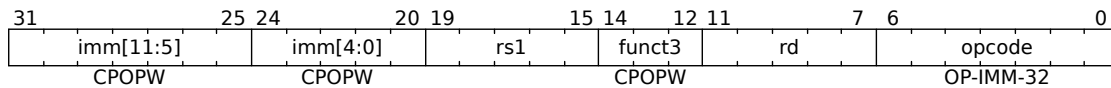


Figure 91: Count Population Instruction (cont.)

Instruction	Description
CPOP rd, rs	Counts the number of 1 bits in the source register
CPOPW rd, rs	Counts the number of 1 bits in the least-significant word of the source register

Table 71: Count Population Instructions Description

Logic-With-Negate Instructions

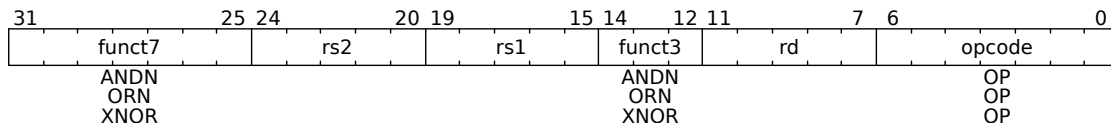


Figure 92: Logic-With-Negate Instructions

Instruction	Description
ANDN rd,rs1,rs2	Performs bitwise logical AND between rs1 and the bitwise inversion of rs2
ORN rd,rs1,rs2	Performs bitwise logical OR between rs1 and the bitwise inversion of rs2
XNOR rd,rs1,rs2	Performs bitwise exclusive-NOR on rs1 and rs2

Table 72: Logic-With-Negate Instructions Description

Comparison Instructions

These instructions are arithmetic R-type instructions that return the smaller or larger value of two operands.

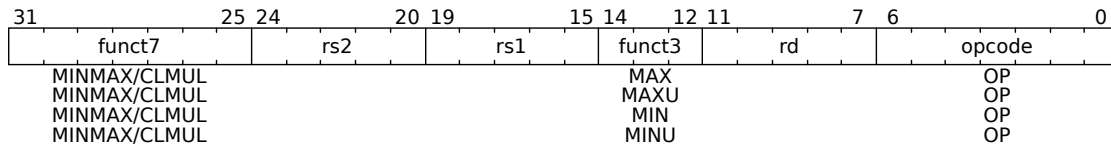


Figure 93: Comparison Instructions

Instruction	Description
MIN rd,rs1,rs2	Returns the smaller of two signed integers
MINU rd,rs1,rs2	Returns the smaller of two unsigned integers
MAX rd,rs1,rs2	Returns the larger of two signed integers
MAXU rd,rs1,rs2	Returns the larger of two unsigned integers

Table 73: Comparison Instructions Description

Sign-Extend and Zero-Extend Instructions

These instructions perform the sign-extension or zero-extension of the least-significant 8 or 16 bits of the source register.

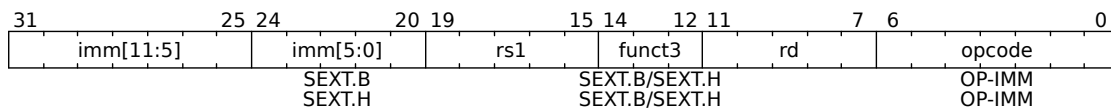


Figure 94: Sign-Extend Instructions

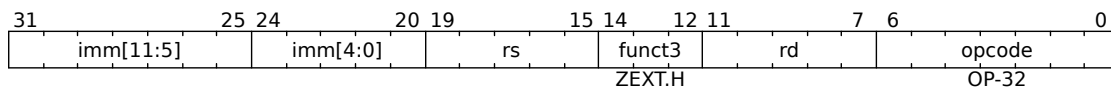


Figure 95: Zero-Extend Instruction

Instruction	Description
SEXT.B rd, rs	Sign-extends the least-significant byte in the source to 64 by copying the most-significant bit in the byte (i.e., bit 7) to all of the more-significant bits
SEXT.H rd, rs	Sign-extends the least-significant halfword in <i>rs</i> to 64 by copying the most-significant bit in the halfword (i.e., bit 15) to all of the more-significant bits
ZEXT.H rd, rs	Zero-extends the least-significant halfword of the source to 64 by inserting 0's into all of the bits more significant than 15

Table 74: Sign- and Zero-Extend Instructions

Bitwise Rotation Instructions

Bitwise rotation instructions are similar to the shift-logical operations from the base ISA specification. However, where the shift-logical instructions shift in zeros, the rotate instructions shift in the bits that were shifted out of the other side of the values.

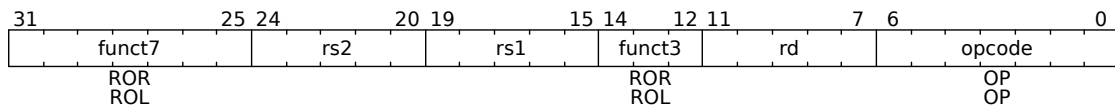


Figure 96: Bitwise Rotation Instructions

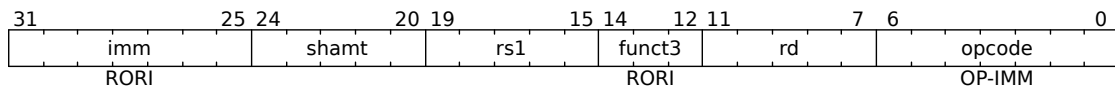


Figure 97: Bitwise Rotation Instructions (cont.)

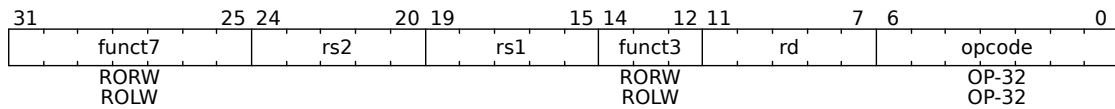


Figure 98: Bitwise Rotation Instructions (cont.)

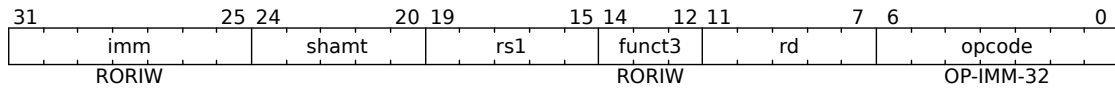


Figure 99: Bitwise Rotation Instructions (cont.)

Instruction	Description
ROR rd,rs1,rs2	Performs a rotate right shift of rs1 by the amount in the least-significant 6 bits of rs2
ROL rd,rs1,rs2	Performs a rotate left shift of rs1 by the amount in the least-significant 6 bits of rs2
RORI rd,rs1,shamt	Performs a rotate right shift of rs1 by the amount in the least significant 6 bits of shamt
RORW rd,rs1,rs2	Performs a rotate right shift of the least-significant word of rs1 by the amount in the least-significant 5 bits of rs2. The resulting word value is sign-extended by copying bit 31 to all of the more-significant bits.
ROLW rd,rs1,rs2	Performs a rotate left shift on the least-significant word of rs1 by the amount in the least-significant 5 bits of rs2. The resulting word value is sign-extended by copying bit 31 to all of the more-significant bits.
RORIW rd,rs1,imm	Performs a rotate right shift on the least-significant word of rs1 by the amount in the least-significant 6 bits of shamt. The resulting word value is sign-extended by copying bit 31 to all of the more significant bits.

Table 75: Bitwise Rotation Instructions Description

OR Combine Instruction

Instruction	Description
ORC.B rd,rs	Combines the bits within every byte through a reciprocal bitwise logical OR. This sets the bits of each byte in the result rd to all zeros if no bit within the respective byte of rs is set, otherwise it sets the bits to all ones if any bit within the respective byte of rs is set.

Table 76: OR Combine Instruction Description

Byte-Reverse Instruction

Instruction	Description
REV8 rd,rs	Reverses the order of the bytes in a register

Table 77: Byte-Reverse Instruction Description

5.9 Zicsr Extension: Control and Status Register Instructions

RISC-V defines a separate address space of 4096 Control and Status registers associated with each hart. The defined instructions access counter, timers and floating-point status registers.

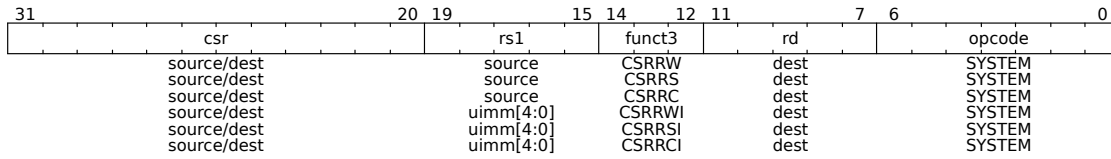


Figure 100: Zicsr Instructions

Instruction	Description
CSRRW rd, rs1 csr	Instruction atomically swaps values in the CSRs and integer registers.
CSRRS rd, rs1 csr	Instruction reads the value of the CSR, zero-extends the value to 64-bits, and writes it to integer register rd. The initial value in integer register rs1 is treated as a bit mask that specifies bit positions to be set in the CSR.
CSRRC rd, rs1 csr	Instruction reads the value of the CSR, zero-extends the value to 64-bits, and writes it to integer register rd. The initial value in integer register rs1 is treated as a bit mask that specifies bit positions to be cleared in the CSR.
CSRRWI rd, rs1 csr	Update the CSR using an 64-bit value obtained by zero-extending a 5-bit unsigned immediate (uimm[4:0]) field encoded in the rs1 field instead of a value from an integer register.
CSRRSI rd, rs1 csr	Update the CSR using an 64-bit value obtained by zero-extending a 5-bit unsigned immediate (uimm[4:0]) field encoded in the rs1 field instead of a value from an integer register.
CSRRCI rd, rs1 csr	If the uimm[4:0] field is zero, then these instructions will not write to the CSR.

Table 78: Control and Status Register Instruction Description

The CSRRWI, CSRRSI, and CSRRCI instructions are similar in kind to CSRRW, CSRRS, and CSRRC respectively, except in that they update the CSR using an 64-bit value obtained by zero-extending a 5-bit unsigned immediate (uimm[4:0]) field encoded in the rs1 field instead of a value from an integer register. For CSRRSI and CSRRCI, these instructions will not write to the CSR if the uimm[4:0] field is zero, and they shall not cause any of the size effects that might otherwise occur on a CSR write. For CSRRWI, if rd = x0, then the instruction shall not read the CSR and shall not cause any of the side effects that might occur on a CSR read. Both CSRRSI and CSRRCI will always read the CSR and cause any read side effects regardless of the rd and rs1 fields.

Table 79 shows if a CSR reads or writes given a particular CSR.

Register Operand				
Instruction	rd	rs1	read CSR?	write CSR?
CSRRW	x0	-	no	yes
CSRRW	!x0	-	yes	yes
CSRRS/C	-	x0	yes	no
CSRRS/C	-	!x0	yes	yes
Immediate Operand				
Instruction	rd	uimm	read CSR?	write CSR?
CSRRWI	x0	-	no	yes
CSRRWI	!x0	-	yes	yes
CSRRS/CI	-	0	yes	no
CSRRS/CI	-	!0	yes	yes

Table 79: CSR Reads and Writes

5.9.1 Control and Status Registers

The control and status registers (CSRs) are only accessible using variations of the CSRR (Read) and CSRRW (Write) instructions. Only the CPU executing the csr instruction can read or write these registers, and they are not visible by software outside of the core they reside on. The standard RISC-V ISA sets aside a 12-bit encoding space (csr[11:0]) for up to 4,096 CSRs. Attempts to access a non-existent CSR raise an illegal instruction exception. Attempts to access a CSR without appropriate privilege level or to write a read-only register also raise illegal instruction. A read/write register might also contain some bits that are read-only, in which case, writes to the read-only bits are ignored. Each core functionality has its own control and status registers which are described in the corresponding section.

5.9.2 Defined CSRs

The following tables describe the currently defined CSRs, categorized by privilege level. The usage of the CSRs below is implementation specific. CSRs are only accessible when operating within a specific access mode (user mode, debug mode, supervisor mode, or machine mode). Therefore, attempts to access a non-existent CSR raise an illegal instruction exception, and attempts to access a CSR without appropriate privilege level or to write a read-only register also raise illegal instruction exceptions.

Number	Privilege	Name	Description
User Trap Setup			
0x000	RW	ustatus	User status register.
0x004	RW	uie	User interrupt-enable register.
0x005	RW	utvec	User trap handler base address.
User Trap Handling			
0x040	RW	uscratch	Scratch register for use trap handlers.
0x041	RW	uepc	User exception program counter.
0x042	RW	ucause	User trap cause.
0x043	RW	ubadaddr	User bad address.
0x044	RW	uip	User interrupt pending.
User Floating-Point CSRs			
0x001	RW	fflags	Floating-Point Accrued Exceptions.
0x002	RW	frm	Floating-Point Dynamic Rounding Mode.
0x003	RW	fcsr	Floating-Point Control and Status Register (frm + fflags).
User Counter/Timers			
0xC00	RO	cycle	Cycle counter for RDCYCLE instruction.
0xC01	RO	time	Timer for RDTIME instruction.
0xC02	RO	instret	Instructions-retired counter for RDINSTRET instruction.
0xC03	RO	hpmcounter3	Performance-monitoring counter.
0xC04	RO	hpmcounter4	Performance-monitoring counter.
		...	
0xC1F	RO	hpmcounter31	Performance-monitoring counter.

Table 80: User Mode CSRs

Number	Privilege	Name	Description
Supervisor Trap Setup			
0x100	RW	sstatus	Supervisor status register.
0x102	RW	sedeleg	Supervisor exception delegation register.
0x103	RW	sideleg	Supervisor interrupt delegation register.
0x104	RW	sie	Supervisor interrupt-enable register.
0x105	RW	stvec	Supervisor trap handler base address.
0x106	RW	scounteren	Supervisor counter enable.
Supervisor Trap Handling			
0x140	RW	sscratch	Scratch register for supervisor trap handlers.
0x141	RW	sepc	Supervisor exception program counter.
0x142	RW	scause	Supervisor trap cause.
0x143	RW	stval	Supervisor bad address or instruction.
0x144	RW	sip	Supervisor interrupt pending.
Supervisor Protection and Translation			
0x180	RW	satp	Supervisor address translation and protection.

Table 81: Supervisor Mode CSRs

Number	Privilege	Name	Description
Machine Information Registers			
0xF11	RO	mvendorid	Vendor ID.
0xF12	RO	marchid	Architecture ID.
0xF13	RO	mimpid	Implementation ID.
0xF14	RO	mhartid	Hardware thread ID.
Machine Trap Setup			
0x300	RW	mstatus	Machine status register.
0x301	RW	misa	ISA and extensions.
0x302	RW	medeleg	Machine exception delegation register.
0x303	RW	mideleg	Machine interrupt delegation register.
0x304	RW	mie	Machine interrupt-enable register.
0x305	RW	mtvec	Machine trap-handler base address.
0x306	RW	mcounteren	Machine counter enable.
Machine Trap Handling			
0x340	RW	mscratch	Scratch register for machine trap handlers.
0x341	RW	mepc	Machine exception program counter.
0x342	RW	mcause	Machine trap cause.
0x343	RW	mtval	Machine bad address or instruction.
0x344	RW	mip	Machine interrupt pending.
Machine Memory Protection			
0x3A0	RW	pmpcfg0	Physical memory protection configuration.
0x3A1	RW	pmpcfg1	Physical memory protection configuration, RV32 only.
0x3A2	RW	pmpcfg2	Physical memory protection configuration.
0x3A3	RW	pmpcfg3	Physical memory protection configuration, RV32 only.
0x3B0	RW	pmpaddr0	Physical memory protection address register.
0x3B1	RW	pmpaddr1	Physical memory protection address register.
		...	
0x3BF	RW	pmpaddr15	Physical memory protection address register.
Machine Counter/Timers			
0xB00	RW	mcycle	Machine cycle counter.
0xB02	RW	minstret	Machine instruction-retired counter.
Machine Counter Setup			
0x320	RW	mcountinhibit	Machine counter-inhibit register.
0x323	RW	mhpmevent3	Machine performance-monitoring event selector.
0x324	RW	mhpmevent4	Machine performance-monitoring event selector.
		...	
0x33F	RW	mhpmevent31	Machine performance-monitoring event selector.
Debug/Trace Register (shared with Debug Mode)			
0x7A0	RW	tselect	Debug/Trace trigger register select.

Table 82: Machine Mode CSRs

Number	Privilege	Name	Description
0x7A1	RW	tdata1	First Debug/Trace trigger data register.
0x7A2	RW	tdata2	Second Debug/Trace trigger data register.
0x7A3	RW	tdata3	Third Debug/Trace trigger data register.

Table 82: Machine Mode CSRs

Number	Privilege	Name	Description
0x7B0	RW	dcsr	Debug control and status register.
0x7B1	RW	dpc	Debug PC.
0x7B2	RW	dscratch	Debug scratch register.

Table 83: Debug Mode Registers

5.9.3 CSR Access Ordering

On a given hart, explicit and implicit CSR access are performed in program order with respect to those instructions whose execution behavior is affected by the state of the accessed CSR. In particular, a CSR access is performed after the execution of any prior instructions in program order whose behavior modifies or is modified by the CSR state and before the execution of any subsequent instructions in program order whose behavior modifies or is modified by the CSR state.

Furthermore, a CSR read access instruction returns the accessed CSR state before the execution of the instruction, while a CSR write access instruction updates the accessed CSR state after the execution of the instruction. Where the above program order does not hold, CSR accesses are weakly ordered, and the local hart or other harts may observe the CSR accesses in an order different from program order. In addition, CSR accesses are not ordered with respect to explicit memory accesses, unless a CSR access modifies the execution behavior of the instruction that performs the explicit memory access or unless a CSR access and an explicit memory access are ordered by either the syntactic dependencies defined by the memory model or the ordering requirements defined by the Memory-Ordering PMAs. To enforce ordering in all other cases, software should execute a FENCE instruction between the relevant accesses. For the purposes of the FENCE instruction, CSR read accesses are classified as device input (I), and CSR write accesses are classified as device output (O). For more about the FENCE instructions, see Section 5.13. For CSR accesses that cause side effects, the above ordering constraints apply to the order of the initiation of those side effects but does not necessarily apply to the order of the completion of those side effects.

5.9.4 SiFive RISC-V Implementation Version Registers

`mvendorid`

The value in `mvendorid` is 0x489, corresponding to SiFive's JEDEC number.

marchid

The value in `marchid` indicates the overall microarchitecture of the core and at SiFive we use this to distinguish between core generators. The RISC-V standard convention separates `marchid` into open-source and proprietary namespaces using the most-significant bit (MSB) of the `marchid` register; where if the MSB is clear, the `marchid` is for an open-source core, and if the MSB is set, then `marchid` is a proprietary microarchitecture. The open-source namespace is managed by the RISC-V Foundation and the proprietary namespace is managed by SiFive.

SiFive's E3 and S5 cores are based on the open-source 3/5-Series microarchitecture, which has a Foundation-allocated `marchid` of 1. Our other generators are numbered according to the core series.

Value	Core Generator
0x8000_0008	P500-Series Processor

Table 84: Core Generator Encoding of `marchid`

mimpid

The value in `mimpid` holds an encoded value that uniquely identifies the version of the generator used to build this implementation. If your release version is not included in Table 85, contact your SiFive account manager for more information.

Value	Generator Release Version
0x0000_0000	Pre-19.02
0x2019_0228	19.02
0x2019_0531	19.05
0x2019_0919	19.08p0p0 / 19.08.00
0x2019_1105	19.08p1p0 / 19.08.01.00
0x2019_1204	19.08p2p0 / 19.08.02.00
0x2020_0423	19.08p3p0 / 19.08.03.00
0x0120_0626	19.08p4p0 / 19.08.04.00
0x0220_0515	koala.00.00-preview and koala.01.00-preview
0x0220_0603	koala.02.00-preview
0x0220_0630	20G1.03.00 / koala.03.00-general
0x0220_0710	20G1.04.00 / koala.04.00-general
0x0220_0826	20G1.05.00 / koala.05.00-general
0x0320_0908	kiwi.00.00-preview
0x0220_1013	20G1.06.00 / koala.06.00-general
0x0220_1120	20G1.07.00 / koala.07.00-general
0x0421_0205	llama.00.00-preview
0x0421_0324	21G1.01.00 / llama.01.00-general
0x0421_0427	21G1.02.00 / llama.02.00-general
0x0521_0528	mongoose.00.00-preview
0x0521_0714	21G2.01.00 / mongoose.01.00-general

Table 85: Generator Release Encoding of mimpid

Reading Implementation Version Registers

To read the mvendorid, marchid, and mimpid registers, simply replace mimpid with mvendorid or marchid as needed.

In C:

```
uintptr_t mimpid;
__asm__ volatile("csrr %0, mimpid" : "=r"(mimpid));
```

In Assembly:

```
csrr a5, mimpid
```

5.9.5 Custom CSRs

SiFive implements some custom CSRs that are specific to the implementation. For these CSRs, including the Feature Disable CSR, consider Chapter 6.

5.10 Base Counters and Timers

RISC-V ISAs provide a set of up to 32×64-bit performance counters and timers that are accessible via unprivileged 64-bit read-only CSR registers 0xc00–0xc1f. The first three of these (CYCLE, TIME, and INSTRET) have dedicated functions; while the remaining counters, if implemented, provide programmable event counting.

The P550 Core Complex implements `mcycle`, `mtime`, and `minstret` counters, which have dedicated functions: cycle count, real-time clock, and instructions-retired, respectively. The timer functionality is based on the `mtime` register. Additionally, the P550 Core Complex implements event counters in the form of `mhpmpcounter`, which is used to monitor user requested events.

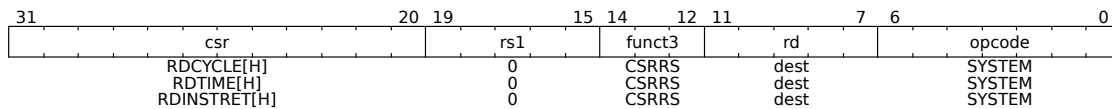


Figure 101: Timer and Counter Pseudoinstructions

Instruction	Description
RDCYCLE rd	Reads the 64-bits of the cycle CSR which holds a count of the number of clock cycles executed by the processor core on which the hart is running from an arbitrary start time in the past.
RDTIME rd	Generates an illegal instruction exception. The <code>mtime</code> register is memory mapped to the CLINT register space and can be read using a regular load instruction.
RDINSTRET rd	Reads the 64-bits of the instret CSR, which counts the number of instructions retired by this hart from some arbitrary start point in the past.

Table 86: Timer and Counter Pseudoinstruction Description

RDCYCLE, RDTIME, and RDINSTRET pseudoinstructions read the full 64 bits of the `cycle`, `time`, and `instret` counters. The RDCYCLE pseudoinstruction reads the low 64-bits of the cycle CSR (`mcycle`), which holds a count of the number of clock cycles executed by the processor core on which the hart is running from an arbitrary start time in the past. The RDTIME pseudoinstruction reads the low 64-bits of the time CSR (`mtime`), which counts wall-clock real time that has passed from an arbitrary start time in the past. The RDINSTRET pseudoinstruction reads the low 64-bits of the instret CSR (`minstret`), which counts the number of instructions retired by this hart from some arbitrary start point in the past. The rate at which the cycle counter advances is `rtc_clock`. To determine the current rate (cycles per second) of instruction execution, call the `metal_timer_get_timebase_frequency` API. The `metal_timer_get_timebase_frequency` and additional APIs are described in Section 5.10.2 below.

Number	Privilege	Name	Description
0xC00	RO	cycle	Cycle counter for RDCYCLE instruction
0xC01	RO	time	Timer for RDTIME instruction
0xC02	RO	instret	Instruction-retired counter for RDINSTRET instruction

Table 87: Timer and Counter CSRs

5.10.1 Timer Register

`mtime` is a 64-bit read-write register that contains the number of cycles counted from the `rtc_toggle` signal described in the P550 Core Complex User Guide. On reset, `mtime` is cleared to zero.

5.10.2 Timer API

The APIs below are used for reading and manipulating the machine timer. Other APIs are described in more detail within the Freedom Metal documentation. <https://sifive.github.io/freedom-metal-docs/>

Functions

`int metal_timer_get_cyclecount(int hartid, unsigned long long *cyclecount)`

Read the machine cycle count.

Return

0 upon success

Parameters

- `hartid`: The hart ID to read the cycle count of
- `cyclecount`: The variable to hold the value

`int metal_timer_get_timebase_frequency(int hartid, unsigned long long *timebase)`

Get the machine timebase frequency.

Return

0 upon success

Parameters

- `hartid`: The hart ID to read the cycle count of
- `timebase`: The variable to hold the value

`int metal_timer_set_tick(int hartid, int second)`

Set the machine timer tick interval in seconds.

Return

0 upon success

Parameters

- `hartid`: The hart ID to read the cycle count of
- `second`: The number of seconds to set the tick interval to

5.11 Privileged Instructions

The RISC-V architecture implements privileged instructions that can only be executed when the P550 Core Complex is operating in a privileged mode. The SYSTEM major opcode is used to encode all of the privileged instructions.

5.11.1 Machine-Mode Privileged Instructions

Environment Call and Breakpoint

These ECALL and EBREAK instructions cause a precise requested trap to the supporting execution environment. The ECALL instruction is used to make a service request to the execution environment. The EBREAK instruction is used to return control to a debugging environment.

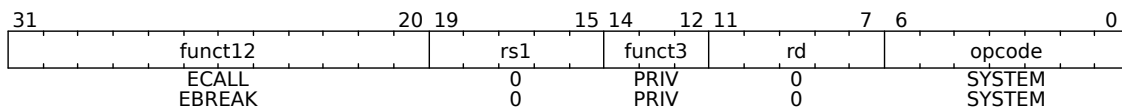


Figure 102: ECALL and EBREAK Instructions

Trap-Return Instructions

To return after handling a trap, there are separate trap return instructions per privilege level: MRET, SRET, and URET. MRET is always provided, while SRET must be provided if the respective privilege mode is supported. URET is only provided if user-mode traps are supported. An xRET instruction can be executed in privilege mode x or higher, where executing a lower-privilege xRET instruction will pop the relevant lower-privilege interrupt enable and privilege mode stack.

Wait for Interrupt

The Wait for Interrupt (WFI) instruction provides a hint to the P550 Core Complex that the current hart can be stalled until an interrupt might need servicing. Execution of the WFI instruction can also be used to inform the hardware platform that suitable interrupts should preferentially be routed to this hart.

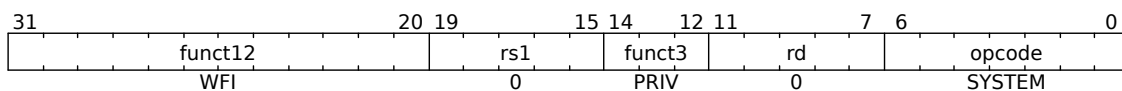


Figure 103: Wait for Interrupt Instruction

If an enabled interrupt is present or later becomes present while the hart is stalled, the interrupt exception will be taken on the following instruction, i.e., execution resumes in the trap handler

and $mepc = pc + 4$. The WFI instruction can also be executed when interrupts are disabled. The operation of WFI must be unaffected by the global interrupt bits in `mstatus` (MIE/SIE/UIE) (i.e., the hart must resume if a locally enabled interrupt becomes pending), but should honor the individual interrupt enables (e.g., MTIE). WFI is also required to resume execution for locally enabled interrupts pending at any privilege level, regardless of the global interrupt enable at each privilege level. If the event that causes the hart to resume execution does not cause an interrupt to be taken, execution will resume at $pc + 4$, and software must determine what action to take, including looping back to repeat the WFI if there was no actionable event.

The suggested way to call WFI is inside an infinite loop as described below.

```
while (1) {
    __asm__ volatile ("wfi");
}
```

The WFI instruction is just a hint, and a legal implementation is to implement WFI as a NOP. In SiFive's implementation of WFI, the WFI instruction is issued and the core goes into internal clock gating state.

5.11.2 Supervisor-Mode Privileged Instructions

For S-Mode enabled cores, such as the P550 Core Complex one new supervisor-level instruction is provided in addition to SRET.

Supervisor Memory-Management Fence Instruction

The supervisor memory-management fence instruction `SFENCE.VMA` is used to synchronize updates to in-memory memory-management data structures with current execution.

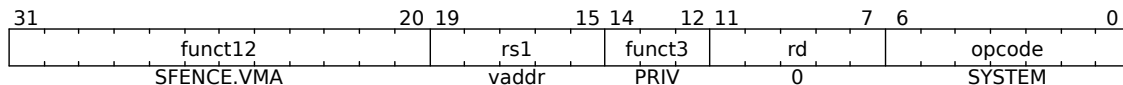


Figure 104: Supervisor Memory-Management Fence Instruction

Instruction execution causes implicit reads and writes to these data structures; however, these implicit references are ordinarily not ordered with respect to loads and stores in the instruction stream. Executing an `SFENCE.VMA` instruction guarantees that any stores in the instruction stream prior to the execution of `SFENCE.VMA` are ordered before all implicit references subsequent to the `SFENCE.VMA`. Furthermore, executing an `SFENCE.VMA` guarantees that any implicit writes caused by instructions prior to the `SFENCE.VMA` are ordered before all loads and stores subsequent to the `SFENCE.VMA`.

5.12 ABI - Register File Usage and Calling Conventions

RV64GCB has 32 x registers that are each 64 bits wide.

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	-
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	-
x4	tp	Thread pointer	-
x5	t0	Temporary / alternate link register	Caller
x6-7	t1-2	Temporaries	Caller
x8	s0/fp	Saved-register / frame-pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments / return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller
Floating-Point Registers			
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP arguments / return values	Caller
f12-17	fa2-7	FP arguments	Caller
f18-27	fa2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller

Table 88: RISC-V Registers

The programmer counter PC hold the address of the current instruction.

- x1 / ra - holds the return address for a call.
- x2 / sp - stack pointer, points to the current routine stack.
- x8 / fp / s0 - frame pointer, points to the bottom of the top stack frame.
- x3 / gp - global pointer, points into the middle of the global data section.
The common definition is: .data + 0x800. RISC-V immediate values are 12-bit signed values, which is +/- 2048 in decimal or +/- 0x800 in hex. So that global pointer relative accesses can reach their full extent, the global pointer point + 0x800 into the data section. The linker can then relax LUI+LW, LUI+SW into gp-relative LW or SW, i.e., shorter instruction sequences and access most global data using LW at gp +/- offset

```
LW t0 , 0x800(gp)
LW t1 , 0x7FF(gp)
```

- x4 / tp - thread pointer, point to thread-local storage (TLS-mostly used in Linux and RTOS).
If you create a variable in TLS, every thread has its own copy of the variable, i.e., changes to the variable are local to the thread. This is a static area of memory that gets copied for each thread in a program. It is also used to create libraries that have thread-safe functions,

because of the fact that each call to a function has its copy of the same global data, so it's safe.

5.12.1 RISC-V Assembly

RISC-V instructions have opcodes and operands.

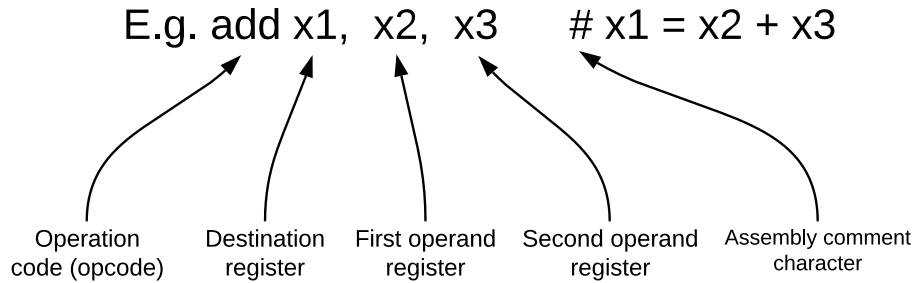


Figure 105: RISC-V Assembly Example

Assembly	C	Description
<code>add x1, x2, x3</code>	<code>a = b + c</code>	<code>a=x1, b=x2, c=x3</code>
<code>sub x3, x4, x5</code>	<code>d = e - f</code>	<code>d=x3, e=x4, f=x5</code>
<code>add x0, x0, x0</code>	NOP	Writes to x0 are always ignored
<code>add x3, x4, x0</code>	<code>f = g</code>	<code>f=x3, g=x4</code>
<code>addi x3, x4, -10</code>	<code>f = g - 10</code>	<code>f=x3, g=x4</code>
<code>lw x10, 12(x13) # 12 = 3x4</code> <code>add x11, x12, x10</code>	<code>int A[100];</code> <code>g = h + A[3];</code>	Reg x10 gets A[3] <code>g=x11, h=x12</code>
<code>lw x10, 12(x13) # 12 = 3x4</code> <code>add x10, x12, x10</code> <code>sw x10, 40(x13) # 40 = 10x4</code>	<code>int A[100];</code> <code>A[10] = h + A[3];</code>	Reg x10 gets A[3] <code>h=x12</code> Reg x10 gets <code>h + A[3]</code>
<code>bne x13, x14, done</code> <code>add x10, x11, x12</code> <code>done:</code>	<code>if (i == j)</code> <code> f = g + h;</code>	<code>f=x10, g=x11, h=x12, i=x13, j=x14</code>
<code>bne x10, x14, else</code> <code>add x10, x11, x12</code> <code>j done</code> <code>else: sub x10, x11, x12</code> <code>done:</code>	<code>if (i == j)</code> <code> f = g + h;</code> <code>else</code> <code> f = g - h;</code>	<code>f=x10, g=x11, h=x12, i=x13, j=x14</code>

Table 89: RISC-V Assembly and C Examples

5.12.2 Assembler to Machine Code

The following flowchart describes how the assembler converts the RISC-V assembly code to machine code.

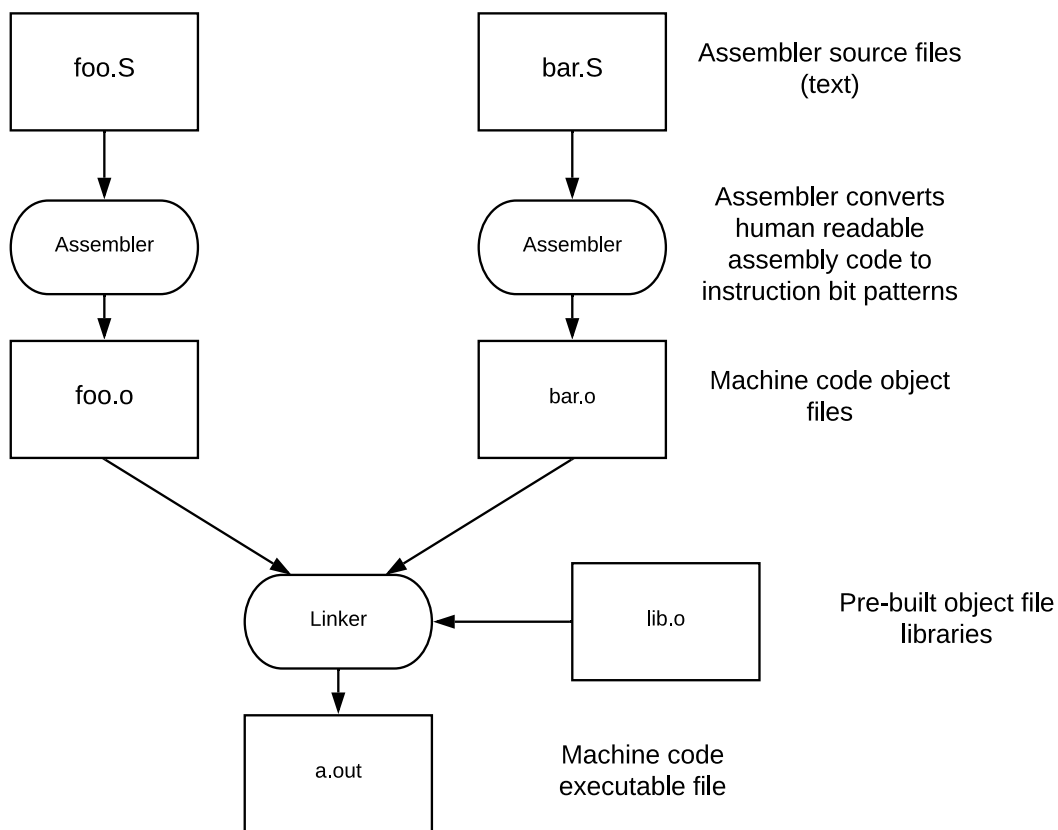


Figure 106: RISC-V Assembly to Machine Code

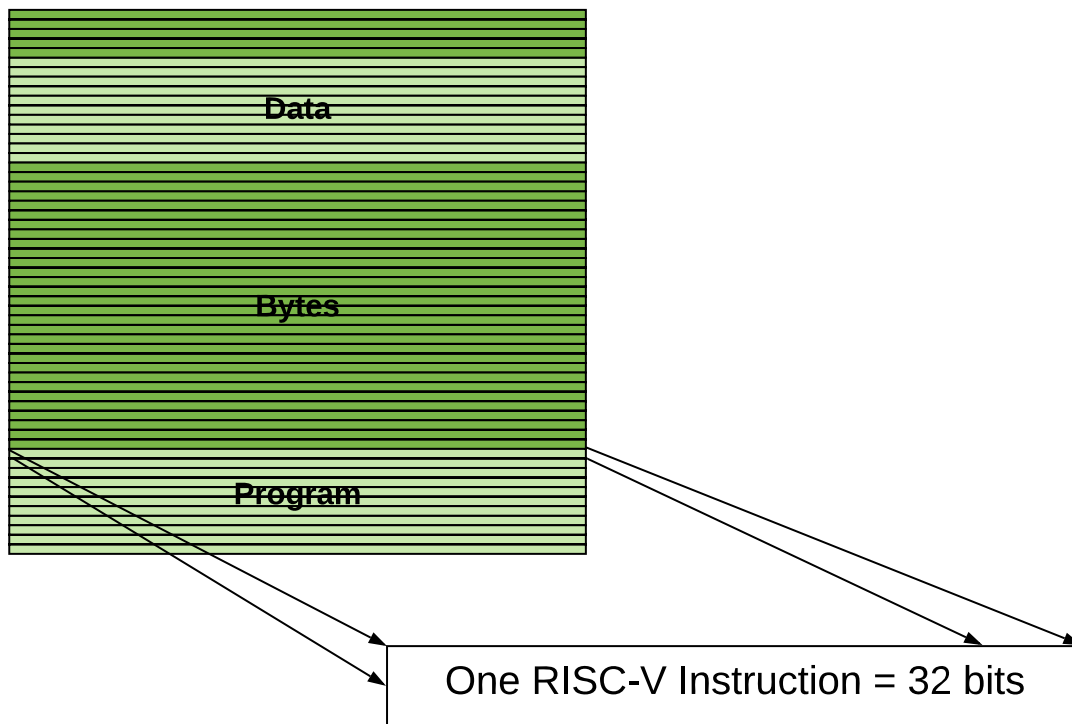


Figure 107: One RISC-V Instruction

5.12.3 Calling a Function (Calling Convention)

1. Put parameters in place where function can access them.
2. Transfer control to function.
3. Acquire local resources needed for function.
4. Perform function task.
5. Place result values where calling code can access and restore any registers might have used.
6. Return control to original caller.

Caller-saved The function invoked can do whatever it likes with the registers. Callee-saved If a function wants to use registers it needs to store and restore them.

Take, for example, the following function:

```
int leaf(int g, int h, int i, int j) {  
    int f;  
    f = (g+h) - (i+j);  
    return f;  
}
```

In this function above, arguments are passed in a0, a1, a2 and a3. The return value is returned in a0.

```
addi sp, sp, -8    # adjust stack for 2 items
sw s1, 4(sp)       # save s1 for use afterwards
sw s0, 0(sp)       # save s0 for use afterwards

add s0,a0,a1       # s0 = g + h
add s1,a2,a3       # s1 = i + j
sub a0,s0,s1       # return value (g + h) - (i + j)

lw s0, 0(sp)       # restore register s0 for caller
lw s1, 4(sp)       # restore register s1 for caller
addi sp, 4(sp)     # adjust stack to delete 2 items
jr ra              # jump back to calling routine
```

In the assembly above, notice that the stack pointer was decremented by 8 to make room to save the registers. Also, s1 and s0 are saved and will be stored at the end.

Nested Functions

In the case of nested function calls, values held in a0-7 and ra will be clobbered.

Take, for example, the following function:

```
int sumSquare(int x, int y) {
    return mult(x,x) + y;
}
```

In the function above, a function called sumSquare is calling mult. To execute the function, there's a value in ra that sumSquare wants to jump back to, but this value will be overwritten by the call to mult.

To avoid this, the sumSquare return address must be saved before the call to mult. To save the return address of sumSquare, the function can utilize stack memory. The user can use stack memory to preserve automatic (local) variables that don't fit within the registers.

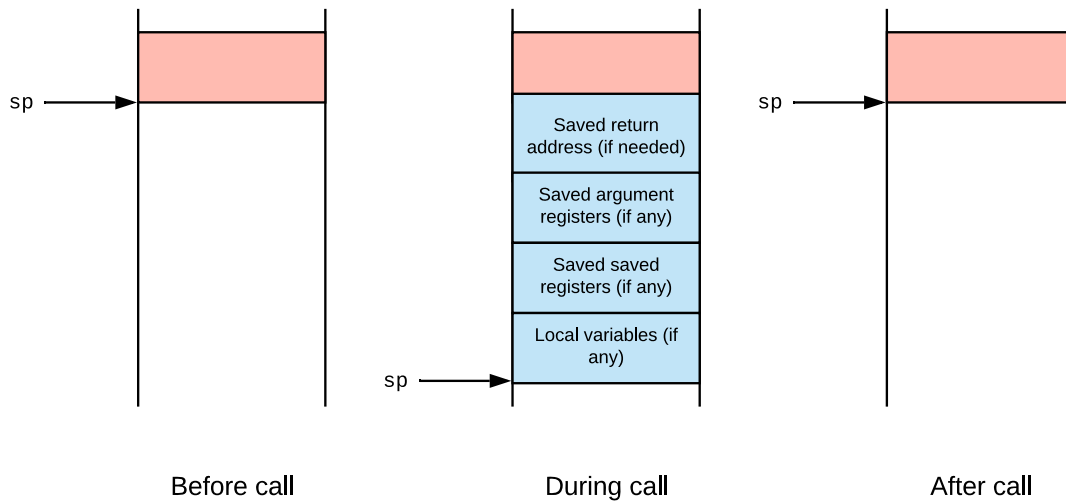


Figure 108: Stack Memory during Function Calls

Consider the assembly for sumSquare below:

```
sumSquare:
addi sp,sp,-8      # reserve space on stack
sw ra, 4(sp)       # save return address
sw a1, 0(sp)       # save y
mv a1,a0           # mult(x,x)
jal mult           # call mult
lw a1, 0(sp)       # restore y
add a0,a0,a1       # mult()+y
lw ra, 4(sp)       # get return address
addi sp,sp,8       # restore stack
mult:...
```

5.13 Memory Ordering - FENCE Instructions

In the RISC-V ISA, each thread, referred to as a hart, observes its own memory operations as if they executed sequentially in program order. RISC-V also has a relaxed memory model, which requires explicit FENCE instructions to guarantee the ordering of memory operations.

The FENCE instructions include FENCE and FENCE.I. The FENCE instruction simply ensures that the memory access instructions before the FENCE instruction get committed before the FENCE instruction is committed. It does not guarantee that those memory access instructions have actually completed. For example, a load instruction before a FENCE instruction can commit without waiting for its value to come back from the memory system. FENCE.I functions the same as FENCE, as well as flushes the instruction cache.

For example, without FENCE instructions:

Hart 1 executes:

Load X
Store Y
Store Z

Because of relaxed memory model, Hart 2 could see stores/loads arranged in any order:

Store Z
Load X
Store Y

With FENCE instructions:

Hart 1 executes:

Load X
Store Y
FENCE
Store Z

Hart 2 sees:

Store Y
Load X
Store Z

With FENCE instructions, Hart 2 is forced to see the Load X and the Store Y prior to the Store Z, but could arbitrarily see Store Y before Load X or Load X before Store Y. Functionally, FENCE instructions order the completion of older memory accesses prior to newer accesses. However, unnecessary FENCE instructions slow processes and can hide bugs, so it is essential to identify where and when FENCE should be used.

5.14 Boot Flow

This process is managed as part of the Freedom Metal source code. The freedom-metal boot code supports single core boot or multi-core boot, and contains all the necessary initialization code to enable every core in the system.

1. ENTRY POINT: File: freedom-metal/src/entry.S, label: `_enter`.
2. Initialize global pointer gp register using the generated symbol `__global_pointer$`.
3. Write mtvec register with `early_trap_vector` as default exception handler.
4. Clear feature disable CSR `0x7c1`.
5. Read mhartid into register a0 and call `_start`, which exists in `crt0.S`.
6. We now transition to File: freedom-metal/gloss/crt0.S, label: `_start`.
7. Initialize stack pointer, sp, with `_sp` generated symbol. Harts with mhartid of one or larger are offset by `(_sp + __stack_size × mhartid)`. The `__stack_size` field is generated in the linker file.

8. Check if `mhartid == __metal_boot_hart` and run the init code if they are equal. All other harts skip init and go to the Post-Init Flow, step #15.
9. Boot Hart Init Flow begins here.
10. Init data section to destination in defined RAM space.
11. Copy ITIM section, if ITIM code exists, to destination.
12. Zero out bss section.
13. Call `atexit` library function that registers the `libc` and `freedom-metal` destructors to run after main returns.
14. Call the `__libc_init_array` library function, which runs all functions marked with `__attribute__((constructor))`.
 - a. For example, PLL, UART, L3 if they exist in the design. This method provides full early initialization prior to entering the main application.
15. Post-Init Flow Begins Here.
16. Call the C routine `__metal_synchronize_harts`, where hart 0 will release all harts once their individual `msip` bits are set. The `msip` bit is typically used to assert a software interrupt on individual harts, however interrupts are not yet enabled, so `msip` in this case is used as a gatekeeping mechanism.
17. Check `misa` register to see if floating-point hardware is part of the design, and set up `mstatus` accordingly.
18. Single or multi-hart design redirection step.
 - a. If design is a single hart only, or a multi-hart design without a C-implemented function `secondary_main`, ONLY the boot hart will continue to `main()`.
 - b. For multi-hart designs, all other CPUs will enter sleep via WFI instruction via the weak `secondary_main` label in `crt0.S`, while boot hart runs the application program.
 - c. In a multi-hart design which includes a C-defined `secondary_main` function, all harts will enter `secondary_main` as the primary C function.

5.15 Linker File

The linker file generates important symbols that are used in the boot code. The linker file options are found in the `freedom-e-sdk/bsp` path.

There are usually three different linker file options:

- `metal.default.lds` — Use flash and RAM sections
- `metal.ramrodata.lds` — Place read only data in RAM for better performance
- `metal.scratchpad.lds` — Places all code + data sections into available RAM location

Each linker option can be selected by specifying LINK_TARGET on the command line.

For example:

```
make PROGRAM=hello TARGET=design-rtl CONFIGURATION=release LINK_TARGET=scratchpad  
software
```

The metal.default.lds linker file is selected by default when LINK_TARGET is not specified. If there is a scenario where a custom linker is required, one of the supplied linker files can be copied and renamed and used for the build. For example, if a new linker file named metal.newmap.lds was generated, this can be used at build time by specifying LINK_TARGET=newmap on the command line.

5.15.1 Linker File Symbols

The linker file generates symbols that are used by the startup code, so that software can use these symbols to assign the stack pointer, initialize or copy certain RAM sections, and provide the boot hart information. These symbols are made visible to software using the PROVIDE keyword.

For example:

```
__stack_size = DEFINED(__stack_size) ? __stack_size : 0x400;  
PROVIDE(__stack_size = __stack_size);
```

Generated Linker Symbols

A description list of the generated linker symbols is shown below.

__metal_boot_hart

This is an integer number to describe which hart runs the main init flow. The mhartid CSR contains the integer value for each hart. For example, hart 0 has mhartid==0, hart 1 has mhartid==1, and so on. An assembly example is shown below, where a0 already contains the mhartid value.

```
/* If we're not hart 0, skip the initialization work */  
la t0, __metal_boot_hart  
bne a0, t0, _skip_init
```

An example on how to use this symbol in C code is shown below.

```
extern int __metal_boot_hart;  
int boot_hart = (int)&__metal_boot_hart;
```

Additional linker file generated symbols, along with descriptions are shown below.

__metal_chicken_bit

Status bit to tell startup code to zero out the Feature Disable CSR. Details of this register are internal use only.

`__global_pointer$`

Static value used to write the gp register at startup.

`__sp`

Address of the end of stack for hart 0, used to initialize the beginning of the stack since the stack grows lower in memory. On a multi-hart system, the start address of the stack for each hart is calculated using $(_sp + _stack_size \times mhartid)$

`metal_segment_bss_target_start`

`metal_segment_bss_target_end`

Used to zero out global data mapped to .bss section.

- Only `__metal_boot_hart` runs this code.

`metal_segment_data_source_start`

`metal_segment_data_target_start`

`metal_segment_data_target_end`

Used to copy data from image to its destination in RAM.

- Only `__metal_boot_hart` runs this code.

`metal_segment_itim_source_start`

`metal_segment_itim_target_start`

`metal_segment_itim_target_end`

Code or data can be placed in itim sections using the `__attribute__((section(".itim")))`.

- When this attribute is applied to code or data, the `metal_segment_itim_source_start`, `metal_segment_itim_target_start`, and `metal_segment_itim_target_end` symbols get updated accordingly, and these symbols allow the startup code to copy code and data into the ITIM area.
 - Only `__metal_boot_hart` runs this code.

Note

At the time of this writing, the boot flow does not support C++ projects

5.16 RISC-V Compiler Flags

5.16.1 arch, abi, and mtune

RISC-V targets are described using three arguments:

1. `-march=ISA`: selects the architecture to target.

2. `-mabi=ABI`: selects the ABI to target.
3. `-mtune=CODENAME`: selects the microarchitecture to target.

-march

This argument controls which instructions and registers are available for the compiler, as defined by the RISC-V user-level ISA specification.

The RISC-V ISA with 32, 32-bit integer registers and the instructions for multiplication would be denoted as RV32IM. Users can control the set of instructions that GCC uses when generating assembly code by passing the lower-case ISA string to the `-march` GCC argument; for example, `-march=rv32im`. On RISC-V systems that don't support particular operations, emulation routines may be used to provide the missing functionality.

Example:

```
double dmul(double a, double b) {  
    return a * b;  
}
```

will compile directly to a FP multiplication instruction when compiled with the D extension:

```
$ riscv64-unknown-elf-gcc test.c -march=rv64imafdc -mabi=lp64d -o- -S -O3  
dmul:  
    fmul.d    fa0,fa0,fa1  
    ret
```

but will compile to an emulation routine without the D extension:

```
$ riscv64-unknown-elf-gcc test.c -march=rv64i -mabi=lp64 -o- -S -O3  
dmul:  
    add      sp,sp,-16  
    sd       ra,8(sp)  
    call     __muldf3  
    ld       ra,8(sp)  
    add      sp,sp,16  
    jr       ra
```

Similar emulation routines exist for the C intrinsics that are trivially implemented by the M and F extensions.

-mabi

`-mabi` selects the ABI to target. This controls the calling convention (which arguments are passed in which registers) and the layout of data in memory. The `-mabi` argument to GCC specifies both the integer and floating-point ABIs to which the generated code complies. Much like how the `-march` argument specifies which hardware generated code can run on, the `-mabi` argument specifies which software-generated code can link against. We use the standard naming scheme for integer ABIs (`ilp32` or `lp64`), with an argumental single letter appended to

select the floating-point registers used by the ABI (ilp32 vs. ilp32f vs. ilp32d). In order for objects to be linked together, they must follow the same ABI.

RISC-V defines two integer ABIs and three floating-point ABIs.

- ilp32: int, long, and pointers are all 32-bits long. long long is a 64-bit type, char is 8-bit, and short is 16-bit.
- lp64: long and pointers are 64-bits long, while int is a 32-bit type. The other types remain the same as ilp32.

The floating-point ABIs are a RISC-V specific addition:

- "" (the empty string): No floating-point arguments are passed in registers.
- f: 32-bit and smaller floating-point arguments are passed in registers. This ABI requires the F extension, as without F there are no floating-point registers.
- d: 64-bit and smaller floating-point arguments are passed in registers. This ABI requires the D extension.

arch/abi Combinations

- march=rv32imafdc -mabi=ilp32d: Hardware floating-point instructions can be generated and floating-point arguments are passed in registers. This is like the -mfloat-abi=hard argument for the Arm® architecture GCC.
- march=rv32imac -mabi=ilp32: No floating-point instructions can be generated and no floating-point arguments are passed in registers. This is like the -mfloat-abi=soft argument for the Arm architecture GCC.
- march=rv32imafdc -mabi=ilp32: Hardware floating-point instructions can be generated, but no floating-point arguments will be passed in registers. This is like the -mfloat-abi=softfp argument for the Arm architecture GCC, and is usually used when interfacing with soft-float binaries on a hard-float system.
- march=rv32imac -mabi=ilp32d: Illegal, as the ABI requires floating-point arguments are passed in registers but the ISA defines no floating-point registers to pass them in.

Example:

```
double dmul(double a, double b) {  
    return b * a;  
}
```

If neither the ABI nor ISA contains the concept of floating-point hardware then the C compiler cannot emit any floating-point-specific instructions. In this case, emulation routines are used to perform the computation and the arguments are passed in integer registers:

```
$ riscv64-unknown-elf-gcc test.c -march=rv32imac -mabi=ilp32 -o- -S -O3  
dmul:  
    mv      a4,a2
```

```

mv      a5,a3
add     sp,sp,-16
mv      a2,a0
mv      a3,a1
mv      a0,a4
mv      a1,a5
sw      ra,12(sp)
call    __muldf3
lw      ra,12(sp)
add     sp,sp,16
jr      ra

```

The second case is the exact opposite of this one: everything is supported in hardware. In this case we can emit a single `fmul.d` instruction to perform the computation.

```

$ riscv64-unknown-elf-gcc test.c -march=rv32imafdc -mabi=ilp32d -o- -S -O3
dmul:
    fmul.d  fa0,fa1,fa0
    ret

```

The third combination is for users who may want to generate code that can be linked with code designed for systems that don't subsume a particular extension while still taking advantage of the extra instructions present in a particular extension. This is a common problem when dealing with legacy libraries that need to be integrated into newer systems. For this purpose, the compiler arguments and multilib paths designed to cleanly integrate with this workflow. The generated code is essentially a mix between the two above outputs: the arguments are passed in the registers specified by the `ilp32` ABI (as opposed to the `ilp32d` ABI, which could pass these arguments in registers) but then once inside the function the compiler is free to use the full power of the RV32IMAFDC ISA to actually compute the result. While this is less efficient than the code the compiler could generate if it was allowed to take full advantage of the D-extension registers, it's a lot more efficient than computing the floating-point multiplication without the D-extension instructions

```

$ riscv64-unknown-elf-gcc test.c -march=rv32imafdc -mabi=ilp32 -o- -S -O3
dmul:
    add     sp,sp,-16
    sw      a0,8(sp)
    sw      a1,12(sp)
    fld     fa5,8(sp)
    sw      a2,8(sp)
    sw      a3,12(sp)
    fld     fa4,8(sp)
    fmul.d  fa5,fa5,fa4
    fsd     fa5,8(sp)
    lw      a0,8(sp)
    lw      a1,12(sp)
    add     sp,sp,16
    jr      ra

```

5.17 Compilation Process

GCC driver script is actually running the preprocessor, then the compiler, then the assembler and finally the linker. If the user runs GCC with the `--save-temps` argument, several intermediate files will be generated.

```
$ riscv64-unknown-linux-gnu-gcc relocation.c -o relocation -O3 --save-temps
```

- `relocation.i`: The preprocessed source, which expands any preprocessor directives (things like `#include` or `#ifdef`).
- `relocation.s`: The output of the actual compiler, which is an assembly file (a text file in the RISC-V assembly format).
- `relocation.o`: The output of the assembler, which is an un-linked object file (an ELF file, but not an executable ELF).
- `relocation`: The output of the linker, which is a linked executable (an executable ELF file).

5.18 Large Code Model Workarounds

RISC-V software currently requires that linked symbols reside within a 32-bit range. There are two types of code models defined for RISC-V, **medlow** and **medany**. The **medany** code model generates `auipc/ld` pairs to refer to global symbols, which allows the code to be linked at any address, while **medlow** generates `lui/ld` pairs to refer to global symbols, which restricts the code to be linked around address zero. They both generate 32-bit signed offsets for referring to symbols, so they both restrict the generated code to being linked within a 2 GiB window. When building software, the code model parameter is passed into the RISC-V toolsuite and it defines a method to generate the necessary instruction combinations to access global symbols within the software program. This is done using `-mcode1=medany/medlow`. For 32-bit architectures, we use the **medlow** code model, while **medany** is used for 64-bit architectures. This is controlled within the 'setting.mk' file in the `freedom-e-sdk/bsp` folder.

The real problem occurs when:

1. Total program size exceeds 2 GiB, which is rare
2. When global symbols within a single compiled image are required to reside in a region outside of the 32-bit space

Example for symbols within 32-bit address space:

```
MEMORY
{
  ram (wxa!ri) : ORIGIN = 0x80000000, LENGTH = 0x4000
  flash (rxai!w) : ORIGIN = 0x20400000, LENGTH = 0x1fc00000
}
```

Example for symbols outside 32-bit address space:

```
MEMORY
```

```
{
ram (wxa!ri) : ORIGIN = 0x100000000, LENGTH = 0x4000 /* Updated ORIGIN from
0x80000000 */
flash (rxai!w) : ORIGIN = 0x20400000, LENGTH = 0x1fc00000
}
```

If a software example uses the above memory map, and uses either medlow or medany code models, it will not link successfully. Generated errors will generally contain the following phrase:

relocation truncated to fit:

A workaround for the linker error “relocation truncated to fit:” is to use `LINK_TARGET=scratchpad` since both the code and data sections get placed into the ram section, as defined by the linker script. Note that this doesn't always solve the problem, since some designs do not have enough memory allocated to the ram section to fit the compiled software example. To solve these cases, SiFive provides support for the compact code model.

5.18.1 RISC-V Code Model Summary

	Medlow	Medany	Compact
Code	Small	Small	Small
Data	Small	Small	Small
Distance	< 4GB	< 2GB (PC to GOT)	No limitation
Address	4GB (absolute)	No limitation	No limitation

Table 90: RISC-V Code Model Table

As shown in the above table, the compact code model option has no limits on the base address, or the distance between, the code and data sections.

5.18.2 Enabling the Compact Code Model

To enable the large code model, follow the steps below:

1. Enable compact code model in settings.mk file to use `RISCV_CMODEL = compact` instead of `RISCV_CMODEL = medany`.
2. Update assembly files to use new instructions if `__riscv_cmodel_compact` is defined by the toolsuite.
3. Update linker alignment.
4. Use the latest GCC+LLVM toolsuite from SiFive, starting with 2021.06.2.

Makefile Update

To enable the toolsuite to generate the proper code sequences for the compact code model, first update settings.mk file, which can be found in the board support package (BSP) path, similar to `freedom-e-sdk/bsp/design-rtl`.

Change: `RISCV_CMODEL = medany` $\Rightarrow$ `RISCV_CMODEL = compact`

The `RISCV_CMODEL` definition gets passed into the toolsuite using the `-mcmode1` switch.

Note

For 32-bit designs, which do not require the use of the compact code model, you will find `RISCV_CMODEL = medlow` in `settings.mk`.

This `-mcmode1=compact` option will enable the symbol `__riscv_cmodel_compact` to be visible within the code, and can be used to determine the correct code sequences to use within assembly files.

Assembly File Updates

Assembly files may need to be hand-edited to support the compact code model if they reference a global symbol. The following freedom-metal source files now support the compact code model option:

1. `freedom-metal/src/entry.S`
2. `freedom-metal/src/scrub.S`
3. `freedom-metal/gloss/crt0.S`

Linker Alignment

The global pointer alignment is required to be: `PROVIDE( __global_pointer$ = ALIGN . + 0x800 ), 16;`

See `metal.default.lds`, or the `*.lds` you plan to use.

5.19 Pipeline Hazards

The pipeline only interlocks on read-after-write and write-after-write hazards, so instructions may be scheduled to avoid stalls.

5.19.1 Read-After-Write Hazards

Read-after-Write (RAW) hazards occur when an instruction tries to read a register before a preceding instruction tries to write to it. This hazard describes a situation where an instruction refers to a result that has not been calculated or retrieved. This situation is possible because even though an instruction was executed after a prior instruction, the prior instruction may only have processed partly through the core pipeline.

Example:

- Instruction 1: $x1 + x3$ is saved in $x2$
- Instruction 2: $x2 + x3$ is saved in $x4$

The first instruction is calculating a value ($x1 + x3$) to be saved in $x2$. The second instruction is going to use the value of $x2$ to compute a result to be saved in $x4$. However, in the core pipeline, when operations are fetched for the second operation, the results from the first operation have not yet been saved.

5.19.2 Write-After-Write Hazards

Write-after-write (WAW) hazards occur when an instruction tries to write an operand before it is written by a preceding instruction.

Example:

- Instruction 1: $x4 + x7$ is saved in $x2$
- Instruction 2: $x1 + x3$ is saved in $x2$

Write-back of instruction 2 must be delayed until instruction 1 finishes executing.

In general, MMIO accesses stall when there is a hazard on the result caused by either RAW or WAW. So, instructions may be scheduled to avoid stalls.

5.20 Reading CSRs

There are several methods for reading the CSRs that are implemented in the P550 Core Complex. A full list of the defined RISC-V CSRs are described in Section 5.9.2.

1. Inline assembly using `csrr` instruction and the register name. For example, reading the `misa` CSR:

```
int misa;
__asm__ volatile("csrr %0, misa" : "=r" (misa));
```

2. Using the Freedom Metal API `METAL_CPU_GET_CSR`. Again, reading the `misa` CSR:

```
int misa_value;
METAL_CPU_GET_CSR(misa, misa_value);
```

In the second method, the first argument is the register name and the second is the variable to store the result in.

Both inline assembly and Freedom Metal API methods can receive the CSR number instead of its name. For example:

```
int mscratch;  
METAL_CPU_GET_CSR(0x340, mscratch_value); // reading mscratch csr
```

Note

Accessing CSRs has to be according to the privilege level you are in. Attempting to access a CSR in a privilege level higher than the current level of operation will result in an exception.

To access a privileged CSR, the user must switch to the appropriate privilege level. This can be done using the following Freedom Metal API:

```
metal_privilege_drop_to_mode(METAL_PRIVILEGE_USER,  
                             my_regfile,  
                             user_mode_entry_point);
```

The Freedom Metal API routines and more examples located in `freedom-e-sdk/software` directory.

Chapter 6

Custom Instructions and CSRs

These custom instructions use the SYSTEM instruction encoding space, which is the same as the custom CSR encoding space, but with funct3=0.

6.1 CFLUSH.D.L1

- Implemented as state machine in L1 data cache, for cores with data caches.
- Only available in M-mode.
- Opcode 0xFC000073, with optional *rs1* field in bits [19:15].
- When *rs1* = x0, CFLUSH.D.L1 writes back and invalidates all lines in the L1 data cache.
- When *rs1* ≠ x0, CFLUSH.D.L1 writes back and invalidates the L1 data cache line containing the virtual address in integer register *rs1*.
- If the effective privilege mode does not have write permissions to the address in *rs1*, then a store access or store page-fault exception is raised.
- If the address in *rs1* is in an uncacheable region with write permissions, the instruction has no effect but raises no exceptions.
- Note that if the PMP scheme write-protects only part of a cache line, then using a value for *rs1* in the write-protected region will cause an exception, whereas using a value for *rs1* in the write-permitted region will write back the entire cache line.

6.2 CDISCARD.D.L1

- Implemented as state machine in L1 data cache, for cores with data caches.
- Only available in M-mode.
- Opcode 0xFC200073, with optional *rs1* field in bits [19:15].
- When *rs1* = x0, CDISCARD.D.L1 invalidates, but does not write back, all lines in the L1 data cache. Dirty data within the cache is lost.
- When *rs1* ≠ x0, CDISCARD.D.L1 invalidates, but does not write back, the L1 data cache line containing the virtual address in integer register *rs1*. Dirty data within the cache line is lost.

- If the effective privilege mode does not have write permissions to the address in `rs1`, then a store access or store page-fault exception is raised.
- If the address in `rs1` is in an uncacheable region with write permissions, the instruction has no effect but raises no exceptions.
- Note that if the PMP scheme write-protects only part of a cache line, then using a value for `rs1` in the write-protected region will cause an exception, whereas using a value for `rs1` in the write-permitted region will invalidate and discard the entire cache line.

6.3 CEASE

- Privileged instruction only available in M-mode.
- Opcode `0x30500073`.
- After retiring CEASE, hart will not retire another instruction until reset.
- Instigates power-down sequence, which will eventually raise the `cease_from_tile_N` signal to the outside of the Core Complex, indicating that it is safe to power down.
- CEASE has no effect on System Bus Access.
- Debug `haltreq` will not work after a CEASE instruction has retired.

6.4 PAUSE

- Opcode `0x0100000F`, which is a FENCE instruction with predecessor set `W` and null successor set. Therefore, PAUSE is a HINT instruction that executes as a no-op on all RISC-V implementations.
- This instruction may be used for more efficient idling in spin-wait loops.
- This instruction causes a stall of up to 32 cycles or until a cache eviction occurs, whichever comes first.

6.5 Branch Prediction Mode CSR

This SiFive custom extension adds an M-mode CSR to control the current branch prediction mode, `bpm` at CSR `0x7C0`.

The P550 Core Complex's branch prediction system includes a Return Address Stack (RAS), a Branch Target Buffer (BTB), and a Branch History Table (BHT). While branch predictors are essential to achieve high performance in pipelined processors, they can also cause undesirable timing variability for hard real-time systems. The `bpm` register provides a means to customize the branch predictor behavior to trade average performance for a more predictable execution time.

Branch Prediction Mode CSR (bpm)			
CSR	0x7C0		
Bits	Field Name	Attr.	Description
0	bdp	WARL	Branch-Direction Prediction. Determines the value returned by the BHT component of the branch prediction system. A zero value indicates dynamic direction prediction, and a non-zero value indicates static-taken direction prediction. The BTB is cleared on any write to bdp, and the RAS is unaffected by writes to bdp.
[63:1]	Reserved	RO	

Table 91: Branch Prediction Mode CSR

6.6 SiFive Feature Disable CSRs

The SiFive custom M-mode Feature Disable CSRs are provided to enable or disable certain microarchitectural features. In the P550 Core Complex, CSRs 0x7C1 and 0x7C2 have been allocated for this purpose. These features are described in Table 92 and Table 94.

Warning

The features that can be controlled by these CSRs are subject to change or removal in future releases. It is not advised to depend on these CSRs for development.

A feature is fully enabled when the associated bit is zero. If a particular core does not support the disabling of a feature, the corresponding bit is hardwired to zero.

On reset, all implemented bits are set to 1, disabling all features. The bootloader is responsible for turning on all required features and can simply write zero to turn on the maximal set of features. SiFive's Freedom Metal bootloader handles turning on these features; when using a custom bootloader, clearing the Feature Disable CSRs must be implemented.

Note that arbitrary toggling of the Feature Disable CSR bits is neither recommended nor supported; they are only intended to be set from 1 to 0. A particular Feature Disable CSR bit is only to be used in a very limited number of situations, as detailed in the **Example Usage** entry in Table 93.

Feature Disable CSR 1	
CSR	0x7C1
Bit	Description
0	Disable data cache clock gating
1	Disable instruction cache clock gating
2	Reserved
3	Disable integer pipeline clock gating
4	Disable floating-point pipeline clock gating
5	Disable load/store unit clock gating
[9:6]	Reserved
10	Disable allowing prefetches to fill miss queue
11	Only allow a single outstanding instruction cache miss
12	Disable instruction cache prefetcher
13	Disable RAS
14	Disable IJTP
15	Disable next-line predictor
16	Disable IJTP hint power optimization
17	Only decode up to one instruction per cycle
18	Disable issue queue bypass
19	Disable RAS forwarding to next-line prefetcher
20	Re-read instruction cache tag array on every fetch
21	Disable speculative instruction cache refill
22	Disable tagged tables of branch direction predictor
23	Disable prediction skip
[29:24]	Reserved
30	Disable aging out MDP usefulness counters
31	Disable MDP training for unnecessarily synchronized single-set loads
32	Disable MDP training for unnecessarily synchronized loads
33	Disable MDP training for MDP predicted dependencies
34	Disable training and lookup of memory dependence predictor
35	Disable updates of branch direction predictor before retirement
36	Assume all instructions may require PCMT allocation. Potential slight performance impact.
37	All memory-type instructions will start and end an ROB group
38	All floating-point instructions will start and end an ROB group
39	All branch and jump instructions will start and end an ROB group
40	Disable all ROB grouping
41	Disable cross-issue feature
42	Disable move elimination
43	Force single-stepping behavior
44	Disable conditional move conversion
45	Disable instruction fusion
46	Disable RDU clock gating

Table 92: SiFive Feature Disable CSR 1

[49:45]	Reserved
50	Disable early issue queue release for integer pipeline
51	Disable early issue of new divide while current divide execution is finishing for integer pipeline
[56:52]	Reserved
57	Disable early issue queue release for floating-point pipeline
[63:58]	Reserved

Table 92: SiFive Feature Disable CSR 1

Feature Disable CSR 1 Usage	
Bit	Description / Usage
21	Disable speculative instruction cache refill Example Usage: A particular integration might require that execution from the System Port range be disallowed. Startup code would first configure PMP to prevent execution from the System Port range, followed by clearing bit 21 of the Feature Disable CSR 1. This would enable speculative instruction cache refill accesses, without allowing those to access the System Port range because PMP would prohibit such accesses.

Table 93: SiFive Feature Disable CSR 1 Usage

Feature Disable CSR 2	
CSR	0x7C2
Bit	Description
0	Disable reporting L2 corrupt or denied errors
1	Disable reporting L1 data cache data array uncorrectable ECC errors
2	Disable reporting L1 data cache data array correctable ECC errors
3	Disable reporting L1 data cache tag array uncorrectable ECC errors
4	Disable reporting L1 data cache tag array correctable ECC errors
5	Force SFENCE to target all ASIDs
6	Force SFENCE to target all addresses
7	Force Noisy Evict to send Release message from any Valid coherence permission state
8	Force Evict Release to always downgrade coherence permission to Invalidate
9	Force Snoop Probe to always downgrade coherence permission to Invalidate
10	Reserved
11	Disable requesting L2 AcquirePerm.toT
12	Reserved
13	Disable pipelining multiple MMIO reads/writes
14	Force all FENCES upgrade to RWIO, and all AMO/LR/SC to SC=AQ+RL
15	Disable MSHR Cleaning timer and Keeper threshold
16	Disable speculative L2 accesses
17	Disable L2 request fast-path
18	Reserved
19	Disable MSHR reuse by multiple accesses
20	Sequential Consistency Memory Ordering
21	Disable Store-to-Load Forwarding from adjacent stage (LSTM+SDFP to LSTM+LDR)
22	Disable Store-to-Load Forwarding from stage ahead (LSTO+SDO to LSTM+LDR)
23	Disable Store-to-Load Forwarding to LSTM+LDR
24	Disable Store-to-Load Forwarding
25	Disable store queue write gathering
26	Disable load data return for potential read-after-write hazard
27	Disable load data return for potential read-after-read hazard
28	Disable load data return for known read-after-read hazard
29	Disable load queue request data return fast-path
30	Disable l2 data return fast-path
31	Force non-speculative accesses (AMO/LR/SC) to return data from the load queue
32	Force loads to return data from the load queue
33	Force loads to resolve from the load queue
34	Force loads to wait to resolve until returning data, to enable precise error reporting on loads
35	Force hazards to conservatively match between all loads and stores

Table 94: SiFive Feature Disable CSR 2

36	Disable PTW waiting after flushing newer instructions
37	Reserved
38	Disable 3-cycle fast load forward to store data
39	Disable 3-cycle fast load forward to load/store address
40	Disable 4-cycle fast load forward from LDF stage
41	Reserved
42	Force self-replay operation to flush newer operations
43	Force self-replay operation to flush itself and newer operations
44	Force AMOs to the L2 Cache
45	Force L1 data cache allocation to way 0
46	Force non-allocating L1 data cache, except for AMO/LR/SC
47	Force store data issue to wait until store address will not replay
48	Allow issuing more store address operations than there are store data pipes
49	Force loads from the stack to wait before issuing for all older stores to the stack
50	Force in-order operations to wait before issuing until all older memory operations will not replay
51	Reserved
52	Force in-order operations to wait before issuing until all older operations are committed and retired
53	Force all memory operations to issue in-order
54	Force all memory operations to single issue
55	L1 allocate for vector store
56	L1 allocate for floating-point store
57	L1 allocate for scalar integer store
58	L1 allocate for stores to the stack
[63:59]	Reserved

Table 94: SiFive Feature Disable CSR 2

6.7 Other Custom Instructions

Other custom instructions may be implemented, but their functionality is not documented further here, and they should not be used in this version of the P550 Core Complex.

Chapter 7

Interrupts and Exceptions

This chapter describes how interrupt and exception concepts in the RISC-V architecture apply to the P550 Core Complex.

7.1 Interrupt Concepts

Interrupts are *asynchronous* events that cause program execution to change to a specific location in the software application to handle the interrupting event. When processing of the interrupt is complete, program execution resumes back to the original program execution location. For example, a timer that triggers every 10 milliseconds will cause the CPU to branch to the interrupt handler, acknowledge the interrupt, and set the next 10 millisecond interval.

The P550 Core Complex supports machine mode and supervisor mode interrupts. By default, all interrupts are handled in machine mode. For harts that support supervisor mode, it is possible to selectively delegate interrupts to supervisor mode.

The Core Complex also has support for the following types of RISC-V interrupts: local and global. Local interrupts are signaled directly to an individual hart with a dedicated interrupt exception code and fixed priority. This allows for reduced interrupt latency as no arbitration is required to determine which hart will service a given request and no additional memory accesses are required to determine the cause of the interrupt. Software and timer interrupts are local interrupts generated by the Core-Local Interruptor (CLINT). The P550 Core Complex contains no other local interrupt sources.

Global interrupts are routed through a Platform-Level Interrupt Controller (PLIC), which can direct interrupts to any hart in the system via the external interrupt. Decoupling global interrupts from the hart allows the design of the PLIC to be tailored to the platform, permitting a broad range of attributes like the number of interrupts and the prioritization and routing schemes.

Chapter 8 describes the CLINT. Chapter 9 describes the global interrupt architecture and the PLIC design.

7.2 Exception Concepts

Exceptions are different from interrupts in that they typically occur *synchronously* to the instruction execution flow, and most often are the result of an unexpected event that results in the pro-

gram to enter an exception handler. For example, if a hart is operating in supervisor mode and attempts to access a machine mode only Control and Status Register (CSR), it will immediately enter the exception handler and determine the next course of action. The exception code in the `mstatus` register will hold a value of 0x2, showing that an illegal instruction exception occurred. Based on the requirements of the system, the supervisor mode application may report an error and/or terminate the program entirely.

There are no specific enable bits to allow exceptions to occur since they are always enabled by default. However, early in the boot flow, software should set up `mtvec.BASE` to a defined value, which contains the base address of the default exception handler. All exceptions will trap to `mtvec.BASE`. Software must read the `mcause` CSR to determine the source of the exception, and take appropriate action.

Synchronous exceptions that occur from within an interrupt handler will immediately cause program execution to abort the interrupt handler and enter the exception handler. Exceptions within an interrupt handler are usually the result of a software bug and should generally be avoided since `mepc` and `mcause` CSRs will be overwritten from the values captured in the original interrupt context.

The RISC-V defined synchronous exceptions have a priority order which may need to be considered when multiple exceptions occur simultaneously from a single instruction. Table 95 describes the synchronous exception priority order.

Priority	Interrupt Exception Code	Description
<i>Highest</i>	3	Instruction Address Breakpoint
	12	Instruction page fault
	1	Instruction access fault
	2	Illegal instruction
	0	Instruction address misaligned
	8, 9, 11	Environment call
	3	Environment break
	3	Load/Store/AMO address breakpoint
	6	Store/AMO address misaligned
	4	Load address misaligned
	15	Store/AMO page fault
	13	Load page fault
<i>Lowest</i>	7	Store/AMO access fault
	5	Load access fault

Table 95: Exception Priority

Refer to Table 103 for the full table of interrupt exception codes.

Data address breakpoints (watchpoints), Instruction address breakpoints, and environment break exceptions (EBREAK) all have the same Exception code (3), but different priority, as shown in the table above.

Instruction address misaligned exceptions (0x0) have lower priority than other instruction address exceptions because they are the result of control-flow instructions with misaligned targets, rather than from instruction fetch.

Some of the helpful CSRs for debugging exceptions and interrupts are described below:

CSR	Description
exception	SiFive Scope signal. Indicates the moment that an exception occurs in the write-back (commit) stage.
mcause	Contains the cause value of the exception/interrupt. See Section 7.7.5 for more description.
mepc	Contains the pc where the exception occurs.
mtval	If the cause is a load/store fault, this register has the value of the problematic address. If it is an invalid instruction, it provides the instruction that the core tried to execute.
mstatus	Contains the interrupt enables, privilege modes, and general status of execution. See Section 7.7.1 for more description.
mtvec	Contains the vector that the core will jump to when an exception occurs. If this is not a valid executable value, you may get a double exception when jumping to the exception handler, so it is important to look at all these registers when the exception FIRST occurs. See Section 7.7.2 for more description.

Table 96: Summary of Exception and Interrupt CSRs

7.3 Trap Concepts

The term trap describes the transfer of control in a software application, where trap handling typically executes in a more privileged environment. For example, a particular hart contains three privilege modes: machine, supervisor, and user. Each privilege mode has its own software execution environment including a dedicated stack area. Additionally, each privilege mode contains separate control and status registers (CSRs) for trap handling. While operating in User mode, a context switch is required to handle an event in Supervisor mode. The software sets up the system for a context switch, and then an ECALL instruction is executed which synchronously switches control to the Environment call-from-User mode exception handler.

The default mode out of reset is Machine mode. Software begins execution at the highest privilege level, which allows all CSRs and system resources to be initialized before any privilege level changes. The steps below describe the required steps necessary to change privilege mode from machine to user mode, on a particular design that also includes supervisor mode.

1. Interrupts should first be disabled globally by writing `mstatus.MIE` to 0, which is the default reset value.
2. Write `mtvec` CSR with the base address of the Machine mode exception handler. This is a required step in any boot flow.
3. Write `mstatus.MPP` to 0 to set the previous mode to User which allows us to *return* to that mode.

4. Setup the Physical Memory Protection (PMP) regions to grant the required regions to user and supervisor mode, and optionally, revoke permissions from machine mode.
5. Write `stvec` CSR with the base address of the supervisor mode exception handler.
6. Write `medeleg` register to delegate exceptions to supervisor mode. Consider ECALL and page fault exceptions.
7. Write `mstatus.FS` to enable floating-point (if supported).
8. Store machine mode user registers to stack or to an application specific frame pointer.
9. Write `mepc` with the entry point of user mode software
10. Execute `mret` instruction to enter user Mode.

Note

There is only one set of user registers (x1 - x31) that are used across all privilege levels, so application software is responsible for saving and restoring state when entering and exiting different levels.

7.4 Interrupt Block Diagram

The P550 Core Complex interrupt architecture is depicted in Figure 109.

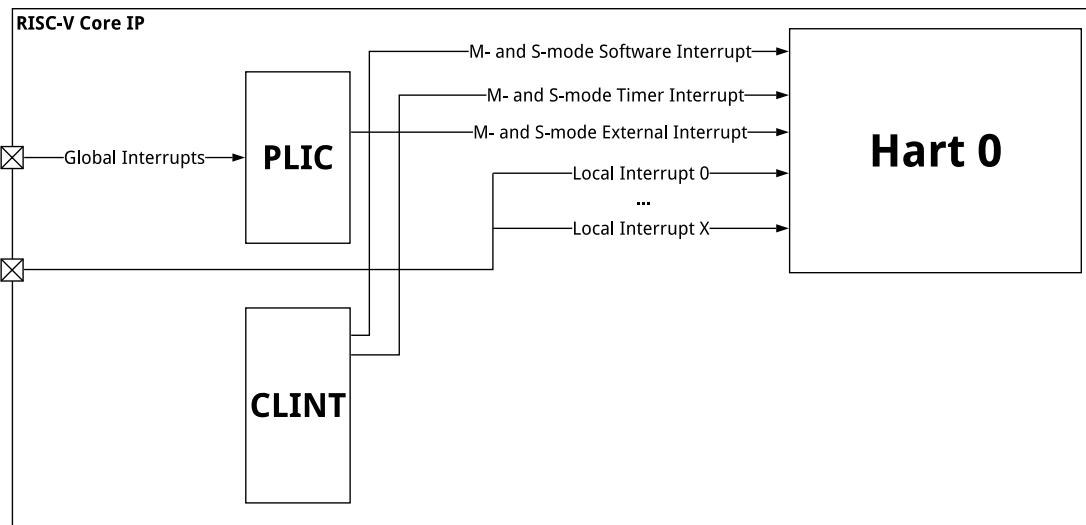


Figure 109: P550 Core Complex Interrupt Architecture Block Diagram

7.5 Local Interrupts

Software interrupts (Interrupt ID #3) are triggered by writing the memory-mapped interrupt pending register `msip` for a particular hart. The `msip` register is described in Table 101.

Timer interrupts (Interrupt ID #7) are triggered when the memory-mapped register `mtime` is greater than or equal to the global timebase register `mtimecmp`, and both registers are part of the CLINT memory map. The `mtime` and `mtimecmp` registers are generally only available in machine mode, unless the PMP grants user or supervisor mode access to the memory-mapped region in which they reside.

Global interrupts are usually first routed to the PLIC, then into the hart using external interrupts (Interrupt ID #11).

7.6 Interrupt Operation

Within a privilege mode *m*, if the associated global interrupt-enable `{ie}` is clear, then no interrupts will be taken in that privilege mode, but a pending-enabled interrupt in a higher privilege mode will preempt current execution. If `{ie}` is set, then pending-enabled interrupts at a higher interrupt level in the same privilege mode will preempt current execution and run the interrupt handler for the higher interrupt level.

When an interrupt or synchronous exception is taken, the privilege mode is modified to reflect the new privilege mode. The global interrupt-enable bit of the handler's privilege mode is cleared.

7.6.1 Interrupt Entry and Exit

When an interrupt occurs:

- The value of `mstatus.MIE` is copied into `mcause.MPIE`, and then `mstatus.MIE` is cleared, effectively disabling interrupts.
- The privilege mode prior to the interrupt is encoded in `mstatus.MPP`.
- The current `pc` is copied into the `mepc` register, and then `pc` is set to the value specified by `mtvec` as defined by the `mtvec.MODE` described in Table 99.

At this point, control is handed over to software in the interrupt handler with interrupts disabled. When an `mret` instruction is executed, the following occurs:

- The privilege mode is set to the value encoded in `mstatus.MPP`.
- The global interrupt enable, `mstatus.MIE`, is set to the value of `mcause.MPIE`.
- The `pc` is set to the value of `mepc`.

At this point, control is handed over to software.

At the software level, interrupt attributes can be applied to interrupt processing functions, as described in Section 8.4.

The Control and Status Registers (CSRs) involved in handling RISC-V interrupts are described in Section 7.7.

7.7 Interrupt Control and Status Registers

The P550 Core Complex specific implementation of interrupt CSRs is described below. For a complete description of RISC-V interrupt behavior and how to access CSRs, please consult *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

7.7.1 Machine Status Register (mstatus)

The mstatus register keeps track of and controls the hart's current operating state, including whether or not interrupts are enabled. A summary of the mstatus fields related to interrupts in the P550 Core Complex is provided in Table 97. Note that this is not a complete description of mstatus as it contains fields unrelated to interrupts. For the full description of mstatus, please consult *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

Machine Status Register (mstatus)			
CSR	0x300		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SIE	RW	Supervisor Interrupt Enable
2	Reserved	WPRI	
3	MIE	RW	Machine Interrupt Enable
4	Reserved	WPRI	
5	SPIE	RW	Supervisor Previous Interrupt Enable
6	Reserved	WPRI	
7	MPIE	RW	Machine Previous Interrupt Enable
8	SPP	RW	Supervisor Previous Privilege Mode
[10:9]	Reserved	WPRI	
[12:11]	MPP	RW	Machine Previous Privilege Mode

Table 97: Machine Status Register (partial)

Interrupts are enabled by setting the MIE bit in mstatus. Prior to writing mstatus.MIE=1, it is recommended to first enable interrupts in mie.

7.7.2 Machine Trap Vector (mtvec)

The mtvec register has two main functions: defining the base address of the trap vector, and setting the mode by which the P550 Core Complex will process interrupts. For Direct and Vectored modes, the interrupt processing mode is defined in the MODE field of the mtvec register. The mtvec register is described in Table 98, and the mtvec.MODE field is described in Table 99.

Machine Trap Vector Register (mtvec)			
CSR	0x305		
Bits	Field Name	Attr.	Description
[1:0]	MODE	WARL	MODE Sets the interrupt processing mode. The encoding for the P550 Core Complex supported modes is described in Table 99.
[63:2]	BASE[63:2]	WARL	Interrupt Vector Base Address. Operating in Direct Mode requires 4-byte alignment. Operating in Vectored Mode requires 256-byte alignment.

Table 98: Machine Trap Vector Register

MODE Field Encoding mtvec.MODE		
Value	Mode	Description
0x0	Direct	All asynchronous interrupts and synchronous exceptions set pc to BASE.
0x1	Vectored	Exceptions set pc to BASE, interrupts set pc to BASE + 4 × mcause.EXCCODE.
≥0x2	Reserved	

Table 99: Encoding of mtvec.MODE

Mode Direct

When operating in direct mode, all interrupts and exceptions trap to the mtvec.BASE address. Inside the trap handler, software must read the mcause register to determine what triggered the trap. The mcause register is described in Table 102.

When operating in Direct Mode, BASE must be 4-byte aligned.

Mode Vectored

While operating in vectored mode, interrupts set the pc to mtvec.BASE + 4 × exception code (mcause.EXCCODE). For example, if a machine timer interrupt is taken, the pc is set to mtvec.BASE + 0x1C. Typically, the trap vector table is populated with jump instructions to transfer control to interrupt-specific trap handlers.

In vectored interrupt mode, BASE must be 256-byte aligned.

All machine external interrupts (global interrupts) are mapped to exception code 11. Thus, when interrupt vectoring is enabled, the pc is set to address mtvec.BASE + 0x2C for any global interrupt.

7.7.3 Machine Interrupt Enable (mie)

Individual interrupts are enabled by setting the appropriate bit in the mie register. The mie register is described in Table 100.

Machine Interrupt Enable Register (mie)			
CSR	0x304		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SSIE	RW	Supervisor Software Interrupt Enable
2	Reserved	WPRI	
3	MSIE	RW	Machine Software Interrupt Enable
4	Reserved	WPRI	
5	STIE	RW	Supervisor Timer Interrupt Enable
6	Reserved	WPRI	
7	MTIE	RW	Machine Timer Interrupt Enable
8	Reserved	WPRI	
9	SEIE	RW	Supervisor External Interrupt Enable
10	Reserved	WPRI	
11	MEIE	RW	Machine External Interrupt Enable
[63:12]	Reserved	WPRI	

Table 100: Machine Interrupt Enable Register

7.7.4 Machine Interrupt Pending (mip)

The machine interrupt pending (mip) register indicates which interrupts are currently pending. The mip register is described in Table 101.

Machine Interrupt Pending Register (mip)			
CSR	0x344		
Bits	Field Name	Attr.	Description
0	Reserved	WIRI	
1	SSIP	RW	Supervisor Software Interrupt Pending
2	Reserved	WIRI	
3	MSIP	RO	Machine Software Interrupt Pending
4	Reserved	WIRI	
5	STIP	RW	Supervisor Timer Interrupt Pending
6	Reserved	WIRI	
7	MTIP	RO	Machine Timer Interrupt Pending
8	Reserved	WIRI	
9	SEIP	RW	Supervisor External Interrupt Pending
10	Reserved	WIRI	
11	MEIP	RO	Machine External Interrupt Pending
[63:12]	Reserved	WIRI	

Table 101: Machine Interrupt Pending Register

7.7.5 Machine Cause (mcause)

When a trap is taken in machine mode, `mcause` is written with a code indicating the event that caused the trap. When the event that caused the trap is an interrupt, the most-significant bit of `mcause` is set to 1, and the least-significant bits indicate the interrupt number, using the same encoding as the bit positions in `mip`. For example, a Machine Timer Interrupt causes `mcause` to be set to `0x8000_0000_0000_0007`. `mcause` is also used to indicate the cause of synchronous exceptions, in which case the most-significant bit of `mcause` is set to 0.

See Table 102 for more details about the `mcause` register. Refer to Table 103 for a list of synchronous exception codes.

Machine Cause Register (mcause)			
CSR	0x342		
Bits	Field Name	Attr.	Description
[9:0]	EXCCODE	WLRL	A code identifying the last exception.
[62:10]	Reserved	WLRL	
63	Interrupt	WARL	1, if the trap was caused by an interrupt; 0 otherwise.

Table 102: Machine Cause Register

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2	Reserved
1	3	Machine software interrupt
1	4	Reserved
1	5	Supervisor timer interrupt
1	6	Reserved
1	7	Machine timer interrupt
1	8	Reserved
1	9	Supervisor external interrupt
1	10	Reserved
1	11	Machine external interrupt
1	12–13	Reserved
1	14	Debug interrupt
1	≥15	Reserved
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	Reserved
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	Debug
0	15	Store/AMO page fault
0	≥16	Reserved

Table 103: mcause Exception Codes

Note that there are scenarios where a misaligned load or store will generate an access exception instead of an address-misaligned exception. The access exception is raised when the misaligned access should not be emulated in a trap handler, e.g., emulating an access in an I/O region, as such emulation could cause undesirable side-effects.

7.7.6 Minimum Interrupt Configuration

The minimum configuration needed to configure an interrupt is shown below.

- Write `mtvec` to configure the interrupt mode and the base address for the interrupt vector table.
- Enable interrupts in memory mapped PLIC register space. The CLINT does not contain interrupt enable bits.
- Write `mie` CSR to enable the software, timer, and external interrupt enables for each privilege mode.
- Write `mstatus` to enable interrupts globally for each supported privilege mode.

7.8 Supervisor Mode Interrupts

The P550 Core Complex supports the ability to selectively direct interrupts and exceptions to supervisor mode, resulting in improved performance by eliminating the need for additional mode changes.

This capability is enabled by the interrupt and exception delegation CSRs; `mideleg` and `medeleg`, respectively. Supervisor interrupts and exceptions can be managed via supervisor versions of the interrupt CSRs, specifically: `stvec`, `sip`, `sie`, and `scause`.

Machine mode software can also directly write to the `sip` register, which effectively sends an interrupt to supervisor mode. This is especially useful for timer and software interrupts as it may be desired to handle these interrupts in both machine mode and supervisor mode.

The delegation and supervisor CSRs are described in the sections below. The definitive resource for information about RISC-V supervisor interrupts is *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

7.8.1 Delegation Registers (`mideleg` and `medeleg`)

By default, all traps are handled in machine mode. Machine mode software can selectively delegate interrupts and exceptions to supervisor mode by setting the corresponding bits in `mideleg` and `medeleg` CSRs. The exact mapping is provided in Table 104 and Table 105 and matches the `mcause` interrupt and exception codes defined in Table 103.

Note that local interrupts may be delegated to supervisor mode.

Machine Interrupt Delegation Register (mideleg)			
CSR	0x303		
Bits	Field Name	Attr.	Description
0	Reserved	WARL	
1	SSIP	RW	Delegate Supervisor Software Interrupt
[4:2]	Reserved	WARL	
5	STIP	RW	Delegate Supervisor Timer Interrupt
[8:6]	Reserved	WARL	
9	SEIP	RW	Delegate Supervisor External Interrupt
[63:10]	Reserved	WARL	

Table 104: Machine Interrupt Delegation Register

Machine Exception Delegation Register (medeleg)		
CSR	0x302	
Bits	Attr.	Description
0	RW	Delegate Instruction Access Misaligned Exception
1	RW	Delegate Instruction Access Fault Exception
2	RW	Delegate Illegal Instruction Exception
3	RW	Delegate Breakpoint Exception
4	RW	Delegate Load Access Misaligned Exception
5	RW	Delegate Load Access Fault Exception
6	RW	Delegate Store/AMO Address Misaligned Exception
7	RW	Delegate Store/AMO Access Fault Exception
8	RW	Delegate Environment Call from U-Mode
9	RW	Delegate Environment Call from S-Mode
[11:0]	WARL	Reserved
12	RW	Delegate Instruction Page Fault
13	RW	Delegate Load Page Fault
14	WARL	Reserved
15	RW	Delegate Store/AMO Page Fault Exception
[63:16]	WARL	Reserved

Table 105: Machine Exception Delegation Register

7.8.2 Supervisor Status Register (sstatus)

Similar to machine mode, supervisor mode has a register dedicated to keeping track of the hart's current state called `sstatus`. `sstatus` is effectively a restricted view of `mstatus`, described in Section 7.7.1, in that changes made to `sstatus` are reflected in `mstatus` and vice-versa, with the exception of the machine mode fields, which are not visible in `sstatus`.

A summary of the `sstatus` fields related to interrupts in the P550 Core Complex is provided in Table 106. Note that this is not a complete description of `sstatus` as it also contains fields unre-

lated to interrupts. For the full description of `sstatus`, consult the *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

Supervisor Status Register (<code>sstatus</code>)			
CSR	0x100		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SIE	RW	Supervisor Interrupt Enable
[4:2]	Reserved	WPRI	
5	SPIE	RW	Supervisor Previous Interrupt Enable
[7:6]	Reserved	WPRI	
8	SPP	RW	Supervisor Previous Privilege Mode
[12:9]	Reserved	WPRI	

Table 106: Supervisor Status Register (partial)

Interrupts are enabled by setting the SIE bit in `sstatus` and by enabling the desired individual interrupt in the `sie` register, described in Section 7.8.3.

7.8.3 Supervisor Interrupt Enable Register (`sie`)

Supervisor interrupts are enabled by setting the appropriate bit in the `sie` register. The P550 Core Complex `sie` register is described in Table 107.

Supervisor Interrupt Enable Register (<code>sie</code>)			
CSR	0x104		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SSIE	RW	Supervisor Software Interrupt Enable
[4:2]	Reserved	WPRI	
5	STIE	RW	Supervisor Timer Interrupt Enable
[8:6]	Reserved	WPRI	
9	SEIE	RW	Supervisor External Interrupt Enable
[63:10]	Reserved	WPRI	

Table 107: Supervisor Interrupt Enable Register

7.8.4 Supervisor Interrupt Pending (`sip`)

The supervisor interrupt pending (`sip`) register indicates which interrupts are currently pending. The P550 Core Complex `sip` register is described in Table 108.

Supervisor Interrupt Pending Register (sip)			
CSR	0x144		
Bits	Field Name	Attr.	Description
0	Reserved	WIRI	
1	SSIP	RW	Supervisor Software Interrupt Pending
[4:2]	Reserved	WIRI	
5	STIP	RW	Supervisor Timer Interrupt Pending
[8:6]	Reserved	WIRI	
9	SEIP	RW	Supervisor External Interrupt Pending
[63:10]	Reserved	WIRI	

Table 108: Supervisor Interrupt Pending Register

7.8.5 Supervisor Cause Register (scause)

When a trap is taken in supervisor mode, scause is written with a code indicating the event that caused the trap. When the event that caused the trap is an interrupt, the most-significant bit of scause is set to 1, and the least-significant bits indicate the interrupt number, using the same encoding as the bit positions in sip. For example, a Supervisor Timer Interrupt causes scause to be set to 0x8000_0000_0000_0005.

scause is also used to indicate the cause of synchronous exceptions, in which case the most-significant bit of scause is set to 0. Refer to Table 110 for a list of synchronous exception codes.

Supervisor Cause Register (scause)			
CSR	0x142		
Bits	Field Name	Attr.	Description
[62:0]	EXCCODE	WLRL	A code identifying the last exception.
63	Interrupt	WARL	1 if the trap was caused by an interrupt; 0 otherwise.

Table 109: Supervisor Cause Register

Supervisor Interrupt Exception Codes		
Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2–4	Reserved
1	5	Supervisor timer interrupt
1	6–8	Reserved
1	9	Supervisor external interrupt
1	≥10	Reserved
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Reserved
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9–11	Reserved
0	12	Instruction page fault
0	13	Load page fault
0	14	Reserved
0	15	Store/AMO Page Fault
0	≥16	Reserved

Table 110: scause Exception Codes

7.8.6 Supervisor Trap Vector (stvec)

By default, all interrupts trap to a single address defined in the `stvec` register. It is up to the interrupt handler to read `scause` and react accordingly. RISC-V and the P550 Core Complex also support the ability to optionally enable interrupt vectors. When vectoring is enabled, each interrupt defined in `sie` will trap to its own specific interrupt handler.

Vectored interrupts are enabled when the `MODE` field of the `stvec` register is set to 1.

Supervisor Trap Vector Register (stvec)			
CSR	0x105		
Bits	Field Name	Attr.	Description
[1:0]	MODE	WARL	MODE determines whether or not interrupt vectoring is enabled. The encoding for the MODE field is described in Table 112.
[63:2]	BASE[63:2]	WARL	Interrupt Vector Base Address. Must be aligned on a 128-byte boundary when MODE=1. Note, BASE[1:0] is not present in this register and is implicitly 0.

Table 111: Supervisor Trap Vector Register

MODE Field Encoding stvec.MODE		
Value	Name	Description
0x0	Direct	All exceptions and interrupts set pc to BASE
0x1	Vectored	Exceptions set pc to BASE, interrupts set pc to BASE + 4 × scause.EXCCODE
≥0x2	Reserved	

Table 112: Encoding of stvec.MODE

If vectored interrupts are disabled (stvec.MODE=0), all interrupts trap to the stvec.BASE address. If vectored interrupts are enabled (stvec.MODE=1), interrupts set the pc to stvec.BASE + 4 × exception code (scause.EXCCODE). For example, if a supervisor timer interrupt is taken, the pc is set to stvec.BASE + 0x14. Typically, the trap vector table is populated with jump instructions to transfer control to interrupt-specific trap handlers.

In vectored interrupt mode, BASE must be 128-byte aligned.

All supervisor external interrupts (global interrupts) are mapped to exception code of 9. Thus, when interrupt vectoring is enabled, the pc is set to address stvec.BASE + 0x24 for any global interrupt.

See Table 111 for a description of the stvec register. See Table 112 for a description of the stvec.MODE field. See Table 110 for the P550 Core Complex supervisor mode interrupt exception code values.

7.8.7 Delegated Interrupt Handling

Upon taking a delegated trap, the following occurs:

- The value of sstatus.SIE is copied into sstatus.SPIE, then sstatus.SIE is cleared, effectively disabling interrupts.

- The current pc is copied into the sepc register, and then pc is set to the value of stvec. In the case where vectored interrupts are enabled, pc is set to stvec.BASE + 4 × exception code (scause.EXCCODE).
- The privilege mode prior to the interrupt is encoded in sstatus.SPP.

At this point, control is handed over to software in the interrupt handler with interrupts disabled. Interrupts can be re-enabled by explicitly setting sstatus.SIE or by executing an SRET instruction to exit the handler. When an SRET instruction is executed, the following occurs:

- The privilege mode is set to the value encoded in sstatus.SPP.
- The value of sstatus.SPIE is copied into sstatus.SIE.
- The pc is set to the value of sepc.

At this point, control is handed over to software.

7.9 Interrupt Priorities

Individual priorities of global interrupts are determined by the PLIC, as discussed in Chapter 9.

P550 Core Complex interrupts are prioritized as follows, in decreasing order of priority:

- Machine external interrupts
- Machine software interrupts
- Machine timer interrupts
- Supervisor external interrupts
- Supervisor software interrupts
- Supervisor timer interrupts

7.10 Interrupt Latency

Interrupt latency for the P550 Core Complex is four external_source_for_core_N_clock cycles, as counted by the number of cycles it takes from signaling of the interrupt to the hart to the first instruction fetch of the handler.

Global interrupts routed through the PLIC incur additional latency of three clock cycles, where the PLIC is clocked by clock. This means that the total latency, in cycles, for a global interrupt is: $4 + 3 \times (\text{external_source_for_core_N_clock Hz} \div \text{clock Hz})$. This is a best-case cycle count and assumes the handler is cached. It does not take into account additional latency from a peripheral source.

7.11 Non-Maskable Interrupt

The `rnmi` (resumable non-maskable interrupt) interrupt signal is a level-sensitive input to the hart. Non-maskable interrupts have higher priority than any other interrupt or exception on the hart and cannot be disabled by software. Specifically, they are not disabled by clearing the `mstatus.mie` register.

7.11.1 Handler Addresses

The NMI has an associated exception trap handler address. This address is set by external input signals, described in the P550 Core Complex User Guide.

7.11.2 RNMI CSRs

These M-mode CSRs enable a resumable non-maskable interrupt (RNMI).

Number	Name	Description
0x350	<code>mnscratch</code>	Resumable Non-maskable scratch register
0x351	<code>mnepc</code>	Resumable Non-maskable EPC value
0x352	<code>mncause</code>	Resumable Non-maskable cause value
0x353	<code>mnstatus</code>	Resumable Non-maskable status

Table 113: RNMI CSRs

- The `mnscratch` CSR holds a 64-bit read-write register, which enables the NMI trap handler to save and restore the context that was interrupted.
- The `mnepc` CSR is a 64-bit read-write register, which, on entry to the NMI trap handler, holds the PC of the instruction that took the interrupt. The lowest bit of `mnepc` is hardwired to zero.
- The `mncause` CSR holds the reason for the NMI, with bit 63 set to 1, and the NMI cause encoded in the least-significant bits, or zero if NMI causes are not supported. The lower bits of `mncause`, defined as the `exception_code`, are as follows:

<code>mncause</code>	NMI Cause	Function
1	Reserved	Reserved
2	RNMI input pin	External <code>rnmi_N</code> input
3	Bus error	RNMI caused by BEU

Table 114: `mncause.exception_code` Fields

- The `mnstatus` CSR holds a two-bit field, which, on entry to the trap handler, holds the privilege mode of the interrupted context encoded in the same manner as `mstatus.mpp`.

7.11.3 MNRET Instruction

This M-mode only instruction uses the values in `mnepc` and `mnstatus` to return to the program counter and privileged mode of the interrupted context, respectively. This instruction also sets the internal `rnmie` state bits.

Encoding is same as MRET except with bit 30 set (i.e., `funct7=0111000`). For example:

```
.word 0x70200073    // opcode for MNRET (return from RNMI)
```

7.11.4 RNMI Operation

When an RNMI interrupt is detected, the interrupted PC is written to the `mnepc` CSR, the type of RNMI to the `mncause` CSR, and the privilege mode of the interrupted context to the `mnstatus` CSR. An internal microarchitectural state bit, `rnmie`, is cleared to indicate that the processor is in an RNMI handler and cannot take a new RNMI interrupt. When clear, the internal `rnmie` bit also disables all other interrupts.

Note

These interrupts are called non-maskable because software cannot mask the interrupts. However, for correct operation, other instances of the same interrupt must be held off until the handler is completed, hence the internal state bit.

The RNMI handler can resume original execution using the MNRET instruction (described in Section 7.11.3), which restores the PC from `mnepc`, the privilege mode from `mnstatus`, and also sets the internal `rnmie` state bit, which re-enables other interrupts.

If the hart encounters an exception while the `rnmie` bit is clear, the exception state is written to `mepc` and `mcause`, `mstatus.mpp` is set to M-mode, and the hart jumps to the RNMI exception handler address.

Note

Traps in the RNMI handler can only be resumed if they occur while the handler was servicing an interrupt that occurred outside of machine mode.

Chapter 8

Core-Local Interruptor (CLINT)

This chapter describes the operation of the Core-Local Interruptor (CLINT). The P550 Core Complex CLINT complies with *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*.

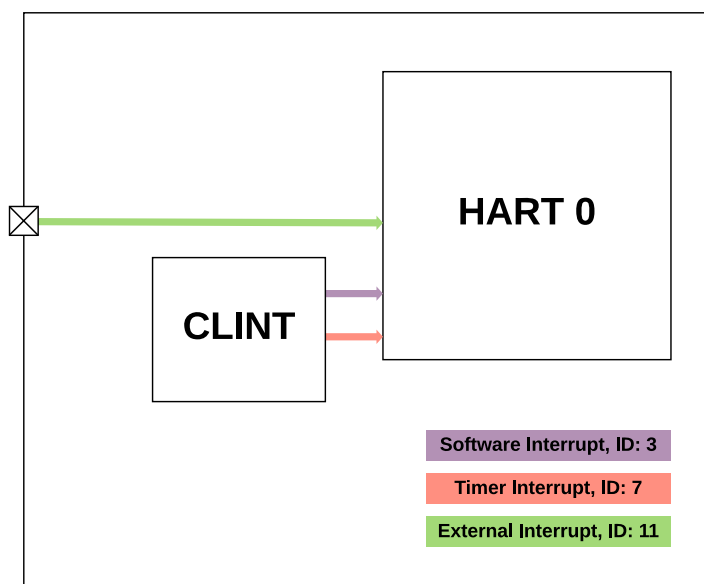


Figure 110: CLINT Block Diagram

The CLINT has a small footprint and provides software, timer, and external interrupts directly to the hart. The CLINT block also holds memory-mapped control and status registers associated with software and timer interrupts.

8.1 CLINT Priorities and Preemption

The CLINT has a fixed priority scheme, described in Section 7.9, and nested interrupts (pre-emption) within a given privilege level is not supported. Higher privilege levels may preempt lower privilege levels, however. The CLINT offers two modes of operation, Direct mode and Vectored mode.

In Direct mode, all interrupts and exceptions trap to `mtvec.BASE`. In Vectored mode, exceptions trap to `mtvec.BASE`, but interrupts will jump directly to their vector table index. See Section 7.7.2 for more information about `mtvec.BASE`.

8.2 CLINT Vector Table

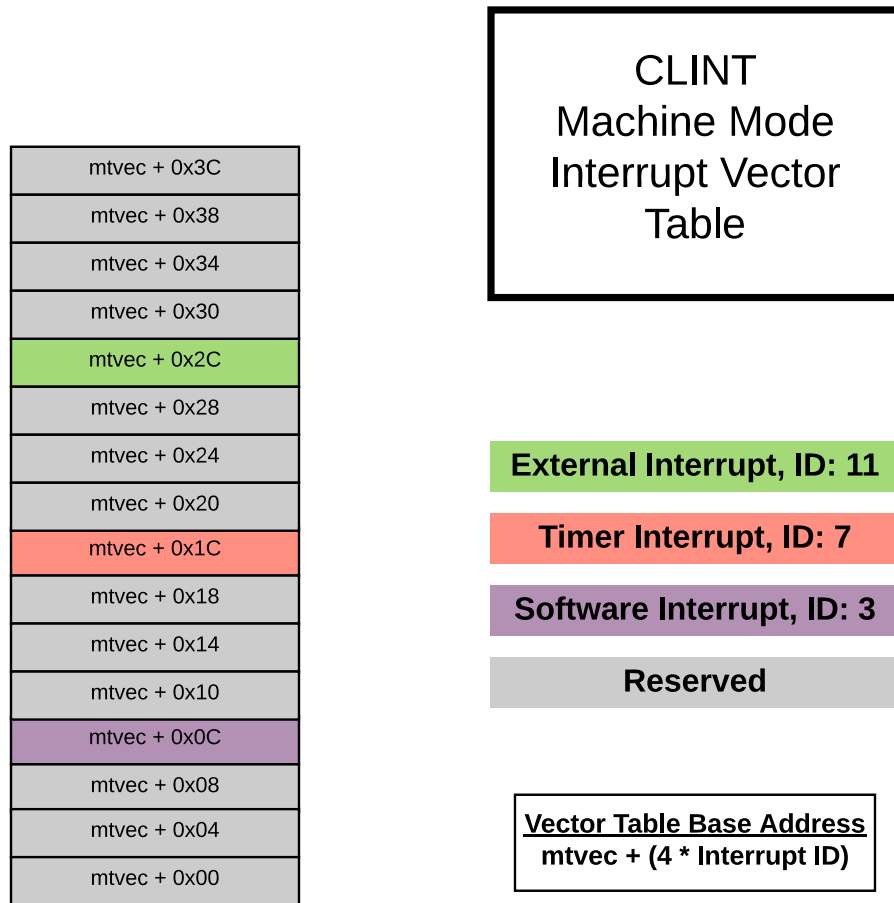


Figure 111: CLINT Interrupts and Vector Table

The CLINT vector table is populated with jump instructions, since hardware jumps to the index in the vector table first, then subsequently jumps to the handler. All exception types trap to the first entry in the table, which is `mtvec.BASE`.

An example CLINT vector table is shown below.

```
.weak default_exception_handler
.balign 4, 0
.global default_exception_handler

.weak software_handler
.balign 4, 0
.global software_handler

.weak timer_handler
.balign 4, 0
.global timer_handler

.weak external_handler
.balign 4, 0
.global external_handler

.option norvc
.weak __mtvec_clint_vector_table
#if __riscv_xlen == 32
.balign 128, 0
#else
.balign 256, 0
#endif
.global __mtvec_clint_vector_table
__mtvec_clint_vector_table:

IRQ_0:
    j default_exception_handler
IRQ_1:
    j default_vector_handler
IRQ_2:
    j default_vector_handler
IRQ_3:
    j software_handler
IRQ_4:
    j default_vector_handler
IRQ_5:
    j default_vector_handler
IRQ_6:
    j default_vector_handler
IRQ_7:
    j timer_handler
IRQ_8:
    j default_vector_handler
IRQ_9:
    j default_vector_handler
IRQ_10:
    j default_vector_handler
IRQ_11:
    j external_handler
IRQ_12:
    j default_vector_handler
IRQ_13:
    j default_vector_handler
IRQ_14:
    j default_vector_handler
IRQ_15:
    j default_vector_handler
```

Figure 112: CLINT Vector Table Example

8.3 CLINT Interrupt Sources

The P550 Core Complex supports the standard RISC-V software, timer, and external interrupts.

CLINT Interrupt IDs are provided in Table 115.

P550 Core Complex Interrupt IDs		
ID	Interrupt	Notes
0	Reserved	
1	ssip	Supervisor Software Interrupt
2	Reserved	
3	msip	Machine Software Interrupt
4	Reserved	
5	stip	Supervisor Timer Interrupt
6	Reserved	
7	mtip	Machine Timer Interrupt
8	Reserved	
9	seip	Supervisor External Interrupt
10	Reserved	
11	meip	Machine External Interrupt

Table 115: P550 Core Complex Interrupt IDs

8.4 CLINT Interrupt Attribute

To help with efficiency of save and restore context, interrupt attributes can be applied to functions used for interrupt handling.

```
void __attribute__((interrupt))
software_handler (void) {
    // handler code
}
```

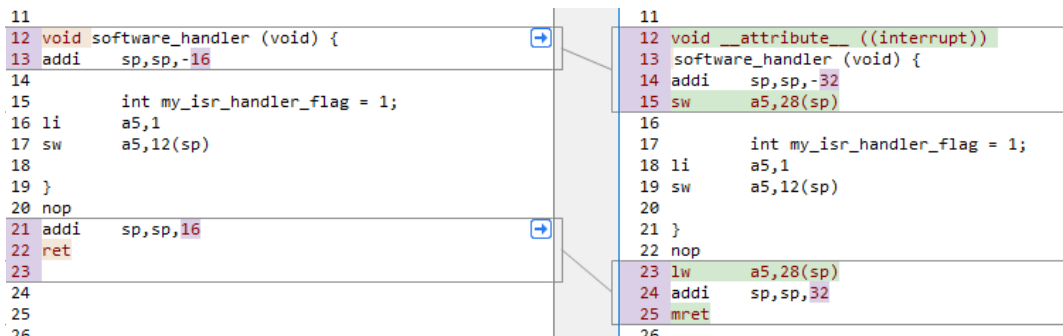


Figure 113: CLINT Interrupt Attribute Example

This attribute will save and restore registers that are used within the handler, and insert an `mret` instruction at the end of the handler.

8.5 CLINT Memory Map

Table 116 shows the memory map for CLINT on the P550 Core Complex. Note that there are no enable bits for specific interrupts within the CLINT memory map, as the enables for these interrupts reside in the `mie` CSR for each interrupt, and the `mstatus.mie` CSR bit, which enables all machine interrupts globally. See Section 7.7.3 for a description of the interrupt enable bits in the `mie` CSR, and Section 7.7.4 for a description of the interrupt pending bits in the `mip` CSR.

Address	Width	Attr.	Description	Notes
0x0200_0000	4B	RW	<code>msip</code> for hart 0	MSIP Register (1-bit wide)
0x0200_0004			Reserved	
...				
0x0200_3FFF				
0x0200_4000	8B	RW	<code>mtimecmp</code> for hart 0	MTIMECMP Register
0x0200_4008			Reserved	
...				
0x0200_BFF7				
0x0200_BFF8	8B	RW	<code>mtime</code>	Timer Register
0x0200_C000			Reserved	

Table 116: CLINT Memory Map

8.6 Register Descriptions

This section describes the functionality of the memory-mapped registers in the CLINT.

8.6.1 MSIP Registers

Machine mode software interrupts are generated by writing to the memory-mapped control register `msip`. The `msip` register is a 32-bit wide **WARL** register, where the upper 31 bits are tied to 0. The least-significant bit is reflected in the MSIP bit of the `mip` CSR. Other bits in the `msip` registers are hardwired to zero. On reset, each `msip` register is cleared to zero.

Software interrupts are most useful for interprocessor communication in multi-hart systems, as harts may write each other's `msip` bits to effect interprocessor interrupts.

8.6.2 Timer Registers

`mtime` is a 64-bit read-write register that contains the number of cycles counted from the `rtc_toggle` signal, which is described in the P550 Core Complex User Guide. A timer interrupt

is pending whenever `mtime` is greater than or equal to the value in the `mtimecmp` register. The timer interrupt is reflected in the `mtip` bit of the `mip` register, described in Chapter 7.

On reset, `mtime` is cleared to zero. The `mtimecmp` registers are not reset.

8.7 Supervisor Mode Delegation

By default, all interrupts trap to machine mode, including timer and software interrupts. In order for supervisor timer and software interrupts to trap directly to supervisor mode, supervisor timer and software interrupts must first be delegated to supervisor mode.

Please see Section 7.8 for more details on supervisor mode interrupts.

Chapter 9

Platform-Level Interrupt Controller (PLIC)

This chapter describes the operation of the Platform-Level Interrupt Controller (PLIC) on the P550 Core Complex. The PLIC complies with *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10* and can support a maximum of 132 external interrupt sources with 7 priority levels.

The P550 Core Complex PLIC resides in the `clock` timing domain, allowing for relaxed timing requirements. The latency of global interrupts, as perceived by a hart, increases with the ratio of the `external_source_for_core_N_clock` frequency and the `clock` frequency.

9.1 Memory Map

The memory map for the P550 Core Complex PLIC control registers is shown in Table 117. The PLIC memory map only supports aligned 32-bit memory accesses.

Address	Width	Attr.	Description	Notes
0x0C00_0000			Reserved	
0x0C00_0004	4B	RW	Source 1 priority	See Section 9.3 for more information
...				
0x0C00_0210	4B	RW	Source 132 priority	
0x0C00_0214			Reserved	
...				
0x0C00_1000	4B	RO	Start of pending array	See Section 9.4 for more information
...				
0x0C00_1010	4B	RO	Last word of pending array	
0x0C00_1014			Reserved	
...				
0x0C00_2000	4B	RW	Start Hart 0 M-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2010	4B	RW	End Hart 0 M-Mode interrupt enables	
0x0C00_2014			Reserved	
...				
0x0C00_2080	4B	RW	Start Hart 0 S-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2090	4B	RW	End Hart 0 S-Mode interrupt enables	
0x0C00_2094			Reserved	
...				
0x0C1F_F000	1B	RW	PLIC global clock gating disable feature	See Section 9.6 for more information
0x0C1F_F001			Reserved	
...				
0x0C20_0000	4B	RW	Hart 0 M-Mode priority threshold	See Section 9.7 for more information
0x0C20_0004	4B	RW	Hart 0 M-Mode claim/complete	See Section 9.8 for more information
0x0C20_0008			Reserved	
...				
0x0C20_1000	4B	RW	Hart 0 S-Mode priority threshold	See Section 9.7 for more information
0x0C20_1004	4B	RW	Hart 0 S-Mode claim/complete	See Section 9.8 for more information
0x0C20_1008			Reserved	
...				
0x1000_0000			End of PLIC Memory Map	

Table 117: PLIC Memory Map

9.2 Interrupt Sources

The P550 Core Complex has a total of 132 global interrupt sources, in addition to the local interrupts described in Table 115. 127 of these are external global interrupts, and the remainder are driven by various on-chip devices as listed in Table 118.

Note

In the *RISC-V Platform-Level Interrupt Controller Specification*, interrupt source 0 (ID 0) is unused, so the first usable PLIC Interrupt ID has a value of 1.

Table 118 describes the global interrupt sources on the P550 Core Complex.

PLIC Interrupt ID	Source
0	<i>Unused</i>
1	L3 Cache DirError
2	L3 Cache DirFail
3	L3 Cache DataError
4	L3 Cache DataFail
5–131	External Global Interrupts
132	Bus-Error Unit

Table 118: PLIC Interrupt Source Mapping

Table 119 describes the mapping of external global interrupts to its corresponding top-level `global_interrupts` signal bit. This signal is positive-level triggered and not configurable. See the P550 Core Complex User Guide for further description of `global_interrupts`.

<code>global_interrupts</code> Signal	PLIC Interrupt ID	PLIC Pending / Enable Register
<code>global_interrupts[0]</code>	5	<code>pending1[5]</code> / <code>enable1[5]</code>
<code>global_interrupts[1]</code>	6	<code>pending1[6]</code> / <code>enable1[6]</code>
<code>global_interrupts[2]</code>	7	<code>pending1[7]</code> / <code>enable1[7]</code>
...		
<code>global_interrupts[126]</code>	131	<code>pending5[3]</code> / <code>enable5[3]</code>

Table 119: Mapping of `global_interrupts` Signal Bits to PLIC Interrupt ID

9.3 Interrupt Priorities

Each PLIC interrupt source can be assigned a priority by writing to its 32-bit memory-mapped priority register. The P550 Core Complex supports 7 levels of priority. A priority value of 0 is reserved to mean "never interrupt" and effectively disables the interrupt. Priority 1 is the lowest active priority, and priority 7 is the highest. Ties between global interrupts of the same priority

are broken by the Interrupt ID; interrupts with the lowest ID have the highest effective priority. See Table 120 for the detailed register description.

PLIC Interrupt Priority Register (priority)				
Base Address		0x0C00_0000 + 4 × Interrupt ID		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Priority	RW	X	Global interrupt priority
[31:3]	Reserved	RO	0x0	

Table 120: PLIC Interrupt Priority Register

9.4 Interrupt Pending Bits

The current status of the interrupt source pending bits in the PLIC core can be read from the pending array, organized as 5 words of 32 bits. The pending bit for interrupt ID N is stored in bit $(N \bmod 32)$ of word $(N/32)$. As such, the P550 Core Complex has 5 interrupt pending registers. Bit 0 of word 0, which represents the non-existent interrupt source 0, is hardwired to zero.

A pending bit in the PLIC core can be cleared by setting the associated enable bit then performing a claim as described in Section 9.8.

PLIC Interrupt Pending Register 1 (pending1)				
Base Address		0x0C00_1000		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 0 Pending	RO	X	Non-existent global interrupt 0 is hardwired to zero
1	Interrupt 1 Pending	RO	X	Pending bit for global interrupt 1
2	Interrupt 2 Pending	RO	X	Pending bit for global interrupt 2
...				
31	Interrupt 31 Pending	RO	X	Pending bit for global interrupt 31

Table 121: PLIC Interrupt Pending Register 1

PLIC Interrupt Pending Register 5 (pending5)				
Base Address		0x0C00_1010		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 128 Pending	RO	X	Pending bit for global interrupt 128
...				
4	Interrupt 132 Pending	RO	X	Pending bit for global interrupt 132
[31:5]	Reserved	WIRI	X	

Table 122: PLIC Interrupt Pending Register 5

9.5 Interrupt Enables

Each global interrupt can be enabled by setting the corresponding bit in the enable registers. The enable registers are accessed as a contiguous array of 5×32 -bit words, packed the same way as the pending bits. Bit 0 of enable word 0 represents the non-existent interrupt ID 0 and is hardwired to 0.

64-bit and 32-bit word accesses are supported by the enables array in SiFive RV64 systems.

PLIC Interrupt Enable Register 1 for Hart 0 M-Mode (enable1)				
Base Address		0x0C00_2000		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 0 Enable	RO	0x0	Non-existent global interrupt 0 is hardwired to zero
1	Interrupt 1 Enable	RW	X	Enable bit for global interrupt 1
2	Interrupt 2 Enable	RW	X	Enable bit for global interrupt 2
...				
31	Interrupt 31 Enable	RW	X	Enable bit for global interrupt 31

Table 123: PLIC Interrupt Enable Register 1 for Hart 0 M-Mode

PLIC Interrupt Enable Register 5 for Hart 0 M-Mode (enable5)				
Base Address		0x0C00_2010		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 128 Enable	RW	X	Enable bit for global interrupt 128
...				
4	Interrupt 132 Enable	RW	X	Enable bit for global interrupt 132
[31:5]	Reserved	RO	0x0	

Table 124: PLIC Interrupt Enable Register 5 for Hart 0 M-Mode

9.6 PLIC Clock Gate Disable

The PLIC implements a clock gating feature to gate the module clock node when not active. PLIC clock gating is disabled out of reset and should be enabled in startup code, unless otherwise specified by SiFive erratum. Once enabled, clock is only available when there is activity on the PLIC control bus or on any interrupt line when the corresponding interrupt is not inflight. Clock gating is further described in the P550 Core Complex User Guide.

PLIC Clock Gate Disable Register (<code>disablePlicClockGateFeature</code>)				
Base Address		0x0C1F_F000		
Bits	Field Name	Attr.	Rst.	Description
0	<code>disablePlicClockGateFeature</code>	RW	0x1	Used to enable/disable PLIC clock gating feature. Clear to enable.
[7:1]	Reserved	RO	0x0	

Table 125: PLIC Clock Gate Disable Register

9.7 Priority Thresholds

The P550 Core Complex supports setting of an interrupt priority threshold via the `threshold` register. The threshold is a **WARL** field, where the P550 Core Complex supports a maximum threshold of 7.

The P550 Core Complex masks all PLIC interrupts of a priority less than or equal to `threshold`. For example, a `threshold` value of zero permits all interrupts with non-zero priority, whereas a value of 7 masks all interrupts. If the `threshold` register contains a value of 5, all PLIC interrupt configured with priorities from 1 through 5 will not be allowed to propagate to the CPU.

PLIC Interrupt Priority Threshold Register (<code>threshold</code>)				
Base Address		0x0C20_0000		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Threshold	WARL	X	Sets the priority threshold
[31:3]	Reserved	RO	0x0	

Table 126: PLIC Interrupt Priority Threshold Register

9.8 Interrupt Claim Process

A P550 Core Complex hart can perform an interrupt claim by reading the `claim_complete` register (Table 127), which returns the ID of the highest-priority pending interrupt or zero if there is no pending interrupt. A successful claim also atomically clears the corresponding pending bit on the interrupt source.

A P550 Core Complex hart can perform a claim at any time, even if the `MEIP` bit in its `mip` (Table 101) register is not set.

The claim operation is not affected by the setting of the priority threshold register.

9.9 Interrupt Completion

A P550 Core Complex hart signals it has completed executing an interrupt handler by writing the interrupt ID it received from the claim to the `claim_complete` register (Table 127). The PLIC does not check whether the completion ID is the same as the last claim ID for that target. If the completion ID does not match an interrupt source that is currently enabled for the target, the completion is silently ignored.

PLIC Claim/Complete Register for Hart 0 M-Mode (<code>claim_complete</code>)				
Base Address		0x0C20_0004		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	Interrupt Claim/Complete for Hart 0 M-Mode	RW*	X	A read of zero indicates that no interrupts are pending. A non-zero read contains the ID of the highest pending interrupt. A write to this register signals completion of the interrupt ID written.
*Consecutive reads will not return the same result.				

Table 127: PLIC Claim/Complete Register for Hart 0 M-Mode

The PLIC cannot forward a new interrupt to a hart that has claimed an interrupt, but has not yet finished the complete step of the interrupt handler. Thus, the PLIC does not support preemption of global interrupts to an individual hart.

Interrupt IDs for global interrupts routed through the PLIC are independent of the interrupt IDs for local interrupts. The PLIC handler may check for additional pending global interrupts once the initial claim/complete process has finished, prior to exiting the handler. This method could save additional PLIC save/restore context for global interrupts.

9.10 Example PLIC Interrupt Handler

Since the PLIC interfaces with the CPU through external interrupt #11, the external handler must contain an additional claim/complete step that is used to handshake with the PLIC logic.

```
void external_handler() {
    //get the highest priority pending PLIC interrupt
    uint32_t int_num = plic.claim_complete;

    //branch to handler
    plic_handler[int_num]();

    //complete interrupt by writing interrupt number back to PLIC
    plic.claim_complete = int_num;

    // Add additional checks for PLIC pending here, if desired
```

}

If a CPU reads `claim_complete` and it returns 0, the interrupt does not require processing, and thus write-back of the claim/complete is not necessary.

The `plic_handler[]()` routine shown above demonstrates one method to implement a software table where the offset of the function that resides within the table is determined by the PLIC interrupt ID. The PLIC interrupt ID is unique to the PLIC, in that it is completely independent of the interrupt IDs of local interrupts.

Chapter 10

TileLink Error Device

The Error Device is a TileLink slave that responds to all requests with a TileLink denied error and all reads with a corrupt error. It has no registers. The entire memory range discards writes and returns zeros on read. Both operation acknowledgements carry an error indication.

The Error Device serves a dual role. Internally, it is used as a landing pad for illegal off-chip requests. However, it is also useful for testing software handling of bus errors.

Chapter 11

Bus-Error Unit

This chapter describes the operation of the SiFive Bus-Error Unit (BEU).

11.1 Bus-Error Unit Overview

The Bus-Error Unit is a per-processor device that records erroneous events and reports them using platform-level and hart-local interrupts. Figure 114 shows the connections from the core to the BEU.

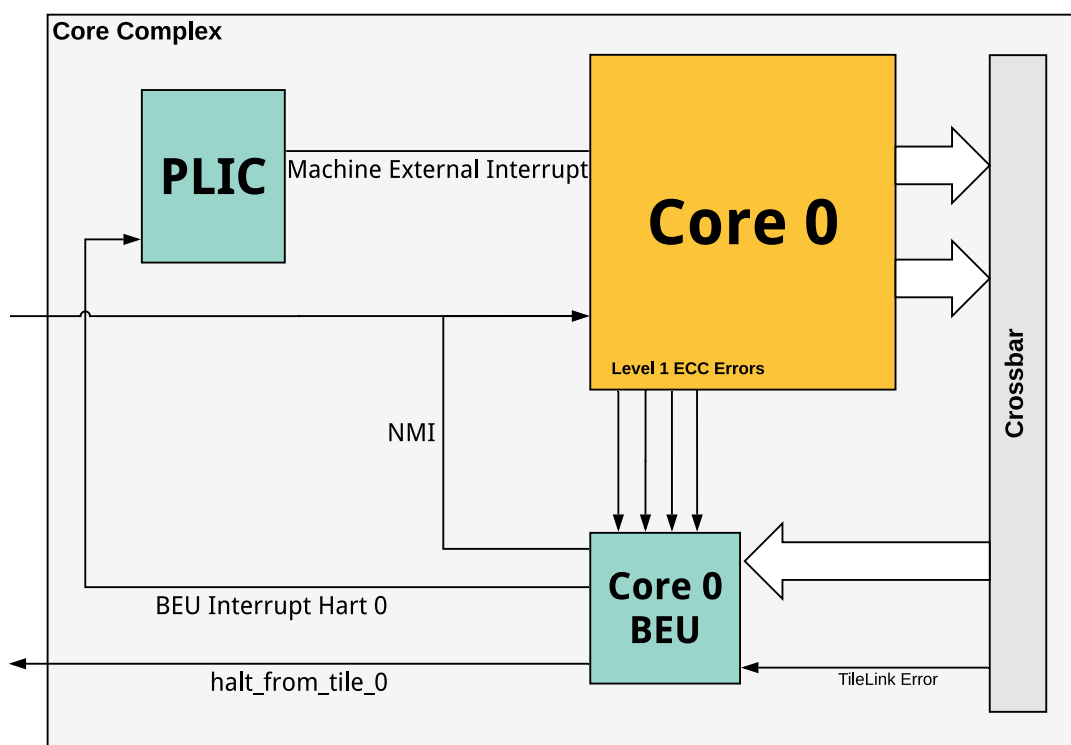


Figure 114: Bus-Error Unit Block Diagram

The BEU can be configured to generate interrupts on correctable memory errors, uncorrectable memory errors, and/or TileLink bus errors. When an error occurs, the BEU will hold the address of the error and a code to describe the error.

11.2 Memory Map

The Bus-Error Unit memory map is shown in Table 128.

Offset	Name	Description
0x00	cause	Cause of error event
0x08	value	Physical address of error event
0x10	enable	Event enable mask
0x18	plic_interrupt	Platform-level interrupt enable mask
0x20	accrued	Accrued event mask
0x28	local_interrupt	Hart-local interrupt-enable mask

Table 128: Bus-Error Unit Memory Map

Each bit in `enable`, `plic_interrupt`, `accrued`, and `local_interrupt` is associated with an error event, described in Table 129. The `cause` register represents which event occurred, and the `value` register contains the address of that error event. The `value` register will be updated when errors occur on memory that supports data cache. For example, the Memory Port has data cache support, where the System and Peripheral Ports do not.

11.3 Reportable Errors

Table 129 lists the events that the Bus-Error Unit may report.

Value	Description
0	<i>No error</i>
1	Instruction cache TileLink bus error
2	Reserved
3	Reserved
4	Reserved
5	Load/Store/PTW TileLink bus error
6	Data cache correctable ECC error
7	Data cache uncorrectable ECC error
8	Reserved
9	Private L2 TileLink bus error
10	Private L2 cache correctable ECC error
11	Private L2 cache uncorrectable ECC error

Table 129: Bus-Error Unit Error Events

11.4 Functional Description

When one of the events listed in Table 129 occurs, the Bus-Error Unit can record information about that event and can generate an interrupt to the PLIC or locally to the hart. The `enable` register contains a mask of which events the BEU can record. Each bit in `enable` corresponds

to an event in Table 129. For example, if `enable[5]` is set, the BEU will record load, store, or PTW TileLink bus errors. If `pllc_interrupt[5]` is also set, then a global interrupt will be asserted to the platform-level interrupt controller (PLIC or CLIC). Alternatively, if `local_interrupt[5]` is set, then a BEU error will result in a local interrupt to the hart. The cause register indicates the event the BEU has most recently recorded, e.g., a value of 5 indicates a load, store, or PTW TileLink bus error was recorded.

The cause value 0 is reserved to indicate "no error". `cause` is only written for events enabled in the `enable` register. Furthermore, `cause` is only written when its current value is 0; that is, if multiple events occur, only the first one is latched, until software clears the cause register.

The value register supplies the physical address that caused the event, or 0 if the address is unknown. The BEU writes the value register whenever it writes the cause register, such as when an event enabled in the `enable` register occurs or when `cause` contains 0.

The bit position in the accrued register indicates which events have occurred since the last time it was cleared by software. Its format is the same as the `enable` register. The BEU sets bits in the accrued register whether or not they are enabled in the `enable` register.

11.4.1 BEU Global Interrupt

The bit position in the `pllc_interrupt` register indicates which accrued events should generate an interrupt into the PLIC. An interrupt is generated when the same bit is set in both `accrued` and `pllc_interrupt`. The PLIC drives the machine external interrupt to the core, which has an interrupt ID of 11 (0xB). The exception code value, located in `mcause` (machine trap cause) CSR, will be 11 (0xB) when BEU interrupts are routed through the PLIC. This exception code value is independent of the PLIC interrupt number used to connect the BEU to the PLIC.

11.4.2 BEU Local Interrupt

The BEU output signal connects to the non-maskable interrupt (NMI) logic, described in Section 7.11. The `local_interrupt` register indicates which accrued events will cause a trap to the RNMI interrupt vector address. Within the NMI handler, the `nmcause` register can be read to determine if it was a BEU error, or an error triggered directly via the NMI `rnmi_N` input signal.

11.4.3 Global Interrupt Configuration

In addition to writing the BEU registers to enable interrupts, the `mstatus.MIE` CSR bit should be written to enable Machine level interrupts globally. The `mie.MEIE` CSR bit should also be configured to enable external interrupts when `pllc_interrupt` is enabled to route the PLIC interrupt from the BEU interrupts to the core. Refer to the local interrupt chapter for more details regarding interrupt configurations.

Chapter 12

Private Level 2 Cache

In addition to the L1 memory system, the P550 Core Complex also includes a per hart Private L2 (pL2) Cache. The pL2 Cache communicates with both the upstream L1 caches and downstream L3 cache or memory, enabling a high-performance cache subsystem. The pL2 Cache is responsible for managing requests from the L1 instruction and data caches of the core. The overall cache structure is depicted in Figure 115. The features of the pL2 Cache are summarized in Table 131.

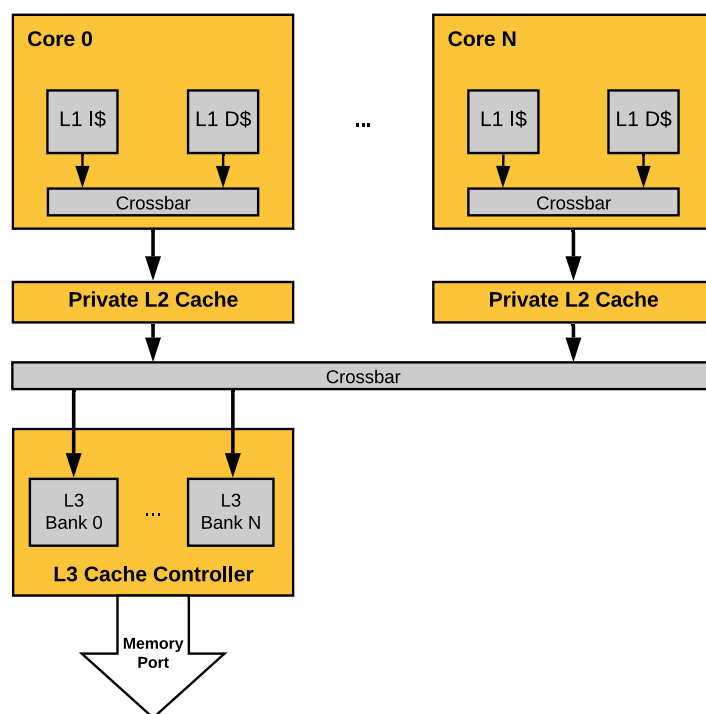


Figure 115: Cache Structure in the P550 Core Complex

The P550 Core Complex contains one pL2 Cache instance. Its address is shown in Table 130.

Instance	Base Address
Hart 0 Private L2 Cache	0x1010_4000

Table 130: Private L2 Cache Instance

12.1 Functional Description

The pL2 Cache is a 256 KiB 8-way cache. It has a line size of 64 bytes and is read/write-allocate with a random replacement policy. The pL2 Cache implements the MESI coherence protocol, and operates in write-back mode.

The pL2 cache line data are stored in the data RAM. The data RAM is composed of 2 banks. This subdivision into banks helps to increase available bandwidth between the CPU and its pL2 Cache. As such, multiple requests to different banks may proceed in parallel. Each bank consists of one or more sub-banks and each sub-bank consists of a group of physical SRAM blocks. SRAM memories are protected by ECC.

The pL2 Cache also has a 11-16 cycle load-to-use latency on L2 hits depending on cache capacity, as well as a high L1/L2 and L2/L3 bandwidth.

Features	Description
Size	256 KiB
Associativity	8 ways
Number of banks	2 banks
Line size	64 bytes
Coherence protocol	MESI Coherence Protocol
ECC protection	SECDED (9 bits ECC)
Latency	11-16 cycle load-to-use latency
Number of MSHRs	32 MSHRs

Table 131: Private L2 Cache Feature Set

12.1.1 Coherence and Ordering

The pL2 Cache supports the classic MESI, invalidate-based cache coherence protocol.

It is inclusive of the L1 data cache and partially inclusive of the L1 instruction cache. When a block of data is allocated to the L1 cache, it is also allocated to the pL2. When a block is evicted from the L1 data cache, the corresponding block in the pL2 is then updated.

The pL2 Cache also supports cacheable atomic memory operations.

The pL2 Cache guarantees ordering of MMIO requests. An MMIO read will follow the cache read pipeline and allocate into a Miss Status Handling Register (MSHR). There are 32 MSHRs in the pL2 Cache.

12.1.2 Cache Maintenance

The pL2 Cache supports cache maintenance messages through an MMIO CSR interface. A cache maintenance message must specify its target and type.

The target can be a specific pair of set/ways of the target cache, a specific cache line address of the target cache, or all cache lines of the target cache.

The type can be one of the following:

- **Refresh:** a read-modify-write performed on the target. This is mainly used for in-line ECC correction.
- **Clean:** if the line is dirty, write-back the dirty data, but keep the line in the cache. This can be used for cache scrubbing.
- **Invalid:** invalidate the cache line without a write-back of the dirty data. This is mainly used for power-down.
- **CleanInvalid:** evict the line out of cache. This is used to enable software coherence.

All legal combinations of targets and types are listed in Section 12.3.4.

The pL2 Cache maintains coherence for any cache maintenance operation (CMO), so the above operations may back probe to the L1 caches, if necessary. The invalid operations do not impact the L3 Cache and clean operations terminate in L3. As the L3 is inclusive, clean operations terminate in L3.

Cache Maintenance Operation Completion

The two CMO completion types are as follows:

- **Early completion:** when the order of requests is established in the pL2
- **Late completion:** when the cache line is evicted out of the targeted cache for the set/way target CMO, and the whole cache subsystem for the physical address target CMO

12.1.3 Error Correction Codes (ECC)

The pL2 Cache supports ECC. ECC is applied to the data RAMs and tag/state RAMs. The data RAMs use Single-Error Correcting, Double-Error Detecting (SECCDED). The tag/state RAMs use Single-Error Correcting, Double-Error Detecting (SECCDED).

The pL2 Cache can observe errors from three sources:

- The core, when the core passes down data with corrupt
- The L3 cache and memory, when the pL2 receives data with corrupt
- PL2 tag/state/data SRAMs, when the pL2 observes ECC errors from these sources

All correctable and uncorrectable ECC errors are reported to the Bus-Error Unit, described in Chapter 11. When any ECC error is detected after a tag/state lookup and/or a data RAM read, the pL2 manages this request and initiates a read-modify-write flow for correctable ECC errors, or a clean/invalid flow for uncorrectable ECC errors.

12.1.4 Private L2 Performance Monitor

Similar to the hardware performance monitor (HPM) for the core, the pL2 Cache also has a pL2 performance monitoring (pL2PM) facility. It consists of a set of event-programmable counters and their event selector registers. The registers are available to control the behavior of the counters. The performance event selector and other control registers are configured in machine mode, and the event-programmable counters can be read in user mode.

PL2PM Event Selector Registers (pmEventSelectX)

The pL2PM event selector CSRs (pmEventSelect0-5) follow the definition of mhpmeventX as defined in the RISC-V ISA. They are 64-bit bit **WARL** registers. To control the event type to count, these CSRs are used to program the corresponding event counters.

Offset	Bits	Access	Description
0x2000	64	RW	pmEventSelect0
0x2008	64	RW	pmEventSelect1
0x2010	64	RW	pmEventSelect2
0x2018	64	RW	pmEventSelect3
0x2020	64	RW	pmEventSelect4
0x2028	64	RW	pmEventSelect5

Table 132: PL2 Performance Monitor Event Selectors

The event selectors are partitioned into two fields: the lower 8 bits select an event class, and the upper bits form a mask of events in that class.

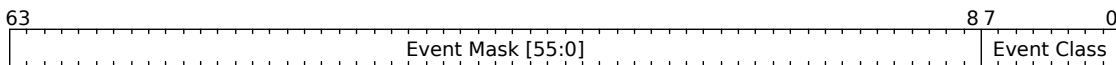


Figure 116: Event Selector Fields

The counter increments if the event corresponding to any set mask bit occurs.

PL2PM Counter Client Filter CSR (pmClientFilter)

The pL2PM counter client filter register pmClientFilter is a 64-bit **WARL** register that controls which client's performance events are excluded from incrementing the performance monitor counters. However, some events might not be attributed to a particular client in a specific implementation (for example, the pL2 release event in SiFive's pL2 Cache design).

Offset	Bits	Access	Description
0x2800	32	RW	pmClientFilter

Table 133: PL2 Performance Monitor Counter Client Filter

The mapping between masters and their pL2 master IDs is shown in Table 134.

Master ID	Description
0	Fetch Unit
1	Load Store Unit Cache 0
2	Load Store Unit Cache 1
3	Load Store Unit IO
4	Load Store Unit HWPF

Table 134: Master IDs in the Private L2 Cache

Note that the default value of `pmClientFilter` is 0, meaning all clients' events are counted. To disable event counting for a particular client, set its corresponding bit in `pmClientFilter`.

PL2 Performance Monitor Counters (`pmEventCounterX`)

The pL2 Performance Monitor counters (`pmEventCounter0-5`), follow the definition of `mhpmcounterX` as defined in the RISC-V ISA. They are 64-bit **WARL** registers.

Offset	Bits	Access	Description
0x3000	64	RW	<code>pmEventCounter0</code>
0x3008	64	RW	<code>pmEventCounter1</code>
0x3010	64	RW	<code>pmEventCounter2</code>
0x3018	64	RW	<code>pmEventCounter3</code>
0x3020	64	RW	<code>pmEventCounter4</code>
0x3028	64	RW	<code>pmEventCounter5</code>

Table 135: PL2 Performance Monitor Counters

Note

If a counter is not implemented, such as when there is no pL2PM, both the counter and its corresponding event selector are hard-wired to 0, meaning that the counter always returns 0.

Event Selector Encodings

Table 136 describes the event selector encodings available. Events are categorized into two classes based on the Event Class field encoded in `pmEventSelectX[7:0]`. One or more events can be programmed by setting the respective Event Mask bit for a given event class. An event selector encoding of 0 means "count nothing". Multiple events will cause the counter to increment any time any of the selected events occur.

pL2 Performance Monitor Event Register		
Inner Transaction Events, pmEventSelectX[7:0]=0x1		
Bits	Description	Can use ClientFilter?
0	PutFullData request from inner cache	Yes
1	PutPartialData request from inner cache	Yes
2	ArithmeticData / LogicalData request from inner cache	Yes
3	Get request from inner cache	Yes
4	PrefetchRead request from inner cache	Yes
5	PrefetchWrite request from inner cache	Yes
6	AcquireBlock.NtoB request from inner cache	Yes
7	AcquireBlock.NtoT request from inner cache	Yes
8	AcquireBlock.BtoT request from inner cache	Yes
9	AcquirePerm.NtoT request from inner cache	Yes
10	AcquirePerm.BtoT request from inner cache	Yes
11	Release.TtoB request from inner cache	Yes
12	Release.TtoN request from inner cache	Yes
13	Release.BtoN request from inner cache	Yes
14	ReleaseData.TtoB request from inner cache	Yes
15	ReleaseData.TtoN request from inner cache	Yes
16	ReleaseData.BtoN request from inner cache	Yes
17	ReleaseData.TtoT request from inner cache	Yes
18	ProbeBlock.toT request from inner cache	Yes
19	ProbeBlock.toB request from inner cache	Yes
20	ProbeBlock.toN request from inner cache	Yes
21	ProbePerm.toN request from inner cache	Yes
22	ProbeAck.TtoB request from inner cache	Yes
23	ProbeAck.TtoN request from inner cache	Yes
24	ProbeAck.BtoN request from inner cache	Yes
25	ProbeAck.TtoT request from inner cache	Yes
26	ProbeAck.BtoB request from inner cache	Yes
27	ProbeAck.NtoN request from inner cache	Yes
28	ProbeAckData.TtoB request from inner cache	Yes
29	ProbeAckData.TtoN request from inner cache	Yes
30	ProbeAckData.TtoT request from inner cache	Yes
pL2 Query Result Events, pmEventSelectX[7:0]=0x2		
Bits	Description	Can use ClientFilter?
0	PutFullData.Hit request from inner cache hits a valid line in pL2	Yes
1	PutPartialData.Hit request from inner cache hits a valid line in pL2	Yes

Table 136: PL2PM pmEventSelect Register

2	ArithmeticData / LogicalData request from inner cache hits a valid line in pL2	Yes
3	Get request from inner cache hits a valid line in pL2	Yes
4	PrefetchRead request from inner cache hits a valid line in pL2	Yes
5	AcquireBlock.NtoB request from inner cache hits a valid line in pL2	Yes
6	AcquirePerm.NtoT request from inner cache hits a valid line in pL2	Yes
7	Release.TtoB request from inner cache hits a valid line in pL2	Yes
8	ReleaseData.TtoB request from inner cache hits a valid line in pL2	Yes
9	ProbeBlock.toT request from outer cache hits a valid line in pL2	No
10	PutFullData request from inner cache hits a shared line in pL2	Yes
11	PutPartialData request from inner cache hits a shared line in pL2	Yes
12	ArithmeticData / LogicalData request from inner cache hits a shared line in pL2	Yes
13	Get request from inner cache hits a shared line in pL2	Yes
14	PrefetchRead request from inner cache hits a shared line in pL2	Yes
15	AcquireBlock.NtoB request from inner cache hits a shared line in pL2	Yes
16	AcquirePerm.NtoT request from inner cache hits a shared line in pL2	Yes
17	ProbeBlock.toT request from outer cache hits a shared line in pL2	No
18	ProbeBlock.toT request from outer cache hits a dirty line in pL2	No
Outer Transaction Events, pmEventSelectX[7:0]=0x3		
Bits	Description	Can use ClientFilter?
0	PutFullData request to outer cache	No
1	PutPartialData request to outer cache	No
2	ArithmeticData / LogicalData request to outer cache	No
3	Get request to outer cache	No
4	PrefetchRead request to outer cache	No
5	PrefetchWrite request to outer cache	No
6	AcquireBlock.NtoB request to outer cache	No
7	AcquireBlock.NtoT request to outer cache	No
8	AcquireBlock.BtoT request to outer cache	No
9	AcquirePerm.NtoT request to outer cache	No
10	AcquirePerm.BtoT request to outer cache	No
11	Release.TtoB request to outer cache	No
12	Release.TtoN request to outer cache	No
13	Release.BtoN request to outer cache	No

Table 136: PL2PM pmEventSelect Register

14	ReleaseData.TtoB request to outer cache	No
15	ReleaseData.TtoN request to outer cache	No
16	ReleaseData.BtoN request to outer cache	No
17	ReleaseData.TtoT request to outer cache	No
18	ProbeBlock.toT request from outer cache	No
19	ProbeBlock.toB request from outer cache	No
20	ProbeBlock.toN request from outer cache	No
21	ProbePerm.toN request from outer cache	No
22	ProbeAck.TtoB request to outer cache	No
23	ProbeAck.TtoN request to outer cache	No
24	ProbeAck.BtoN request to outer cache	No
25	ProbeAck.TtoT request to outer cache	No
26	ProbeAck.BtoB request to outer cache	No
27	ProbeAck.NtoN request to outer cache	No
28	ProbeAckData.TtoB request to outer cache	No
29	ProbeAckData.TtoN request to outer cache	No
30	ProbeAckData.TtoT request to outer cache	No
Other pL2 Microarchitectural Events, pmEventSelectX[7:0]=0x4		
Bits	Description	Can use ClientFilter?
0	Hint request from inner cache hits an inflight miss request	Yes
1	Acquire / Get request from inner cache hits an inflight miss request	Yes
2	Put / ArithmeticData / LogicalData request from inner cache hits an inflight miss request	Yes
3	Hint / Acquire / Get request from inner cache replays	Yes
4	Put / ArithmeticData / LogicData request from inner cache replays	Yes
5	Probe request from outer cache replays	No
Note: "Inner cache" refers to the inner side of the pL2 cache, i.e., transactions from the core. "Outer cache" refers to the other side of the pL2 Cache, i.e., read/write transactions from the pL2 and probe requests from outside.		

Table 136: PL2PM pmEventSelect Register

12.2 Memory Map

The pL2 Cache memory map is shown in Table 137.

Offset	Name	Description
0x0000	Config	Information about the cache configuration
0x0040	ECCInjectError	Inject an ECC error
0x0060	EccInjectErrorDone	ECC inject is done
0x0200	CFlush64	MMIO cache flush command register
0x0208	FlushCount	The number of outstanding CFLUSH requests
0x1008	pL2Config0	Config bits of private L2 cache
0x2000	pmEventSelect0	Performance monitor event select 0
0x2008	pmEventSelect1	Performance monitor event select 1
0x2010	pmEventSelect2	Performance monitor event select 2
0x2018	pmEventSelect3	Performance monitor event select 3
0x2020	pmEventSelect4	Performance monitor event select 4
0x2028	pmEventSelect5	Performance monitor event select 5
0x2800	pmClientFilter	Performance counter client disable mask
0x3000	pmEventCounter0	Performance monitor event counter 0
0x3008	pmEventCounter1	Performance monitor event counter 1
0x3010	pmEventCounter2	Performance monitor event counter 2
0x3018	pmEventCounter3	Performance monitor event counter 3
0x3020	pmEventCounter4	Performance monitor event counter 4
0x3028	pmEventCounter5	Performance monitor event counter 5

Table 137: Private L2 Cache Memory Map

12.3 Register Descriptions

This section describes the functionality of the memory-mapped registers in the Private L2 Cache.

12.3.1 PL2 Cache Configuration Register (Config)

The Config Register can be used to programmatically determine information regarding the cache size and organization.

PL2 Cache Configuration Register (Config)				
Register Offset		0x0		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	Banks	RO	0x2	Number of banks in the cache
[15:8]	Ways	RO	0x8	Number of ways per bank
[23:16]	lgSets	RO	0x9	Base-2 logarithm of the total number of sets
[31:24]	lgBlockBytes	RO	0x6	Base-2 logarithm of the bytes per cache block

Table 138: PL2 Cache Configuration Register

12.3.2 ECC Error Injection Register (ECCInjectError)

The ECCInjectError register can be used to inject single-bit or double-bit ECC errors into either the tag/state or data SRAM. This function can be used to test error correction logic, measurement, and recovery. Note that when an uncorrectable ECC error is injected into either tag/state or data SRAM, the program may lose data coherence and consistency, which may impact normal behavior.

ECC Error Injection Register (ECCInjectError)				
Register Offset		0x40		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	ECCToggleBit	RW	0x0	Toggle (corrupt) this bit index on the next cache operation
[15:8]	Reserved			
[17:16]	InjectionTarget	RW	0x0	Inject error in 0=data or 1=tag/state or 2=lru
18	InjectOnWrite	RW	0x0	Inject error on (0) read data or (1) write data
19	InjectionType	RW	0x0	Inject correctable (0) or uncorrectable (1) error
[31:20]	Reserved			

Table 139: ECC Error Injection Register

ECC injection only occurs on the next cacheable read or write. If software wants to know the status of the injection, it needs to poll the EccInjectErrorDone register.

12.3.3 ECC Error Injection Done Register (EccInjectErrorDone)

ECCInjectErrorDone indicates if the ECC error injection is complete.

ECC Error Injection Done Register (EccInjectErrorDone)				
Register Offset		0x60		
Bits	Field Name	Attr.	Rst.	Description
0	ECCToggleDone	RO	0x1	Toggle (corrupt) is done
[7:1]	Reserved			

Table 140: ECC Error Injection Done Register

12.3.4 PL2 Cache Flush Register (Cflush64)

Writes to the Flush64_Command field generates a cache flush request; the response indicates that the pL2 cache has sunk the request, but may or may not perform it. This behavior is to enable multiple outstanding cache flush requests. If software wants to know the status of previously issued requests, it needs to poll the FlushCount register.

PL2 Cache Flush Register (Cflush64)				
Register Offset		0x200		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	Flush64_Address_Low	WO	0x0	The lower 32-bit address for physical address operations or the cache set number for set/way operations
[55:32]	Flush64_Address_High	WO	0x0	The upper 24-bit address for physical address operations or the cache way number for set/way operations
[63:56]	Flush64_Command	WO	0x0	The command of cache flush

Table 141: PL2 Cache Flush Register

Table 142 describes the defined cache flush commands.

Message	Bits		Description
	[5:3]	[2:0]	
Reserved	0	0	
CFlush_PA_Inv	0	1	Invalidate the cache line of the target physical address
CFlush_PA_Clean	0	2	If the target cache line has dirty data, write back the dirty data but keep the cache line
CFlush_PA_CleanInv	0	3	Evict the cache line of the target physical address
CFlush_SetWay_Refresh	1	0	Read the cache line of the specified set/way from cache, and write it back into cache
CFlush_SetWay_Inv	1	1	Invalidate the cache line of the target set/way
CFlush_SetWay_Clean	1	2	If the target cache line has dirty data, write back the dirty data but keep the cache line
CFlush_SetWay_CleanInv	1	3	Evict the cache line of the target set/way
CFlush_Cache_Refresh	2	0	Read the cache line of each set/way from cache, and write it back into cache
CFlush_Cache_Inv	2	1	Invalidate the cache line of each set/way
CFlush_Cache_Clean	2	2	Write back all dirty data in the cache, but keep all valid cache lines
CFlush_Cache_CleanInv	2	3	Evict all valid cache lines

Table 142: Cache Flush Commands

When an illegal command is received, or an illegal set or way is given for a set/way target request, it is treated as a nop.

12.3.5 Flush Count Register (FlushCount)

FlushCount is a read-only register that contains the number of outstanding cache flush requests. 0 indicates that all previously issued cache flush requests have been completed.

PL2 Flush Count Register (FlushCount)				
Register Offset		0x208		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	FlushCount	RO	0x0	The number of outstanding cflush requests

Table 143: PL2 Flush Count Register

12.3.6 PL2 Configuration Register (pL2Config)

This register contains bits to configure various features of the pL2 cache.

PL2 Configuration 0 Register (pL2Config0)				
Register Address		0x1008		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Reserved	RO	0x0	
3	cleanEvictEnable	RW	0x1	When set, enable clean eviction from pL2 to an outer cache/ directory. This bit will affect the L3 cache miss rate.
[8:4]	Reserved	RO	0x0	
9	l2AvoidL1LineDisable	RW	0x1	When set, L2 victim can pick L1 cache lines as well. This bit will affect pL2 cache miss rate.
[15:10]	Reserved	RO	0x0	
16	softwareEccInjectEnable	RW	0x0	When set, enable software ECC injection
17	errInjectOnWriteEnable	RW	0x1	When set, enable error injecton on tag/data RAM write (latent error). When clear, inject only on the next read. This is useful to test ECC correction logic.
18	errInjectOnReadEnable	RW	0x0	When set, enable error injection on tag/data RAM read.
19	dataUceInjectEnable	RW	0x0	When set, enable uncorrectable data error injection. When clear, inject only correctable data errors.
20	dirUceInjectEnable	RW	0x0	When set, enable uncorrectable metadata error injection. When clear, inject only correctable metadata errors.
[31:21]	Reserved	RO	0x0	

Table 144: PL2 Configuration 0 Register

Chapter 13

Level 3 Cache Controller

This chapter describes the functionality of the Level 3 Cache Controller used in the P550 Core Complex.

13.1 Level 3 Cache Controller Overview

The SiFive Level 3 Cache Controller is used to provide access to fast copies of memory for masters in a Core Complex. The Level 3 Cache Controller also acts as a directory-based coherency manager.

The SiFive Level 3 Cache Controller offers extensive flexibility, as it allows for several features in addition to the Level 3 Cache functionality. These include memory-mapped access to L3 Cache RAM for disabled cache ways, scratchpad functionality, way masking and locking, ECC support with error tracking statistics, error injection, and interrupt signaling capabilities.

These features are described in Section 13.2.

13.2 Functional Description

The P550 Core Complex L3 Cache is a 2 MiB 8-way set-associative cache. It has a line size of 64 bytes and is read/write-allocate with a random replacement policy. The cache operates in write-back mode only. The L3 Cache is composed of 4 banks. This subdivision into banks helps facilitate increased available bandwidth between CPU masters and the L3 Cache, as each bank has its own dedicated 256-bit TL-C inner port. As such, multiple requests to different banks may proceed in parallel.

The outer port of the L3 Cache Controller is a 128-bit TL-C port shared among all banks and typically connected to a DDR controller. The outer Memory Port of the L3 Cache Controller is shared among all banks and typically connected to cacheable memory. The overall organization of the L3 Cache Controller is depicted in Figure 117.

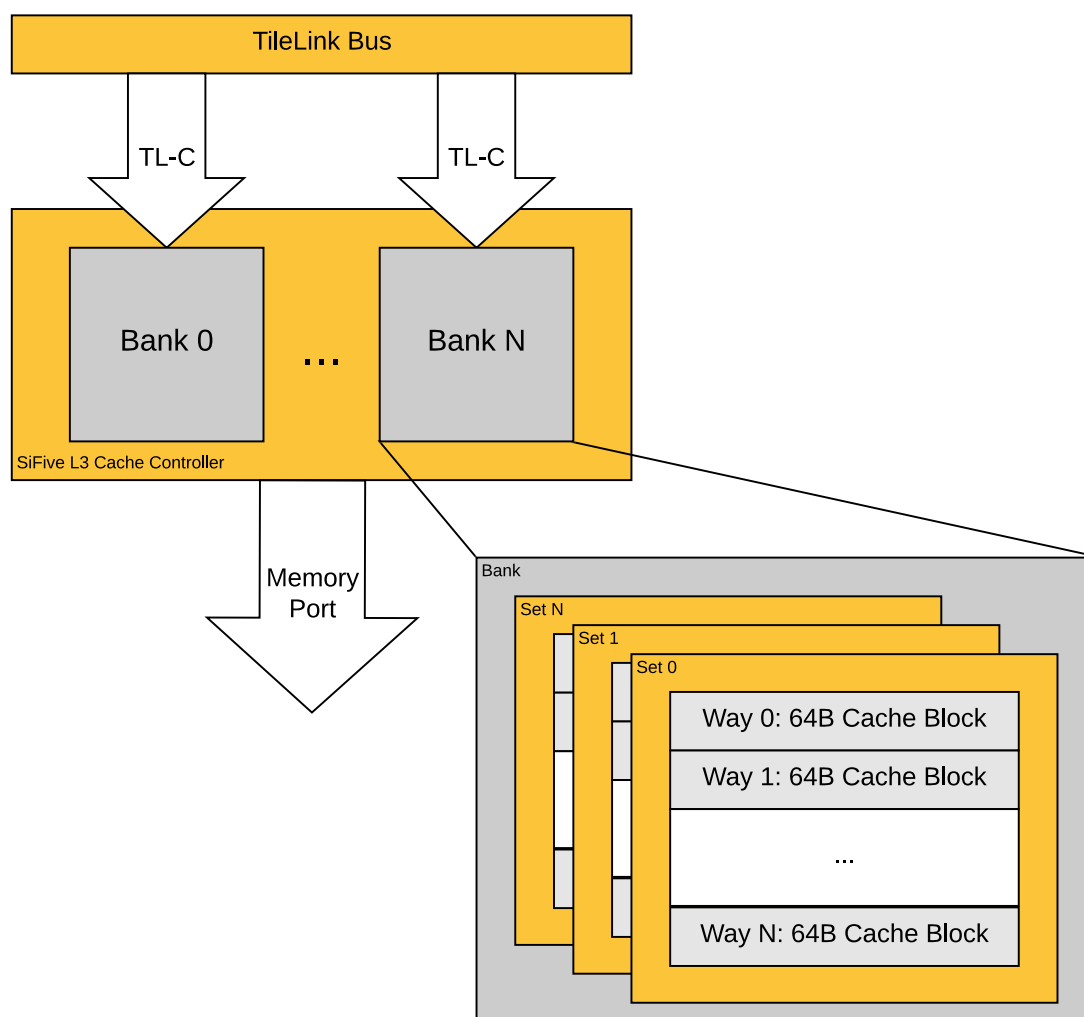


Figure 117: Organization of the SiFive L3 Cache Controller

13.2.1 Way Enable and the L3 Loosely-Integrated Memory (L3 LIM)

The SiFive Level 3 Cache Controller allows for its SRAMs to act either as direct-addressed memory in the Core Complex address space or as a cache that is controlled by the L3 Cache Controller, which can contain a copy of any cacheable address.

When cache ways are disabled, they are addressable in the L3 Loosely-Integrated Memory (L3 LIM) address space as described in the P550 Core Complex memory map in Section 4.2. The L3 LIM is an uncacheable port into unused L3 SRAM and provides deterministic access time. It is neither cached by the L1 data cache nor memory backed, as it is just a dedicated software-addressable, low latency, uncached memory. Fetching instructions or data from the L3 LIM provides deterministic behavior equivalent to an L3 Cache hit, with no possibility of a cache miss. Accesses to the L3 LIM are always given priority over cache way accesses, which target the same L3 Cache bank.

Out of reset, all ways, except for way 0, are disabled. Cache ways can be enabled by writing to the wayEnable register described in Section 13.4.2. Once a cache way is enabled, it cannot be disabled unless the P550 Core Complex is reset. The highest numbered L3 cache way is mapped to the lowest L3 LIM address space, and way 1 occupies the highest L3 LIM address range. As L3 cache ways are enabled, the size of the L3 LIM address space shrinks. The mapping of L3 cache ways to L3 LIM address space is shown in Figure 118, where N is the number of L3 cache ways, each of size 256 KiB (0x0004_0000).

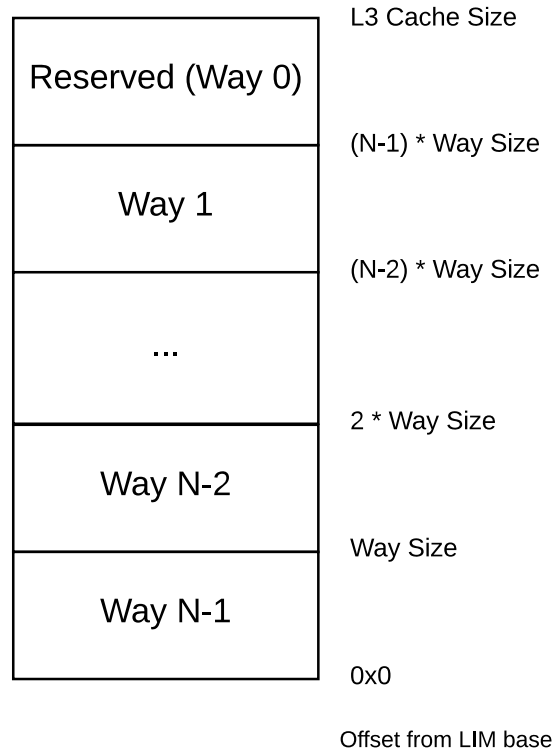


Figure 118: Mapping of L3 Cache Ways to L3 LIM Addresses

13.2.2 Way Masking and Locking

The SiFive L3 Cache Controller can control the amount of cache memory a CPU master is able to allocate into by using the wayMaskN register described in Section 13.4.10. Note that wayMaskN registers only affect allocations and reads can still occur to ways that are masked. As such, it becomes possible to lock down specific cache ways by masking them in all wayMaskN registers. In this scenario, all masters can still read data in the locked cache ways but cannot evict data.

13.2.3 L3 Zero Device

The SiFive L3 Cache Controller has a dedicated scratchpad address region that allows for allocation into the cache using an address range that is not memory backed. This address region is

denoted as the L3 Zero Device in the Section 4.2 memory map. Writes to the scratchpad region allocate into cache ways that are enabled and not masked.

A Zero Device ignores write data and always returns zero on reads. The P550 Core Complex provides a Zero Device behind the L3 Cache, similar to the Memory Port. When combined with locked L3 cache ways, which prevent eviction, locations within a Zero Device's address range appear to retain their value. This provides a mechanism to create L1 and L2 cacheable memory that are essentially backed by L3 SRAM until the way is released (and the value resets to zero). The L3 Zero Device is cacheable like the Memory Port. However, if dirty data is evicted and a write-back to the L3 Zero Device occurs, the Zero Device will discard the write data. Therefore, care must be taken with the scratchpad, as there is no memory backing this address space. Cache evictions from addresses in the scratchpad result in data loss.

The main advantage of the L3 Zero Device over the L3 LIM is that it is a cacheable region allowing for data stored to the scratchpad to also be cached in a master's L1 data cache and pL2 cache, which results in faster access.

To understand the difference between the L3 LIM and the L3 Zero Device, consider Figure 119. Notice that the L3 LIM accesses the same blocks of memory as the main path into the L3 Cache, whereas the L3 Zero Device sits behind L3 Cache much like the Memory Port:

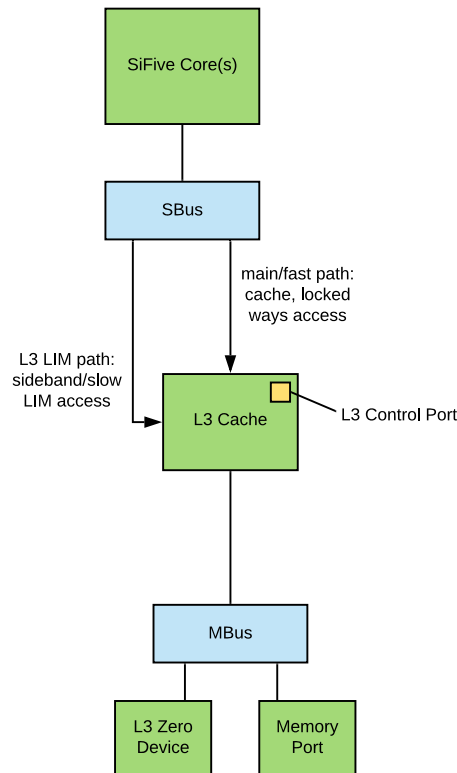


Figure 119: Difference between L3 LIM and L3 Zero Device

The recommended procedure for using the L3 Zero Device is as follows:

1. Use the wayEnable register to enable the desired cache ways
2. Designate a single master that will allocate into the scratchpad. For this procedure, we designate this master as Master S. All other masters (CPU and non-CPU) are denoted as Masters N.
3. Masters N: Write to the wayMaskN register to mask the ways that are to be used for the scratchpad. This prevents Masters N from evicting cache lines in the designated scratchpad ways.
4. Master S: Write to the wayMaskN register to mask all ways *except* the ways that are to be used for the scratchpad. At this point, Master S should only be able to allocate into the cache ways meant to be used as a scratchpad.
5. Master S: Write scratchpad data into the L3 Zero Device address range
6. Master S: Repeat steps 4 and 5 for each way to be used as scratchpad
7. Master S: Use the wayMaskN register to mask the scratchpad ways for Master S so that it is no longer able to evict cache lines from the designated scratchpad ways
8. At this point, the scratchpad ways should contain the scratchpad data, with all masters able to read, write, and execute from this address space, and no masters able to evict the scratchpad contents

13.2.4 L3 Features Access Summary

Table 145 describes the L3 features as a function of Way Enable and Way Mask.

Way Enable	Way Mask	Access Base Address	L3 Feature
0	X	0x0800_0000	LIM
1	0	0x8000_0000	Locked Ways — Fast Read access
1	0	0x0A00_0000	Zero Device — Fast Read access from scratchpad
1	1	0x8000_0000	Functional Cache, Locked Ways — Write data mode
1	1	0x0A00_0000	Zero Device — Write data to scratchpad

Table 145: L3 Features Access Summary

13.2.5 L3 Performance Monitor

Similar to the hardware performance monitor (HPM) for the core, the L3 Cache also has an L3 performance monitoring (L3PM) facility. It consists of a set of event-programmable counters and their event selector registers. The registers are available to control the behavior of the counters. The performance event selector and other control registers are configured in machine mode, and the event-programmable counters can be read in user mode.

L3PM Event Selector Registers (pmEventSelectX)

The L3PM event selector CSRs (pmEventSelect0-5) follow the definition of mhpmeventX as defined in the RISC-V ISA. They are 64-bit **WARL** registers. To control the event type to count, these CSRs are used to program the corresponding event counters.

Offset	Bits	Access	Description
0x2000	64	RW	pmEventSelect0
0x2008	64	RW	pmEventSelect1
0x2010	64	RW	pmEventSelect2
0x2018	64	RW	pmEventSelect3
0x2020	64	RW	pmEventSelect4
0x2028	64	RW	pmEventSelect5

Table 146: L3 Performance Monitor Event Selectors

The event selectors are partitioned into two fields: the lower 8 bits select an event class, and the upper bits form a mask of events in that class.

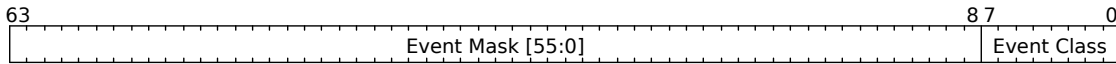


Figure 120: Event Selector Fields

The counter increments if the event corresponding to any set mask bit occurs.

L3PM Counter Client Filter CSR (pmClientFilter)

The L3PM counter client filter register pmClientFilter is a 64-bit **WARL** register, which controls which client's performance events are excluded from incrementing the performance monitor counters. However, some events might not be attributed to a particular client in a specific implementation (for example, the L3 release event in SiFive's L3 Cache design).

Offset	Bits	Access	Description
0x2800	32	RW	pmClientFilter

Table 147: L3 Performance Monitor Counter Client Filter

L3 Performance Monitor Counters (pmEventCounterX)

The L3 Performance Monitor counters (pmEventCounter0-5), follow the definition of mhpmcouterX as defined in the RISC-V ISA. They are 64-bit **WARL** registers.

Offset	Bits	Access	Description
0x3000	64	RW	pmEventCounter0
0x3008	64	RW	pmEventCounter1
0x3010	64	RW	pmEventCounter2
0x3018	64	RW	pmEventCounter3
0x3020	64	RW	pmEventCounter4
0x3028	64	RW	pmEventCounter5

Table 148: L3 Performance Monitor Counters

Note

If a counter is not implemented, such as when there is no L3PM, both the counter and its corresponding event selector are hard-wired to 0, meaning that the counter always returns 0.

Event Selector Encodings

Table 149 describes the event selector encodings available. Events are categorized into two classes based on the Event Class field encoded in `pmEventSelectX[7:0]`. One or more events can be programmed by setting the respective Event Mask bit for a given event class. An event selector encoding of 0 means "count nothing". Multiple events will cause the counter to increment any time any of the selected events occur.

L3 Performance Monitor Event Register		
Transaction Events, pmEventSelectX[7:0]=0x1		
Bits	Description	Can use ClientFilter?
0	PutFullData request from inner cache	Yes
1	PutPartialData request from inner cache	Yes
2	AtomicData request from inner cache	Yes
3	Get request from inner cache	Yes
4	PrefetchRead request from inner cache	Yes
5	PrefetchWrite request from inner cache	Yes
6	AcquireBlock.NtoB request from inner cache	Yes
7	AcquireBlock.NtoT request from inner cache	Yes
8	AcquireBlock.BtoT request from inner cache	Yes
9	AcquirePerm.NtoT request from inner cache	Yes
10	AcquirePerm.BtoT request from inner cache	Yes
11	Release.TtoB request from inner cache	Yes
12	Release.TtoN request from inner cache	Yes
13	Release.BtoN request from inner cache	Yes
14	ReleaseData.TtoB request from inner cache	Yes
15	ReleaseData.TtoN request from inner cache	Yes
16	ReleaseData.BtoN request from inner cache	Yes
17	ProbeBlock.toT request from outer cache	No
18	ProbeBlock.toB request from outer cache	No
19	ProbeBlock.toN request from outer cache	No
L3 Query Result Events, pmEventSelectX[7:0]=0x2		
Bits	Description	Can use ClientFilter?
0	PutFullData request from inner cache hits a valid line in L3	Yes
1	PutPartialData request from inner cache hits a valid line in L3	Yes
2	AtomicData request from inner cache hits a valid line in L3	Yes
3	Get request from inner cache hits a valid line in L3	Yes
4	Prefetch request from inner cache hits a valid line in L3	Yes
5	AcquireBlock request from inner cache hits a valid line in L3	Yes
6	AcquirePerm request from inner cache hits a valid line in L3	Yes
7	Release request from inner cache hits a valid line in L3	Yes
8	ReleaseData request from inner cache hits a valid line in L3	Yes
9	Probe request from outer cache hits a valid line in L3	No
10	PutFullData request from inner cache hits a shared line in L3	Yes

Table 149: L3PM pmEventSelect Register

11	PutPartialData request from inner cache hits a shared line in L3	yes
12	AtomicData request from inner cache hits a shared line in L3	Yes
13	Get request from inner cache hits a shared line in L3	Yes
14	Prefetch request from inner cache hits a shared line in L3	Yes
15	AcquireBlock request from inner cache hits a shared line in L3	Yes
16	AcquirePerm request from inner cache hits a shared line in L3	Yes
17	Probe request from outer cache hits a shared line in L3	No
18	Probe request from outer cache hits a dirty line in L3	No
L3 Request Events, pmEventSelectX[7:0]=0x3		
Bits	Description	Can use ClientFilter?
0	AcquireBlock.NtoB request to outer cache, miss	No
1	AcquireBlock.NtoT request to outer cache, miss	No
2	AcquireBlock.BtoT request to outer cache, miss	No
3	AcquirePerm.NtoT request to outer cache, miss	No
4	AcquirePerm.BtoT request to outer cache, miss	No
5	Release.TtoB request to outer cache, eviction	No
6	Release.TtoN request to outer cache, eviction	No
7	Release.BtoN request to outer cache, eviction	No
8	ReleaseData.TtoB request to outer cache, not applicable	No
9	ReleaseData.TtoN request to outer cache, dirty eviction	No
10	ReleaseData.BtoN request to outer cache, not applicable	No
11	ProbeBlock.toT request to inner cache, code miss hits other harts	No
12	ProbeBlock.toB request to inner cache, load miss hits other harts	No
13	ProbeBlock.toN request to inner cache, store miss hits other harts	No
Other Events, pmEventSelectX[7:0]=0x4		
Bits	Description	Can use ClientFilter?
0	Hint request from inner cache hits an inflight miss request	Yes
Note: "Inner cache" refers to the inner side of the L3 cache, i.e., transactions from the core and Front Port. "Outer cache" refers to the other side of the L3 Cache, i.e., read/write transactions from the L3 and probe requests from outside.		

Table 149: L3PM pmEventSelect Register

Setting up the pmClientFilter Register

Table 155 shows the mapping of L3 Cache masters and their client ID.

Note that the default value of `pmClientFilter` is 0, meaning all clients' events are counted. To disable event counting for a particular client, set its corresponding bit in `pmClientFilter`.

Programming the `pmEventSelect` registers

The following example shows the use of 6 performance counters:

```
// directory lookup events: L1 miss
*pmEventSelect0 = 0x01 | // Event Set 1
    ( (0x01 << 6 ) | // innerAcquireBlockNtoB
      (0x01 << 7 ) | // innerAcquireBlockNtoT
      (0x01 << 8 ) | // innerAcquireBlockBtoT
      (0x01 << 9 ) | // innerAcquirePermNtoT
      (0x01 << 10) // innerAcquirePermB2T
    ) << 8;

// directory lookup results: L1 miss hit L3
*pmEventSelect1 = 0x02 | // Event Set 2
    ( (0x01 << 5) | // innerAcquireBlock_Hit
      (0x01 << 6) // innerAcquirePerm_Hit
    ) << 8;

// directory lookup events: prefetch
*pmEventSelect2 = 0x01 | // Event Set 1
    ( (0x01 << 4 ) | // innerPrefetchRead
      (0x01 << 5 ) // innerPrefetchWrite
    ) << 8;

// prefetch hits L3
*pmEventSelect3 = 0x02 | // Event Set 2
    ( (0x01 << 4) // innerPrefetch_Hit
    ) << 8;

// L1 request misses L3
*pmEventSelect4 = 0x03 | // Event Set3
    ( (0x01 << 0) | // outerAcquireBlockNtoB
      (0x01 << 1) | // outerAcquireBlockNtoT
      (0x01 << 2) | // outerAcquireBlockBtoT
      (0x01 << 3) | // outerAcquirePermNtoT
      (0x01 << 4) // outerAcquirePermBtoT
    ) << 8;
```

13.2.6 Error Correction Codes (ECC)

The SiFive Level 3 Cache Controller supports ECC. ECC is applied to both categories of SRAM used, the data SRAMs and the metadata SRAMs (index, tag, and directory information). The data SRAMs use Single-Error Correcting, Double-Error Detecting (SECDDED). The metadata SRAMs use Single-Error Correcting, Double-Error Detecting (SECDDED).

Whenever a correctable error is detected, the cache immediately repairs the corrupted bit and writes it back to SRAM. This corrective procedure is completely invisible to application software. However, to support diagnostics, the cache records the address of the most recently corrected metadata and data errors. Whenever a new error is corrected, a counter is increased, and an

interrupt is raised. There are independent addresses, counters, and interrupts for correctable metadata and data errors.

DirError, DirFail, DataError, and DataFail signals are used to indicate that an L3 metadata, uncorrectable L3 metadata, L3 data, or uncorrectable L3 data error has occurred, respectively. These signals are connected to the PLIC, as described in Chapter 9, and are cleared upon reading their respective count registers.

13.2.7 Coherence

The SiFive L3 Cache is partially inclusive of the L1 instruction cache and is inclusive of the L1 data cache. When a block of data is allocated to the L1 cache, it is also allocated to the L3 Cache. When a block is evicted from the L1, the corresponding block in the L3 is then updated and marked dirty.

To understand how coherence is managed differently in the L3 Cache with respect to the L1 instruction and data caches, consider the following rules:

1. Only an instruction cache allocation from the Memory Port will land in the L3 Cache
2. An eviction from the L3 Cache does not cause an eviction from the instruction cache
3. An eviction from the instruction cache does not cause L3 Cache eviction either
4. A discard from the data cache does not invalidate the L3 Cache
5. Following a flush in the L3 Cache, the L3 Cache will back probe lines in L1 data cache

13.3 Memory Map

The L3 Cache Controller memory map is shown in Table 150.

Offset	Name	Description
0x0000	Config	Information about the Cache Configuration
0x0008	WayEnable	The index of the largest way which has been enabled. May only be increased.
0x0040	ECCInjectError	Inject an ECC Error
0x0100	DirECCFixLow	The low 32-bits of the most recent address to fail ECC
0x0104	DirECCFixHigh	The high 32-bits of the most recent address to fail ECC
0x0108	DirECCFixCount	Reports the number of times an ECC error occurred
0x0120	DirECCFailLow	The low 32-bits of the most recent address to fail ECC
0x0124	DirECCFailHigh	The high 32-bits of the most recent address to fail ECC
0x0128	DirECCFailCount	Reports the number of times an ECC error occurred
0x0140	DatECCFixLow	The low 32-bits of the most recent address to fail ECC
0x0144	DatECCFixHigh	The high 32-bits of the most recent address to fail ECC
0x0148	DatECCFixCount	Reports the number of times an ECC error occurred
0x0160	DatECCFailLow	The low 32-bits of the most recent address to fail ECC
0x0164	DatECCFailHigh	The high 32-bits of the most recent address to fail ECC
0x0168	DatECCFailCount	Reports the number of times an ECC error occurred
0x0200	Flush64	Flush the physical address equal to the 64-bit written data from the cache
0x0240	Flush32	Flush the physical address equal to the 32-bit written data << 4 from the cache
0x0800	WayMask0	Master 0 way enable mask register
0x0808	WayMask1	Master 1 way enable mask register
0x0810	WayMask2	Master 2 way enable mask register
0x0818	WayMask3	Master 3 way enable mask register
0x0820	WayMask4	Master 4 way enable mask register
0x0828	WayMask5	Master 5 way enable mask register
0x0830	WayMask6	Master 6 way enable mask register
0x2000	pmEventSelect0	Performance monitor event select 0
0x2008	pmEventSelect1	Performance monitor event select 1
0x2010	pmEventSelect2	Performance monitor event select 2
0x2018	pmEventSelect3	Performance monitor event select 3
0x2020	pmEventSelect4	Performance monitor event select 4
0x2028	pmEventSelect5	Performance monitor event select 5
0x2800	pmClientFilter	Performance counter client disable mask
0x3000	pmEventCounter0	Performance monitor event counter 0
0x3008	pmEventCounter1	Performance monitor event counter 1
0x3010	pmEventCounter2	Performance monitor event counter 2
0x3018	pmEventCounter3	Performance monitor event counter 3
0x3020	pmEventCounter4	Performance monitor event counter 4
0x3028	pmEventCounter5	Performance monitor event counter 5

Table 150: L3 Cache Controller Memory Map

13.4 Register Descriptions

This section describes the functionality of the memory-mapped registers in the Level 3 Cache Controller.

13.4.1 L3 Cache Configuration Register (Config)

The Config Register can be used to programmatically determine information regarding the cache size and organization.

L3 Cache Configuration Register (Config)				
Register Offset		0x0		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	Banks	RO	0x4	Number of banks in the cache
[15:8]	Ways	RO	0x8	Number of ways per bank
[23:16]	lgSets	RO	0xA	Base-2 logarithm of the sets per bank
[31:24]	lgBlockBytes	RO	0x6	Base-2 logarithm of the bytes per cache block

Table 151: L3 Cache Configuration Register

13.4.2 Way Enable Register (WayEnable)

The wayEnable register determines which ways of the Level 3 Cache Controller are enabled as cache. Cache ways that are not enabled are mapped into the P550 Core Complex's L3 LIM (Loosely-Integrated Memory) as described in the memory map in Section 4.2.

This register is initialized to 0 on reset and may only be increased. This means that, out of reset, only a single L3 cache way is enabled, as one cache way must always remain enabled. Once a cache way is enabled, the only way to map it back into the L3 LIM address space is by a reset.

Way Enable Register (WayEnable)				
Register Offset		0x8		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	WayEnable	WARL	0x0	The index of the largest way which has been enabled. May only be increased.

Table 152: Way Enable Register

13.4.3 ECC Error Injection Register (ECCInjectError)

The ECCInjectError register can be used to insert an ECC error into either the backing data or metadata SRAM. This function can be used to test error correction logic, measurement, and recovery.

ECC Error Injection Register (ECCInjectError)				
Register Offset		0x40		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	ECCToggleBit	RW	0x0	Toggle (corrupt) this bit index on the next cache operation
[15:8]	Reserved			
16	ECCToggleType	RW	0x0	Toggle (corrupt) a bit in 0=data or 1=directory
[31:17]	Reserved			

Table 153: ECC Error Injection Register

13.4.4 ECC Directory Fix Registers (DirECCFix*)

The DirECCFixHigh and DirECCFixLow registers are read-only registers that contain the address of the most recently corrected L2 metadata error. This field supplies only the portions of the address that correspond to the affected set, since all ways are corrected together.

The DirECCFixCount register is a read-only register that contains the number of corrected L3 metadata errors. Reading this register clears the DirError interrupt signal described in Section 13.2.6.

13.4.5 ECC Directory Fail Registers (DirECCFail*)

The DirECCFailLow and DirECCFailHigh registers are read-only registers that contains the address of the most recent uncorrected L3 metadata error.

The DirECCFailCount register is a read-only register that contains the number of uncorrected L3 metadata errors. Reading this register clears the DirFail interrupt signal described in Section 13.2.6.

13.4.6 ECC Data Fix Registers (DataECCFix*)

The DataECCFixLow and DataECCFixHigh registers are read-only registers that contain the address of the most recently corrected L3 data error.

The DataECCFixCount register is a read-only register that contains the number of corrected L2 data errors. Reading this register clears the DataError interrupt signal described in Section 13.2.6.

13.4.7 ECC Data Fail Registers (DataECCFail*)

The DataECCFailLow and DataECCFailHigh registers are a read-only registers that contain the address of the most recent uncorrected L3 data error.

The `DirECCFailCount` register is a read-only register that contains the number of uncorrected L2 data errors. Reading this register clears the `DataFail` interrupt signal described in Section 13.2.6.

13.4.8 L3 Cache ECC Error Injection and Correction

Writes to the `ECCInjectError` register, described in Section 13.4.3, specifies a bit position within the combined data+ECC (or directoryEntry+ECC) granule to toggle or corrupt exactly 1 bit the next time an L3 entry is written.

The `ECCToggleBit` register specifies a hex value corresponding to which bit to toggle. For example, if you write a value of `0x47`, bit 71 will be toggled (consequently, for a write to an L3 data RAM that has 64 bits of data and 8 bits of ECC, writing a value $> 0x47$ will have no effect).

The error will be injected (i.e., the bit will be toggled/corrupted) on the next write. When there are multiple data+ECC granules comprising a cache line, error injection will always be applied to the first of these granules. For instance, when you have a 64+8-bit granule and a 64-byte line, it takes eight granules to make up a full cache line, and errors will be injected into the specified bit in the first of these eight granules. For metadata, this is a non-issue since there is only one directoryEntry+ECC granule for each cache line.

The error will not be reported until the corrupted entry is subsequently read, and the error is detected/corrected; that is, this will be at an arbitrary time after the `ECCInjectError` Register is written, unless a test specifically ensures that it reads the particular set and way that was corrupted due to that write.

The address reported in the fix/fail address registers (`DirECCFix` and `DirECCFail`, respectively) will always be the cache-line-size-aligned address of the start of the cache line in which the error was detected.

For error injection on the data, if the L1-L3 interface (and consequently, the width of the L3 data R/W interface) is larger than the ECC granule, the designated bit will actually be toggled on multiple granules; the number of granules being as many granules as make up the data interface. For example, if the L3 data interface is 128-bits wide and the data+ECC granule is 72-bits, two bits will get toggled—one per granule. This is, however, invisible to software since, when these bits are read, only a single corrected error will be reported, and the address will be the start of the cache line as stated above.

For the reported fix address (`DirECCFix`) on corrected errors in the metadata, only the address bits corresponding to the set (i.e., the index) are captured. In contrast, address bits corresponding to the tag are not captured and they will read out as 0.

13.4.9 Cache Flush Registers (Flush*)

The P550 Core Complex L3 Cache Controller provides two registers that can be used for flushing specific cache blocks.

Flush64 is a 64-bit write-only register that flushes the cache block containing the address written. Flush32 is a 32-bit write-only register that flushes a cache block containing the written address left shifted by 4 bytes. In both registers, all bits must be written in a single access for the flush to take effect.

The flush operation performs a write-back and invalidate, meaning the contents are written to memory and L3, pL2, and L1 cache lines are then invalidated.

13.4.10 Way Mask Registers (wayMask*)

The wayMaskN register allows a master connected to the L3 Cache Controller to specify which L3 Cache ways can be evicted by Master N. Masters can still access memory cached in masked ways. The mapping between masters and their L3 master IDs is shown in Table 155.

At least one cache way must be enabled. It is recommended to set/clear bits in this register using atomic operations.

Way Mask 0 Register (wayMask0)				
Register Offset		0x800		
Bits	Field Name	Attr.	Rst.	Description
0	WayMask0_0	RW	0x1	Enable way 0 for Master 0
1	WayMask0_1	RW	0x1	Enable way 1 for Master 0
2	WayMask0_2	RW	0x1	Enable way 2 for Master 0
3	WayMask0_3	RW	0x1	Enable way 3 for Master 0
4	WayMask0_4	RW	0x1	Enable way 4 for Master 0
5	WayMask0_5	RW	0x1	Enable way 5 for Master 0
6	WayMask0_6	RW	0x1	Enable way 6 for Master 0
7	WayMask0_7	RW	0x1	Enable way 7 for Master 0

Table 154: Way Mask 0 Register

Master ID	Description
0	Debug
1	AXI4 Front Port ID#0
2	AXI4 Front Port ID#1
3	AXI4 Front Port ID#2
4	AXI4 Front Port ID#3
5	Trace Sink SBA
6	Hart 0 Private L2 Cache

Table 155: Master IDs in the L3 Cache Controller

13.5 Procedure to Flush the L3 Cache

This section describes how to flush the L3 Cache using the Zero Device. Alternatively, there is a flush-by-address function in the L3 Controller space, described in Section 13.4.9. Note that using the Zero Device to flush the entire cache is faster than flushing by address.

To flush a single index+way:

1. Write wayMaskN to allow evictions from only the specified way
2. Issue a load (or store) to an address in the L3 Zero Device region that corresponds to the specified index

To flush the entire L3 cache:

1. Write wayMaskN to allow evictions from only way 0
2. Issue a series of loads (or stores) to addresses in the L3 Zero Device region that correspond to each L3 index (i.e., one load/store per 64 B, total of (way-size-in-bytes/64) loads or stores).
3. Write wayMaskN to allow evictions from only way 1
4. Repeat step 2
5. Repeat steps 3 and 4, moving through each way of the cache, until all ways have been flushed.

To flush a range of physical addresses much larger than a cache way:

1. Flush the whole cache as shown above

To flush a range of physical addresses not much larger than a cache way:

1. Use the existing flush-by-address mechanism and iterate over the addresses, or write wayMaskN to allow evictions from only way 0
2. Issue a series of loads (or stores) to addresses in the L3 Zero Device region that correspond to the L3 index associated with each 64 B chunk within the specified address range (i.e., one load/store per 64 B, total of (specified-address-range-in-bytes/64) load or stores).
3. Write wayMaskN to allow evictions from only way 1
4. Repeat step 2
5. Repeat steps 3 and 4, moving through each way of the cache, until all ways have been flushed (this should all be done with no intervening stores that could create new dirty lines)

Chapter 14

Power Management

The following chapter describes power modes and establishes flows for powering up, powering down, and resetting the hardware of the P550 Core Complex.

14.1 Power Modes

Power modes include normal run mode with the Power Dial option and wait-for-interrupt clock gating mode using the `wFI` instruction. Additionally, there is a full power down mode supported via the `CEASE` instruction. These modes are covered in detail below.

14.2 Run Mode

The hart is fully operational in run mode, and SiFive designs include the option to include coarse-grained architectural clock gating. When this feature is enabled in the hart, the I-Cache, D-Cache, integer pipeline, Debug Logic, and Floating-Point Unit (FPU) each contain their own clock gate module. The clock gating feature will enable automatic clock gating of functional units when they are inactive and allow the hart to gate its own clock(s) based on activity.

14.2.1 Power Control

To further reduce power while in run mode, users may choose to reduce `external_source_for_core_N_clock`, which is required to be changed synchronously to the rest of the clocks in the system. It is important to note that the clock relationships with the rest of the system must still be maintained if `external_source_for_core_N_clock` is reduced.

To limit maximum power without frequency changes, Power Dial provides a method of scaling down dynamic power in a core. Power is reduced by restricting cycles allowed to advance instructions into the execution pipeline. This feature can typically be used for throttling high CPU usage applications.

Power Dial Register (PowerDial1)			
Register Address		0x7C8	
Bits	Field Name	Rst.	Description
[3:0]	IssueRate	0x0	(1 - value/16) portion of the peak instruction throughput (i.e., value=0 is no reduction)
[31:4]	Reserved	R0	

Table 156: Power Dial Register

The Power Dial Register may only be programmed in M-mode. When written with a non-zero value, the register restricts peak instruction throughput to the indicated rate. A value of 0 has no effect on instruction throughput. The rate is calculated per 256-cycle period, so a Power Dial value of 1 restricts instruction throughput at a common point in the pipeline to 240 cycles of each 256-cycle period. Reducing the peak rate reduces the worst-case power while minimizing the impact on performance.

14.3 WFI Clock Gate Mode

WFI clock gating mode can be entered by executing the WFI instruction. The assembly-level instruction is simply `wfi` and executing the C method using the GCC compiler can be accomplished with `asm("WFI")`.

14.3.1 WFI Wake Up

Wake up from a WFI occurs when the hart receives any interrupt. Depending on the software configuration, the hart will either immediately enter the interrupt handler, or resume execution on the instruction immediately after the WFI.

If interrupts are enabled and `mstatus.MIE=1`, then the hart will wake when an interrupt is enabled and becomes pending, and immediately enter the interrupt handler. Upon exit from the interrupt handler, program execution will resume at the instruction following the WFI.

If interrupts are enabled but `mstatus.MIE=0`, then the hart will wake when an interrupt is enabled and becomes pending but will not enter the interrupt handler. It will simply resume at the instruction immediately after the WFI in this case.

To prevent an interrupt source from waking a hart, the enable bit for that interrupt must be written to 0 prior to executing the WFI instruction. If any interrupts are pending upon executing a WFI instruction, then the WFI is effectively treated as a NOP instruction.

Refer to Chapter 7 for more detail on interrupt configuration.

14.4 CEASE Instruction for Power Down

To fully power down, follow the steps described in Section 14.9, where the last step is to execute a CEASE instruction. Once the CEASE instruction is executed, the core will not retire another instruction until reset. The CEASE opcode is 0x30500073 and can be implemented in either assembly or C. To create an assembly-level function using GCC, consider the following example.

```
.global _cease
.type      _cease, @function
_cease:
    .word 0x30500073
    ret
```

The next example demonstrates how to implement the CEASE instruction within a function in C.

```
static inline void cease()
{
    __asm__ __volatile__ (".word 0x30500073" : : : "memory"); // CEASE
}
```

14.5 Hardware Reset

The following list summarizes the hardware reset values required by *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10* and applies to all SiFive designs.

1. Privilege mode is set to machine mode.
2. mstatus.MIE and mstatus.MPRV are required to be 0.
3. The misa register holds the full set of supported extensions for that implementation, and misa.MXL defaults to the widest supported ISA available, referred to as MXLEN.
4. The pc is set to the implementation specific reset vector.
5. The mcause register is set to 0x0 at reset.
6. The PMP configuration fields for address matching mode (A) and Lock (L) are set to 0, which defaults to no protection for any privilege level.

The internal state of the rest of the system should be completed by software early in the boot flow.

14.6 Early Boot Flow

For the early stages of boot, some of the first things software must consider are listed below:

- The global pointer (gp or x3) user register should be initialized to the `__global_pointer$` linker generated symbol and not changed at any point in the application program.

- The stack pointer (sp or x2) user register should be also set up as a standard part of the boot flow.
- All other user registers (x1, x4 - x31) can be written to 0 upon initial power-on.
- The mtvec register holds the default exception handler base address, so it is important to set up this register early in the boot flow, so it points to a properly aligned, valid exception handler location.
- Zero out the bss section and copy data sections into RAM areas as needed.

14.7 Interrupt State During Early Boot

Since `mstatus.MIE` defaults to 0, all interrupts are disabled globally out of reset. Prior to enabling interrupts globally through `mstatus.MIE`, consider the following:

- Ensure no timer interrupts are pending by checking the `mip.MTIP` bit. The `mtime` register is 0 out of reset and starts running immediately. However, the `mtimecmp` register does not have a reset value.

If no timer interrupt is required, leave `mie.MTIE` equal to 0 prior to enabling global interrupt with `mstatus.MIE`.

If the application requires a timer interrupt, write `mtimecmp` to a value in the future for the next timer interrupt before enabling `mstatus.MIE`.

- Write the remaining bits in the `mie` CSR to the desired value to enable interrupts based on the requirements of the system. This register is not defined to have a reset value.
- Each `msip` register in the Core-Local Interruptor (CLINT) or Core-Local Interrupt Controller (CLIC) address space is reset to 0, so no specific initialization is required for local software interrupts.

Since `msip` is memory-mapped, any hart in the system may trigger a software interrupt on another hart, so this should be considered during the boot flow on a multi-hart system.

- If a Platform-Level Interrupt Controller (PLIC) exists, check the PLIC pending status. The PLIC memory mapped pending bits are read-only, so the pending status should be cleared at the source if they reset to a non-zero status. Then, enable the PLIC interrupts as required by the system prior to enabling interrupts in the system via `mstatus.MIE`.

If an L3 Cache or Bus-Error Unit (BEU) is present, these interrupt IDs begin at 128, so the enable bits may lie in a different region of the memory map than other PLIC enable bits in the design.

- Wipe down memory if enabled with ECC. This can be done by writing 0x0 to memory with either store instructions issued by the CPU or using a DMA controller. ECC errors are reported via the Bus-Error Unit (BEU).
- Check BEU registers to ensure no errors are reported and that the enable bits reflect the requirements of the application.

14.8 Other Boot Time Considerations

- Ensure the remaining bits in the `mstatus` CSR are written to the desired application specific configuration at boot time.
- If a design includes user and supervisor privilege levels, initialize `medeleg` and `mideleg` registers to 0 until supervisor-level trap handling is set up correctly using `stvec`.
- The `mcause`, `mepc`, and `mtval` registers hold important information in the event of a synchronous exception. If the synchronous exception handler forces reset in the application, the contents of these registers can be checked to understand root cause.
- The PMP address and configuration CSRs are required to be initialized if user or supervisor privilege levels are part of the design. By default, user and supervisor modes have no permissions to the memory map unless explicitly granted by the PMP.
- The `mcycle` CSR is a 64-bit counter on both RV32 and RV64 systems, and it counts the number of cycles executed by the hart. It has an arbitrary value after reset and can be written as needed by the application.
- Instructions retired can be counted by the `minstret` register, and this also has an arbitrary value after reset. This can be written to any given value.
- The `mhpmeventX` CSR selects which hardware events to count, where the count is reflected in `mhpmcouterX`. At any point, the `mhpmcouterX` registers can be directly written to reset their value when the `mhpmeventX` register has the proper event selected.
- For cores with an MMU, ensure the `satp` register holds the correct configuration for address translation.
- There is no requirement for boot time initialization to any of the registers within the Debug Module, unless there is an application specific reason to do so.
- All other CSRs during boot time initialization should be considered based on system and application requirements.

14.9 Power-Down Flow

For SiFive Core IP, coordination with an "External Agent" is required.

1. External Agent: Wait for communication from the core to initiate the following steps:
 - a. Stop sending inbound traffic (both transactions and interrupts) into the Core Complex.
 - b. Wait until all outstanding requests to the Core Complex are completed, then
 - c. Wait until `cease_from_tile_N` is high for the core.
 - d. Once `cease_from_tile_N` is high for the core, apply reset to the entire Core Complex.
2. Core:

- a. The following sequence should be executed in machine mode and NOT out of a remote ITIM/DTIM.
- b. Communicate with external agent to initiate cease power-down sequence.
- c. Poll external agent until steps 1.a and 1.b are completed.
- d. Disable all interrupts except those related to bus errors/memory corruption.
 - i. Copy contents of any TIMs/LIMs into external memory.
 - ii. Core: if there is an L3 cache, flush it (all addresses at which cacheable physical memory exists).
 - iii. If there is no L3 cache, but there is a data cache, flush it using full-cache variant of CFLUSH.D.L1 if available, or per-line variant if not.
- e. Disable all interrupts.
- f. Execute CEASE instruction.

Chapter 15

Debug

This chapter describes the operation of SiFive debug hardware, which follows *The RISC-V Debug Specification, Version 1.0*. Currently, only interactive debug and hardware breakpoints are supported.

15.1 Debug Module

The Debug Module (DM) handles nearly all of the functions related to debugging. It is a slave to both the Debug Module Interface (DMI) coming from the probe, and a TileLink bus coming from the core. From the perspective of the core, the DM appears as a 4K block in the memory map. The DM memory map as seen from the perspective of the core is shown in Table 158, and the register map from the perspective of the DMI is shown in Table 157.

Most of the DM is clocked by `debug_clock`. The `dmcontrol` register is accessible when `debug_clock` is not running, mainly to be able to write to `haltreq` while the core is in reset due to `ndreset`. Doing so generates a debug interrupt and will interrupt the selected core immediately once it is out of reset or during a WFI instruction.

DMI Address	Name	Description
0x11	dmstatus	Debug Module Status. See Table 170 for more information.
0x10	dmcontrol	Debug Module Control. See Table 171 for more information.
0x12	hartinfo	Hart Information. See Table 172 for more information.
0x40	haltsum0	Read-only. Halt Summary 0. Bit n reads 1 if hart n is halted.
0x13	haltsum1	Read-only. Halt Summary 1. Only present on systems with >32 harts. Not used by SiFive.
0x16	abstractcs	Abstract Control and Status. See Table 173 for more information.
0x18	abstractauto	Selects whether access to particular DATA or PROGBUF locations will re-execute the last command. Used for block transfers or other repeating commands. See Table 175 for more information.
0x17	command	Initiate abstract command. See Table 174 for more information.
0x04-0x0F	data0-data11	Read/Write DATA registers. 32-bit SiFive cores have 1 data register, 64-bit cores have 2.
0x20-0x2F	progbuf0-progbuf15	Read/Write PROGBUF registers.
0x32	dmcs2	Fields to set up and read back Halt Group or Resume Group configuration. Present by default on systems with more than 1 hart or with any external triggers. See Table 176 for more information.
0x37-0x3F	sbXXXX	Read/Write. System Bus Access.

Table 157: Debug Module Memory Map Seen from the Debug Module Interface

From the point of view of the core, the DM appears as a 4K block of memory. It is mapped into low memory so that memory references can use addresses relative to the \$zero register.

TL Address	Name	Attr.	Description
0x100	HALTED	WO	Written with hartid by ROM code when hart gets a debug interrupt or reenters ROM due to EBREAK. Sets halted[hartid]. If an abstract command was running, writing this also clears busy.
0x104	GOING	WO	Written by ROM code when it begins executing a command started by FLAGS[hartid].go. Clears FLAGS[hartid].go.
0x108	RESUMING	WO	Written with hartid by hart when it is about to resume. Sets resumeack[hartid] and clears halted[hartid] and FLAGS[hartid].resume.
0x10C	EXCEPTION	WO	Written by hart when it encounters an exception in debug mode. Sets cmderr to "exception".
0x300	WHERE0	RO	JAL to ABSTRACT. This opcode is constructed by DM hardware and is needed because ABSTRACT is not a fixed address (depends on number of PROGBUF words selected in the configuration).
contiguous	ABSTRACT	RO	2 words constructed by DM hardware based on abstract command written from DTM. +0: If transfer set, construct instruction to load/store specific register to/from DATA[0] (32 bits) or DATA[1:0] (64 bits), else NOP. +4: If postexec set, then NOP to fall thru and execute PROGBUF, else EBREAK to return to ROM park loop.
contiguous	PROGBUF	RW	Configurable number (typically 16, max 16) of R/W words to be filled in by debugger and executed by hart.
0x380-0x3BF	DATA	RW	Configurable number (1 for 32-bit or 2 for 64-bit, max 12) of R/W words intended for use for data transfer between debugger and hart. Since it is contiguous with PROGBUF, the debugger may use DATA as an extension of PROGBUF.
0x400-0x7FF	FLAGS	RO	One byte flag per hart. Bit 0 (go): Set by writing an abstract command, cleared by ROM write to GOING. ROM will jump to WHERE0. Bit 1 (resume): Set by writing 1 to resumereq[hartid]. Cleared by ROM write of hartid to RESUMING. ROM restores s0 then executes dret.
0x800-0xFFF	ROM	RO	Debug interrupt or EBREAK enters at 0x800, saves s0, writes hartid to HALTED, then busy-waits for FLAGS[hartid] > 0. If FLAGS[hartid].go, write 0 to GOING, then jump to WHERE0.

Table 158: Debug Module Memory Map from the Perspective of the Core

TL Address	Name	Attr.	Description
			Else write hartid to RESUMING, then execute dret to return to user program. ROM Source Code: https://github.com/chipsalliance/rocket-chip/blob/master/scripts/debug_rom/debug_rom.S

Table 158: Debug Module Memory Map from the Perspective of the Core

15.2 Debug and Trigger Registers

This section describes the per hart debug and trigger registers, which are mapped into the CSR space as follows:

CSR	Name	Allowed Access Mode	Description
0x7B0	dcsr	Debug	Debug Control and Status Register
0x7B1	dpc	Debug	Debug PC. Stores execution address just before debug exception and to return to at dret.
0x7B2	dscratch0	Debug	Debug Scratch Register 0
0x7A0	tselect	Debug, Machine	Trigger Select. Most configs implement 2, 4, or 8 triggers. Triggers are all type 2 (address/data).
0x7A1	tdata1	Debug, Machine	Trigger Data 1, mcontrol
0x7A2	tdata2	Debug, Machine	Trigger Data 2, the address for comparison
0x7A3	tdata3	Debug, Machine	Trigger Data 3

Table 159: Debug and Trigger Registers

15.2.1 Debug Control and Status Register (dcsr)

This register gives information about debug capabilities and status. Its detailed functionality is described in *The RISC-V Debug Specification, Version 1.0*.

Debug Control and Status Register (dcsr)			
CSR	0x7B0		
Bits	Field Name	Attr.	Description
[1:0]	prv	RW	Privilege level of processor prior to debug exception and to return to at dret.
2	step	RW	Set to 0x1 to single-step.
3	nmip	RO	Non-maskable interrupt pending. Not used by SiFive.
4	mprven	WARL	Not used by SiFive.
5	Reserved		
[8:6]	cause	RO	Indicates cause of most recent debug exception.
9	stoptime	WARL	0x1 will stop timers in debug mode. Not used by SiFive (timers continue).
10	stopcount	WARL	0x1 will stop counters in debug mode. Not used by SiFive (counters continue).
11	stepie	WARL	Enable interrupts when stepping. Not used by SiFive (interrupts disabled).
12	ebreaku	RW	EBREAK instructions in U-mode enter debug mode (vs. breakpoint exception).
13	ebreaks	RW	EBREAK instructions in S-mode enter debug mode.
14	Reserved		
15	ebreakm	RW	EBREAK instructions in M-mode enter debug mode.
[27:16]	Reserved		
[31:28]	xdebugver	RO	Version

Table 160: Debug Control and Status Register

15.2.2 Debug PC (dpc)

When entering debug mode, the current PC is copied here. When leaving debug mode, execution resumes at this PC.

15.2.3 Debug Scratch (dscratch)

This register is generally reserved for use by Debug ROM in order to save registers needed by the code in Debug ROM. The debugger may use it as described in *The RISC-V Debug Specification, Version 1.0*.

15.2.4 Trigger Select Register (tselect)

To support a large and variable number of triggers for tracing and breakpoints, they are accessed through one level of indirection where the `tselect` register selects which bank of three `tdata1-3` registers are accessed via the other three addresses.

The `tselect` register has the format shown below:

Trigger Select Register (tselect)			
CSR	0x7A0		
Bits	Field Name	Attr.	Description
[63:0]	index	WARL	Selection index of triggers

Table 161: Trigger Select Register

The `index` field is a **WARL** field that does not hold indices of unimplemented triggers. Even if `index` can hold a trigger index, it does not guarantee the trigger exists. The `type` field of `tdata1` must be inspected to determine whether the trigger exists.

15.2.5 Trigger Data Registers (tdata1-3)

The `tdata1-3` registers are 64-bit read/write registers selected from a larger underlying bank of triggers by the `tselect` register.

Trigger Data Register 1 (tdata1)			
CSR	0x7A1		
Bits	Field Name	Attr.	Description
[58:0]	Trigger-Specific Data		
59	dmode	WARL	Selects between debug mode (dmode=1) and machine mode (dmode=0) views of the registers, where only debug mode code can access the debug mode view of the triggers
[63:60]	type	WARL	The type of trigger selected by <code>tselect</code> 0x0 - No such trigger 0x1 - Reserved 0x2 - Address/Data Match Trigger ≥0x3 - Reserved

Table 162: Trigger Data Register 1

Trigger Data Registers 2 and 3 (tdata2/3)			
CSR	0x7A2 - 0x7A3		
Bits	Field Name	Attr.	Description
[63:0]	Trigger-Specific Data		

Table 163: Trigger Data Registers 2 and 3

Any attempt to read/write the tdata1-3 registers in machine mode when TSELECT.dmode=1 raises an illegal-instruction exception.

15.3 Breakpoints

The P550 Core Complex supports four hardware breakpoint registers per hart, which can be flexibly shared between debug mode and machine mode.

When a breakpoint register is selected with tselect, the other CSRs access the following information for the selected breakpoint:

CSR Name	Breakpoint Alias	Description
tselect	tselect	Breakpoint selection index
tdata1	mcontrol	Breakpoint match control
tdata2	maddress	Breakpoint match address
tdata3	N/A	Reserved

Table 164: Trigger CSRs When Used as Breakpoints

15.3.1 Breakpoint Match Control Register (mcontrol)

Each breakpoint control register is a read/write register laid out in Table 165. This register is accessible as tdata1 when type is 0x2.

Breakpoint Match Control Register (mcontrol1)				
CSR	0x7A1			
Bits	Field Name	Attr.	Rst.	Description
0	R	WARL	X	Address match on LOAD
1	W	WARL	X	Address match on STORE
2	X	WARL	X	Address match on Instruction FETCH
3	U	WARL	X	Address match on user mode
4	S	WARL	X	Address match on supervisor mode
5	Reserved	WPRI	X	
6	M	WARL	X	Address match on machine mode
[10:7]	match	WARL	X	Breakpoint Address Match type • 0x0 - Single address • 0x1 - Power-of-2 range, limited to 64 bytes in SiFive implementations • 0x2 - ≥ address • 0x3 - < address • Others not supported by SiFive
11	chain	WARL	0x0	Chain adjacent conditions. When set, this trigger and the next must match at the same time to fire. Typically used for a range breakpoint using 2 triggers, one with match=0x2 and one with match=0x3. This is not a sequential trigger.
[15:12]	action	WARL	0x0	Breakpoint action to take
[17:16]	szelo	WARL	0x0	Size of the breakpoint. Fixed at 0, meaning accesses of any size that cover any part of the trigger address range will fire.
18	timing	WARL	0x0	Timing of the breakpoint. Fixed at 0, meaning breaks happen just before the event.
19	select	WARL	0x0	Perform match on address or data. Fixed at 0, meaning all triggers compare addresses only (no data value).
[52:20]	Reserved	WPRI	X	
[58:53]	maskmax	RO	0x4	Largest supported NAPOT range
59	dmode	RW	0x0	Debug-Only access mode
[63:60]	type	RO	0x2	Address/Data match type, always 0x2

Table 165: Breakpoint Match Control Register

The type field is a 4-bit read-only field holding the value 0x2 to indicate this is a breakpoint containing address match logic.

The action field is a 4-bit read-write **WARL** field that specifies the available actions when the address match is successful. The value 0 generates a breakpoint exception. The value 1 enters debug mode. The value 2 starts tracing. The value 3 stops tracing. The value 4 inserts a program trace sync message. The value 5 generates a trace external trigger out. Other actions are not implemented.

The R/W/X bits are individual **WARL** fields, and if set, indicate an address match should only be successful for loads, stores, and instruction fetches, respectively. All combinations of implemented bits must be supported.

The M/S/U bits are individual **WARL** fields, and if set, indicate that an address match should only be successful in the machine, supervisor, and user modes, respectively. All combinations of implemented bits must be supported.

The match field is a 4-bit read-write **WARL** field that encodes the type of address range for breakpoint address matching. Three different match settings are currently supported: exact, NAPOT, and arbitrary range. A single breakpoint register supports both exact address matches and matches with address ranges that are naturally aligned powers-of-two (NAPOT) in size. Breakpoint registers can be paired to specify arbitrary exact ranges, with the lower-numbered breakpoint register giving the byte address at the bottom of the range and the higher-numbered breakpoint register giving the address 1 byte above the breakpoint range and using the chain bit to indicate both must match for the action to be taken.

NAPOT ranges make use of low-order bits of the associated breakpoint address register to encode the size of the range as follows:

address	Match Type and Size
a...aaaaaa	Exactly 1 byte
a...aaaaa0	2-byte NAPOT range
a...aaaa01	4-byte NAPOT range
a...aaa011	8-byte NAPOT range
a...aa0111	16-byte NAPOT range
a...a01111	32-byte NAPOT range
...	
a01...1111	2 ³¹ -byte NAPOT range

Table 166: NAPOT Size Encoding

The maskmax field is a 6-bit read-only field that specifies the largest supported NAPOT range. The value is the logarithm base 2 of the number of bytes in the largest supported NAPOT range. A value of 0 indicates that only exact address matches are supported (1-byte range). A value of 31 corresponds to the maximum NAPOT range, which is 2³¹ bytes in size. The largest range is

encoded in `maddress` with the 30 least-significant bits set to 1, bit 30 set to 0, and bit 31 holding the only address bit considered in the address comparison.

To provide breakpoints on an exact range, two neighboring breakpoints can be combined with the `chain` bit. The first breakpoint can be set to match on an address using `action` of 2 (greater than or equal). The second breakpoint can be set to match on address using `action` of 3 (less than). Setting the `chain` bit on the first breakpoint prevents the second breakpoint from firing unless they both match.

15.3.2 Breakpoint Match Address Register (`maddress`)

Each breakpoint match address register is a 64-bit read/write register used to hold significant address bits for address matching and also the unary-encoded address masking information for NAPOT ranges.

15.3.3 Breakpoint Execution

Breakpoint traps are taken precisely. Implementations that emulate misaligned accesses in software will generate a breakpoint trap when either half of the emulated access falls within the address range. Implementations that support misaligned accesses in hardware must trap if any byte of an access falls within the matching range.

Debug mode breakpoint traps jump to the debug trap vector without altering machine mode registers.

Machine mode breakpoint traps jump to the exception vector with "Breakpoint" set in the `mcause` register and with `badaddr` holding the instruction or data address that caused the trap.

15.3.4 Sharing Breakpoints Between Debug and Machine Mode

When debug mode uses a breakpoint register, it is no longer visible to machine mode (that is, the `tdrtype` will be 0). Typically, a debugger will leave the breakpoints alone until it needs them, either because a user explicitly requested one or because the user is debugging code in ROM.

15.4 Debug Memory Map

This section describes the Debug Module's memory map when accessed via the regular system interconnect. The Debug Module is only accessible to debug code running in debug mode on a hart (or via a Debug Transport Module). The following addresses are offsets from the base address of the Debug Module. Note that the PMP must allow M-mode access to the Debug Module address range for debugging to be possible.

15.4.1 Debug RAM and Program Buffer (0x300–0x3FF)

The P550 Core Complex has 16 32-bit words of program buffer for the debugger to direct a hart to execute arbitrary RISC-V code. Its location in memory can be determined by executing `aiupc` instructions and storing the result into the program buffer.

The P550 Core Complex has two 32-bit words of debug data RAM. Its location can be determined by reading the `DM.hartinfo` register, as described in *The RISC-V Debug Specification, Version 1.0*. This RAM space is used to pass data for the Access Register abstract command, as described in *The RISC-V Debug Specification, Version 1.0*. The P550 Core Complex supports only general-purpose register access when harts are halted. All other commands must be implemented by executing from the debug program buffer.

In the P550 Core Complex, both the program buffer and debug data RAM are general-purpose RAM and are mapped contiguously in the Core Complex memory space. Therefore, additional data can be passed in the program buffer, and additional instructions can be stored in the debug data RAM.

Debuggers must not execute program buffer programs that access any Debug Module memory except defined program buffer and debug data addresses.

15.4.2 Debug ROM (0x800–0xFFFF)

This ROM region holds the debug routines on SiFive systems. The actual total size may vary between implementations.

15.4.3 Debug Flags (0x100–0x110, 0x400–0x7FF)

The flag registers in the Debug Module are used for the Debug Module to communicate with each hart. These flags are set and read used by the debug ROM and should not be accessed by any program buffer code. The specific behavior of the flags is not further documented here.

15.4.4 Safe Address

In the P550 Core Complex, the Debug Module contains the Debug Module address range in the memory map. Memory accesses to these addresses raise access exceptions, unless the hart is in debug mode. This property allows a "safe" location for unprogrammed parts, as the default `mtvec` location is `0x0`.

15.5 Instruction Trace Interface

This section describes the interface between a SiFive core and its trace unit. The trace interface conveys information about instruction-retirement and exception events.

Table 168 lists the fields in a trace packet. The `valid` signal is 1 if and only if an instruction retires or traps (either by generating a synchronous exception or taking an interrupt). The remaining fields in the packet are only defined when `valid` is 1.

The `iaddr` field holds the address of the instruction that retired or trapped. If address translation is enabled, it is a virtual address, else it is a physical address. Virtual addresses narrower than 64 bits are sign-extended, and physical addresses narrower than 64 bits are zero-extended.

The `insn` field holds the instruction that retired or trapped. For instructions narrower than the maximum width, e.g., those in the RISC-V C extension, the unused high-order bits are zero-filled. The length of the instruction can be determined by examining the low-order bits of the instruction, as described in *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1*. The width of the `insn` field, `ILEN`, is 32 bits for current implementations.

The `priv` field indicates the privilege mode at the time of instruction execution. On an exception, the **next** valid trace packet's `priv` field gives the privilege mode of the activated trap handler. The width of the `priv` field, `PRIVLEN`, is 3, and it is encoded as shown in Table 167.

Value*	Description
0x0	User mode
0x1	Supervisor mode
0x3	Machine mode
0x4-0x7	Debug mode
*Unspecified values are reserved.	

Table 167: Encoding of `priv`

The `exception` field is 0 if this packet corresponds to a retired instruction, or 1 if it corresponds to an exception or interrupt. In the former case, the `cause` and `interrupt` fields are undefined and the `tval` field is zero. In the latter case, the fields are set as follows:

- `interrupt` is 0 for synchronous exceptions and 1 for interrupts.
- `cause` supplies the exception or interrupt cause, as would be written to the lower `CAUSELEN` bits of the `mcause` CSR. For current implementations, `CAUSELEN` is $\log_2(64)$.
- `tval` supplies the associated trap value, e.g., the faulting virtual address for address exceptions, as would be written to the `mtval` CSR.

Future optional extensions may define `tval` to provide ancillary information in cases where it currently supplies zero.

For cores that can retire `N` instructions per clock cycle, this interface is replicated `N` times. Lower-numbered entries correspond to older instructions. If fewer than `N` instructions retire, the valid packets need not be consecutive, i.e., there may be invalid packets between two valid packets. If one of the instructions is an exception, no younger instruction will be valid.

Name	Description
valid	Indicates an instruction has retired or trapped.
iaddr[63:0]	The address of the instruction.
insn[31:0]	The instruction.
priv[2:0]	Privilege mode during execution.
exception	0, if the instruction retired; 1, if it trapped.
interrupt	0, if the exception was synchronous; 1, if interrupt.
cause[5:0]	Exception cause.
tval[63:0]	Exception data.

Table 168: SiFive Instruction Trace Interface Fields

15.6 Debug Module Interface

The SiFive Debug Module (DM) conforms to *The RISC-V Debug Specification, Version 1.0*. A debug probe or agent connects to the Debug Module through the Debug Module Interface (DMI). The following sections describe notable spec options used in the implementation and should be read in conjunction with *The RISC-V Debug Specification, Version 1.0*.

DMI is a simple read/write bus whose master is the DTM (if it exists, otherwise DMI passes through to customer logic) and whose slave is the Debug Module. The master sends a request to the slave and the slave responds with a response. A request is considered sent if req_ready=1 indicating the master is sending a request and req_valid=1 indicating the slave is accepting the request on this cycle. Similarly, the response is sent when both resp_valid=1 indicating the slave is sending a response and resp_ready=1 indicating the master is accepting it.

Note

It is the responsibility of the debugger to simulate virtual address accesses by accessing the page tables directly, then sending the translated physical address to hardware when doing the access.

Note

The Debug Module registers are not directly accessible from the core.

Group	Signal	Source	Description
System	clock	system	All signals timed to this clock. With JTAG DTM, this clock is the JTAG TCK.
	reset	system	Synchronous reset. Generated by power-on reset circuit.
Request Bus	req_ready	slave	Slave ready to receive request.
	req_valid	master	Master's request valid.
	req_addr	master	Configurable width address bus. 0x7 for SiFive.
	req_data	master	32-bit write data bus.
	req_op	master	• 0x0 = None • 0x1 = Read • 0x2 = Write • 0x3 = Reserved
Response Bus	resp_ready	master	Master is ready to receive response.
	resp_valid	slave	Slave response is valid.
	resp_data	slave	32-bit read data bus.
	resp_op	slave	• 0x0 = Success • 0x1 = Failure • 0x2 = Not used • 0x3 = Reserved

Table 169: Debug Module Interface Signals

15.6.1 Debug Module Status Register (dmstatus)

dmstatus holds the DM version number and other implementation information. Most importantly, it contains status bits that indicate the current state of the selected hart(s).

Debug Module Status Register (dmstatus)				
DMI Address		0x11		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	version	RO	0x3	Implementation version number
4	Reserved	RO	0x0	
5	hasresethaltreq	RO	0x1	1 if resethaltreq exists
6	authbusy	RO	0x0	0=the Authentication Module is ready, 1=busy
7	authenticated	RO	0x1	0=Debug Module locked, 1=the authentication check has passed
8	anyhalted	RO	0x0	Any currently selected hart is halted
9	allhalted	RO	0x0	All currently selected harts are halted
10	anyrunning	RO	0x1	Any currently selected hart is running
11	allrunning	RO	0x1	All currently selected harts are running
12	anyunavail	RO	0x0	Any currently selected hart is not available (i.e., is powered down). DM supports it, but not currently used by SiFive cores.
13	allunavail	RO	0x0	All currently selected harts are not available (i.e., is powered down). DM supports it, but not currently used by SiFive cores.
14	anynonexistent	RO	0x0	Any currently selected hart does not exist in the system
15	allnnonexistent	RO	0x0	All currently selected harts do not exist in the system
16	anyresumeack	RO	0x1	Any currently selected hart has resumed execution
17	allresumeack	RO	0x1	All currently selected harts have resumed execution
18	anyhavereset	RO	0x0	Any currently selected hart has been reset, but reset has not been acknowledged
19	allhavereset	RO	0x0	All currently selected harts have been reset, but reset has not been acknowledged
[21:20]	Reserved	RO	0x0	
22	impebreak	RO	0x0	1 if PROGBUF is followed by implicit EBREAK. Generally, 1 for E2 cores, 0 otherwise.
[31:23]	Reserved	RO	0x0	

Table 170: Debug Module Status Register

15.6.2 Debug Module Control Register (dmcontrol)

A debugger performs most hart controls through the dmcontrol register.

Debug Module Control Register (dmcontrol)				
DMI Address		0x10		
Bits	Field Name	Attr.	Rst.	Description
0	dmactive	RW	0x0	0 disables the DM and sets DMI registers to their reset state, 1 puts the DM in operational mode. Drives dmactive output that could be used by a system power controller to maintain power to the DM while it is being used. When 1, dmcontrol should be read back until dmactive=1, which indicates that the Debug Module is fully operational. When 0, the DM TileLink clock is gated off to save power.
1	ndmreset	RW	0x0	Write 1 to reset system (assert ndreset output). Write 0 to operate normally.
2	clrresethaltreq	WO	0x0	Write 1 to clear the reset-halt-request bit
3	setresethaltreq	WO	0x0	When written to 1, the core will halt upon the next deassertion of its reset
[27:4]	Reserved	RW	0x0	
28	ackhavereset	WO	0x0	Write 1 to acknowledge that a reset occurred on the selected hart
29	Reserved	RO	0x0	
30	resumereq	WO	0x0	Write 1 to request selected hart to resume, cleared to 0 automatically when hart resumes
31	haltreq	RW	0x0	Write 1 to request selected hart to halt. Generates debug interrupt to the core. Write 0 once halted has been set by the DM.

Table 171: Debug Module Control Register

15.6.3 Hart Info Register (hartinfo)

hartinfo contains information about the currently selected hart.

Hart Info Register (hartinfo)				
DMI Address		0x12		
Bits	Field Name	Attr.	Rst.	Description
[11:0]	dataaddr	RO	0x380	Address of DATA registers in hart memory map. 0x380 for SiFive.
[15:12]	datasize	RO	0x2	Number of DATA registers. 0x1 for 32-bit, 0x2 for 64-bit SiFive cores.
16	dataaccess	RO	0x1	DATA registers are shadowed in the hart memory map. 1 for SiFive.
[19:17]	Reserved	RO	0x0	
[23:20]	nscratch	RO	0x1	Number of dscratch registers available for debugger. 1 for SiFive.
[31:24]	Reserved	RO	0x0	

Table 172: Hart Info Register

15.6.4 Abstract Control and Status Register (abstractcs)

Abstract Control and Status Register (abstractcs)				
DMI Address		0x16		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	datacount	RO	0x2	Number of DATA registers. 0x1 for 32-bit, 0x2 for 64-bit SiFive cores.
[7:4]	Reserved	RO	0x0	
[10:8]	cmderr	RW1C	0x0	Non-zero value indicates an abstract command error. Remains set until cleared by writing all ones. If set, no abstract commands are accepted. • 0x0 - No error • 0x1 - Busy. Abstract command or register was accessed while command was running. • 0x2 - Not supported. Abstract command type not supported by hardware was attempted. • 0x3 - Exception. An exception occurred during execution of an abstract command. • 0x4 - Halt/resume. Abstract command attempted while hart was running or unavailable. • 0x5 - Bus. Bus error occurred during abstract command. Not used by SiFive. • 0x7 - Other. Abstract command failed for another reason. Not used by SiFive.
11	Reserved	RO	0x0	
12	busy	RO	0x0	Reads as 1 while Abstract command is running, 0 if not.
[23:13]	Reserved	RO	0x0	
[28:24]	progbufsize	RO	0x10	Number of 32-bit words in PROGBUF. P550 Core Complex has 16 words.
[31:29]	Reserved	RO	0x0	

Table 173: Abstract Control and Status Register

15.6.5 Abstract Command Register (command)

Abstract Command Register (command)			
DMI Address		0x17	
Bits	Field Name	Attr.	Description
[15:0]	regno	RW	Select which register to read/write. SiFive only supports GPRs: 0x1000-0x101F.
16	write	RW	1=write register, 0=read register. Only done if transfer=1.
17	transfer	RW	1=do the register read/write, 0=don't.
18	postexec	RW	1=execute PROGBUF after the command, 0=don't.
19	aarpostincrement	RW	Not supported by SiFive.
[22:20]	aarsize	RW	0x2, 0x3, 0x4 select 32, 64, 128 bits, respectively.
23	Reserved	RO	0x0
	[31:24]	cmdtype	RW

Table 174: Abstract Command Register**15.6.6 Abstract Command Autoexec Register (abstractauto)**

Abstract Command Autoexec Register (abstractauto)				
DMI Address		0x18		
Bits	Field Name	Attr.	Rst.	Description
[11:0]	autoexecdata	RW	0x0	Bitmap of DATA registers [11:0]. 1 indicates DATA access initiates command.
[15:12]	Reserved	RO	0x0	
[31:16]	autoexecprogbuf	RW	0x0	Bitmap of PROGBUF words [15:0]. 1 indicates PROGBUF access initiates command.

Table 175: Abstract Command Autoexec Register**15.6.7 Debug Module Control and Status 2 Register (dmcs2)**

Table 176 describes the Debug Module Control and Status 2 Register dmcs2. If halt/resume groups are not implemented, then group will always read back as 0. The Debug Module external triggers may be allocated as needed between halt and resume groups.

Debug Module Control and Status 2 Register (dmcs2)				
DMI Address		0x32		
Bits	Field Name	Attr.	Rst.	Description
0	hgselect	RW	0x0	0=operate on harts, 1=operate on external triggers.
1	hgwrite	WO	X	When written with 1, the selected harts or external trigger is assigned to group group.
[6:2]	group	RW	0x0	Specify the halt group or resume group number that the selected harts or external triggers will be assigned to.
[10:7]	Reserved	RO	0x0	
11	grouptype	RW	0x0	0=operate on Halt Group configuration, 1=operate on Resume Group configuration.
[31:12]	Reserved	RO	0x0	

Table 176: Debug Module Control and Status 2 Register

15.6.8 Abstract Commands

Abstract commands provide a debugger with a path to read and write processor state and are used for extracting and modifying processor state such as registers and memory. Register `s0` is saved by the ROM and is available for use by the abstract command code. An abstract command is started by the debugger writing to `command`. In `command`, the debugger selects whether to load/store a register, execute `PROGBUF`, or both. Only GPR register transfers are supported currently. Many aspects of Abstract Commands are optional in *The RISC-V Debug Specification, Version 1.0* and are implemented as described below.

cmdtype	Feature	Support
Access Register	GPR registers	Access Register command, register number 0x1000 - 0x101F
	CSR registers	Not supported. CSRs are accessed using the Program Buffer.
	FPU registers	Not supported. FPU registers are accessed using the Program Buffer.
	Autoexec	Both <code>autoexecprogbuf</code> and <code>autoexecdata</code> are supported.
	Post-increment	Not supported.
	Core Register Access	Not supported.
Quick Access		Not supported.
Access Memory		Not supported. Memory access is accomplished using the Program Buffer.

Table 177: Debug Abstract Commands

The use of abstract commands is outlined in the following example, describing how to read a word of target memory:

1. The debugger writes opcodes to PROGBUF to accomplish the desired function.
2. The debugger writes the desired memory address to DATA[0].
3. The debugger requests an abstract command specifying to load s0 from DATA[0], then execute PROGBUF. Writing to command while hart n is selected has the side effect of setting FLAGS[n].go. Writing to command also sets busy which is readable from the debugger, and indicates that an abstract command is in progress.
4. The ROM busy-wait loop being executed by hart n sees FLAGS[n].go set.
5. ROM code writes 0 to GOING which has the effect of clearing FLAGS[n].go.
6. ROM code jumps to WHERETO, then ABSTRACT which contains the opcode lw s0, 0(DATA) to load s0 from DATA[0]. Opcodes in ABSTRACT are constructed by DM hardware from command. If command.transfer=0, no register transfer is done and instead ABSTRACT[0] reads as NOP.
7. If a register read/write is all that is needed, the debugger would set command.postexec to 0. ABSTRACT[1] would then read as EBREAK.
8. If command.postexec=1, ABSTRACT[1] reads as NOP and execution falls through to PROGBUF which will have been previously written by the debugger with the opcodes lw s0, 0(s0), then sw s0, DATA(zero), then EBREAK.
9. EBREAK reenters ROM at address 0x800. ROM writes hartid to HALTED which has the side effect of clearing busy, telling the debugger that the abstract command is finished.
10. The debugger reads the result from DATA[0].

The autoexec feature of Abstract Commands is supported by SiFive hardware (and is used by OpenOCD for memory block read and write). Once an abstract command has been completed, the debugger can read or write a particular DATA or PROGBUF location to run the command again. For example, fast download can be accomplished by setting up PROGBUF for memory write, then repeatedly writing words to DATA[0]. Each write re-executes the register transfer and PROGBUF to store the word into memory. For a 32-bit block write, the abstract command would be set up like this:

ABSTRACT	regno=s1, write=1, transfer=1, postexec=1. DM constructs the instructions lw s1,0(DATA) // load s1 from debugger NOP // fall thru to PROGBUF
PROGBUF	sw s1, 0(s0) // store s1 to memory addi s0, s0, 4 // increment memory pointer ebreak // done

Table 178: Abstract Command Example for 32-bit Block Write

15.6.9 System Bus Access

System Bus Access (SBA) provides an alternative method to access memory. SBA operation conforms to *The RISC-V Debug Specification, Version 1.0* and its description is not duplicated here. It implements a bus master that connects with the bus crossbar to allow access to the device's physical address space without involving a hart to perform accesses. SBA is controlled from the DMI using registers in the range 0x37 - 0x3F. By default, the maximum bus width supported by SBA is 64. Comparing Program Buffer memory access and SBA:

Program Buffer Memory Access	SBA Memory Access
Physical Address	Physical Address
Subject to Physical Memory Protection (PMP)	Not subject to PMP
Cache coherent	Cache coherent
Hart must be halted	Hart may be halted or running

Table 179: System Bus vs. Program Buffer Comparison

15.7 Debug Module Operational Sequences

The sections below describe the flow for entering into and exiting from debug mode. The user can halt and resume more than one hart at a time using the hart array mask.

15.7.1 Entering Debug Mode

To use debug mode, the DM must be enabled by writing 0x0000_0001 to `dmcontrol`.

The debugger can request a halt by writing 0x8000_0001 to `dmcontrol` to set `haltreq`. This generates a debug interrupt to the core.

The core enters debug mode and jumps to the debug interrupt handler located at 0x800 and serviced from the DM.

ROM code at 0x800 writes `hartid` into the HALTED register which has the effect of setting the halted bit for this hart. Halted bits are readable from the debugger and generally will be continually polled to check for breakpoints when a hart is running.

ROM code then busy-waits checking its hart-specific FLAGS register.

15.7.2 Exiting Debug Mode

The debugger writes 1 to `resumereq` in the `dmcontrol` register to restart execution. This clears `resumeack` and sets bit 1 of the FLAGS register for the selected hart.

The ROM busy-wait loop being executed by hart `n` sees `FLAGS[n].resume` set.

ROM code writes `hartid` to `RESUMING`, which has the effect of clearing `FLAGS[n].resume`, setting `resumeack`, and clearing `halted` for the hart.

ROM code then executes `dret` which returns to user code at the address currently in `dpc`.

The debugger sees `resumeack` and knows the resume was successful.

Chapter 16

SiFive Trace System

16.1 Introduction

Instruction Trace is a system that collects a history of processor execution, along with other events. The trace system is set up and controlled using a TileLink interface, typically connected to the Peripheral Crossbar. Core execution activity appears on the Trace Bundle and is compressed and formatted into trace messages which are transmitted via TileLink to a Trace Sink. The system includes an SRAM sink with a 4 KiB on-chip trace buffer, an off-chip interface to a trace probe, and a System Memory sink.

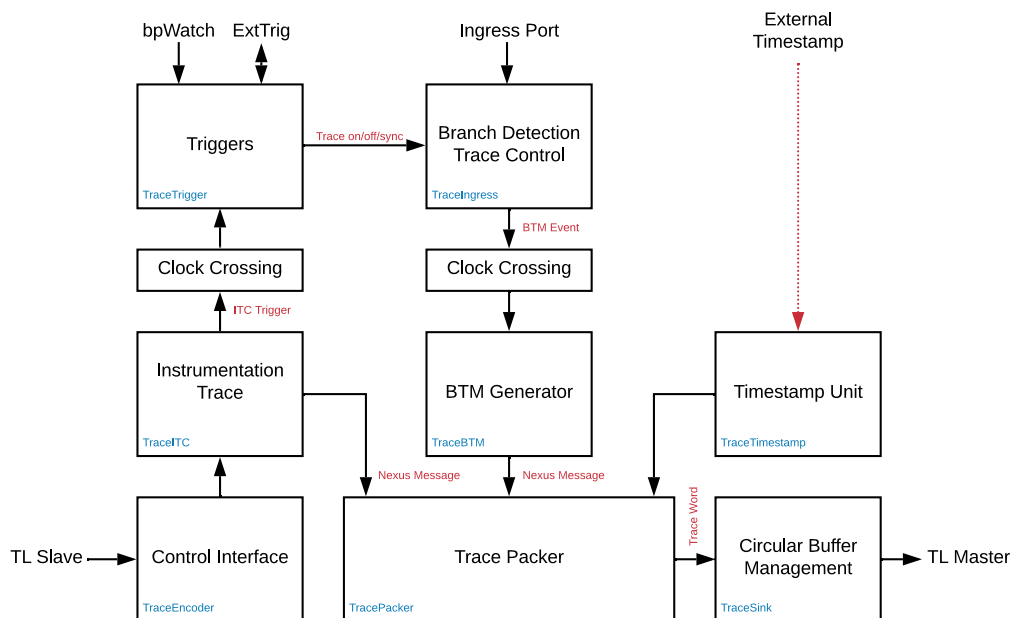


Figure 121: Trace Encoder Block Diagram

16.2 Trace Control Interface

The Trace Encoder (TE) control interface is a TileLink slave node typically connected to the CBUS crossbar, so it appears in the system physical memory space. The control interface is used to set up and control a trace session, generate messages from instrumentation writes,

retrieve collected trace history and control any trace system components directly connected to this Trace Encoder.

If the Debug Module is implemented with System Bus Access (SBA), a JTAG probe can control and monitor the trace system without requiring intervention by any of the cores.

Mapping the control interface into physical memory also allows one of the cores to manage a trace session—for example, collecting crash information in the field—or to modify trace collection parameters during execution. If desired, a Physical Memory Protection (PMP) range can be configured to restrict access to the trace system from core(s). If PMP is used, the ITC stimulus registers would normally be enabled for access.

Offset	TE Name	TF Name	Description
0x0000	teControl	tfControl	Trace Encoder/Funnel control register
0x0004	teImpl	tfImpl	Trace Encoder/Funnel implementation information
0x0010	teSinkBase	teSinkBase	Base address of circular trace buffer
0x0014	teSinkBaseHigh	teSinkBaseHigh	Bits [N:32] of the circular buffer address
0x0018	teSinkLimit	teSinkLimit	End address of circular trace buffer
0x001C	teSinkWP	teSinkWP	Current write location for trace data in circular buffer
0x0020	teSinkRP	teSinkRP	Access pointer for SRAM readback
0x0024	teSinkData	teSinkData	Read/write access to SRAM
0x0040	tsControl	—	Timestamp control register
0x0044	tsLower	—	Lower 32 bits of timestamp counter
0x0048	tsUpper	—	Upper bits of timestamp counter
0x0050	xtiControl	—	External Trigger Input control register
0x0058	wpControl	—	Core watchpoint control register
0x0060	itcTraceEnable	—	ITC trace enable bits, 1 bit per virtual stimulus register
0x0080–0x00FF	itcStimulus	—	Instrumentation Trace stimulus 0 to 31
0x0100–0x01FF	pcSample	—	Register interface to PC Sampling Unit, if present
0x0F00–0x0FFF	pibSink	pibSink	Control registers for PIB trace sink

Table 180: Trace Encoder Memory Map

16.3 Instruction Trace Encoder

16.3.1 Ingress Port

The Instruction Trace Encoder (ITE) monitors the activity of the core and determines whether and what type of branches are taken. ITE encodes a trace message for each taken branch or exception. Trace Messages conform to the IEEE ISTO-5001-2012 (Nexus) Standard.

Table 181 shows the interface from the core to the Trace Encoder.

Trace Spec Signal	TE Signal	Description
—	clock	Core clock
reset	reset	Core is in reset (does not reset TE)
iretire	valid	1 indicates an instruction has retired
iexception	exception	1 indicates an exception
interrupt	interrupt	1 indicates an interrupt
iaddr[63:0]	iaddr[63:0]	If iretire=1, the instruction address
instr[31:0]	insn[31:0]	If iretire=1, the instruction opcode
priv[2:0]	priv[2:0]	If iretire=1, the privilege mode: • 0x0 - User mode • 0x1 - Supervisor mode • 0x3 - Machine mode • 0x4-0x7 - Debug mode
—	bpwatch[n:0]	Core watchpoint actions, 4 bits per watchpoint: • valid: this watchpoint is matching in this cycle • action[2:0]: action field for this watchpoint
stall	teStall	Stall input to core

Table 181: Core to Trace Encoder Interface

16.3.2 BTM Generator

By partially decoding the opcode and comparing successive addresses, the ITE determines whether and what type of branches are taken. The ITE encodes a trace message for each taken branch or exception.

Trace Messages are Nexus Branch Trace Messages (BTM) and are generated according to the following rules:

Ingress Port	Condition	Action	Notes
iretire=1	—	pred.iaddr = iaddr + ilen(instr) flags.db = (instr is direct branch)	Compute and store predicted next address. Decode instr and store type.
iexception=1	—	flags.exception=1	Exception occurring, handler begins at next iretire
iretire=1 iaddr=pred.iaddr	flags.exception=0	None	Not a branch or conditional branch not taken
iretire=1 iaddr≠pred.iaddr	flags.exception=0 flags.db=1	Direct Branch Message: I-CNT = pred.iaddr - base.iaddr base.iaddr = iaddr	Direct branch to iaddr
iretire=1 iaddr≠pred.iaddr	flags.exception=0 flags.db=0	Indirect Branch Message: B-TYPE=Indirect Branch U-ADDR=iaddr I-CNT = pred.iaddr - base.iaddr base.iaddr=iaddr	Indirect branch to iaddr
iretire=1	flags.exception=1	Indirect Branch Message: B-TYPE=Exception U-ADDR=iaddr I-CNT = pred.iaddr - base.iaddr base.iaddr=iaddr flags.exception=0	Exception handler began at iaddr

Table 182: Branch Trace Messages Description

The Direct and Indirect Branch Message normally contain a compressed address (U-ADDR) to reduce bandwidth. The ITE emits a Branch with Sync Message containing the complete instruction address under certain conditions. This message type is a variant of the Direct or Indirect Branch Message and includes a full address (F-ADDR) and a field indicating the reason for the Sync (SYNC). In addition, various conditions can cause the ITE to put out a Program Trace Synchronization (PT Sync) Message which communicates the full address of a retired instruction but does not indicate that a branch occurred.

In each variant of Sync message, the timestamp value, if present, is not compressed, though leading zeroes are omitted. The I - CNT field of a Sync message emitted after a gap (all odd-numbered SYNC codes) is set to 0 since there is no previous execution address to reference. The SYNC types used by SiFive are:

SYNC	Description	I - CNT	Notes
0x0	External trigger	valid	PT Sync message sent when External Trigger Input occurs with xtiAction=4
0x1	Exit from reset	0	PT Sync message sent at first instruction retired after reset
0x2	Periodic message counter	valid	Include full address in every nth branch message
0x3	Exit from debug mode	0	PT Sync Message sent at first instruction retired following a gap in recording
0x4	Sequential instruction counter	valid	PT Sync message sent when I - CNT reaches a limit (such as in a long section of straight-line code)
0x5	Trace enable	0	PT Sync message sent at first instruction retired after enabling trace
0x6	Watchpoint	valid	PT Sync message sent when core watchpoint occurs with action=4
0x7	Restart after FIFO overrun	0	PT Sync message sent at first instruction retired after a trace FIFO overflow
0xF	PC sample (vendor-defined)	0	PT Sync message sent when Trace Encoder is in Sync-only mode

Table 183: SYNC Types

When trace stops, the ITE will emit a Program Trace Correlation Message (PTCM) indicating the reason in the EVCODE field and an I - CNT field which is used to compute the last instruction executed prior to the event. Trace is disabled in debug mode. Upon exiting debug mode, the ITE will emit a PT Sync Message indicating the first instruction to be executed. On entering debug mode, the ITE will emit a Program Trace Correlation Message with EVCODE 0.

EVCODE	Description	Notes
0x0	Entry into debug mode	Processor just entered debug mode.
0x4	Trace disabled	Caused by writing to teTracing, external trigger, or watchpoint
0x8	Entry into reset	Processor is being reset (vendor-defined Nexus code)

Table 184: EVCODE and PTCM Description

16.3.3 In-Circuit Trace Message

In certain conditions, the Trace Encoder needs to report the address of the most recent retired instruction. The In-circuit Trace message (ICT) is used for this reporting. The address is either U-ADDR or F-ADDR and this is indicated by the TCODE (either ICT or ICT with Sync). When the message is ICT, the address is compressed and refers to the full address of the most recent message with an address and the timestamp is compressed normally. When the Sync form of ICT is used, the address is F-ADDR and, the timestamp is uncompressed. However, ICT with Sync cannot be used by the trace decoder as a starting point because there may be previous ICNT history that cannot be reported with ICT.

CKSRC	Description	Notes
8	External Trigger	Used when an external trigger with <code>action=4</code> occurs at a time when no instructions are being retired or when a PTSYNC External Trigger message cannot be sent due to encoder resource limitation. Always uses ICT.
14	Watchpoint	Used to report a watchpoint with <code>action=4</code> that occurs at a time when a PTSYNC Watchpoint message cannot be sent due to encoder resource limitation. The watchpoint is noted and sent as soon as possible. ICT is used for this reporting when there are no instructions being retired. Always uses ICT.
15	PC sample	Message sent when TE is in Periodic Sample mode (<code>teInstruction=1</code>). Normally uses ICT, but an ICT with Sync is sent at a rate determined by <code>teSyncMaxBTM</code> .

Table 185: CKSRC Description

16.3.4 TE Trace Packer

The Trace Ingress Port operates in the core clock domain. Minimal processing is done prior to crossing into the `external_source_for_trace_com_clock` domain, where BTMs are generated. If `external_source_for_trace_com_clock` and `external_source_for_trace_N_clock` are the same, then this async crossing is omitted.

The Trace Packer receives the BTM and packs its Nexus slices (see below) into words. Completed words are sent to the TL master interface. If a message ends in the middle of a word and no additional messages are pending, then the Packer retains the partial word until another message is received. The Packer also receives formatted Nexus messages from the ITC unit and watchpoints. BTM has priority, followed by watchpoints, then ITC. The Packer may generate more than one trace word per cycle; these are stored in a shift register until the TileLink master can consume them. When TE is disabled (`teEnable → 0`), any partial word left in the trace packer is filled with idle slices and sent.

The Trace Packer adds timestamps to the formatted Nexus messages prior to saving in the shift register. Timestamps are added at this point so that the timestamp compression thread follows through all message types, whether BTM, ITC, or watchpoints.

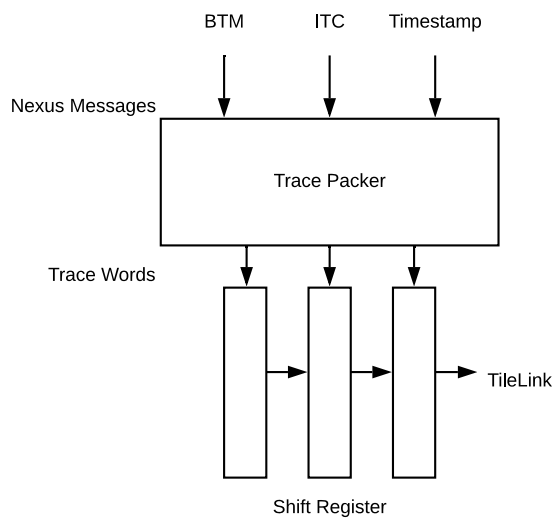


Figure 122: Trace Packer

16.3.5 Register Interface

This section describes the functionality of the Trace Encoder registers.

Trace Encoder Control Register (teControl1)				
Address Offset		0x0000		
Bits	Field Name	Attr.	Rst.	Description
0	teActive	RW	0x0	Master enable for trace system. 0 resets the TE and clocks may be gated off. teActive must be set to 1 before any other registers (except pcControl1), including other fields in teControl1, are programmed.
1	teEnable	RW	0x0	1=TE enabled. Allows teTracing to turn all tracing on and off. Setting teEnable to 0 flushes any queued trace data to the designated sink.
2	teTracing	RW	0x0	1=Trace is being generated. Written from tool or controlled by triggers.
3	teEmpty	RO	0x1	Reads as 1 when all generated trace has been emitted.
[6:4]	teInstruction	WARL	0x0	Instruction trace generation parameters: • 0x0 - Don't generate BTM or Sync (no instruction trace) • 0x1 - Generate Sync only (periodically record current PC - period is teSyncMaxInst) • 0x2 - Reserved • 0x3 - Reserved • 0x4-0x5 - Reserved • 0x6 - Generate HTM and Sync without HTM optimizations • 0x7 - Generate HTM and Sync with full optimization
[8:7]	teInstrumentation	RW	0x0	ITC trace generation parameters for itcStimulus registers whose corresponding itcTraceEnable=1. • 0x0 - ITC trace disabled • 0x1 - Generate ITC messages for all itcStimulus registers • 0x2 - Generate Ownership message for itcStimulus[15, 31], disable other ITC trace

Table 186: Trace Encoder Control Register

Trace Encoder Control Register (teControl)				
				0x3 - Generate Ownership message for itcStimulus[15, 31] and ITC messages for all other itcStimulus registers
[11:9]	Reserved	RO	0x0	
12	teStallOrOverflow	RW	0x0	Written to 1 by hardware when an overflow message is generated or when the TE requests a core stall. Debugger can write 0 to clear.
13	teStallEnable	RW	0x0	0=If TE cannot accept a message, an overflow is generated. 1=If TE cannot accept a message, the core is stalled until it can.
14	teStopOnWrap	RW	0x0	Disable trace (teEnable → 0) when circular buffer fills for the first time. Present only in systems with SRAM or System Memory sinks.
15	Reserved	RO	0x0	
[19:16]	teSyncMaxBTM	WARL	0x3	Maximum number of BTMs between periodic Sync messages. A Sync emitted for another reason will reset this timer. 0x0-0xB - Generate Sync after $2^{(teSyncMaxBTM + 5)}$ BTM's. Range is 32 to 65536. Default is 256. 0xC-0xF - Reserved
[23:20]	teSyncMaxInst	WARL	0xA	Maximum I-CNT between BTMs. A BTM emitted for another reason will reset this timer. 0x0-0x5 - Generate Sync when running I-CNT reaches $2^{(teSyncMaxInst + 5)}$ bytes. Range is 32 to 1024 bytes. Default is 1024. 0x6-0xF - Reserved
[26:24]	teSliceFormat	RO	0x1	Nexus Slice Format 0x0 - Reserved 0x1 - 6 MDO + 2 MSEO 0x2-0x7 - Reserved

Table 186: Trace Encoder Control Register

Trace Encoder Control Register (teControl1)				
27	teSinkError	RW1C	0x0	Hardware sets this to 1 when a trace sink TileLink transaction results in an error. This can happen with a System Memory sink when the address or size is invalid or with PIB, ATB, and Funnel sinks when the destination device is disabled. Write 1 to clear.
[31:28]	teSink	WARL	0x4	Which sink to send trace to. If there is only one sink type, this field may read as either 0 or that sink's type number: • 0x0 - Send trace to the one sink connected • 0x1-0x3 - Reserved • 0x5 - SRAM Sink • 0x5 - ATB Sink • 0x6 - PIB Sink • 0x7 - SBA Sink • 0x8 - Funnel Sink • 0x9-0xF - Reserved

Table 186: Trace Encoder Control Register

Trace Encoder Implementation Register (teImp1)				
Address Offset		0x0004		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	version	RO	0x0	TE Version (0 is this version)
4	hasSRAMSink	RO	0x1	1 if this TE has an on-chip SRAM sink. Size of SRAM may be determined by writing all 1s to teSinkWP, then reading the value back.
5	hasATBSink	RO	0x0	1 if this TE has an ATB sink.
6	hasPIBSink	RO	0x1	1 if this TE has an off-chip trace port via a Probe Interface Block.
7	hasSBASink	RO	0x1	1 if this TE has an on-chip bus master through the Front Port.
8	hasFunnelSink	RO	0x0	1 if this TE feeds into a Trace Funnel device.
[15:9]	Reserved	RO	0x0	
[17:16]	sinkBytes	RO	0x1	Width of this TE's trace sink. • 0x0 - 32-bit • 0x1 - 64-bit • 0x2 - 128-bit • 0x3 - 256-bit
[19:18]	crossingType	RO	preset	Type of clock crossing between Ingress and BTM Generator. • 0x0 - Synchronous • 0x1 - Rational • 0x2 - Asynchronous
[23:20]	srcID	RO	preset	This TE's source ID. If nSrcBits>0 and SRC is not disabled by teInhibitSrc, then Nexus messages will all include a SRC field of nSrcBits bits. Messages from this TE will have this value in SRC.
[26:24]	nSrcBits	RO	preset	The number of bits in the Nexus SRC field, unless disabled by teInhibitSrc.
27	Reserved	RO	0x0	
[31:28]	hartid	RO	preset	The hartid of the hart this TE is connected to.

Table 187: Trace Encoder Implementation Register

Trace Encoder Sink Base Register (teSinkBase)				
Address Offset		0x0010		
Bits	Field Name	Attr.	Rst.	Description
[1:0]	Reserved	RO	0x0	
[N:2]	teSinkBase	RW	0x0	Base address of trace circular buffer relative to TE TileLink master node. Fixed to the natural boundary implied by width of trace sink.

Table 188: Trace Encoder Sink Base Register

Trace Encoder Sink Base High Bits Register (teSinkBaseHigh)				
Address Offset		0x0014		
Bits	Field Name	Attr.	Rst.	Description
[N:0]	teSinkBaseH	RW	0x0	High order bits [N:32] of address of trace circular buffer in System Memory.

Table 189: Trace Encoder Sink Base High Bits Register

Trace Encoder Sink Limit Register (teSinkLimit)				
Address Offset		0x0018		
Bits	Field Name	Attr.	Rst.	Description
[1:0]	Reserved	RO	0x0	
[N:2]	teSinkLimit	RW	0x0	Highest address of trace circular buffer relative to TE TileLink master node. Fixed to natural boundary.

Table 190: Trace Encoder Sink Limit Register

Trace Encoder Sink Write Pointer Register (teSinkWP)				
Address Offset		0x001C		
Bits	Field Name	Attr.	Rst.	Description
0	teSinkWrapped	RW	0x0	Set to 1 by hardware when teSinkWP wraps
1	Reserved	RO	0x0	
[N:2]	teSinkWP	RW	0x0	Address in trace sink where next trace message will be written. Fixed to natural boundary. When using a System Memory Sink and a trace word write occurs while teSinkWP=teSinkLimit, teSinkWP is set to teSinkBase. When using an SRAM sink, teSinkWP wraps back to 0 when the SRAM has been filled.

Table 191: Trace Encoder Sink Write Pointer Register

Trace Encoder SRAM Sink Access Pointer Register (teSinkRP)				
Address Offset		0x0020		
Bits	Field Name	Attr.	Rst.	Description
[1:0]	Reserved	RO	0x0	
[N:2]	teSinkRP	RW	0x0	Address in trace circular buffer visible through teSinkData. Auto-increments following an access to teSinkData.

Table 192: Trace Encoder SRAM Sink Access Pointer Register

Trace Encoder SRAM Sink Data Register (teSinkData)				
Address Offset		0x0024		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	teSinkData	RW	X	Read or write value for SRAM access. SRAM is always accessed by words through this path regardless of the width of the Sink bus.

Table 193: Trace Encoder SRAM Sink Data Register

16.4 Instrumentation Trace Component (ITC)

ITC appears as a block of 32×32 -bit stimulus registers in the Trace Encoder control interface.

A bitmap enable register selectively enables any subset of ITC registers to cause trace or trigger events. If trace is enabled, any write to an ITC stimulus register generates a Data Acquisition Message (DQM) containing the data and byte offset of the ITC register that was written. Writes of 8, 16, or 32 bits are supported. The read value of a Stimulus register reflects the number of available write slots. Before writing to an ITC Stimulus register, software must first read a Stimulus register to obtain the number of available slots. Software may then write up to that number of items to ITC Stimulus before checking again.

16.4.1 ITC Register Interface

Instrumentation Trace Enable Register (itcTraceEnable)				
Address Offset		0x0060		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	itcTraceEnable	RW	0x0	1=write to corresponding stimulus register generates DQM or Ownership message if enabled by teInstrumentation

Table 194: Instrumentation Trace Enable Register

Instrumentation Trace Stimulus Registers (itcStimulus)				
Address Offset		0x0080 - 0x00FF		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	itcStimulus	RW	0x1	Writes generate Nexus Data Acquisition Message (DQM) with data value and byte offset if enabled by teInstrumentation and itcTraceEnable. Messages generated by writing to registers 0-15 (byte offset 0-63) do not include timestamps. Registers 16-31 include a timestamp if ITC timestamping is enabled by teInstrumentation. The byte offset generated by registers 16-31 will be 0-63. ITC supports full and partial word writes. To transmit a single byte through DQM, write the byte to offset 3 (bits [31:24]) in a register. To transmit 2 bytes, do a halfword write to offset 2 (bits [31:16]). The byte offset field in the DQM message will thereby indicate the size of the data value. Registers 15 and 31 may be set to generate Ownership trace message or DQM using teInstrumentation.

Table 195: Instrumentation Trace Stimulus Register

16.4.2 Using the ITC

This section provides a code example how to use the ITC. This example assumes that the trace encoder has already been enabled.

Consider a configuration with a CLINT based at address 0x1820000 and a Trace Encoder at 0x4000. If the user would like to read mtime, a memory-mapped register in the CLINT and insert the value into the trace, you can use the following sequence:

```
Store 0x4060, 0xffffffff // enable trace for all ITC stimulus
Load  x1, 0x182bff8      // read low half of memory-mapped mtime register
Load  x2, 0x182bffc      // read high half of memory-mapped mtime register
Store 0x4080, x1          // send mtime_low to ITC
Store 0x4080, x2          // send mtime_high to ITC
Store 0x4060, 0x00000000 // disable trace for all ITC stimulus
```

16.5 Timestamp Unit

Timestamp is an optional feature. A Trace Encoder instance includes one of four types of timestamp unit: Internal, External, Internal with Core Clock, or Slave. The P550 Core Complex includes the following timestamp unit(s):

- Internal timestamp uses the `external_source_for_trace_com_clock` to increment the timestamp counter.
- The timestamp counter or input is 40 bits.

An Internal Timestamp Unit includes a prescale divider, which can extend the range of a narrower timestamp and uses less power but has less resolution.

Since the Internal and External Timestamp clocks are system-dependent, it may be useful to measure the clock frequency for calibration. To do this, debug software can read the count value, wait a known interval, then read the count value again. $\text{Frequency} = (\text{count difference}) / (\text{time interval})$. Note when reading the count value that two reads are required to retrieve the complete value and that the counter continues to run between the two. Software can definitively determine the full count value by reading `tsUpper`, `tsLower`, then `tsUpper` again. If the two reads from `tsUpper` are the same, then $\text{count} = \{\text{tsUpper}, \text{tsLower}\}$. If different, then examine the MSB of `tsLower`; if 0, then use the second `tsUpper`, and if 1, use the first `tsUpper`.

It may be desirable to stop the counter when the hart is in debug mode. A control bit is provided for this purpose.

16.5.1 Timestamp Register Interface

Timestamp Control Register (tsControl1)				
Address Offset		0x0040		
Bits	Field Name	Attr.	Rst.	Description
0	tsActive	RW	0x0	Master reset/enable for timestamp unit. tsActive must be set to 1 before any other fields in tsControl1 are programmed.
1	tsCount	RW	0x0	1 for counter runs, 0 for counter stopped
2	tsReset	WO	X	Write 1 to reset the timestamp counter
3	tsDebug	RW	0x0	1 for counter runs in debug mode, 0 for stopped
[6:4]	tsType	RO	0x2	Type of Timestamp unit • 0x0 - None • 0x1 - External • 0x2 - Internal, bus clock • 0x3 - Internal, core clock • 0x4 - Slave • 0x5-0x7 - Reserved
[9:8]	tsPrescale	RW	0x0	Prescale timestamp clock by 2^{2n} (1, 4, 16, 64)
[14:10]	Reserved	RO	0x0	
15	tsEnable	RW	0x0	Enable timestamps and add timestamp field to Sync messages
[17:16]	tsBranch	WARL	0x0	Add timestamp field to branch messages if tsEnable=1. • 0x0 - No timestamp on branch messages • 0x1 - Timestamp all indirect branch and exception messages • 0x2 - Reserved • 0x3 - Timestamp all branch, exception, PTCM, and Error messages
18	tsInstrumentation	RW	0x0	Add timestamp field to Instrumentation messages if tsEnable=1
19	tsOwnership	RW	0x0	Add timestamp field to Ownership messages if tsEnable=1
[23:20]	Reserved	RO	0x0	
[31:24]	tsWidth	RO	0x28	Width of timestamp in bits

Table 196: Timestamp Control Register

Timestamp Lower Bits Register (tsLower)				
Address Offset		0x0044		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	tsLower	RO	X	Lower 32 bits of timestamp counter

Table 197: Timestamp Lower Bits Register

Timestamp Upper Bits Register (tsUpper)				
Address Offset		0x0048		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	tsUpper	RO	X	Upper bits of timestamp counter, zero-extended

Table 198: Timestamp Upper Bits Register

16.6 External Triggers

The TE is configured with 2 external trigger inputs for controlling trace. These are in addition to the external triggers present in the Debug Module when Halt Groups are configured. A trigger input is a 2-signal pair consisting of a request from the system and an acknowledgment handshake from the TE.

This protocol is compatible with the Arm Cross Trigger Interface or may connect to another TE or other system logic, even in another clock domain.

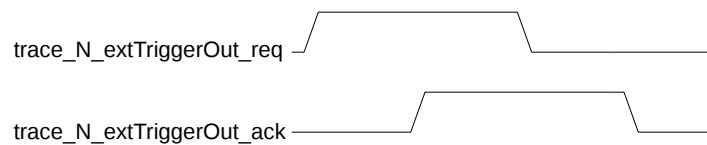


Figure 123: Trace Encoder External Trigger Handshake

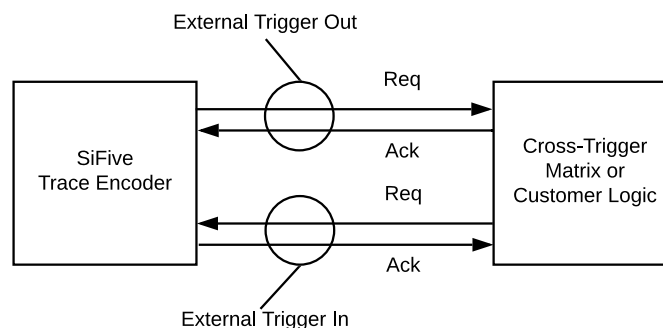


Figure 124: Trace Encoder External Triggers

trace_N_extTriggerIn_req is synchronized to external_source_for_trace_N_clock and is defined to have fired when a 0-to-1 transition occurs between two successive external_source_for_trace_N_clock cycles. The synchronized request is normally fed back to the system immediately on trace_N_extTriggerIn_ack. Note that if external triggers occur too often, some may be missed.

16.6.1 Register Descriptions

This section describes the functionality of the External Trigger registers.

External Trigger Input Control Register (xtiControl)				
Address Offset		0x0050		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	xtiAction0	WARL	0x0	Select action to perform when external trigger input 0 fires. If external trigger input 0 does not exist, then its action is fixed at 0. • 0x0 - No action • 0x1 - Reserved • 0x2 - Start trace (teTracing → 1) • 0x3 - Stop trace (teTracing → 0) • 0x4 - Record Program Trace Sync message • 0x5-0xF - Reserved
[31:4]	xtiActionN	WARL	0x0	Select actions for external triggers 1 through 7. If an external trigger input does not exist, then its action is fixed at 0.

Table 199: External Trigger Input Control Register

16.6.2 Using External Trigger Inputs

This section describes how to use the external trigger inputs described above.

Each external trigger input is configured with a 4-bit field in xtiControl located at Trace Encoder address offset 0x50. The value in the field defines what action to take when an external trigger input fires (i.e., the req signal transitions from 0 to 1). For example, to turn trace on when external trigger 0 fires, you would write the value 0x2 to bits [3:0] of xtiControl. To turn trace off when external trigger 1 fires, write 0x3 to bits [7:4].

For the code example below, consider a design with a Trace Encoder located at 0x4000.

```
mww 0x4000 0x01530033 // Enable trace encoder with trace off
```

```
mww 0x4050 0x00000032 // Set xtiControl as described above
```

16.7 Core Watchpoints

Watchpoints are signals from the core that a breakpoint was hit, but the action associated with that breakpoint is a trace-related action. Action identifiers 2-5 are reserved for trace actions in the RISC-V Debug Spec, where breakpoints are defined. The desired action is written to the action field of the `mcontrol` CSR (0x7A1). Not all cores support trace actions; the debugger should read back `mcontrol` after setting one of these actions to verify that the option exists. Once synchronized, watchpoint actions take effect unconditionally, assuming the TE is enabled with `teEnable`.

The start and stop trace actions can operate in two modes, selected by `wpControl`.

- Edge mode: `iretire & (wpAction[n]=2)` sets `teTracing` to 1. `iretire & (wpAction[m]=3)` sets `teTracing` to 0.
- Range mode: `iretire & (wpAction[n]=2)` sets `teTracing` to 1. `iretire & (wpAction[n]≠2)` sets `teTracing` to 0. Range mode allows using only one address range watchpoint to enable tracing in a range of instruction addresses.

wpAction	Effect
0	Breakpoint exception
1	Debug exception
2	Start trace (<code>teTracing</code> → 1)
3	Stop trace (<code>teTracing</code> → 0)
4	Record Program Trace Sync message
5	Reserved

Table 200: Watchpoint Action Identifiers

16.7.1 Register Descriptions

This section describes the functionality of the Core Watchpoint registers.

Core Watchpoint Control Register (<code>wpControl</code>)				
Address Offset		0x0058		
Bits	Field Name	Attr.	Rst.	Description
0	<code>wpRange0</code>	RW	0x0	Select Edge (0) or Range (1) mode for core watchpoint 0
[N:1]	<code>wpRangeN</code>	RW	0x0	Select Edge (0) or Range (1) mode for core watchpoints 1 to N. Fixed at 0 for any watchpoints that don't exist.

Table 201: Core Watchpoint Control Register

16.8 Watchpoint Precedence

Core watchpoints and External Trigger Inputs operate under a specific precedence when simultaneous conflicting actions occur.

- For trace start and stop actions, the precedence is: core watchpoints 0 through N, then external triggers 0 through N.
 - The active watchpoint with the highest precedence that is programmed for trace start or stop is selected. All lower active watchpoints are ignored.
 - If the selected watchpoint has trace start action and trace is currently off, then trace is turned on and a Trace-On message is sent.
 - If the selected watchpoint has trace stop action and trace is currently on, then trace is turned off and a Trace-Off message is sent.
 - If a watchpoint in Range mode is active and a lower precedence watchpoint or trigger with Edge action occurs, the Edge action is ignored.
 - If a watchpoint in Range mode is active and a higher precedence Range watchpoint occurs, the lower one is disabled until it deactivates. A message is sent only if the action associated with the new Range watchpoint alters the state of tracing.
 - If a watchpoint in Range mode is active and a higher precedence Edge watchpoint occurs, the Edge action is executed, and the range trigger is disabled until it deactivates. A message is sent only if the action associated with the Edge watchpoint alters the state of tracing.
 - A disabled Range watchpoint must traverse to the inactive state (`wpAction[n]≠2`) before it can activate again.
- Core or external watchpoint with `action=4` (record Sync message) is executed if trace is enabled (`teTracing=1`) and no message was produced by above rules. If more than one watchpoint occurs simultaneously, only one Sync message is generated. If the TE can't insert the Sync due to resource limitation, the Sync message request is held until the Sync can be generated. If another Sync request occurs during this time, the second request will be ignored.

16.9 PC Sampling Unit

Since the Trace Encoder directly connects to the core's `TracedInstruction` output bundle, it is relatively easy to add a PC Sampling Unit that provides a read-only interface to the current execution address. In some hang scenarios, this information is useful to be able to pinpoint what the core was doing just prior to the hang. This interface can also be used for non-intrusive statistical profiling.

In systems with a watchdog timer that resets the core upon timeout, it is also useful to capture the address of the last retired instruction so that it can be read out as part of a crash analysis. The `pcCapture` registers capture a new sample on each occurrence of the core reset.

16.9.1 Register Descriptions

This section describes the functionality of the PC Sampling Unit registers.

PC Sample Control Register (pcControl)				
Address Offset		0x0100		
Bits	Field Name	Attr.	Rst.	Description
0	pcActive	RW	0x0	Master enable/reset for PC Sampling Unit. pcActive must be set to 1 before any other fields in pcControl are programmed. This bit is reset to 0 by the trace reset input and must be set to 0x1 in order to perform sampling. The clock to the PC Sampling Unit may be gated off when pcActive=0. Note that pcActive is not dependent on teActive, so the PC Sampling Unit can be enabled while the rest of the trace encoder is disabled.
[31:1]	Reserved	RO	0x0	

Table 202: PC Sample Control Register

PC Capture/Sample Implementation Register (pcImp1)				
Address Offset		0x0104		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	pcCaptureFields	RO	preset	Bitmap indicating which core registers other than PC are present in the PC Capture register range. PC is always present. Capture registers are populated downward from 0x134 in the order listed (bit 0 to 15) with no gaps. Core registers in this list that are more than 32 bits occupy two capture register locations with the least-significant word at the higher address. • Bit 0 = MEPC • Bits [15:1] = Reserved
[31:16]	pcSampleFields	RO	preset	Bitmap indicating which core registers other than PC are present in the PC Sample register range. PC is always present. Sample registers are populated downward from 0x174 in the order listed (bit 0 to 15) with no gaps. Core registers in this list that are more than 32 bits occupy two sample register locations with the least-significant word at the higher address. • Bits [15:0] = Reserved

Table 203: PC Capture and Sample Implementation Register

PC Capture Register (pcCapture)				
Address Offset		0x013C		
Bits	Field Name	Attr.	Rst.	Description
0	pcCaptureValid	RO	0x0	The pcCapture/pcCaptureH registers contain a valid capture. This bit is reset to 0 when pcActive=0 and becomes 1 when there is an instruction retirement followed sometime later by a core reset.
[31:1]	pcCapture	RO	x	Address of the last instruction retired prior to the most recent core reset

Table 204: PC Capture Register

PC Capture High Register (pcCaptureH)				
Address Offset		0x0138		
Bits	Field Name	Attr.	Rst.	Description
[N:0]	pcCaptureH	RO	X	If the core address is more than 32 bits, this register reflects bits [N:32] of the address of the PC Capture register. Reads as zero in systems without PC Sampling or with an address width of 32 bits or less.

Table 205: PC Capture High Register

PC Sample Register (pcSample)				
Address Offset		0x017C		
Bits	Field Name	Attr.	Rst.	Description
0	pcSampleValid	RO	0x0	The pcSample/pcSampleH registers contain a valid sample that has not yet been read. This bit is reset to 0 when pcActive=0 and becomes 1 when the core retires an instruction. This bit is also reset when pcSample is read.
[31:1]	pcSample	RO	X	Address of most recent retired instruction

Table 206: PC Sample Register

PC Sample High Register (pcSampleH)				
Address Offset		0x0178		
Bits	Field Name	Attr.	Rst.	Description
[N:0]	pcSampleH	RO	X	If the core address is more than 32 bits, this register reflects bits [N:32] of the address of the most recent retired instruction. Reads as zero in systems without PC Sampling or with an address width of 32 bits or less. Since a full PC sample covers more than one register, hardware is provided to latch the full sample when any one of the sample registers below pcSample is read. In this way, reading an address range starting with a lower address and ending with pcSample will provide the values of all the fields at the time the first register was read. There is space in the address map below pcSampleH for additional fields to be added in the future.

Table 207: PC Sample High Register

16.10 Nexus Messages

16.10.1 Nexus Message Formats

The following tables show the message formats generated by the Trace Encoder and are ordered with LSBs at the bottom. The TCODE in the LSBs indicates the message type.

Direct Branch Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	I - CNT	Number of instruction units (half-words) executed since the last BTM or Sync
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 3

Table 208: Direct Branch Message Description

Indirect Branch Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	U-ADDR	Compressed address of target of branch
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync
2	B-TYPE	Branch type, 0=indirect branch, 1=exception
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 4

Table 209: Indirect Branch Message Description

Direct Branch with Sync Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	F-ADDR	Full target address for a taken branch. High-order zeroes are not transmitted.
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 11

Table 210: Direct Branch with Sync Message Description

Indirect Branch with Sync Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	F-ADDR	Full target address for a taken branch. High-order zeroes are not transmitted.
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync
2	B-TYPE	Branch type, 0=indirect branch, 1=exception
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 12

Table 211: Indirect Branch with Sync Message Description

Program Trace Synchronization Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	F-ADDR	Full target address of current PC. High-order zeroes are not transmitted.
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync. Valid only if SYNC[0]=0.
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 9

Table 212: Program Trace Synchronization Message Description

Error Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	pad	Pad ETYPE to end of slice so timestamp begins on slice boundary
4	ETYPE	Error Type, 0=Queue Overrun
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 8

Table 213: Error Message Description

Program Trace Correlation Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync.
2	CDF	Number of CDATA fields (always 0x0)
4	EVCODE	Reason for PTCM
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 33

Table 214: Program Trace Correlation Message Description

In-Circuit Trace Message		
Bits	Name	Description
var	TSTAMP	Timestamp value
var	CKDATA	Address of most recent retired instruction, U-ADDR or F-ADDR
2	CKDF	Number of CKDATA fields (always 0, meaning 1 CKDATA follows)
4	CKSRC	Reason for ICT. Possible values described in Table 185.
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 34 (compressed) or 35 (uncompressed)

Table 215: In-Circuit Trace Message Description (Reports PC)

Data Acquisition Message (from ITC)		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	DQDATA	Write data value to ITC, zero extended.
var	IDTAG	Write data index (byte offset) in ITC, zero extended
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 7

Table 216: Data Acquisition Message (from ITC) Description

Ownership Trace Message (from ITC)		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	PROCESS	Write data value to ITC, zero extended
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 2

Table 217: Ownership Trace Message (from ITC) Description

16.10.2 Address Compression

Address transmissions are compliant with the Nexus spec. Address fields begin with bit 1 since all execution addresses are on 2-byte boundaries. Addresses sent in U-ADDR compressed form are computed based on a reference address sent by or computed from the most recent preceding BTM or Sync message containing an address field. Starting with an F-ADDR, each U-ADDR modifies the reference address used for the next BTM. A U-ADDR is generated by XORing the full address with the reference address and sending the result starting with bit 1 and with high-order zeroes suppressed. The reverse process is used by software to recover the original full address.

Full Address	Compression		New Reference
0x003FC04		-	0x003FC04
0x003F368	ref	0000 0000 0011 1111 1100 0000 0100	0x003F368
	F-ADDR	0000 0000 0011 1111 0011 0110 1000	
	XOR	0000 0000 0000 0000 1111 0110 1100	
	U-ADDR	1111 0110 110	
0x003E100	ref	0000 0000 0011 1111 0011 0110 1000	0x003E100
	F-ADDR	0000 0000 0011 1110 0001 0000 0000	
	XOR	0000 0000 0000 0001 0010 0110 1000	
	U-ADDR	1 0010 0110 100	

Table 218: Address Compression Example

16.10.3 Timestamp Compression

If timestamp recording is enabled, Sync messages all include a full timestamp value with upper zeroes suppressed. Other message types with timestamps use a compressed value using the same XOR algorithm as in address compression. To reconstruct the full timestamp, software begins at a Sync message and stores the TSTAMP value found there, zero-extended to the full timestamp width. Software extracts the compressed TSTAMP from each message thereafter and XORs it with the previous decompressed timestamp to obtain the full timestamp value associated with each message.

Message	TSTAMP	Computed Timestamp
Sync	00 1111 1100 0001 1001	0x0000_0000FC19
BTM	11 1110	0x0000_0000FC27
BTM	0000 0100 0010	0x0000_0000FC65

Table 219: Timestamp Compression Description

16.10.4 Nexus Transmission Protocol

The Nexus standard defines a messaging protocol using a number of Message Data Out (MDO) signals and two flag signals known as MSE0[1:0] (Message Start/End Out). A Nexus message is sent or stored in slices comprised of MDO and MSE0. The first slice of a message sends the LSBs of the message and is indicated by MSE0=0x0. A variable-length field in a message always ends on a slice boundary (zero extended as needed) and the last slice of a variable field is indicated by MSE0=0x1. The last slice of a message is indicated by MSE0=0x3. Most messages have more than one variable field. TSTAMP is an optional field and may be present in some messages and not in others. When not present, the variable field just preceding TSTAMP will end with MSE0=0x3.

The Trace Encoder uses a slice format consisting of 6 MDO bits and 2 MSE0 bits. These slices are packed into words, LSB first, for storage in SRAM or sending to another destination. Following is an example of slice packing for an Indirect Branch Message with timestamp.

	MDO							MSEO		
Slice	5	4	3	2	1	0		1	0	Notes
	X	X	X	X	X	X		1	1	Idle or end of previous message
0	T5	T4	T3	T2	T1	T0		0	0	Start of message
1	I3	I2	I1	I0	B1	B0		0	0	Normal transfer
2	0	0	0	0	I5	I4		0	1	End of var field I -CNT zero-extended to slice boundary
3	0	A5	A4	A3	A2	A1		0	1	End of var field U -ADDR zero-extended to slice boundary
4	P5	P4	P3	P2	P1	P0		0	0	Normal transfer
5	0	0	P9	P8	P7	P6		1	1	End of message (implies end of var field)
Tx=TCODE, Bx=BTYPE, Ix=I -CNT, Ax=U -ADDR, Px=TSTAMP										

Table 220: Nexus Transmission Protocol Description

16.11 PIB Trace Sink

Trace data may be sent to chip pins through an interface called the Probe Interface Block (PIB). This interface typically operates at data rates of up to 100-400 MHz and can sometimes be higher with careful constraints and board layout. PIB may consist of just one signal and in this configuration may be called SWT (Serial-wire Trace). Alternative configurations include a trace reference clock and between 1 and 8 trace data signals timed to that reference.

16.11.1 Register Descriptions

This section describes the functionality of the PIB Trace Sink registers.

PIB Sink Control Register (<code>pibControl</code>)				
Address Offset		0x0F00		
Bits	Field Name	Attr.	Rst.	Description
0	<code>pibActive</code>	RW	0x0	Master enable/reset for PIB Sink block
1	<code>pibEnable</code>	RW	0x0	Enable PIB to produce output
2	Reserved	RO	0x0	
3	<code>pibEmpty</code>	RO	0x1	Reads 1 when PIB internal buffers are empty
[7:4]	<code>pibMode</code>	WARL	0x0	Select mode for output pins. Do not change while <code>pibActive</code> = 1.
8	<code>pibRefCenter</code>	WARL	0x0	In parallel modes, adjust <code>tref</code> timing to center of bit period. This can be set only if <code>pibMode</code> selects one of the parallel protocols.
9	<code>pibCalibrate</code>	RW	0x0	Set this to 1 to generate a repeating calibration pattern to help tune a probe's signal delays, bit rate, etc. The calibration pattern is described in Table 223.
[15:10]	Reserved	RO	0x0	
[28:16]	<code>pibDivider</code>	RW	0x0	Timebase selection for the PIB module. The bus clock is divided by <code>pibDivider</code> + 1. PIB data is sent at either this quotient rate or 1/2 of this rate, depending on <code>pibMode</code> .
[31:29]	Reserved	RO	0x0	

Table 221: PIB Sink Control Register

PIB port widths up to 9 may be selected. This width and all narrower modes may then be selected during circuit operation. Software can determine what modes are available by attempting to write each mode setting to the WARL field `pibControl.pibMode` while `pibActive`=0 and reading back to see if the value was accepted.

				Supported in PIB Port Width Configurations				
Mode	pibMode	pibREFCenter	Bit Rate	1	2	3	5	9
Off	0	X	-	✓	✓	✓	✓	✓
SWT Manchester	4	X	1/2	✓	✓	✓	✓	✓
SWT UART	5	X	1	✓	✓	✓	✓	✓
tref + 1 data	8	0	1	-	✓	✓	✓	✓
tref + 2 data	9	0	1	-	-	✓	✓	✓
tref + 4 data	10	0	1	-	-	-	✓	✓
tref + 8 data	11	0	1	-	-	-	-	✓
tref + 1 data	8	1	1/2	-	✓	✓	✓	✓
tref + 2 data	9	1	1/2	-	-	✓	✓	✓
tref + 4 data	10	1	1/2	-	-	-	✓	✓
tref + 8 data	11	1	1/2	-	-	-	-	✓

Table 222: PIB Port Width Configuration Description

16.11.2 Programming Recommendations

Since the PIB supports many different modes, it is necessary to follow a particular programming sequence.

1. Activate the PIB by setting pibActive.
2. Set the pibMode, pibDivider, pibRefCenter, and pibCalibrate fields. This will set the tdata outputs to the quiescent state (whether that is high or low depends on pibMode) and start tref running.
3. Activate the receiving device, such as a trace probe. Allow time for PLL to sync up, if using a PLL with a parallel PIB mode.
4. Set pibEnable. This enables the PIB to generate output either immediately (calibration mode) or when the trace encoder begins sending trace messages.

16.11.3 Calibration Mode

In calibration mode, the PIB transmits a repeating pattern. Probes can use this to automatically tune input delays due to skew on different PIB signal lines and to adjust to the transmitter's data rate. Each pattern is 4 slices in length and is transmitted as a BTM would be – LSB first, complete with start/stop framing bits. Each pattern has been chosen so that it both exercises the interface and follows Nexus message rules. In UART and Manchester modes, the pattern is repeated starting every 80 cycles of the divided clock. This leaves a 12-cycle gap in Manchester and a 40-cycle gap in UART mode. There is no gap in the parallel modes.

Mode	Calibration Slices	Wire Sequence
2-bit parallel	66 66 CC 33	2, 1, 2, 1, 2, 1, 2, 1, 0, 3, 0, 3, 3, 0, 3, 0
4-bit parallel	5A 5A F0 0F	A, 5, A, 5, 0, F, F, 0
8-bit parallel	AA 55 00 FF	AA, 55, 00, FF
UART, Manchester, 1-bit parallel	AA 55 00 FF	Alternating 1/0, then all 0, then all 1

Table 223: Calibration Mode Description

16.11.4 SWT Manchester Protocol

In this mode, the PIB outputs complete Nexus messages encapsulated between a start bit and a stop bit. Each bit period is divided into 2 phases and the sequential values of the `tdata[0]` pin during those 2 phases denote the bit value. Bits in each Nexus slice are transmitted LSB first, and the oldest slice is transmitted first. The idle state of `tdata[0]` is low in this mode.

Bit	Phase 1	Phase 2
Start	1	0
Logic 0	0	1
Logic 1	1	0
Stop/idle	0	0

Table 224: SWT Manchester Protocol Description

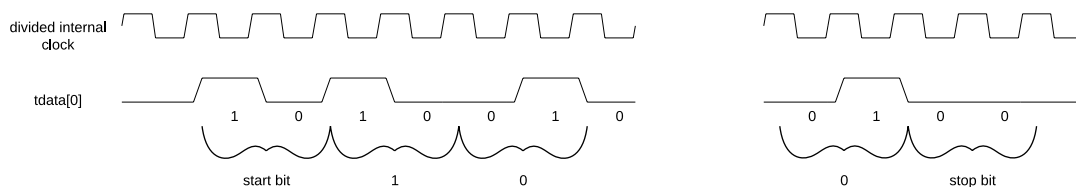


Figure 125: SWT Manchester Protocol

16.11.5 SWT UART Protocol

In UART protocol, the PIB outputs Nexus slices encapsulated in a 10-bit packet consisting of a low start bit, 8 data bits, LSB first, and a high stop bit. Another packet may begin immediately following the stop bit or there may be an idle period between packets. Slices comprising a Nexus message will be output back-to-back. When no data is being sent, `tdata[0]` is high in this mode.

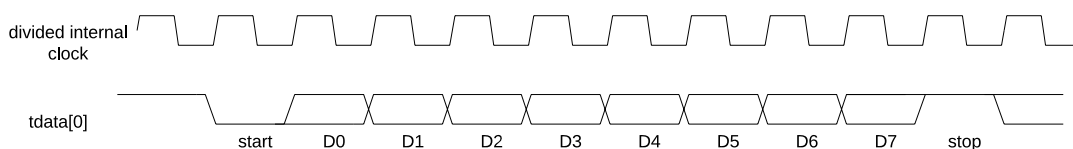


Figure 126: SWT UART Protocol

16.11.6 PIB Parallel Protocol

Traditionally, off-chip trace has used this protocol. There are a number of parallel data signals and one continuously running clock reference. The data rate of several parallel signals can be much higher than either of the serial-wire protocols.

As with SWT modes, this protocol is oriented to Nexus messages rather than fixed-width trace words. The idle state of `tdata` is all-ones. A transmission begins with `tdata[0]` driven low during a bit period. Since the first bits driven out include the `MSE0` bits of the first slice, there will always be a zero starting any message. This sample and possibly the next few (depending on the width of `tdata`) are collected until a complete Nexus slice (8 bits) have been received. Slices are transmitted LSB first, with `tdata[0]` being the LSB in each beat of data. The receiver continues collecting slices until a complete slice has been collected that has `MSE0=1`, denoting the end of the message. The data signals may then go their idle state (all ones), or a new message may begin in the next bit period (one or more zeroes).

The clock reference, `tref`, normally has edges coincident with the `tdata` edges. Typically, a probe will use a PLL to recover a sampling clock that is twice the frequency of `tref` and shifted 90 degrees so that its rising edges occur near the center of each bit period. The debugger can set `pibRefCenter` to change the timing of `tref` so that there is a `tref` edge at the center of each bit period on `tdata`. Note that this option cuts the data rate in half relative to normal parallel mode and still requires the probe to sample `tdata` on both edges of `tref`.

This example shows 8-bit parallel mode with `pibRefCenter=0` transmitting a 5-slice message followed by a 2-slice message.

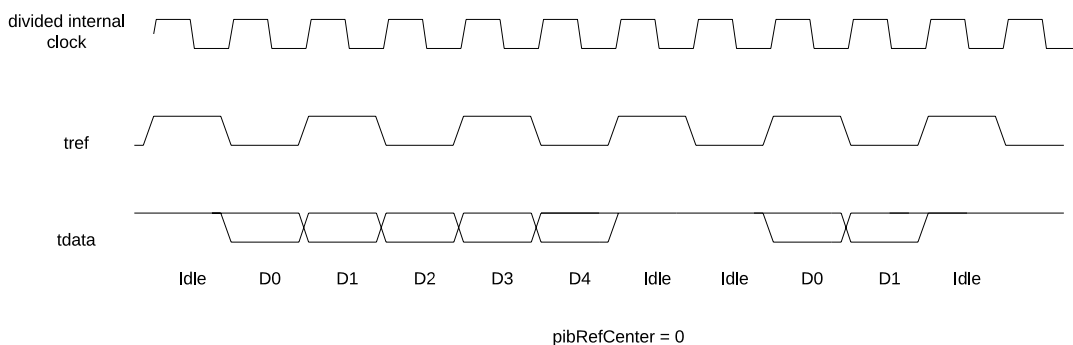


Figure 127: PIB Parallel Protocol

And an example showing 8-bit parallel mode transmitting a 4-slice packet with `pibRefCenter=1`.

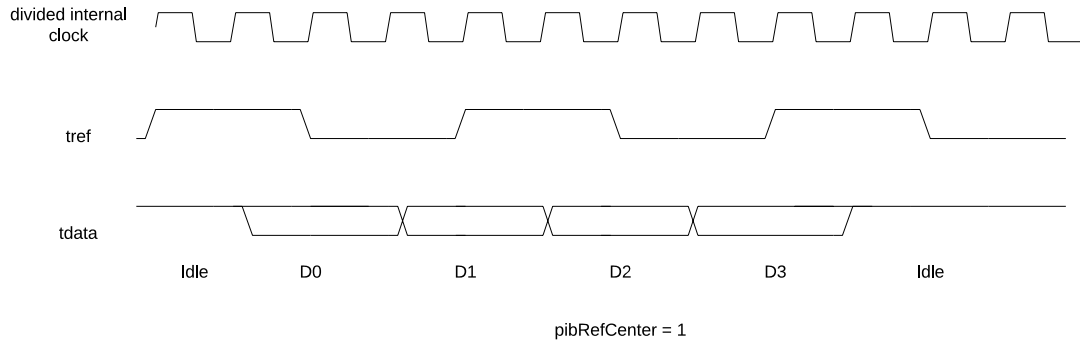


Figure 128: PIB Parallel Protocol, `pibRefCenter=1`

16.12 Nexus Branch History Trace Encoder

The Trace Encoder (TE) is a hardware block that converts execution status signals from the core to formatted Nexus trace messages. There is one TE per core and the outputs from several encoders are combined in a Trace Funnel to produce the final trace message stream.

16.12.1 History Buffer

A branch or jump is considered "inferable" if its destination can be determined solely by examining the program image. Jumps whose destination depends on processor state such as a register value are said to be "uninferable".

Rather than reporting every branch, the History Trace Mode (HTM) Trace Encoder only reports the minimal information needed by the Trace Decoder to track execution history. Unconditional inferable jumps do not generate a trace message since the decoder can follow execution flow through this type of branch by disassembling it and computing the destination address.

Conditional branches are always inferable in the RISC-V architecture.

Instead of reporting them as direct branches as in Branch Trace Mode (BTM), conditional branches are gathered into a history buffer as they are executed.

A taken branch is represented by 1 and a not-taken branch by 0. An indirect jump or sync event will generate an Indirect Branch History Message (IBHM) that includes the target address of the jump, current contents of the history buffer (HIST), and an instruction unit count (I-CNT) that includes all of the instructions executed since the previous IBHM, including unconditional jumps and all the conditional branches represented by the history buffer.

HIST is a variable length field that includes a single stop bit with value 1. Since var fields are padded to the next slice boundary, the stop bit is needed in order for the decoder to determine

where data ends and the pad begins. For example, a taken branch followed by a not-taken branch would result in a padded HIST field of 000110. Reading from the MSB (oldest instruction), three 0s are the pad, then 1 for the stop bit, 1 indicates taken branch, and 0 indicates not-taken branch.

The TE has hardware limitations on the number of conditional branches that can be conveyed in the HIST field and on the number of instructions between IBHMs. If either of these limits is exceeded, the TE will transmit a Resource Full message. The message will indicate whether it applies to HIST or I-CNT and will include the HIST field or I-CNT up to that point.

When trace turns off, a Program Trace Correlation Message (PTCM) is sent. PTCM always includes an I-CNT, but in Branch History Trace, it may also include a HIST field. Per the Nexus spec, the CDF field of the PTCM indicates the number of variable CDATE fields in the message. When CDF=1, then CDATE contains a HIST field that indicates the conditional branch history up to the point where the trace turned off. If a PTCM is sent and there is no HIST to report, then the simpler form of PTCM is used (i.e., CDF=0).

16.12.2 Return-Address Stack Optimization

Uninferable jumps are generally reported with an IBHM. However, in some cases, an uninferable jump target can be determined and uninferable jumps are not reported. The Trace Encoder will track the current depth of the return-address stack (RAS) of the core. The RAS is initialized when the Trace Encoder transmits a Sync message. Thereafter, when a call occurs, the count is incremented and when a return occurs, the count is decremented. If a return is executed and the count is non-zero, that return is considered inferable and, therefore, does not generate an IBHM. The RAS counter is "saturating"; that is, it sticks at a maximum value if too many calls are encountered and sticks at 0 if too many returns are encountered.

Consider the example below showing starting execution at 1000 and hitting a breakpoint at 1008:

Addr	Instruction	BTM	HTM	Notes
1000	c.add	PTSync, FADDR=1000	PTSync, FADDR=1000	Exit debug mode
1002	jal x1,2000	Direct, ICNT=3	—	Inferable call (no HTM message)
2000	c.sub	—	—	
2002	c.beqz	Direct, ICNT=2	—	1 (taken branch) pushed to HIST
2010	c.jr x1	Indirect, ICNT=1, UADDR=6	—	Return inferable due to RAS
1006	c.add	—	—	
1008	—	PTCM, ICNT=1	PTCM, ICNT=7, HIST=1	Enter debug mode (breakpoint)

Table 225: RAS Optimization Example

16.12.3 Repeat Branch Optimization

Another relatively easy optimization involves reporting certain cases of HIST in a compressed format using a form of the Resource Full message. A long sequence of taken branches or a long sequence of not-taken branches are common in embedded software when polling hardware registers and would normally result in multiple Resource Full messages with HIST being either all 1s or all 0s. For these cases, a vendor-defined RCODE indicates that the RDATA field of the Resource Full message contains a count rather than a bitmap of conditional branch history.

Example 1: Consider a polling loop such as `while (*uart_status & 1) { }`. Suppose the loop reads 1 (busy) 10,000 times before finally reading 0.

Addr	Instruction	BTM	HTM
1000	<code>lw x6, 0(x2)</code>	—	—
1004	<code>andi x6, x6, 1</code>	—	—
1008	<code>bnez x6, 1000</code>	Direct, ICNT=6	HIST = (HIST << 1)

Table 226: Repeat Branch Optimization Example 1

In BTM, this generates 10,000 direct branch messages occupying **20,000** bytes of trace buffer space. If the HIST buffer capacity of the HTM Encoder is 32 bits, this loop would generate 312 Resource Full messages, all indicating 32 taken branches, using **2184 bytes**. With the Repeat Branch optimization, one Resource Full message with RCODE=9 is emitted, and the final not-taken branch is loaded into the HIST buffer to be reported the next time HIST is sent. This uses only **4 bytes** of trace buffer space.

Example 2: A for-loop such as `for (i = 0; i < 10000; i++) { buf[i] = 0 }` typically compiles into an instruction sequence with a conditional branch at the top and an unconditional jump at the bottom. The conditional branch is repeatedly not taken until the loop exits.

Addr	Instruction	BTM	HTM
1000	<code>lui x6, 0</code>	—	—
1004	<code>lui x7, 10000</code>	—	—
1008	<code>bge x6, x7, 101c</code>	—	HIST = (HIST << 1) 0
100c	<code>add x28, x3, x6</code>	—	—
1010	<code>sw buf(x28), x0</code>	—	—
1014	<code>addi x6, x6, 1</code>	—	—
1018	<code>jal x0, 1008</code>	Direct, ICNT=10	Inferable jump (no message)

Table 227: Repeat Branch Optimization Example 2

Without Repeat Branch optimization, this loop would generate 312 Resource Full messages, all indicating 32 not-taken branches. With the Repeat Branch optimization, one Resource Full message with RCODE=8 is emitted, and the final taken branch is loaded into the HIST buffer to be reported the next time it is sent.

16.12.4 Sync Messages

The Program Trace Sync message (PT Sync) does not have a HIST field, so in Branch History Trace, certain Sync events generate an IBHM+Sync (IBHSync) instead of a PT Sync. The F-ADDR field can be used to determine whether an IBHSync represents an indirect branch. If an uninferable jump and sync marker event occur simultaneously, only one IBHMSync will be emitted, and SYNC will indicate the marker reason and F-ADDR will point to the discontinuous jump target. PT Sync is still used for those Sync events that indicate trace is resuming after a gap, meaning there is no previous history to transmit.

SYNC	Description	Message	Notes
0	External trigger	IBHSync	Sent when External Trigger Input occurs with xtiAction=4
1	Exit from reset	PTSync	PT Sync message sent at first iretire after reset
2	Periodic message counter	IBHSync	Include full address in every nth branch message
3	Exit from debug mode	PTSync	PT Sync Message sent at first iretire following a gap in recording
4	Sequential instruction counter	—	Not used in HTM (Resource Full is used instead)
5	Trace enable	PTSync	PT Sync message sent at first iretire following a gap in recording
6	Watchpoint	IBHSync	IBHM sent when core watchpoint occurs with action=4
7	Restart after FIFO overrun	PTSync	PT Sync message sent at first iretire following a gap in recording

Table 228: Sync Messages

16.12.5 HTM Nexus Message Formats

Bits	Name	Description
var	TSTAMP	Timestamp value
var	HIST	Conditional branch history
var	U-ADDR	Compressed address of target of branch
var	I - CNT	Number of instruction units (halfwords) executed since the last BTM or Sync.
2	B - TYPE	Branch type, 0=indirect branch, 1=exception
n	SRC	Source of this message (width is telmpl.nSrcBits)
6	TCODE	Value = 28

Table 229: Indirect Branch History Message

Note

If there is no branch history to transmit, then the Indirect Branch message is emitted instead (TCODE=4, no HIST).

Bits	Name	Description
var	TSTAMP	Timestamp value
var	HIST	Conditional branch history
var	F-ADDR	Full target address for a taken branch. High-order zeroes are not transmitted.
var	I - CNT	Number of instruction units (halfwords) executed since the last BTM or Sync.
2	B - TYPE	Branch type, 0=indirect branch, 1=exception
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is telmpl.nSrcBits)
6	TCODE	Value = 29

Table 230: Indirect Branch History with Sync Message

Note

If there is no branch history to transmit, then the Indirect Branch with Sync message is emitted instead (TCODE=12, no HIST).

Bits	Name	Description
var	TSTAMP	Timestamp value
var	RDATA	I-CNT (RCODE=0), HIST (RCODE=1), or count (RCODE=8 or 9)
4	RCODE	Resource code (0=I-CNT, 1=HIST, 8=HIST_NOTTAKEN, 9=HIST_TAKEN)
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 27

Table 231: Resource Full Message

Bits	Name	Description
var	TSTAMP	Timestamp value
var	HIST	Conditional branch history (field present only if CDF=1)
var	I-CNT	Number of instruction units (halfwords) executed since the last BTM or Sync
2	CDF	Number of CDATA fields (0 or 1, 0 is backward-compatible with BTM)
4	EVCODE	Reason for PTCM
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 33

Table 232: Program Trace Correlation Message (HTM format)

Chapter 17

Error Correction Codes (ECC)

Error correction codes (ECC) are implemented on various memories within the P550 Core Complex, allowing for the detection and, in some cases, correction of memory errors. The following SRAM blocks on the P550 Core Complex support ECC: data cache, instruction cache, private L2 cache, and L3 cache.

The minimal case of an ECC error is a single-bit error that is detected, reported via interrupt handler, and corrected automatically by hardware without any software intervention. More difficult scenarios involve double or multi-bit errors that are still reported and tracked in hardware but are not correctable. The ECC hardware includes logic for detection and correction, in addition to 7 redundant bits per 32-bit codeword or 8 redundant bits per 64-bit codeword.

Name	Protection Type
Branch Predictor	None
D-Cache Data	SECDED ECC (32+7b)
D-Cache Tag	SECDED ECC
D-Cache Way Predictor	None
I-Cache Data	Parity-Only (1b)
I-Cache Tag	Parity-Only (1b)
I-Cache Way Predictor	None
Indirect-Jump Target Predictor	None
L3 Cache Data	SECDED ECC (64+8b)
L3 Directory Tag	SECDED ECC
Private L2 Cache Data	SECDED ECC (128+9b)
Private L2 Directory Tag	SECDED ECC
UTLB	None

Table 233: Memory Protection Summary

17.1 ECC Configuration

All blocks with ECC support are enabled globally through the Bus-Error Unit (BEU) configuration registers. The BEU is used to configure ECC reporting and enable interrupt handling via the global or local interrupt controller. The global interrupt controller is the Platform-Level Interrupt Controller (PLIC). The local interrupt controller is the Core-Local Interruptor (CLINT). The BEU

registers `plic_interrupt` and `local_interrupt` are used to route the errors to the respective interrupt controller. Additionally, the BEU can be used for TileLink bus errors.

17.1.1 ECC Initialization

Any SRAM block containing ECC functionality needs to be initialized prior to use. This does not include cache memory, since an internal state machine initializes data cache valid bits, and instruction cache valid bits are flops with reset. ECC will correct defective bits based on memory contents, so if memory is not first initialized to a known state, then the ECC will not operate as expected. It is recommended to use a DMA, if available, to write the entire SRAM or cache to zeros prior to enabling ECC reporting. If no DMA is present, use store instructions issued from the processor. Initializing memory with ECC from an external bus is not recommended. After initialization, ECC-related registers can be written to zero, and then ECC reporting can be enabled. 64-bit aligned writes are recommended.

The startup code in the `freedom-metal` repository provides a method to automatically initialize memory with ECC. This is accomplished using an assembly-level function `_metal_memory_scrub`, located in file `scrub.S`. The linker script provides the symbol `__metal_eccscrub_bit` as a flag to enable the startup code to initialize memory with ECC. It is important to note that this memory initialization is limited to 64 KB to support RTL simulation run times. If unexpected ECC errors occur, check the range of the startup initialization to ensure it covers the region used by the software application.

17.2 ECC Interrupt Handling and Error Injection

Single-bit errors are automatically repaired by the hardware.

BEU errors are always enabled and thus do not have a control bit in `mie` (Machine Interrupt Enable) CSR. Likewise, there is no dedicated control bit for BEU errors in the `mideleg` (Machine Interrupt Delegation) CSR, so it cannot be delegated to a lower privilege mode than M-mode. Error injection, and thus software handling of errors, can be accomplished manually by writing the BEU cause register. The BEU is further described in Chapter 11.

Monitoring overall ECC events can be accomplished in software via the interrupt handler.

The L3 Cache Controller contains hardware counters to track ECC events, and optionally inject ECC errors to test the software handling of ECC events. The L3 Cache Controller is further described in Chapter 13.

The exception code value is located in the `mcause` (Machine Trap Cause) CSR. When BEU interrupts are routed through the PLIC, the default exception code value will be 11 (0xB).

When ECC interrupts are routed through the CLINT, the default exception code value will be 128 (0x80). These exception codes are further detailed in Section 7.7.5.

17.3 Hardware Operation Upon ECC Error

Hardware will operate differently depending on which memory type encounters an ECC error:

- Instruction Cache: The error is corrected, and the cache line is flushed
- Data Cache: The error is corrected and the cache line is invalidated and written back to the next level of memory
- Private L2 Cache: Single-bit correction and double-bit detection for private L2 data and tag/state RAMs
- L3 Cache: Single-bit correction for L3 data and metadata (metadata includes index, tag, and directory information). Double-bit detection only on the L3 data array.

Double-bit errors are reported at the Core Complex boundary via the `halt_from_tile_N` signal that, if asserted, remains high until reset.

Appendix A

SiFive Core Complex Configuration Options

This section provides a reference of the key configuration options of the SiFive P550 Series cores and the larger Core Complex. The file `docs/core_complex_configuration.txt` lists the features and options configured in the P550 Core Complex.

A.1 P550 Series

The P550 Series comes with the following set of configuration options. Note that the configuration may be limited to a fixed set of discrete options.

Modes and ISA:

- Configurable number of Cores (1, 2, or 4). In the case where more than one core is selected, all cores are configured the same.

On-Chip Memory:

- L3 Cache with the following options:
 - Configurable size (1, 2, or 4 MiB), associativity (8- or 16-way), and banks (2 or 4)

Ports:

- Memory Port, System Port, optional Peripheral Port, and Front Port
 - Each port has a configurable base address and size (64 KB up to max. supported address)
 - Memory Port has a configurable width (128- or 256-bit)
 - Front Port has a configurable width (64- or 128-bit)

Security:

- Optional Hardware Cryptographic Accelerator (HCA)

SiFive Insight Debug and Trace:

- Optional Nexus Trace Encoder with the following options:
 - Trace Sink (SRAM, System Memory, and/or PIB)

Interrupts:

- Platform-Level Interrupt Controller (PLIC) with configurable number of interrupts (1 to 511)

Design For Test:

- Configurable SRAM user-defined inputs (0 to 1024)
- Configurable SRAM user-defined outputs (0 to 1024)
- Optional SRAM Macro Extraction
- Optional Clock Gate Extraction
- Optional Grouping and Wrapping of extracted macros

Note that the SRAM user-defined feature is mutually exclusive to the macro extraction features.

RTL Options:

- Optional custom RTL module name prefix

Appendix B

SiFive RISC-V Implementation Registers

This section provides a reference to the SiFive RISC-V implementation version registers `marchid` and `mimpid`.

B.1 Machine Architecture ID Register (`marchid`)

Value	Core Generator
0x8000_0008	P500-Series Processor

Table 234: Core Generator Encoding of `marchid`

B.2 Machine Implementation ID Register (mimpid)

Value	Generator Release Version
0x0000_0000	Pre-19.02
0x2019_0228	19.02
0x2019_0531	19.05
0x2019_0919	19.08p0p0 / 19.08.00
0x2019_1105	19.08p1p0 / 19.08.01.00
0x2019_1204	19.08p2p0 / 19.08.02.00
0x2020_0423	19.08p3p0 / 19.08.03.00
0x0120_0626	19.08p4p0 / 19.08.04.00
0x0220_0515	koala.00.00-preview and koala.01.00-preview
0x0220_0603	koala.02.00-preview
0x0220_0630	20G1.03.00 / koala.03.00-general
0x0220_0710	20G1.04.00 / koala.04.00-general
0x0220_0826	20G1.05.00 / koala.05.00-general
0x0320_0908	kiwi.00.00-preview
0x0220_1013	20G1.06.00 / koala.06.00-general
0x0220_1120	20G1.07.00 / koala.07.00-general
0x0421_0205	llama.00.00-preview
0x0421_0324	21G1.01.00 / llama.01.00-general
0x0421_0427	21G1.02.00 / llama.02.00-general
0x0521_0528	mongoose.00.00-preview
0x0521_0714	21G2.01.00 / mongoose.01.00-general

Table 235: Generator Release Encoding of mimpid

Appendix C

Revision History

This section describes the changes in this document between release versions.

Version	Date	Document Changes
21G2.01.00	July 21, 2021	• Initial release

Table 236: P550 Core Complex Manual Revision History

References

Visit the SiFive forums for support and answers to frequently asked questions:
<https://forums.sifive.com>

[1] A. Waterman and K. Asanovic, Eds., The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.2, June 2019. [Online]. Available: <https://riscv.org/specifications/>

[2] —, The RISC-V Instruction Set Manual Volume II: Privileged Architecture, Version 1.11, June 2019. [Online]. Available: <https://riscv.org/specifications/privileged-isa/>

[3] SiFive TileLink Specification Version 1.8.0, August 2019. [Online]. Available: <https://sifive.com/documentation/tilelink/tilelink-spec>

[4] A. Chang, D. Barbier, and P. Dabbelt, RISC-V Platform-Level Interrupt Controller (PLIC) Specification. [Online]. Available: <https://github.com/riscv/riscv-plic-spec>

[5] E. Edgar and T. Newsome, Eds., RISC-V Debug Support, Version 1.0, May 2021. [Online]. Available: <https://github.com/riscv/riscv-debug-spec>

[6] C. Wolf, Ed., RISC-V Bitmanip Extension, Version 0.94, December 2020. [Online]. Available: <https://github.com/riscv/riscv-bitmanip>